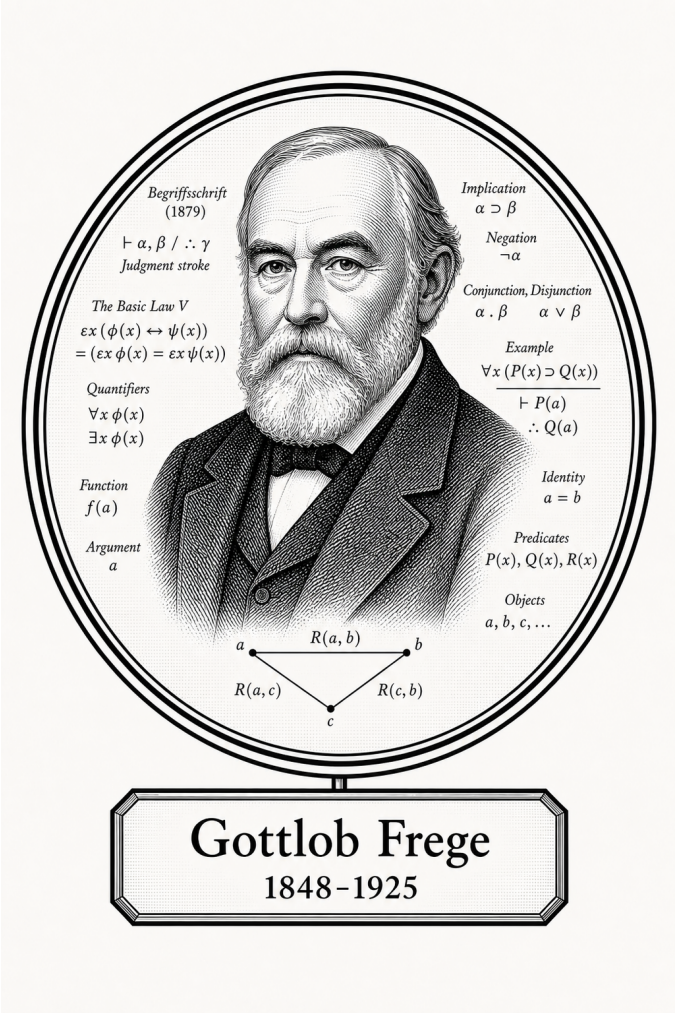
From Cantor to Ito

Logic, Sets, and Proof

Mathematical Logic and Proof



Gottlob Frege

1848-1925

Self-Study Notes

William Sollers

June 30, 2026

For every reader learning to make mathematics their own.

Contents

Chapter 1

Propositional Logic

1.1 Model-Theoretic View

Toolkit

This section presents classical propositional logic as a syntax-plus-semantics model, with the syntactic language connected to the two-valued semantics by the shared free-algebra evaluation machinery.

Theory (Propositional Logic)

Depends on: `Propositional language; formula; bridge:consequence;` $\mathrm{Cn}_{\vdash}$

Language

$$\mathcal{L}_{\mathsf{P}} = (\mathsf{P}, \neg, \wedge, \vee, \rightarrow, \leftrightarrow), \qquad \mathsf{Form}_{\mathsf{P}}.$$

Presented Theory

$$T := \mathrm{Cn}_{\vdash}(\Sigma), \qquad \Sigma \subseteq \mathsf{Form}_{\mathsf{P}}, \qquad \mathrm{CPL} := \mathrm{Cn}_{\vdash}(\varnothing).$$

Theorems and Consequences

1.
$$\Gamma \vdash_{\mathrm{CPL}} \varphi \iff \Gamma \models_{\mathrm{CPL}} \varphi \qquad \text{(completeness)}.$$

Model (Classical Propositional Logic)

Model 1.1.1 (Classical Propositional Logic)**. Depends on:** `Propositional language; valuation; bridge:free-sigma-algebra-evaluation; bridge:consequence`

Note

The seed is the model datum itself.

$$v : \mathsf{P} \to |\mathbf{2}|, \qquad |\mathbf{2}| = \{0, 1\}.$$

Binding

$$\Sigma := \Omega, \qquad X := \mathsf{P}, \qquad \text{target} := \mathbf{2}, \qquad \sigma := v, \qquad \widehat{v} := \mathrm{eval}(\mathbf{2})(v).$$

Syntax–Semantics Bridge

1.
$$v \models \varphi \quad :\Longleftrightarrow \quad \widehat{v}(\varphi) = 1.$$

Theorems and Consequences

1. Connective recursion is inherited from *Free Σ-Algebra and Evaluation*.
2. Semantic consequence, validity, and $\mathrm{Th}(v)$ are inherited from *Consequence*.

Definitional Extension (Propositional Logic)

Depends on: `theory:propositional-logic; truth-function; conservative extension`

Admissibility

A connective $\star$ of arity n is admissible by a defining equivalence:

$$\star(\bar{\varphi}) \leftrightarrow \theta(\bar{\varphi}), \qquad \widehat{v}(\star(\bar{\varphi})) = F_\star(\widehat{v}(\varphi_1), \ldots, \widehat{v}(\varphi_n)).$$

There is no $\exists!$ obligation because formulas already evaluate to truth values.

Instances

$$\top :\equiv (p_0 \to p_0), \qquad \bot :\equiv \neg\top,$$
$$\varphi \oplus \psi :\equiv ((\varphi \vee \psi) \wedge \neg(\varphi \wedge \psi)).$$

Theorems and Consequences

1.
$$\mathrm{CPL}^+ \vdash \sigma \quad \Longleftrightarrow \quad \mathrm{CPL} \vdash \sigma \qquad (\sigma \in \mathcal{L}_\mathsf{P}).$$

2. Every extended formula has a base-language equivalent.

1.2 Axiom Schemas

Propositional Axiom Schemas

The propositional-logic development uses the following classical Hilbert-style axiom schemas as logical starting points. They are schemas: each substitution of formulas for the displayed schematic letters gives an axiom instance.

Axiom Schema (Propositional Weakening)

Axiom 1.2.1 (Propositional Weakening)**.** For all formulas P and Q,

$$P \Longrightarrow (Q \Longrightarrow P).$$

Remark (Interpretation)*.* If P is available, then it remains available under any additional assumption Q.

Axiom Schema (Implication Distribution)

Axiom 1.2.2 (Implication Distribution)**.** For all formulas P, Q, and R,

$$\big(P \Longrightarrow (Q \Longrightarrow R)\big) \Longrightarrow \big((P \Longrightarrow Q) \Longrightarrow (P \Longrightarrow R)\big).$$

Remark (Interpretation)*.* Implications may be composed under a common hypothesis.

Axiom Schema (Classical Contraposition)

Axiom 1.2.3 (Classical Contraposition)**.** For all formulas P and Q,

$$(\neg Q \Longrightarrow \neg P) \Longrightarrow (P \Longrightarrow Q).$$

Remark (Interpretation)*.* This is the classical contrapositive principle. It is the axiom schema that marks the background logic as classical rather than intuitionistic.

1.3 Notation and Conventions

Toolkit

This notation ledger records the propositional symbols, metavariables, semantic notation, and normal-form abbreviations used in the chapter.

Remark (Exposition)*.* The notation section fixes the symbolic vocabulary used throughout the chapter. It identifies which symbols are global conventions, which are local to propositional logic, and where each item is introduced by the formal development.

Symbol	Name	Meaning	Introduced by
$\mathbb{B}$	truth-value set	the set of truth values (global, canonical)	Truth Value
Prop	propositional variable set	the set of propositional variables (global, canonical)	Propositional Language
$\neg$	negation	unary propositional connective (global, canonical)	Propositional Language
$\wedge$	conjunction	binary propositional connective (global, canonical)	Propositional Language
$\vee$	disjunction	binary propositional connective (global, canonical)	Propositional Language
$\to$	conditional	binary propositional connective (global, canonical)	Propositional Language
$\leftrightarrow$	biconditional	binary propositional connective (global, canonical)	Propositional Language
P, Q, R	propositional variables	schematic propositional variables (chapter, canonical)	Propositional Variable
$P_1, \ldots, P_n$	finite list of propositional variables	a finite schematic list of propositional variables (chapter, local)	Propositional Variable
$\circ$	generic binary connective	an arbitrary member of $\{\wedge, \vee, \to, \leftrightarrow\}$ (global, canonical)	Logical Connective
WFF	well-formed formula set	the set of propositional well-formed formulas (chapter, canonical)	Well-Formed Formula
$\varphi, \psi, \chi, \theta$	formula metavariables	metavariables ranging over propositional well-formed formulas (global, canonical)	Well-Formed Formula
α, β	additional formula metavariables	formula metavariables used in comparison statements (chapter, local)	Constructor Injectivity for Propositional Formulas
$\mathrm{Tree}(\varphi)$	syntax tree	parse tree or formation tree of a formula (chapter, canonical)	Parse Tree
$\mathrm{Var}(\varphi)$	variable set of a formula	the set of propositional variables occurring in a formula (chapter, canonical)	Variable Set of a Formula
$\mathrm{Sub}(\varphi)$	subformula set	the set of subformulas of a formula (chapter, canonical)	Subformula
$\mathrm{depth}(\varphi)$	formula depth	height, rank, or complexity of the formula formation tree (chapter, canonical)	Formula Depth

$\mathrm{conn}(\varphi)$	connective count	the number of connective occurrences in a formula (chapter, canonical)	Connective Count
v	truth assignment	a truth assignment on propositional variables (chapter, canonical)	Truth Assignment
w	second truth assignment	truth assignment on propositional variables (global, canonical)	Truth Assignment
$\widehat{v}$	extended truth assignment	unique extension of a truth assignment to formulas (chapter, canonical)	Unique Extension of a Truth Assignment
$\neg, \wedge, \vee, \to, \leftrightarrow$	standard connectives	standard propositional connectives (global, canonical)	Unique Extension of a Truth Assignment
$v \models \varphi$	satisfaction of a formula	truth assignment satisfies a formula (chapter, canonical)	Satisfaction of a Formula
$\models \varphi$	tautology notation	formula valid under every truth assignment (global, canonical)	Satisfaction of a Formula
$v \models \Gamma$	satisfaction of a set	truth assignment satisfies each formula in a set (chapter, canonical)	Satisfaction of a Set of Formulas
Γ	set of premise formulas	a set of propositional formulas (chapter, canonical)	Satisfaction of a Set of Formulas
$\Gamma \models \varphi$	semantic consequence	semantic consequence from premises to conclusion (chapter, canonical)	Logical Consequence
$\varphi \equiv \psi$	logical equivalence	same truth value under every truth assignment (chapter, canonical)	Logical Equivalence
$f : \mathbb{B}^n \to \mathbb{B}$	n-ary Boolean function	function from truth-value tuples to a truth value (chapter, canonical)	Boolean Function
f_φ	truth function induced by a formula	Boolean function represented by a formula (chapter, canonical)	Truth Function Induced by a Formula
$\equiv$	logical equivalence symbol	logical equivalence relation between formulas (global, canonical)	Propositional Equivalence Law
$\top$	tautological formula	a formula chosen as a true constant (chapter, canonical)	Identity and Domination Laws

$\bot$	contradictory formula	a formula chosen as a false constant (chapter, canonical)	Identity and Domination Laws
$C[-]$	formula context	a one-hole propositional formula context (chapter, canonical)	Formula Context
$C[\varphi]$	context substitution	formula obtained by filling a context with a formula (chapter, canonical)	Formula Context

1.4 Syntax

Toolkit

This section uses truth values, propositional variables, connectives, well-formed formulas, formation trees, and subformulas to build syntax.

Definitional Root (Truth Value)

Definition 1.4.1 (Truth Value)**.** The set of truth values is

$$\mathbb{B} := \{\text{False}, \text{True}\}.$$

Some sources identify this set with $\{0, 1\}$, with 0 read as false and 1 read as true.

Remark (Standard quantified statement)*.*

$$\mathbb{B} = \{\text{False}, \text{True}\}.$$

Remark (Interpretation)*.* Truth values are the semantic targets of propositional formulas. Syntax determines which strings are formulas; semantics later assigns elements of $\mathbb{B}$ to formulas.

Definitional Root (Propositional Language)

Definition 1.4.2 (Propositional Language)**.** A propositional language consists of:

1. a set Prop of propositional variables;
2. logical connectives;
3. punctuation symbols such as parentheses;
4. recursive formation rules specifying the well-formed formulas.

Remark (Standard quantified statement)*.*

$$\mathcal{L}_{\text{prop}} = (\mathsf{Prop}, \{\neg, \wedge, \vee, \to, \leftrightarrow\}, \text{punctuation}, \text{formation rules}).$$

Remark (Interpretation). The language supplies symbols and grammar. It does not yet assign truth values, truth conditions, or logical consequence.

Definition (Propositional Variable)

Definition 1.4.3 (Propositional Variable)**.** A propositional variable is an atomic symbol intended to stand for a proposition. A standard supply of variables is

$$\mathsf{Prop} = \{P_1, P_2, P_3, \ldots\}.$$

Informally, variables are often written $P, Q, R, S, \ldots$.

Remark (Standard quantified statement).

$$\text{PropositionalVariable}(P) \iff P \in \mathsf{Prop}.$$

Remark (Definition predicate reading). PropositionalVariable(P) means $P \in \mathsf{Prop}$.

Remark (Interpretation). A propositional variable has no internal syntactic structure in propositional logic. It is a basic symbol from which more complex formulas are formed.

Remark (Examples). P, Q, R, P_1, P_2 are propositional variables when they belong to Prop.

Remark (Non-examples). $\neg P$, $P \wedge Q$, and $(P \to Q)$ are not propositional variables; they are compound formulas.

Remark (Dependencies).

- Propositional Language

Definition (Logical Connective)

Definition 1.4.4 (Logical Connective)**.** The standard primitive propositional connectives are

$$\neg, \qquad \wedge, \qquad \vee, \qquad \to, \qquad \leftrightarrow .$$

The connective $\neg$ is unary. The connectives $\wedge, \vee, \to, \leftrightarrow$ are binary.

Symbol	Name	Arity	Formation pattern	Reading
$\neg$	negation	unary	$\neg\varphi$	not φ
$\wedge$	conjunction	binary	$(\varphi \wedge \psi)$	φ and ψ
$\vee$	disjunction	binary	$(\varphi \vee \psi)$	φ or ψ
$\to$	conditional	binary	$(\varphi \to \psi)$	if φ, then ψ
$\leftrightarrow$	biconditional	binary	$(\varphi \leftrightarrow \psi)$	φ iff ψ

Remark (Standard quantified statement).

$$\text{UnaryConnective}(\neg) \quad \text{and} \quad \text{BinaryConnective}(\wedge) \wedge \text{BinaryConnective}(\vee) \wedge \text{BinaryConnective}(\to) \wedge \text{Binary}$$

Remark (Interpretation). Arity records how many formulas a connective uses to form a new formula. The truth behavior of these connectives belongs to semantics, not to the syntactic definition.

Remark (Dependencies).

- Propositional Language

Definition (Well-Formed Formula)

Definition 1.4.5 (Well-Formed Formula)**.** The set WFF of well-formed formulas is the smallest set of strings satisfying the following formation rules:

1. Every propositional variable $P \in \mathsf{Prop}$ belongs to WFF.
2. If $\varphi \in \mathsf{WFF}$, then $\neg\varphi \in \mathsf{WFF}$.
3. If $\varphi, \psi \in \mathsf{WFF}$ and $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$, then $(\varphi \circ \psi) \in \mathsf{WFF}$.
4. No string is a well-formed formula unless it is obtained by finitely many applications of the previous clauses.

Remark (Standard quantified statement).

$$\mathsf{WFF} = \bigcap \big\{ S : \mathsf{Prop} \subseteq S,\ (\forall \varphi \in S)(\neg\varphi \in S),\ (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \to, \leftrightarrow\})\big((\varphi \circ \psi) \in S\big) \big\}.$$

Remark (Definition predicate reading). WellFormedFormula(φ) means $\varphi \in \mathsf{WFF}$, read as "φ is a well-formed formula."

Remark (Negated quantified statement).

$$\sigma \notin \mathsf{WFF} \iff \exists S\, [\mathsf{Prop} \subseteq S \wedge (\forall \varphi \in S)(\neg\varphi \in S) \wedge (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \to, \leftrightarrow\})((\varphi \circ \psi) \in S) \wedge \sigma \notin S]$$

Remark (Negation predicate reading). $\neg$ WellFormedFormula(σ) means the string σ is excluded by at least one formation-closed set containing Prop; equivalently, it is not generated by finitely many applications of the formation rules.

Remark (Failure modes). A string fails to be well formed when it violates the formation grammar: an operand may be missing, a connective may be missing, parentheses may be unmatched, or the string may not be generated by finitely many formation steps.

Remark (Failure mode decomposition).

$$\underbrace{P\wedge}_{\text{missing right operand}} \qquad \underbrace{(P\ Q)}_{\text{missing binary connective}} \qquad \underbrace{\neg \vee P}_{\text{negation applied to a non-formula}}.$$

Remark (Interpretation). Well-formed formulas are exactly the strings to which the semantic and proof-theoretic machinery of propositional logic applies.

Remark (Examples). P, $\neg P$, $(P \to Q)$, and $(P \to (Q \vee R))$ are well-formed formulas.

Remark (Non-examples). $P\wedge$, (PQ), and $\neg \vee P$ are not well-formed formulas.

Remark (Dependencies).

- Propositional Variable

- Logical Connective

Lemma (Closure Under Formation)

Lemma 1.4.6 (Closure Under Formation)**.** *The set* WFF *is closed under the propositional formation rules.*
That is, if $\varphi \in \mathsf{WFF}$*, then*

$$\neg\varphi \in \mathsf{WFF}.$$

If $\varphi, \psi \in \mathsf{WFF}$ *and*

$$\circ \in \{\wedge, \vee, \to, \leftrightarrow\},$$

then

$$(\varphi \circ \psi) \in \mathsf{WFF}.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall\varphi \in \mathsf{WFF})(\neg\varphi \in \mathsf{WFF})$$

and

$$(\forall\varphi, \psi \in \mathsf{WFF})(\forall\circ \in \{\wedge, \vee, \to, \leftrightarrow\})((\varphi \circ \psi) \in \mathsf{WFF}).$$

Remark (Interpretation)*.* Closure says that every legal use of a formation rule produces another well-formed formula. It is the forward direction of the recursive grammar.

Remark (Dependencies)*.*

- Well-Formed Formula

Theorem (Minimality of Well-Formed Formulas)

Theorem 1.4.7 (Minimality of Well-Formed Formulas)**.** *Let* S *be a set of strings over the propositional alphabet. Suppose:*

1. $\mathsf{Prop} \subseteq S$*;*
2. *if* $\varphi \in S$*, then* $\neg\varphi \in S$*;*
3. *if* $\varphi, \psi \in S$ *and* $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$*, then* $(\varphi \circ \psi) \in S$*.*

Then

$$\mathsf{WFF} \subseteq S.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall S)\Big[\mathsf{Prop} \subseteq S \wedge (\forall\varphi \in S)(\neg\varphi \in S) \\ \wedge (\forall\varphi, \psi \in S)(\forall\circ \in \{\wedge, \vee, \to, \leftrightarrow\})((\varphi \circ \psi) \in S)\Big] \to \mathsf{WFF} \subseteq S.$$

Remark (Interpretation). Minimality is the formal content of the clause "nothing else is a wff." It says that WFF is contained in every set of strings that contains the propositional variables and is closed under the formation rules.

Remark (Dependencies).

- Well-Formed Formula

Definition (Atomic Formula)

Definition 1.4.8 (Atomic Formula)**.** An atomic formula is a well-formed formula that is a propositional variable.

Remark (Standard quantified statement).

$$\text{AtomicFormula}(\varphi) \iff \varphi \in \mathsf{WFF} \land \varphi \in \mathsf{Prop}.$$

Remark (Definition predicate reading). AtomicFormula(φ) means that φ is a propositional variable viewed as a well-formed formula.

Remark (Interpretation). Atomic formulas are the base cases of the recursive grammar. They contain no connective structure.

Remark (Dependencies).

- Well-Formed Formula
- Propositional Variable

Definition (Molecular Formula)

Definition 1.4.9 (Molecular Formula)**.** A molecular formula is a well-formed formula that is not atomic. Equivalently, it is a well-formed formula formed using at least one logical connective.

Remark (Standard quantified statement).

$$\text{MolecularFormula}(\varphi) \iff \varphi \in \mathsf{WFF} \land \neg\, \text{AtomicFormula}(\varphi).$$

Remark (Definition predicate reading). MolecularFormula(φ) means that φ has nontrivial connective structure.

Remark (Interpretation). Molecular formulas are exactly the formulas obtained by applying negation or a binary connective at least once.

Remark (Dependencies).

- Well-Formed Formula
- Atomic Formula
- Logical Connective

Definition (Main Connective)

Definition 1.4.10 (Main Connective)**.** The main connective of a molecular formula is the connective introduced at the final formation step:

1. for $\neg\varphi$, the main connective is $\neg$;
2. for $(\varphi \circ \psi)$, the main connective is $\circ$.

Atomic formulas have no main connective.

Remark (Standard quantified statement)*.*

$$\text{MainConnective}(\neg\varphi, \neg) \quad \text{and} \quad \text{MainConnective}((\varphi \circ \psi), \circ).$$

Remark (Definition predicate reading)*.* MainConnective(φ, c) means that c is the outermost connective introduced in the final formation step of φ.

Remark (Interpretation)*.* The main connective identifies the outermost syntactic operation. It is the first connective used when analyzing a formula from the outside inward.

Remark (Dependencies)*.*

- Well-Formed Formula
- Molecular Formula
- Logical Connective

Theorem (Structural Induction for Propositional Formulas)

Theorem 1.4.11 (Structural Induction for Propositional Formulas)**.** *Let $A(\varphi)$ be a property of well-formed formulas. Suppose:*

1. *$A(P)$ holds for every $P \in \mathsf{Prop}$;*
2. *if $A(\varphi)$ holds, then $A(\neg\varphi)$ holds;*
3. *if $A(\varphi)$ and $A(\psi)$ hold, then $A((\varphi \circ \psi))$ holds for every $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$.*

Then $A(\theta)$ holds for every $\theta \in \mathsf{WFF}$.
Go to proof.

Remark (Standard quantified statement)*.*

$$\big[(\forall P \in \mathsf{Prop})A(P) \wedge (\forall\varphi \in \mathsf{WFF})(A(\varphi) \to A(\neg\varphi)) \wedge (\forall\varphi, \psi \in \mathsf{WFF})(\forall\circ \in \{\wedge, \vee, \to, \leftrightarrow\})\big((A(\varphi) \wedge A(\psi)) -$$

Remark (Interpretation)*.* To prove a statement for all formulas, it suffices to prove it for propositional variables and show it is preserved by each formula-forming operation.

Remark (Dependencies)*.*

- Well-Formed Formula

Theorem (Structural Recursion for Propositional Formulas)

Theorem 1.4.12 (Structural Recursion for Propositional Formulas)**.** *To define a function F on* WFF*, it suffices to specify:*

1. *$F(P)$ for every $P \in$ Prop;*
2. *$F(\neg\varphi)$ in terms of $F(\varphi)$;*
3. *$F((\varphi \circ \psi))$ in terms of $F(\varphi)$, $F(\psi)$, and $\circ$.*

When these clauses respect the formation rules, they determine a unique function on WFF*.*
Go to proof.

Remark (Standard quantified statement)*.*

recursive data on the formation clauses of $\mathsf{WFF} \Longrightarrow \exists! F : \mathsf{WFF} \to X$ satisfying those clauses.

Remark (Interpretation)*.* Structural recursion justifies recursive definitions such as variables of a formula, subformulas, depth, connective count, syntax tree, and truth evaluation.

Remark (Dependencies)*.*

- Well-Formed Formula
- Structural Induction for Propositional Formulas

Lemma (Constructor Disjointness for Propositional Formulas)

Lemma 1.4.13 (Constructor Disjointness for Propositional Formulas)**.** *The three outer formation cases for propositional formulas are disjoint:*

1. *no propositional variable is a negation formula;*
2. *no propositional variable is a binary formula;*
3. *no negation formula is a binary formula.*

Go to proof.

Remark (Standard quantified statement)*.*

$$P \in \mathsf{Prop} \Rightarrow [(\forall \varphi \in \mathsf{WFF})(P \neq \neg\varphi) \wedge (\forall \varphi, \psi \in \mathsf{WFF})(\forall \circ)(P \neq (\varphi \circ \psi))]$$

and

$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})(\forall \circ)\big(\neg\varphi \neq (\psi \circ \chi)\big).$$

Remark (Predicate reading).

$$\underbrace{P}_{\text{atomic}} \qquad \underbrace{\neg\varphi}_{\text{negation case}} \qquad \underbrace{(\varphi \circ \psi)}_{\text{binary case}}$$

belong to three distinct outer-form classes.

Remark (Interpretation). A formula cannot be parsed as two different outer formation types.

Remark (Dependencies).

- Well-Formed Formula
- Atomic Formula
- Main Connective

Lemma (Constructor Injectivity for Propositional Formulas)

Lemma 1.4.14 (Constructor Injectivity for Propositional Formulas)**.** *The propositional formula constructors are injective:*

1. *if* $\neg\varphi = \neg\psi$*, then* $\varphi = \psi$*;*
2. *if* $(\varphi \circ \psi) = (\alpha \star \beta)$*, then* $\varphi = \alpha$*,* $\psi = \beta$*, and* $\circ = \star$*.*

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})(\neg\varphi = \neg\psi \to \varphi = \psi)$$

and

$$(\forall \varphi, \psi, \alpha, \beta \in \mathsf{WFF})(\forall \circ, \star \in \{\wedge, \vee, \to, \leftrightarrow\})\left[(\varphi \circ \psi) = (\alpha \star \beta) \to (\varphi = \alpha \wedge \psi = \beta \wedge \circ = \star)\right].$$

Remark (Interpretation). If two formulas are built by the same outer constructor and are equal as strings, then their immediate constituents are equal.

Remark (Dependencies).

- Well-Formed Formula
- Logical Connective

Theorem (Unique Decomposition for Propositional Formulas)

Theorem 1.4.15 (Unique Decomposition for Propositional Formulas)**.** *Every* $\varphi \in \mathsf{WFF}$ *has exactly one outermost form:*

1. $\varphi = P$ *for a unique* $P \in \mathsf{Prop}$*;*
2. $\varphi = \neg\psi$ *for a unique* $\psi \in \mathsf{WFF}$*;*
3. $\varphi = (\psi \circ \chi)$ *for unique* $\psi, \chi \in \mathsf{WFF}$ *and a unique binary connective* $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$*.*

Go to proof.

Remark (Standard quantified statement)*.*

$$\forall \varphi \in \mathsf{WFF}, \quad \text{exactly one of the following holds:}$$

$$\varphi \in \mathsf{Prop}, \qquad \exists! \psi \in \mathsf{WFF}\,(\varphi = \neg\psi), \qquad \exists! \psi, \chi \in \mathsf{WFF}\ \exists! \circ \in \{\wedge, \vee, \to, \leftrightarrow\}\,(\varphi = (\psi \circ \chi)).$$

Remark (Predicate reading)*.*

$$\varphi \in \underbrace{\mathsf{Prop}}_{\text{atomic case}} \quad \text{or} \quad \varphi = \underbrace{\neg\psi}_{\text{negation case}} \quad \text{or} \quad \varphi = \underbrace{(\psi \circ \chi)}_{\text{binary case}},$$

and exactly one of these alternatives holds.

Remark (Interpretation)*.* Unique decomposition makes recursive definitions and induction on formulas unambiguous.

Remark (Dependencies)*.*

- Well-Formed Formula
- Constructor Disjointness for Propositional Formulas
- Constructor Injectivity for Propositional Formulas

Definition (Parse Tree)

Definition 1.4.16 (Parse Tree)**.** The parse tree, or syntax tree, of a well-formed formula is the tree representation of its recursive construction:

1. a propositional variable is represented by a single leaf;
2. a formula $\neg\varphi$ is represented by a root labeled $\neg$ with one child, the parse tree of φ;
3. a formula $(\varphi \circ \psi)$ is represented by a root labeled $\circ$ with two children, the parse trees of φ and ψ.

Remark (Standard quantified statement).

$$\text{Tree} : \mathsf{WFF} \to \{\text{finite rooted labeled trees}\}$$

is defined recursively by the three formation clauses for propositional formulas.

Remark (Interpretation). The parse tree records the same structure described by unique decomposition. It turns the recursive construction of a formula into a visible tree.

Remark (Dependencies).

- Well-Formed Formula
- Unique Decomposition for Propositional Formulas

Theorem (Unique Syntax Tree for Propositional Formulas)

Theorem 1.4.17 (Unique Syntax Tree for Propositional Formulas)**.** *Every propositional well-formed formula has a unique parse tree.*
Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \in \mathsf{WFF}, \quad \exists! T \ (T = \text{Tree}(\varphi)).$$

Remark (Interpretation). A formula has exactly one tree representation of its recursive construction. This is the tree form of unique decomposition.

Remark (Dependencies).

- Parse Tree
- Unique Decomposition for Propositional Formulas

Definition (Variable Set of a Formula)

Definition 1.4.18 (Variable Set of a Formula)**.** The variable set $\text{Var}(\varphi)$ of a formula φ is defined recursively:

1. if $\varphi = P \in \mathsf{Prop}$, then $\text{Var}(\varphi) = \{P\}$;
2. if $\varphi = \neg\psi$, then $\text{Var}(\varphi) = \text{Var}(\psi)$;
3. if $\varphi = (\psi \circ \chi)$, then
$$\text{Var}(\varphi) = \text{Var}(\psi) \cup \text{Var}(\chi).$$

Remark (Standard quantified statement).

$$\text{Var}(P) = \{P\}, \qquad \text{Var}(\neg\psi) = \text{Var}(\psi), \qquad \text{Var}((\psi \circ \chi)) = \text{Var}(\psi) \cup \text{Var}(\chi).$$

Remark (Interpretation). The variable set records exactly which propositional variables occur in a formula.

Remark (Dependencies).

- Well-Formed Formula
- Structural Recursion for Propositional Formulas

Definition (Subformula)

Definition 1.4.19 (Subformula)**.** The set $\mathrm{Sub}(\varphi)$ of subformulas of a formula φ is defined recursively:

1. if φ is atomic, then $\mathrm{Sub}(\varphi) = \{\varphi\}$;
2. if $\varphi = \neg\psi$, then
$$\mathrm{Sub}(\varphi) = \{\varphi\} \cup \mathrm{Sub}(\psi);$$
3. if $\varphi = (\psi \circ \chi)$, then
$$\mathrm{Sub}(\varphi) = \{\varphi\} \cup \mathrm{Sub}(\psi) \cup \mathrm{Sub}(\chi).$$

Remark (Standard quantified statement).

$$\mathrm{Sub}(P) = \{P\}, \qquad \mathrm{Sub}(\neg\psi) = \{\neg\psi\} \cup \mathrm{Sub}(\psi),$$
$$\mathrm{Sub}((\psi \circ \chi)) = \{(\psi \circ \chi)\} \cup \mathrm{Sub}(\psi) \cup \mathrm{Sub}(\chi).$$

Remark (Interpretation). Subformulas are the formulas that occur at nodes of the parse tree.

Remark (Dependencies).

- Well-Formed Formula
- Unique Decomposition for Propositional Formulas

Definition (Proper Subformula)

Definition 1.4.20 (Proper Subformula)**.** Let $\varphi \in \mathsf{WFF}$. A *proper subformula* of φ is a formula ψ such that
$$\psi \in \mathrm{Sub}(\varphi) \quad \text{and} \quad \psi \neq \varphi.$$

Remark (Standard quantified statement).

$$\mathrm{ProperSubformula}(\psi, \varphi) \iff \psi \in \mathrm{Sub}(\varphi) \wedge \psi \neq \varphi.$$

Remark (Interpretation). Proper subformulas are the strict syntactic parts of a formula. They exclude the whole formula itself.

Remark (Dependencies).

- Subformula

- Well-Formed Formula

Definition (Formula Depth)

Definition 1.4.21 (Formula Depth)**.** The depth of a formula φ, denoted $\text{depth}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\text{depth}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then

$$\text{depth}(\varphi) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$

Remark (Standard quantified statement)*.*

$$\text{depth}(P) = 0, \qquad \text{depth}(\neg\psi) = \text{depth}(\psi) + 1,$$

$$\text{depth}((\psi \circ \chi)) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$

Remark (Interpretation)*.* Formula depth measures the height of the syntax tree. Some authors call this the rank, complexity, or formation depth of the formula.

Remark (Dependencies)*.*

- Well-Formed Formula
- Parse Tree

Definition (Connective Count)

Definition 1.4.22 (Connective Count)**.** The connective count of a formula φ, denoted $\text{conn}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\text{conn}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\text{conn}(\varphi) = \text{conn}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then

$$\text{conn}(\varphi) = \text{conn}(\psi) + \text{conn}(\chi) + 1.$$

Remark (Standard quantified statement)*.*

$$\text{conn}(P) = 0, \qquad \text{conn}(\neg\psi) = \text{conn}(\psi) + 1,$$

$$\text{conn}((\psi \circ \chi)) = \text{conn}(\psi) + \text{conn}(\chi) + 1.$$

Remark (Interpretation)*.* Formula depth measures the height of the syntax tree. Connective count measures the number of connective nodes in the syntax tree.

Remark (Dependencies).

- Well-Formed Formula
- Parse Tree

Lemma (Proper Subformula Depth)

Lemma 1.4.23 (Proper Subformula Depth)**.** *If ψ is a proper subformula of φ, then*

$$\operatorname{depth}(\psi) < \operatorname{depth}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\psi \in \operatorname{Sub}(\varphi) \wedge \psi \neq \varphi \rightarrow \operatorname{depth}(\psi) < \operatorname{depth}(\varphi)\right].$$

Remark (Interpretation). Every descent into a proper subformula lowers formula depth. This justifies induction on formula depth and recursive proofs over subformulas.

Remark (Dependencies).

- Subformula
- Formula Depth

Lemma (Finiteness of Variables in a Formula)

Lemma 1.4.24 (Finiteness of Variables in a Formula)**.** *For every $\varphi \in \mathsf{WFF}$, the set $\operatorname{Var}(\varphi)$ is finite.*
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF}) \quad \operatorname{Var}(\varphi) \text{ is finite.}$$

Remark (Interpretation). Every formula depends on only finitely many propositional variables, even though the language may contain infinitely many variables.

Remark (Dependencies).

- Variable Set of a Formula
- Structural Induction for Propositional Formulas

Lemma (Finiteness of Subformulas)

Lemma 1.4.25 (Finiteness of Subformulas)**.** *For every $\varphi \in \mathsf{WFF}$, the set $\operatorname{Sub}(\varphi)$ is finite.*
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF}) \quad \mathrm{Sub}(\varphi) \text{ is finite.}$$

Remark (Interpretation). Every formula has only finitely many syntactic parts.

Remark (Dependencies).

- Subformula
- Structural Induction for Propositional Formulas

1.5 Semantics

Toolkit

This section uses truth assignments, recursive extensions, satisfaction, truth tables, and logical equivalence to assign meanings to formulas.

Remark (Exposition). The semantics section assigns truth conditions to the syntactic formulas introduced earlier. Truth assignments, recursive extensions, satisfaction, and truth tables provide the machinery for validity, consequence, equivalence, and semantic classification.

Truth Assignments

Definition (Truth Assignment)

Definition 1.5.1 (Truth Assignment)**.** A *truth assignment* is a function

$$v : \mathsf{Prop} \to \mathbb{B}.$$

For each propositional variable $P \in \mathsf{Prop}$, the value $v(P)$ is the truth value assigned to P.

Remark (Standard quantified statement).

$$\mathrm{TruthAssignment}(v) \iff v : \mathsf{Prop} \to \mathbb{B}.$$

Remark (Definition predicate reading).

$$\mathrm{TruthAssignment}(v)$$

means that v assigns exactly one truth value to each propositional variable.

Remark (Interpretation). A truth assignment gives semantic values to the atomic formulas. The recursive truth conditions for the connectives then determine the truth value of every well-formed formula.

Remark (Examples). If $\mathsf{Prop} = \{P_1, P_2, P_3, \ldots\}$, then a truth assignment may satisfy

$$v(P_1) = \mathsf{True}, \qquad v(P_2) = \mathsf{False}, \qquad v(P_3) = \mathsf{True},$$

with values assigned to every remaining propositional variable.

Remark (Non-Examples). A partial assignment that gives values only to P_1 and P_2 is not a truth assignment on Prop. It may be useful for local computations, but it is not a function $\mathsf{Prop} \to \mathbb{B}$.

Remark (Dependencies).

- Truth Value
- Propositional Variable
- Well-Formed Formula

Definition (Domain of a Truth Assignment)

Definition 1.5.2 (Domain of a Truth Assignment)**.** The *domain* of a truth assignment

$$v : \mathsf{Prop} \to \mathbb{B}$$

is the set Prop of propositional variables.

Remark (Standard quantified statement).

$$\mathrm{TruthAssignment}(v) \Longrightarrow \mathrm{dom}(v) = \mathsf{Prop}.$$

Remark (Interpretation). A truth assignment is global: it assigns truth values to all propositional variables in the language, even when a particular formula uses only finitely many of them.

Remark (Dependencies).

- Truth Assignment

Extension of a Truth Assignment

Definition (Extension of a Truth Assignment to Formulas)

Definition 1.5.3 (Extension of a Truth Assignment to Formulas)**.** Let

$$v : \mathsf{Prop} \to \mathbb{B}$$

be a truth assignment. The *extension* of v to well-formed formulas is the function

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

defined recursively as follows:

$$\widehat{v}(P) = v(P) \quad \text{for every } P \in \mathsf{Prop},$$

$$\widehat{v}(\neg\varphi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{False},$$
$$\widehat{v}(\varphi \wedge \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{True},$$
$$\widehat{v}(\varphi \vee \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ or } \widehat{v}(\psi) = \mathsf{True},$$
$$\widehat{v}(\varphi \to \psi) = \mathsf{False} \iff \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{False},$$

and

$$\widehat{v}(\varphi \leftrightarrow \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \widehat{v}(\psi).$$

Remark (Standard quantified statement). For every truth assignment $v : \mathsf{Prop} \to \mathbb{B}$, its extension is a function

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables, negation, conjunction, disjunction, material implication, and biconditional.

Remark (Definition predicate reading).

$$\mathrm{ExtendedTruthAssignment}(\widehat{v}, v)$$

means that $\widehat{v}$ is the recursively defined truth-value function on WFF induced by the atomic truth assignment v.

Remark (Interpretation). The original assignment v gives truth values only to propositional variables. The extension $\widehat{v}$ evaluates every formula by following the formula's syntactic construction.

Remark (Examples).

φ	$\neg\varphi$
True	False
False	True

φ	ψ	$\varphi \wedge \psi$
True	True	True
True	False	False
False	True	False
False	False	False

φ	ψ	$\varphi \vee \psi$
True	True	True
True	False	True
False	True	True
False	False	False

φ	ψ	$\varphi \to \psi$
True	True	True
True	False	False
False	True	True
False	False	True

φ	ψ	$\varphi \leftrightarrow \psi$
True	True	True
True	False	False
False	True	False
False	False	True

Remark (Examples). The truth tables are not additional formation rules. They specify how each primitive connective acts on truth values once the formula has already been formed syntactically.

Remark (Dependencies).

- Truth Assignment
- Well-Formed Formula
- Structural Recursion for Propositional Formulas
- Unique Decomposition for Propositional Formulas

Theorem (Unique Extension of a Truth Assignment)

Theorem 1.5.4 (Unique Extension of a Truth Assignment)**.** *Every truth assignment*

$$v : \mathsf{Prop} \to \mathbb{B}$$

has a unique extension

$$\widehat{v} : \mathsf{WFF} \to \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables and the primitive connectives.
Go to proof.

Remark (Standard quantified statement).

$$(\forall v : \mathsf{Prop} \to \mathbb{B})(\exists! \widehat{v} : \mathsf{WFF} \to \mathbb{B})$$

such that $\widehat{v}$ agrees with v on Prop and satisfies the recursive truth clauses for

$$\neg, \wedge, \vee, \to, \leftrightarrow .$$

Remark (Predicate reading).

$$\mathrm{TruthAssignment}(v) \Longrightarrow \exists! \widehat{v}\ \mathrm{ExtendedTruthAssignment}(\widehat{v}, v).$$

Remark (Interpretation). This theorem guarantees that semantic evaluation is well-defined. Once the truth values of the propositional variables are fixed, every well-formed formula has exactly one truth value.

Remark (Dependencies).

- Truth Assignment
- Extension of a Truth Assignment to Formulas
- Structural Recursion for Propositional Formulas
- Unique Decomposition for Propositional Formulas

Definition (Satisfaction of a Formula)

Definition 1.5.5 (Satisfaction of a Formula)**.** Let v be a truth assignment and let $\varphi \in \mathsf{WFF}$. We say that v satisfies φ, written

$$v \models \varphi,$$

if

$$\widehat{v}(\varphi) = \text{True}.$$

Remark (Standard quantified statement).

$$v \models \varphi \iff \text{TruthAssignment}(v) \land \varphi \in \mathsf{WFF} \land \widehat{v}(\varphi) = \text{True}.$$

Remark (Definition predicate reading).

$$\text{SatisfiesFormula}(v, \varphi)$$

means $v \models \varphi$.

Remark (Negated quantified statement).

$$v \not\models \varphi \iff \widehat{v}(\varphi) = \text{False}.$$

Remark (Interpretation). The notation $v \models \varphi$ means that the formula φ is true under the truth assignment v.

Remark (Dependencies).

- Truth Assignment
- Unique Extension of a Truth Assignment
- Well-Formed Formula

Definition (Satisfaction of a Set of Formulas)

Definition 1.5.6 (Satisfaction of a Set of Formulas)**.** Let $\Gamma \subseteq \mathsf{WFF}$. A truth assignment v satisfies Γ, written

$$v \models \Gamma,$$

if $v \models \gamma$ for every $\gamma \in \Gamma$.

Remark (Standard quantified statement)*.*

$$v \models \Gamma \iff \mathrm{TruthAssignment}(v) \wedge \Gamma \subseteq \mathsf{WFF} \wedge (\forall \gamma \in \Gamma)(v \models \gamma).$$

Remark (Definition predicate reading)*.*

$$\mathrm{SatisfiesSet}(v, \Gamma)$$

means v satisfies every formula in Γ.

Remark (Negated quantified statement)*.*

$$v \not\models \Gamma \iff \exists \gamma \in \Gamma \text{ such that } v \not\models \gamma.$$

Remark (Interpretation)*.* Satisfaction of a set means simultaneous satisfaction of every formula in the set.

Remark (Dependencies)*.*

- Satisfaction of a Formula
- Truth Assignment

Definition (Truth Table)

Definition 1.5.7 (Truth Table)**.** Let $\varphi \in \mathsf{WFF}$ and suppose the variables occurring in φ are among $P_1, \ldots, P_n$. A truth table for φ is a finite table with one row for each assignment

$$a : \{P_1, \ldots, P_n\} \to \mathbb{B}$$

and a final column recording the value of $\widehat{v}(\varphi)$ for any truth assignment v extending a.

Remark (Standard quantified statement)*.*

$$\mathrm{TruthTable}(\varphi) \iff \varphi \in \mathsf{WFF} \wedge \mathrm{Var}(\varphi) = \{P_1, \ldots, P_n\} \wedge \text{the table has } 2^n \text{ assignment rows.}$$

Remark (Interpretation)*.* A truth table is finite because every formula contains only finitely many variables. It records the complete truth behavior of a formula.

Remark (Dependencies)*.*

- Variable Set of a Formula
- Finiteness of Variables in a Formula

- Unique Extension of a Truth Assignment

Definition (Tautology)

Definition 1.5.8 (Tautology)**.** A formula $\varphi \in \mathsf{WFF}$ is a tautology, written

$$\models \varphi,$$

if every truth assignment satisfies φ.

Remark (Standard quantified statement)*.*

$$\models \varphi \iff \varphi \in \mathsf{WFF} \land (\forall v : \mathsf{Prop} \to \mathbb{B})(v \models \varphi).$$

Remark (Definition predicate reading)*.*

$$\text{Tautology}(\varphi)$$

means $\models \varphi$.

Remark (Interpretation)*.* A tautology is true under every possible assignment of truth values to propositional variables.

Remark (Dependencies)*.*

- Satisfaction of a Formula
- Truth Assignment

Definition (Contradiction)

Definition 1.5.9 (Contradiction)**.** A formula $\varphi \in \mathsf{WFF}$ is a contradiction if no truth assignment satisfies φ.

Remark (Standard quantified statement)*.*

$$\text{Contradiction}(\varphi) \iff \varphi \in \mathsf{WFF} \land (\forall v : \mathsf{Prop} \to \mathbb{B})(v \not\models \varphi).$$

Remark (Definition predicate reading)*.*

$$\text{Contradiction}(\varphi)$$

means that φ is false under every truth assignment.

Remark (Interpretation)*.* A contradiction has no true row in its truth table.

Remark (Dependencies)*.*

- Satisfaction of a Formula
- Truth Assignment

Definition (Satisfiable Formula)

Definition 1.5.10 (Satisfiable Formula)**.** A formula $\varphi \in \mathsf{WFF}$ is satisfiable if at least one truth assignment satisfies it.

Remark (Standard quantified statement)*.*

$$\text{SatisfiableFormula}(\varphi) \iff \varphi \in \mathsf{WFF} \land \exists v : \mathsf{Prop} \to \mathbb{B} \text{ such that } v \models \varphi.$$

Remark (Definition predicate reading)*.*

$$\text{SatisfiableFormula}(\varphi)$$

means that φ is true under at least one truth assignment.

Remark (Interpretation)*.* A satisfiable formula has at least one true row in its truth table.

Remark (Dependencies)*.*

- Satisfaction of a Formula
- Truth Assignment

Definition (Contingency)

Definition 1.5.11 (Contingency)**.** A formula $\varphi \in \mathsf{WFF}$ is a contingency if it is satisfiable but not a tautology.

Remark (Standard quantified statement)*.*

$$\text{Contingency}(\varphi) \iff \text{SatisfiableFormula}(\varphi) \land \neg\, \text{Tautology}(\varphi).$$

Remark (Definition predicate reading)*.*

$$\text{Contingency}(\varphi)$$

means that φ is true under some truth assignments and false under others.

Remark (Interpretation)*.* A contingency has at least one true row and at least one false row in its truth table.

Remark (Dependencies)*.*

- Satisfiable Formula
- Tautology

Definition (Logical Consequence)

Definition 1.5.12 (Logical Consequence)**.** Let $\Gamma \subseteq \mathsf{WFF}$ and $\varphi \in \mathsf{WFF}$. The formula φ is a logical consequence of Γ, written

$$\Gamma \models \varphi,$$

if every truth assignment satisfying Γ also satisfies φ.

Remark (Standard quantified statement).

$$\Gamma \models \varphi \iff \Gamma \subseteq \mathsf{WFF} \land \varphi \in \mathsf{WFF} \land (\forall v : \mathsf{Prop} \to \mathbb{B})\,[v \models \Gamma \to v \models \varphi]\,.$$

Remark (Definition predicate reading).

$$\mathrm{LogicalConsequence}(\Gamma, \varphi)$$

means $\Gamma \models \varphi$.

Remark (Negated quantified statement).

$$\Gamma \not\models \varphi \iff \exists v : \mathsf{Prop} \to \mathbb{B}\,[v \models \Gamma \land v \not\models \varphi]\,.$$

Remark (Interpretation). Logical consequence means truth preservation from premises to conclusion. There is no truth assignment under which all premises are true and the conclusion is false.

Remark (Dependencies).

- Satisfaction of a Set of Formulas
- Satisfaction of a Formula

Definition (Logical Equivalence)

Definition 1.5.13 (Logical Equivalence)**.** Formulas $\varphi, \psi \in \mathsf{WFF}$ are logically equivalent, written

$$\varphi \equiv \psi,$$

if they have the same truth value under every truth assignment.

Remark (Standard quantified statement).

$$\varphi \equiv \psi \iff \varphi, \psi \in \mathsf{WFF} \land (\forall v : \mathsf{Prop} \to \mathbb{B})\,[\widehat{v}(\varphi) = \widehat{v}(\psi)]\,.$$

Remark (Definition predicate reading).

$$\mathrm{LogicalEquivalence}(\varphi, \psi)$$

means $\varphi \equiv \psi$.

Remark (Interpretation). Logically equivalent formulas have identical truth-table columns. They are interchangeable for semantic purposes.

Remark (Dependencies).

- Unique Extension of a Truth Assignment
- Truth Assignment

Lemma (Coincidence Lemma for Propositional Logic)

Lemma 1.5.14 (Coincidence Lemma for Propositional Logic)**.** *Let $v, w : \mathsf{Prop} \to \mathbb{B}$ be truth assignments. If v and w agree on every propositional variable occurring in φ, then they assign the same truth value to φ:*

$$v|_{\mathrm{Var}(\varphi)} = w|_{\mathrm{Var}(\varphi)} \Longrightarrow \widehat{v}(\varphi) = \widehat{w}(\varphi).$$

Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi \in \mathsf{WFF})(\forall v, w : \mathsf{Prop} \to \mathbb{B})\left[(\forall P \in \mathrm{Var}(\varphi))(v(P) = w(P)) \to \widehat{v}(\varphi) = \widehat{w}(\varphi)\right].$$

Remark (Interpretation)*.* The truth value of a formula depends only on the variables actually occurring in that formula.

Remark (Dependencies)*.*

- Variable Set of a Formula
- Unique Extension of a Truth Assignment
- Structural Induction for Propositional Formulas

Theorem (Finite Truth-Table Theorem)

Theorem 1.5.15 (Finite Truth-Table Theorem)**.** *Every propositional formula has a finite truth table. More precisely, if $\mathrm{Var}(\varphi)$ has n elements, then φ has a truth table with 2^n rows.*
Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi \in \mathsf{WFF})\left[|\mathrm{Var}(\varphi)| = n \to \mathrm{TruthTable}(\varphi) \text{ has } 2^n \text{ rows}\right].$$

Remark (Interpretation)*.* Truth tables are finite because every formula contains only finitely many propositional variables.

Remark (Dependencies)*.*

- Truth Table
- Finiteness of Variables in a Formula

Definition (Unary Boolean Function)

Definition 1.5.16 (Unary Boolean Function)**.** A unary Boolean function is a function

$$f : \mathbb{B} \to \mathbb{B}.$$

Remark (Definition predicate reading).

$$UnaryBooleanFunction(f)$$

means that f takes one truth value as input and returns one truth value.

Remark (Interpretation). A unary Boolean function has one input row coordinate. It is the semantic shape of a one-place truth operation such as negation.

Remark (Dependencies).

- Truth Value

Definition (Binary Boolean Function)

Definition 1.5.17 (Binary Boolean Function)**.** A binary Boolean function is a function

$$f : \mathbb{B}^2 \to \mathbb{B},$$

equivalently a function

$$f : \mathbb{B} \times \mathbb{B} \to \mathbb{B}.$$

Remark (Definition predicate reading).

$$BinaryBooleanFunction(f)$$

means that f takes an ordered pair of truth values as input and returns one truth value.

Remark (Interpretation). A binary Boolean function assigns one output truth value to each of the four input rows

$$(T, T), \quad (T, F), \quad (F, T), \quad (F, F).$$

Remark (Dependencies).

- Truth Value
- Unary Boolean Function

Definition (Boolean Function)

Definition 1.5.18 (Boolean Function)**.** An n-ary Boolean function is a function

$$f : \mathbb{B}^n \to \mathbb{B}.$$

Remark (Standard quantified statement).

$$BooleanFunction_n(f) \iff f : \mathbb{B}^n \to \mathbb{B}.$$

Remark (Definition predicate reading).

$$BooleanFunction_n(f)$$

means that f takes n truth values as input and returns one truth value.

Remark (Interpretation). Boolean functions are the abstract truth-functions that formulas may represent.

Remark (Rows versus Boolean functions). An input to an n-ary Boolean function is a tuple $(b_1, \dots, b_n) \in \mathbb{B}^n$. These tuples are the rows of the truth table, and there are 2^n of them. The Boolean function is not a single row; it is the full rule assigning exactly one truth value in $\mathbb{B}$ to every row.

Remark (Dependencies).

- Truth Value
- Unary Boolean Function
- Binary Boolean Function

Definition (Truth Function Induced by a Formula)

Definition 1.5.19 (Truth Function Induced by a Formula)**.** Let $\varphi \in \mathsf{WFF}$, and suppose $\mathrm{Var}(\varphi) \subseteq \{P_1, \dots, P_n\}$. The truth function induced by φ is the Boolean function

$$f_\varphi : \mathbb{B}^n \to \mathbb{B}$$

defined by

$$f_\varphi(b_1, \dots, b_n) = \widehat{v}(\varphi),$$

where $v(P_i) = b_i$ for each i.

Remark (Standard quantified statement).

$$f_\varphi(b_1, \dots, b_n) = \widehat{v}(\varphi) \quad \text{whenever } v(P_i) = b_i \text{ for } 1 \le i \le n.$$

Remark (Definition predicate reading).

$$\mathrm{TruthFunctionOfFormula}(f_\varphi, \varphi)$$

means that f_φ records the truth behavior of φ.

Remark (Interpretation). The induced truth function packages the truth table of a formula as a Boolean function.

Remark (Dependencies).

- Boolean Function
- Variable Set of a Formula
- Unique Extension of a Truth Assignment
- Coincidence Lemma for Propositional Logic

Theorem (Counting Boolean Functions)

Theorem 1.5.20 (Counting Boolean Functions)**.** *There are exactly*

$$2^{2^n}$$

Boolean functions $\mathbb{B}^n \to \mathbb{B}$.
Go to proof.

Remark (Standard quantified statement)*.*

$$|\{f : \mathbb{B}^n \to \mathbb{B}\}| = 2^{2^n}.$$

Remark (Interpretation)*.* The domain $\mathbb{B}^n$ has 2^n elements. A Boolean function chooses one of two truth values for each input row.

Remark (Dependencies)*.*

- Boolean Function

1.6 Algebra of Propositions

Toolkit

This section uses logical equivalence, equivalence laws, formula contexts, and replacement of logical equivalents to treat propositions algebraically.

Remark (Exposition)*.* The algebra section treats logically equivalent formulas as interchangeable for calculation. Equivalence laws and replacement principles turn semantic agreement under every truth assignment into a controlled rewrite calculus for propositional formulas.

Definition (Propositional Equivalence Law)

Definition 1.6.1 (Propositional Equivalence Law)**.** A propositional equivalence law is a valid schema of the form

$$\varphi \equiv \psi,$$

with $\varphi, \psi \in \mathsf{WFF}$. Each instance of the schema says that the two formulas have the same truth value under every truth assignment.

Remark (Standard quantified statement)*.*

$$\mathrm{PropositionalEquivalenceLaw}(\varphi, \psi) \iff \varphi, \psi \in \mathsf{WFF} \land \varphi \equiv \psi.$$

Remark (Definition predicate reading)*.*

$$\mathrm{PropositionalEquivalenceLaw}(\varphi, \psi)$$

means that $\varphi \equiv \psi$ is a valid logical-equivalence schema.

Remark (Interpretation). Equivalence laws are the algebraic rewrite rules of propositional logic. They allow formulas to be transformed without changing truth conditions.

Remark (Dependencies).

- Logical Equivalence
- Well-Formed Formula

Theorem (Logical Equivalence is an Equivalence Relation)

Theorem 1.6.2 (Logical Equivalence is an Equivalence Relation)**.** *Logical equivalence is reflexive, symmetric, and transitive on* WFF*:*

$$\varphi \equiv \varphi, \qquad \varphi \equiv \psi \Longrightarrow \psi \equiv \varphi,$$

and

$$(\varphi \equiv \psi \land \psi \equiv \chi) \Longrightarrow \varphi \equiv \chi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})(\varphi \equiv \varphi),$$
$$(\forall \varphi, \psi \in \mathsf{WFF})(\varphi \equiv \psi \to \psi \equiv \varphi),$$
$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})((\varphi \equiv \psi \land \psi \equiv \chi) \to \varphi \equiv \chi).$$

Remark (Interpretation). Logical equivalence behaves like equality at the semantic level.

Remark (Dependencies).

- Logical Equivalence

Theorem (Basic Boolean Equivalence Laws)

Theorem 1.6.3 (Basic Boolean Equivalence Laws)**.** *For all formulas* $\varphi, \psi, \chi \in \mathsf{WFF}$*, the following logical equivalences hold.*
Commutativity:

$$\varphi \land \psi \equiv \psi \land \varphi, \qquad \varphi \lor \psi \equiv \psi \lor \varphi.$$

Associativity:

$$(\varphi \land \psi) \land \chi \equiv \varphi \land (\psi \land \chi), \qquad (\varphi \lor \psi) \lor \chi \equiv \varphi \lor (\psi \lor \chi).$$

Idempotence:

$$\varphi \land \varphi \equiv \varphi, \qquad \varphi \lor \varphi \equiv \varphi.$$

Double negation:

$$\neg\neg\varphi \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi \in \mathsf{WFF}) \begin{bmatrix} \varphi \wedge \psi \equiv \psi \wedge \varphi, & \varphi \vee \psi \equiv \psi \vee \varphi, \\ (\varphi \wedge \psi) \wedge \chi \equiv \varphi \wedge (\psi \wedge \chi), & (\varphi \vee \psi) \vee \chi \equiv \varphi \vee (\psi \vee \chi), \\ \varphi \wedge \varphi \equiv \varphi, & \varphi \vee \varphi \equiv \varphi, \quad \neg\neg\varphi \equiv \varphi \end{bmatrix}.$$

Remark (Interpretation). These are the basic algebraic laws for conjunction, disjunction, and negation.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Identity and Domination Laws)

Theorem 1.6.4 (Identity and Domination Laws)**.** *Let $\top$ be any tautological formula and let $\bot$ be any contradictory formula. Then for every $\varphi \in \mathsf{WFF}$:*

$$\varphi \wedge \top \equiv \varphi, \qquad \varphi \vee \bot \equiv \varphi,$$

and

$$\varphi \wedge \bot \equiv \bot, \qquad \varphi \vee \top \equiv \top.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} &(\text{Tautology}(\top) \wedge \text{Contradiction}(\bot)) \to (\forall \varphi \in \mathsf{WFF}) \\ &\quad [(\varphi \wedge \top \equiv \varphi) \wedge (\varphi \vee \bot \equiv \varphi) \wedge (\varphi \wedge \bot \equiv \bot) \wedge (\varphi \vee \top \equiv \top)]. \end{aligned}$$

Remark (Interpretation). Tautologies act like true constants; contradictions act like false constants.

Remark (Dependencies).

- Tautology
- Contradiction
- Logical Equivalence

Theorem (De Morgan Laws for Propositional Logic)

Theorem 1.6.5 (De Morgan Laws for Propositional Logic)**.** *For all $\varphi, \psi \in \mathsf{WFF}$,*

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi,$$

and

$$\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\,[\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi]\,,$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\,[\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi]\,.$$

Remark (Predicate reading).

$$\underbrace{\neg(\varphi \wedge \psi)}_{\text{not both}} \equiv \underbrace{\neg\varphi \vee \neg\psi}_{\text{at least one fails}}\,, \qquad \underbrace{\neg(\varphi \vee \psi)}_{\text{neither}} \equiv \underbrace{\neg\varphi \wedge \neg\psi}_{\text{both fail}}\,.$$

Remark (Interpretation). Negation swaps conjunction with disjunction when pushed inward.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Distributive Laws for Propositional Logic)

Theorem 1.6.6 (Distributive Laws for Propositional Logic)**.** *For all* $\varphi, \psi, \chi \in \mathsf{WFF}$*,*

$$\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi),$$

and

$$\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})\,[\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi)]\,,$$
$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})\,[\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi)]\,.$$

Remark (Interpretation). Conjunction distributes over disjunction, and disjunction distributes over conjunction. These laws are the main algebraic engine behind CNF and DNF conversion.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Absorption Laws)

Theorem 1.6.7 (Absorption Laws)**.** *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \wedge (\varphi \vee \psi) \equiv \varphi,$$

and

$$\varphi \vee (\varphi \wedge \psi) \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\,[\varphi \wedge (\varphi \vee \psi) \equiv \varphi]\,,$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\,[\varphi \vee (\varphi \wedge \psi) \equiv \varphi]\,.$$

Remark (Interpretation). Once φ is already required, adding $\varphi \vee \psi$ contributes no additional constraint. Dually, once φ is already allowed, adding $\varphi \wedge \psi$ contributes no additional possibility.

Remark (Dependencies).

- Logical Equivalence
- Basic Boolean Equivalence Laws
- Distributive Laws for Propositional Logic

Theorem (Conditional Equivalence Laws)

Theorem 1.6.8 (Conditional Equivalence Laws)**.** *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \to \psi \equiv \neg\varphi \vee \psi,$$

and

$$\neg(\varphi \to \psi) \equiv \varphi \wedge \neg\psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\,[\varphi \to \psi \equiv \neg\varphi \vee \psi]\,,$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\,[\neg(\varphi \to \psi) \equiv \varphi \wedge \neg\psi]\,.$$

Remark (Predicate reading).

$$\underbrace{\varphi \to \psi}_{\text{no counterexample}} \equiv \underbrace{\neg\varphi \vee \psi}_{\text{premise false or conclusion true}}, \qquad \underbrace{\neg(\varphi \to \psi)}_{\text{counterexample}} \equiv \underbrace{\varphi \wedge \neg\psi}_{\text{premise true and conclusion false}}.$$

Remark (Interpretation). A conditional fails exactly when the antecedent is true and the consequent is false.

Remark (Dependencies).

- Logical Equivalence
- Unique Extension of a Truth Assignment

Theorem (Biconditional Equivalence Laws)

Theorem 1.6.9 (Biconditional Equivalence Laws)**.** *For all* $\varphi, \psi \in \mathsf{WFF}$*,*

$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi)\right],$$
$$(\forall \varphi, \psi \in \mathsf{WFF})\left[\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi)\right].$$

Remark (Predicate reading)*.*

$$\varphi \leftrightarrow \psi \quad \Longleftrightarrow \quad (\varphi \to \psi) \wedge (\psi \to \varphi)$$

and also

$$\varphi \leftrightarrow \psi \quad \Longleftrightarrow \quad (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

The biconditional says that the two formulas have the same truth value.

Remark (Interpretation)*.* A biconditional says that two formulas have the same truth value: both true or both false.

Remark (Dependencies)*.*

- Logical Equivalence
- Conditional Equivalence Laws

Definition (Formula Context)

Definition 1.6.10 (Formula Context)**.** A formula context $C[-]$ is a well-formed formula with one distinguished placeholder occurrence $[-]$. For each well-formed formula φ, the expression $C[\varphi]$ denotes the formula obtained by replacing the placeholder with φ.

Remark (Standard quantified statement)*.*

$$\text{FormulaContext}(C[-]) \Longrightarrow (\forall \varphi \in \mathsf{WFF})(C[\varphi] \in \mathsf{WFF}).$$

Remark (Definition predicate reading)*.*

$$\text{FormulaContext}(C[-])$$

means that $C[-]$ is a one-hole propositional formula context.

Remark (Interpretation). A formula context is a syntactic environment into which a formula can be inserted. Contexts make precise the idea of replacing a subformula inside a larger formula.

Remark (Examples). If

$$C[-] := ([-] \wedge R),$$

then

$$C[P] = (P \wedge R)$$

and

$$C[Q \vee S] = ((Q \vee S) \wedge R).$$

Remark (Dependencies).

- Well-Formed Formula
- Subformula

Theorem (Replacement of Logical Equivalents)

Theorem 1.6.11 (Replacement of Logical Equivalents)**.** *Replacing logically equivalent formulas inside a formula context preserves logical equivalence:*

$$\varphi \equiv \psi \Longrightarrow C[\varphi] \equiv C[\psi].$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})(\forall C[-])\left[\text{FormulaContext}(C[-]) \wedge \varphi \equiv \psi \to C[\varphi] \equiv C[\psi]\right].$$

Remark (Predicate reading).

$$\varphi \equiv \psi \quad \Longrightarrow \quad C[\varphi] \equiv C[\psi].$$

A logical equivalence may be used inside a larger formula context without changing the truth behavior of the whole formula.

Remark (Interpretation). Logical equivalence is substitutive: equivalent formulas may be replaced inside larger formulas without changing truth behavior.

Remark (Exposition). The equivalence laws in this section may be used as replacement rules inside larger formulas. If a formula contains a subformula matching one side of a listed equivalence law, that subformula may be replaced by the other side.

Common replacement rules include:

$$\neg\neg\varphi \equiv \varphi,$$

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi, \qquad \neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi,$$

$$\varphi \to \psi \equiv \neg\varphi \vee \psi,$$

$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \wedge (\psi \to \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Remark (Exposition). The laws in this section allow formulas to be manipulated algebraically. They are not new formation rules; they are semantic equivalences. Later sections use these laws to transform formulas into normal forms and to reduce the set of primitive connectives.

Remark (Exposition). The algebra of propositions is the bridge from semantics to computation: once formulas are equivalent, they can be rewritten without changing truth behavior.

Remark (Dependencies).

- Formula Context
- Logical Equivalence
- Coincidence Lemma for Propositional Logic
- Unique Extension of a Truth Assignment

1.7 Normal Forms

Toolkit

This section uses logical equivalence, truth assignments, literals, clauses, and algebraic rewrite laws to motivate normal forms.

Remark (Exposition). The normal-forms section studies controlled syntactic shapes for formulas. Logical equivalence is the governing invariant: conversion may change a formula's shape, but it must preserve truth under every assignment while exposing regular literal, clause, DNF, CNF, and canonical structure.

Orientation

Remark (Section map). This section uses the preceding syntax, semantics, and algebra of propositions as its starting point. It first isolates literals and clauses, then introduces negation normal form, disjunctive normal form, conjunctive normal form, canonical normal forms, and semantic classification uses. Later arguments will use these forms to organize satisfiability questions, proof search, functional completeness, and resolution-style methods.

Remark (Why normal forms matter). Normal forms trade arbitrary expression for regular structure. In a truth-table argument, regular structure helps classify formulas as tautologies, contradictions, satisfiable formulas, or contingencies. In proof search and satisfiability procedures, regular structure turns a formula into a finite combinatorial object that can be searched. In functional completeness, normal forms show how complicated truth functions can be built from simpler connectives.

Definition (Literal)

Definition 1.7.1 (Literal). A *literal* is either a propositional variable P or the negation $\neg P$ of a propositional variable P.

Remark (Standard quantified statement). For a formula $\ell \in \mathsf{WFF}$,

$$\ell \text{ is a literal} \iff (\exists P \in \mathsf{Prop})(\ell = P \text{ or } \ell = \neg P).$$

Remark (Definition predicate reading).

$$\mathrm{Literal}(\ell)$$

means that ℓ is an atomic propositional variable or the negation of one atomic propositional variable.

Remark (Interpretation). Literals are the indivisible signed atoms used in normal forms. They remember which propositional variable is being tested and whether it appears positively or negatively.

Remark (Examples). The formulas P, Q, $\neg P$, and $\neg Q$ are literals.

Remark (Non-Examples). The formulas $P \wedge Q$, $P \vee Q$, and $\neg(P \wedge Q)$ are not literals. The last formula is negated, but it negates a compound formula rather than a single propositional variable.

Remark (Dependencies).

- Propositional Variable
- Logical Connective

Definition (Positive Literal)

Definition 1.7.2 (Positive Literal)**.** A *positive literal* is a literal of the form P, where P is a propositional variable.

Remark (Standard quantified statement). For a formula $\ell \in \mathsf{WFF}$,

$$\ell \text{ is a positive literal} \iff (\exists P \in \mathsf{Prop})(\ell = P).$$

Remark (Definition predicate reading).

$$\mathrm{PositiveLiteral}(\ell)$$

means that ℓ is a literal whose sign is positive.

Remark (Interpretation). A positive literal asserts the propositional variable directly. It carries no leading negation.

Remark (Examples). The formulas P and Q are positive literals.

Remark (Dependencies).

- Literal

Definition (Negative Literal)

Definition 1.7.3 (Negative Literal)**.** A *negative literal* is a literal of the form $\neg P$, where P is a propositional variable.

Remark (Standard quantified statement). For a formula $\ell \in \mathsf{WFF}$,

$$\ell \text{ is a negative literal} \iff (\exists P \in \mathsf{Prop})(\ell = \neg P).$$

Remark (Definition predicate reading).

$$\mathrm{NegativeLiteral}(\ell)$$

means that ℓ is a literal whose sign is negative.

Remark (Interpretation). A negative literal denies exactly one propositional variable. In normal forms, negation is pushed down until it appears only at this literal level.

Remark (Examples). The formulas $\neg P$ and $\neg Q$ are negative literals.

Remark (Dependencies).

- Literal

Definition (Complementary Literals)

Definition 1.7.4 (Complementary Literals)**.** Two literals are *complementary* if they are P and $\neg P$ for the same propositional variable P.

Remark (Standard quantified statement). For literals ℓ_1, ℓ_2,

$$\ell_1 \text{ and } \ell_2 \text{ are complementary} \iff (\exists P \in \mathsf{Prop})\big((\ell_1 = P \wedge \ell_2 = \neg P) \vee (\ell_1 = \neg P \wedge \ell_2 = P)\big).$$

Remark (Definition predicate reading).

$$\mathrm{ComplementaryLiterals}(\ell_1, \ell_2)$$

means that the two literals name the same propositional variable with opposite signs.

Remark (Interpretation). Complementary literals are the local contradiction pattern inside clauses and terms. The pair $P, \neg P$ is tied to one variable, not merely to two formulas with opposite truth values under a particular assignment.

Remark (Examples). The literals P and $\neg P$ are complementary, as are Q and $\neg Q$.

Remark (Non-Examples). The literals P and $\neg Q$ are not complementary unless $P = Q$.

Remark (Dependencies).

- Literal

Definition (Clause)

Definition 1.7.5 (Clause)**.** A *clause* is a finite disjunction of literals.

Remark (Standard quantified statement). A formula $C \in \mathsf{WFF}$ is a clause if there are literals $\ell_1, \ldots, \ell_n$ with $n \geq 1$ such that

$$C = \ell_1 \vee \cdots \vee \ell_n.$$

The empty clause is treated separately as the empty disjunction.

Remark (Definition predicate reading).

$$\text{Clause}(C)$$

means that C is built by finitely disjoining literals.

Remark (Interpretation). A clause records a finite list of literal-level alternatives. It is satisfied when at least one of its literals is true.

Remark (Examples). The formulas $P \vee Q$, $P \vee \neg Q$, and $\neg P \vee Q \vee R$ are clauses.

Remark (Non-Examples). The formula $P \wedge Q$ is not a clause, because its top-level connective is a conjunction. The formula $\neg(P \vee Q)$ is not a clause, because its negation has not been pushed down to the literal level.

Remark (Dependencies).

- Literal

Definition (Term)

Definition 1.7.6 (Term)**.** A *term*, also called a *conjunction term*, is a finite conjunction of literals.

Remark (Standard quantified statement). A formula $T \in \mathsf{WFF}$ is a term if there are literals $\ell_1, \ldots, \ell_n$ with $n \geq 1$ such that

$$T = \ell_1 \wedge \cdots \wedge \ell_n.$$

The empty conjunction is treated separately.

Remark (Definition predicate reading).

$$\text{Term}(T)$$

means that T is built by finitely conjoining literals.

Remark (Interpretation). A term records a finite list of literal-level requirements. It is satisfied when all of its literals are true.

Remark (Examples). The formulas $P \wedge Q$, $P \wedge \neg Q$, and $\neg P \wedge Q \wedge R$ are terms.

Remark (Non-Examples). The formula $P \vee Q$ is not a term, because its top-level connective is a disjunction. The formula $\neg(P \wedge Q)$ is not a term, because its negation has not been pushed down to the literal level.

Remark (Dependencies).

- Literal

Definition (Empty Clause)

Definition 1.7.7 (Empty Clause)**.** The *empty clause* is the empty disjunction of literals. Semantically, it is read as false.

Remark (Standard quantified statement). The empty clause is the disjunction over no literals and has truth value False under every truth assignment.

Remark (Definition predicate reading).

$$\text{EmptyClause}(C)$$

means that C is the clause with no disjuncts.

Remark (Interpretation). A nonempty clause is true when at least one listed literal is true. With no literals listed, there is no witness to make the disjunction true, so the empty clause is the false boundary case for clauses.

Remark (Dependencies).

- Clause
- Contradiction

Definition (Empty Conjunction)

Definition 1.7.8 (Empty Conjunction)**.** The *empty conjunction* is the conjunction over no literals. Semantically, it is read as true.

Remark (Standard quantified statement). The empty conjunction is the conjunction over no literals and has truth value True under every truth assignment.

Remark (Definition predicate reading).

$$\text{EmptyConjunction}(T)$$

means that T is the term with no conjuncts.

Remark (Interpretation). A nonempty term is true when every listed literal is true. With no literals listed, there is no requirement to violate, so the empty conjunction is the true boundary case for terms.

Remark (Dependencies).

- Term
- Tautology

Definition (Negation Normal Form)

Definition 1.7.9 (Negation Normal Form)**.** A formula is in *negation normal form* if it uses only propositional variables, negated propositional variables, conjunction, and disjunction, and every negation occurs immediately before a propositional variable.

Remark (Standard quantified statement). For a formula $\varphi \in \mathsf{WFF}$,

$$\varphi \text{ is in negation normal form} \iff \varphi \text{ is built from literals using only } \wedge \text{ and } \vee.$$

Remark (Definition predicate reading).

$$\text{NegationNormalForm}(\varphi)$$

means that φ has no conditionals, no biconditionals, and no negation outside the literal level.

Remark (Interpretation). Negation normal form is the first cleanup stage for propositional formulas. It does not force a formula to be a disjunction of terms or a conjunction of clauses; it only restricts the available connectives and pushes negation down to propositional variables.

Remark (Examples). The formulas P, $\neg P$, $P \wedge (\neg Q \vee R)$, and $(P \vee Q) \wedge (\neg R \vee S)$ are in negation normal form.

Remark (Non-Examples). The formulas $P \to Q$, $P \leftrightarrow Q$, and $\neg(P \wedge Q)$ are not in negation normal form. The first two use connectives not allowed in negation normal form, and the last has a negation applied to a compound formula.

Remark (Dependencies).

- Literal
- Well-Formed Formula
- Logical Equivalence

Definition (NNF Conversion Procedure)

Definition 1.7.10 (NNF Conversion Procedure)**.** The *NNF conversion procedure* converts a propositional formula to an equivalent formula in negation normal form by the following steps:

1. eliminate biconditionals using biconditional equivalence laws;
2. eliminate conditionals using conditional equivalence laws;
3. eliminate double negations;
4. push negations inward using De Morgan laws.

Remark (Standard quantified statement). Starting with $\varphi \in \mathsf{WFF}$, repeatedly apply the equivalences

$$\alpha \leftrightarrow \beta \equiv (\alpha \to \beta) \wedge (\beta \to \alpha), \qquad \alpha \to \beta \equiv \neg\alpha \vee \beta,$$
$$\neg\neg\alpha \equiv \alpha, \qquad \neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta, \qquad \neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta,$$

until negations occur only immediately before propositional variables and only $\wedge$ and $\vee$ remain above the literal level.

Remark (Definition predicate reading).

$$\text{NNFConversionProcedure}(\varphi, \psi)$$

means that ψ is obtained from φ by eliminating biconditionals and conditionals, removing double negations, and pushing remaining negations inward.

Remark (Interpretation). The procedure is a syntax-directed rewrite process. Each rewrite step is backed by a logical equivalence law, so the output formula has the same truth conditions as the input formula.

Remark (Dependencies).

- Conditional Equivalence Laws
- Biconditional Equivalence Laws
- De Morgan Laws for Propositional Logic
- Basic Boolean Equivalence Laws

Theorem (Existence of Negation Normal Form)

Theorem 1.7.11 (Existence of Negation Normal Form)**.** *For every formula* $\varphi \in \mathsf{WFF}$*, there exists a formula* $\psi \in \mathsf{WFF}$ *such that* ψ *is in negation normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})(\exists \psi \in \mathsf{WFF})\big(\psi \text{ is in negation normal form} \wedge \varphi \equiv \psi\big).$$

Remark (Predicate reading).

$$\mathrm{NegationNormalFormExists}(\varphi)$$

means that φ has a logically equivalent representative in negation normal form.

Remark (Interpretation). Every propositional formula can be rewritten so that negation appears only at the variable level. This makes negation normal form a semantic normalizing step: it changes shape while preserving logical equivalence.

Remark (Dependencies).

- Negation Normal Form
- NNF Conversion Procedure
- Replacement of Logical Equivalents
- Structural Induction for Propositional Formulas

Definition (Elementary Conjunction)

Definition 1.7.12 (Elementary Conjunction)**.** An *elementary conjunction* is a finite conjunction of literals. That is, it is a term of the form

$$L_1 \wedge L_2 \wedge \cdots \wedge L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Remark (Standard quantified statement).

$$L_1 \wedge L_2 \wedge \cdots \wedge L_k, \qquad L_i \text{ is a literal for each } i.$$

Remark (Interpretation). Elementary conjunctions are the conjunction terms used as the building blocks of disjunctive normal form. They record simultaneous literal requirements.

Example (Examples).

$$P, \qquad P \wedge \neg Q, \qquad \neg P \wedge R \wedge S$$

are elementary conjunctions.

Remark (Dependencies).

- Term
- Literal

Definition (Minterm)

Definition 1.7.13 (Minterm)**.** Let $P_1, \ldots, P_n$ be a finite list of propositional variables. A *minterm* relative to $P_1, \ldots, P_n$ is an elementary conjunction that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \ldots, n$.

Remark (Standard quantified statement).

$$M = L_1 \wedge \cdots \wedge L_n,$$

where for each i, exactly one of P_i or $\neg P_i$ appears among the literals $L_1, \ldots, L_n$.

Remark (Interpretation). A minterm relative to $P_1, \ldots, P_n$ specifies one complete truth-value assignment on those variables: choosing P_i requires P_i to be true, and choosing $\neg P_i$ requires P_i to be false.

Example (Examples). Relative to P, Q, R, the formula

$$P \wedge \neg Q \wedge R$$

is a minterm. Relative to P, Q, R, the formula $P \wedge R$ is not a minterm because it omits both Q and $\neg Q$.

Remark (Dependencies).

- Elementary Conjunction
- Variable Set of a Formula

Definition (Disjunctive Normal Form)

Definition 1.7.14 (Disjunctive Normal Form)**.** A formula is in *disjunctive normal form*, abbreviated *DNF*, when it is a finite disjunction of elementary conjunctions. Equivalently, it has the shape

$$C_1 \vee C_2 \vee \cdots \vee C_m,$$

where each C_j is an elementary conjunction.

Remark (Standard quantified statement).

$$\varphi \text{ is in DNF} \iff \varphi = C_1 \vee C_2 \vee \cdots \vee C_m$$

for finitely many elementary conjunctions $C_1, \ldots, C_m$.

Remark (Interpretation). DNF is a syntactic form, but it is used semantically: once a formula is replaced by a logically equivalent DNF formula, questions about satisfiability reduce to questions about individual conjunction terms.

Example (Examples).

$$(P \wedge Q) \vee (\neg P \wedge R) \vee S$$

is in DNF because each disjunct is an elementary conjunction.

Remark (Dependencies).

- Elementary Conjunction
- Logical Equivalence
- Satisfiable Formula

Theorem (Existence of Disjunctive Normal Form)

Theorem 1.7.15 (Existence of Disjunctive Normal Form). *For every formula $\varphi \in \mathsf{WFF}$, there exists a formula $\psi \in \mathsf{WFF}$ such that ψ is in disjunctive normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})(\exists \psi \in \mathsf{WFF})\big(\psi \text{ is in DNF} \wedge \varphi \equiv \psi\big).$$

Remark (Interpretation). One first converts φ to negation normal form, then repeatedly applies distributive laws to move conjunctions below disjunctions. Replacement of logical equivalents preserves equivalence at each step.

Remark (Dependencies).

- Disjunctive Normal Form
- Negation Normal Form
- Distributive Laws for Propositional Logic
- Replacement of Logical Equivalents
- Existence of Negation Normal Form

Theorem (DNF Satisfiability Criterion)

Theorem 1.7.16 (DNF Satisfiability Criterion)**.** *Let ψ be a formula in disjunctive normal form. Then ψ is satisfiable if and only if at least one conjunction term in ψ contains no complementary pair of literals. Go to proof.*

Remark (Standard quantified statement)*.*

ψ is a satisfiable DNF formula $\iff$ some conjunction term of ψ has no complementary literals.

Remark (Interpretation)*.* A DNF formula is true exactly when at least one of its conjunction terms is true. A conjunction term can be made true exactly when it does not require both a variable and its negation.

Remark (Dependencies)*.*

- Disjunctive Normal Form
- Complementary Literals
- Satisfiable Formula

Definition (Elementary Disjunction)

Definition 1.7.17 (Elementary Disjunction)**.** An *elementary disjunction* is a finite disjunction of literals. That is, it is a clause of the form

$$L_1 \vee L_2 \vee \cdots \vee L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Remark (Standard quantified statement)*.*

$$L_1 \vee L_2 \vee \cdots \vee L_k, \qquad L_i \text{ is a literal for each } i.$$

Remark (Interpretation)*.* Elementary disjunctions are the clause-level building blocks used in conjunctive normal form. They record finite literal alternatives.

Example (Examples)*.*

$$P, \qquad P \vee \neg Q, \qquad \neg P \vee R \vee S$$

are elementary disjunctions.

Remark (Dependencies)*.*

- Clause
- Literal

Definition (Maxterm)

Definition 1.7.18 (Maxterm)**.** Let $P_1, \ldots, P_n$ be a finite list of propositional variables. A *maxterm* relative to $P_1, \ldots, P_n$ is an elementary disjunction that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \ldots, n$.

Remark (Standard quantified statement).

$$M = L_1 \vee \cdots \vee L_n,$$

where for each i, exactly one of P_i or $\neg P_i$ appears among the literals $L_1, \ldots, L_n$.

Remark (Interpretation). A maxterm relative to $P_1, \ldots, P_n$ specifies one literal-level alternative for every variable in the list. It is false under exactly the assignment that makes each listed literal false.

Example (Examples). Relative to P, Q, R, the formula

$$P \vee \neg Q \vee R$$

is a maxterm. Relative to P, Q, R, the formula $P \vee R$ is not a maxterm because it omits both Q and $\neg Q$.

Remark (Dependencies).

- Elementary Disjunction
- Variable Set of a Formula

Definition (Conjunctive Normal Form)

Definition 1.7.19 (Conjunctive Normal Form)**.** A formula is in *conjunctive normal form*, abbreviated *CNF*, when it is a finite conjunction of clauses. Equivalently, it has the shape

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each C_j is a clause.

Remark (Standard quantified statement).

$$\varphi \text{ is in CNF} \iff \varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

for finitely many clauses $C_1, \ldots, C_m$.

Remark (Interpretation). CNF is a syntactic form used to expose clause-by-clause constraints. Replacing a formula by a logically equivalent CNF formula turns global validity and contradiction questions into questions about the interaction of its clauses.

Example (Examples).

$$(P \vee Q) \wedge (\neg P \vee R) \wedge S$$

is in CNF because each conjunct is a clause.

Remark (Dependencies).

- Clause
- Logical Equivalence

- Contradiction

Theorem (Existence of Conjunctive Normal Form)

Theorem 1.7.20 (Existence of Conjunctive Normal Form)**.** *For every formula* $\varphi \in \mathsf{WFF}$*, there exists a formula* $\psi \in \mathsf{WFF}$ *such that* ψ *is in conjunctive normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi \in \mathsf{WFF})(\exists \psi \in \mathsf{WFF})\big(\psi \text{ is in CNF} \wedge \varphi \equiv \psi\big).$$

Remark (Interpretation)*.* One first converts φ to negation normal form, then repeatedly applies distributive laws to move disjunctions below conjunctions. Replacement of logical equivalents preserves equivalence through every local rewrite.

Remark (Dependencies)*.*

- Conjunctive Normal Form
- Negation Normal Form
- Distributive Laws for Propositional Logic
- Replacement of Logical Equivalents
- Existence of Negation Normal Form

Remark (Equivalence-preserving conversion)*.* Normal form conversion is a rewrite process that replaces a formula by a logically equivalent formula with a prescribed syntactic shape. Each local rewrite is justified by an equivalence law, and each replacement inside a larger formula is justified by Replacement of Logical Equivalents.

The goal is not to preserve the exact parse tree of the original formula. The goal is to preserve truth value under every valuation while moving the formula toward negation normal form, disjunctive normal form, or conjunctive normal form.

Remark (Step 1: eliminate biconditionals)*.* Replace every biconditional by an equivalent formula using the Biconditional Equivalence Laws. A standard replacement is

$$\alpha \leftrightarrow \beta \quad \rightsquigarrow \quad (\alpha \to \beta) \wedge (\beta \to \alpha).$$

After this step, no occurrence of $\leftrightarrow$ remains.

Remark (Step 2: eliminate conditionals)*.* Replace every conditional by an equivalent disjunction using the Conditional Equivalence Laws. The standard replacement is

$$\alpha \to \beta \quad \rightsquigarrow \quad \neg\alpha \vee \beta.$$

After this step, the only connectives still present are $\neg$, $\wedge$, and $\vee$.

Remark (Step 3: push negations inward). Use De Morgan Laws for Propositional Logic and double-negation simplification to move every negation down to the literal level:

$$\neg(\alpha \wedge \beta) \rightsquigarrow \neg\alpha \vee \neg\beta, \qquad \neg(\alpha \vee \beta) \rightsquigarrow \neg\alpha \wedge \neg\beta, \qquad \neg\neg\alpha \rightsquigarrow \alpha.$$

When this step terminates, the result is in negation normal form: negations occur only immediately in front of propositional variables.

Remark (Step 4A: distribute for DNF). To obtain DNF from negation normal form, distribute conjunction over disjunction using the Distributive Laws for Propositional Logic:

$$\alpha \wedge (\beta \vee \gamma) \quad \rightsquigarrow \quad (\alpha \wedge \beta) \vee (\alpha \wedge \gamma),$$

and similarly

$$(\beta \vee \gamma) \wedge \alpha \quad \rightsquigarrow \quad (\beta \wedge \alpha) \vee (\gamma \wedge \alpha).$$

Repeatedly applying these rewrites moves disjunctions outward until the formula is a finite disjunction of conjunction terms.

Remark (Step 4B: distribute for CNF). To obtain CNF from negation normal form, distribute disjunction over conjunction:

$$\alpha \vee (\beta \wedge \gamma) \quad \rightsquigarrow \quad (\alpha \vee \beta) \wedge (\alpha \vee \gamma),$$

and similarly

$$(\beta \wedge \gamma) \vee \alpha \quad \rightsquigarrow \quad (\beta \vee \alpha) \wedge (\gamma \vee \alpha).$$

Repeatedly applying these rewrites moves conjunctions outward until the formula is a finite conjunction of clauses.

Remark (DNF conversion workflow). To convert a formula to DNF:

1. eliminate biconditionals;
2. eliminate conditionals;
3. push negations inward to obtain negation normal form;
4. distribute conjunction over disjunction until the result is a finite disjunction of elementary conjunctions.

The resulting formula is logically equivalent to the original formula, but it need not be the shortest or only possible DNF representation.

Remark (CNF conversion workflow). To convert a formula to CNF:

1. eliminate biconditionals;
2. eliminate conditionals;
3. push negations inward to obtain negation normal form;
4. distribute disjunction over conjunction until the result is a finite conjunction of clauses.

The resulting formula is logically equivalent to the original formula, but it need not be the shortest or only possible CNF representation.

Remark (Termination). The first three stages terminate because they remove or strictly simplify specific connective patterns. Eliminating biconditionals reduces the number of $\leftrightarrow$ symbols. Eliminating conditionals reduces the number of $\rightarrow$ symbols. Pushing negations inward reduces the distance between each negation and the propositional variables it governs, while double negations shorten the formula.

The distribution stages terminate when guided by a target measure: for DNF, keep applying distributions that move $\vee$ outward past $\wedge$; for CNF, keep applying distributions that move $\wedge$ outward past $\vee$. Each application lowers the number of target-obstructing connective patterns, even though it may duplicate subformulas.

Remark (Correctness). Correctness is by equivalence preservation. Each rewrite instance is one of the listed logical equivalence laws: biconditional laws, conditional laws, De Morgan laws, or distributive laws. Replacement of logical equivalents permits the rewrite to occur inside an arbitrary larger formula. Therefore the final normal form is logically equivalent to the original formula.

Remark (Non-uniqueness). Ordinary DNF and CNF are not unique. Reordering terms or clauses, reordering literals inside a term or clause, duplicating a term, deleting a redundant term, or applying absorption-style simplifications can produce different formulas with the same truth table and the same normal-form type.

Canonical normal forms impose extra conventions to remove this freedom. The ordinary conversion procedures in this topic do not impose those conventions.

Remark (Size blowup). Distribution can duplicate subformulas. As a result, converting a compact formula to DNF or CNF can produce a formula whose size is much larger than the original. This size blowup is a syntactic cost of forcing a formula into a flat disjunction-of-terms or conjunction-of-clauses shape.

Remark (Comparison).

Target form	Final outer connective pattern	Distribution used
NNF	negations only at literals	none
DNF	disjunction of conjunction terms	$\wedge$ over $\vee$
CNF	conjunction of clauses	$\vee$ over $\wedge$

All three workflows use equivalence-preserving rewrites. DNF and CNF share the same first three stages and differ only in the final distribution direction.

Definition (Truth-Table Row Conjunction)

Definition 1.7.21 (Truth-Table Row Conjunction)**.** Fix a finite variable list $P_1, \dots, P_n$ and a row r of the truth table for those variables. The *truth-table row conjunction* for r is the conjunction

$$L_1 \wedge L_2 \wedge \cdots \wedge L_n,$$

where

$$L_i = \begin{cases} P_i, & \text{if row } r \text{ makes } P_i \text{ true,} \\ \neg P_i, & \text{if row } r \text{ makes } P_i \text{ false.} \end{cases}$$

Remark (Standard quantified statement). For a fixed row r on variables $P_1, \dots, P_n$,

$$\bigwedge_{i=1}^{n} L_i, \qquad L_i = \begin{cases} P_i, & r(P_i) = \mathsf{True}, \\ \neg P_i, & r(P_i) = \mathsf{False}. \end{cases}$$

Remark (Interpretation). The row conjunction names exactly one truth-table row. It is true on that row and false on every row that disagrees with it on at least one listed variable.

Remark (Dependencies).

- Truth Table
- Literal
- Variable Set of a Formula

Definition (Truth-Table Row Clause)

Definition 1.7.22 (Truth-Table Row Clause)**.** Fix a finite variable list $P_1, \dots, P_n$ and a row r of the truth table for those variables. The *truth-table row clause* for r is the clause

$$M_1 \vee M_2 \vee \cdots \vee M_n,$$

where

$$M_i = \begin{cases} P_i, & \text{if row } r \text{ makes } P_i \text{ false,} \\ \neg P_i, & \text{if row } r \text{ makes } P_i \text{ true.} \end{cases}$$

Remark (Standard quantified statement). For a fixed row r on variables $P_1, \dots, P_n$,

$$\bigvee_{i=1}^{n} M_i, \qquad M_i = \begin{cases} P_i, & r(P_i) = \mathsf{False}, \\ \neg P_i, & r(P_i) = \mathsf{True}. \end{cases}$$

Remark (Interpretation). The row clause excludes exactly one truth-table row. It is false on row r and true on every row that disagrees with r on at least one listed variable.

Remark (Dependencies).

- Truth Table

- Clause
- Variable Set of a Formula

Definition (Canonical Disjunctive Normal Form)

Definition 1.7.23 (Canonical Disjunctive Normal Form)**.** Let $\varphi \in \mathsf{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ. The *canonical disjunctive normal form* of φ relative to this ordering is the disjunction of the truth-table row conjunctions corresponding to the rows on which φ is true.

Remark (Standard quantified statement). For $\mathrm{Var}(\varphi) = \{P_1, \dots, P_n\}$, the canonical disjunctive normal form is

$$\bigvee_{\widehat{v}(\varphi)=\mathsf{True}} (L_1 \wedge \cdots \wedge L_n),$$

where each row conjunction is determined by the truth-table row of v.

Remark (Interpretation). Canonical DNF records exactly the truth-table rows on which the formula is true, using one row conjunction for each true row.

Remark (Examples). If φ is a contradiction, then it has no true rows. Its canonical DNF is the empty disjunction, represented by the empty clause.

Remark (Dependencies).

- Disjunctive Normal Form
- Truth-Table Row Conjunction
- Truth Table
- Empty Clause

Definition (Canonical Conjunctive Normal Form)

Definition 1.7.24 (Canonical Conjunctive Normal Form)**.** Let $\varphi \in \mathsf{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ. The *canonical conjunctive normal form* of φ relative to this ordering is the conjunction of the truth-table row clauses corresponding to the rows on which φ is false.

Remark (Standard quantified statement). For $\mathrm{Var}(\varphi) = \{P_1, \dots, P_n\}$, the canonical conjunctive normal form is

$$\bigwedge_{\widehat{v}(\varphi)=\mathsf{False}} (M_1 \vee \cdots \vee M_n),$$

where each row clause is determined by the truth-table row of v.

Remark (Interpretation). Canonical CNF records exactly the truth-table rows on which the formula is false, using one row clause for each false row.

Remark (Examples). If φ is a tautology, then it has no false rows. Its canonical CNF is the empty conjunction, represented by the empty conjunction.

Remark (Dependencies).

- Conjunctive Normal Form
- Truth-Table Row Clause
- Truth Table
- Empty Conjunction

Theorem (Canonical Disjunctive Normal Form)

Theorem 1.7.25 (Canonical Disjunctive Normal Form Theorem)**.** *Every formula* $\varphi \in \mathsf{WFF}$ *is logically equivalent to its canonical disjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CDNF}_{P_1,\dots,P_n}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})\, \varphi \equiv \mathrm{CDNF}_{P_1,\dots,P_n}(\varphi),$$

where $P_1, \dots, P_n$ is a fixed ordering of the finitely many variables of φ.

Remark (Interpretation). Canonical DNF rebuilds the formula from exactly the truth-table rows where the formula is true. Each row conjunction recognizes one true row, and the final disjunction accepts any of those rows.

Remark (Dependencies).

- Canonical Disjunctive Normal Form
- Finite Truth-Table Theorem
- Coincidence Lemma for Propositional Logic
- Logical Equivalence

Theorem (Canonical Conjunctive Normal Form)

Theorem 1.7.26 (Canonical Conjunctive Normal Form Theorem)**.** *Every formula* $\varphi \in \mathsf{WFF}$ *is logically equivalent to its canonical conjunctive normal form relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \mathrm{CCNF}_{P_1,\dots,P_n}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathsf{WFF})\, \varphi \equiv \mathrm{CCNF}_{P_1,\ldots,P_n}(\varphi),$$

where $P_1, \ldots, P_n$ is a fixed ordering of the finitely many variables of φ.

Remark (Interpretation). Canonical CNF rebuilds the formula by excluding exactly the truth-table rows where the formula is false. Each row clause rejects one false row, and the final conjunction requires all false rows to be rejected.

Remark (Dependencies).

- Canonical Conjunctive Normal Form
- Finite Truth-Table Theorem
- Coincidence Lemma for Propositional Logic
- Logical Equivalence

Theorem (Semantic Classification by Normal Forms)

Theorem 1.7.27 (Semantic Classification by Normal Forms)**.** *Normal forms give finite certificates for semantic classification. For any formula $\varphi \in \mathsf{WFF}$:*

1. *φ is satisfiable if and only if some DNF formula equivalent to φ has a satisfiable conjunction term.*
2. *φ is a contradiction if and only if some DNF formula equivalent to φ has no satisfiable conjunction term.*
3. *φ is a tautology if and only if some CNF formula equivalent to φ has no falsifiable clause.*
4. *φ is a contingency if and only if it is satisfiable and not a tautology.*

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} \varphi \text{ is satisfiable} &\iff \text{some equivalent DNF term is satisfiable,} \\ \varphi \text{ is a contradiction} &\iff \text{no equivalent DNF term is satisfiable,} \\ \varphi \text{ is a tautology} &\iff \text{no equivalent CNF clause is falsifiable.} \end{aligned}$$

Remark (Interpretation). The truth-table method classifies φ by evaluating every row directly. The normal-form method first rewrites φ into an equivalent DNF or CNF formula and then checks the finite terms or clauses. Canonical normal forms connect the two approaches: canonical DNF is built from true rows, and canonical CNF is built from false rows.

Remark (Exposition). DNF is naturally witness-producing. A satisfiable term gives an explicit way to make the whole disjunction true. CNF is naturally suited for clause checking: each clause is a local obstruction to falsifying the whole conjunction. This clause-centered view is also the starting point for later resolution methods.

Remark (Dependencies).

- Disjunctive Normal Form
- Conjunctive Normal Form
- Satisfiable Formula
- Contradiction
- Tautology
- Contingency
- DNF Satisfiability Criterion
- Canonical Disjunctive Normal Form Theorem
- Canonical Conjunctive Normal Form Theorem

Summary

Remark (Literal vocabulary). The vocabulary begins at the literal level. A literal is a propositional variable or the negation of one propositional variable. Positive literals and negative literals record the sign of that occurrence. Complementary literals are the opposed pair P and $\neg P$. Clauses are finite disjunctions of literals, while terms are finite conjunctions of literals. The empty clause is the false boundary case, and the empty conjunction is the true boundary case.

Remark (Negation normal form). Negation normal form is the first normalization target. It permits only $\neg$, $\wedge$, and $\vee$, with every negation pushed down to the literal level. Its purpose is to remove conditionals and biconditionals and to make negation local. Once a formula is in NNF, the remaining work is a matter of distributing $\wedge$ and $\vee$ in the direction required by the target normal form.

Remark (Disjunctive normal form). Disjunctive normal form writes a formula as a finite disjunction of terms. It is naturally witness-oriented: to make a DNF formula true, it is enough to make one term true, and a satisfiable term directly records compatible literal requirements. This is why DNF is especially useful for satisfiability checks and for building formulas from true rows of a truth table.

Remark (Conjunctive normal form). Conjunctive normal form writes a formula as a finite conjunction of clauses. It is naturally obstruction-oriented: to falsify a CNF formula, one must falsify a clause, and each clause gives a local disjunctive condition. This clause structure makes CNF the preferred shape for later clause checking, resolution-style arguments, and systematic proof search.

Remark (Canonical normal forms). Ordinary DNF and CNF are not unique. Reordering, duplicating, or simplifying terms and clauses can produce different formulas with the same truth table. Canonical normal forms add truth-table discipline. Canonical DNF records the true rows by minterm-style conjunctions, and canonical CNF records the false rows by maxterm-style clauses. These canonical forms show that every finite truth behavior can be represented syntactically.

Remark (Conversion workflow). The conversion workflow has a common beginning. First eliminate biconditionals. Then eliminate conditionals. Then push negations inward until the formula is in negation normal form. From there, distribute conjunction over disjunction to obtain DNF, or distribute disjunction over conjunction to obtain CNF. Each stage is justified by logical equivalence laws and by replacement of logical equivalents inside larger formulas.

Remark (Back to the algebra of propositions). Normal forms are an application of the algebra of propositions. The conversion rules are not new semantic principles; they are uses of earlier equivalence laws such as De Morgan laws, conditional and biconditional equivalences, distributivity, and replacement of equivalents. The normal-form viewpoint therefore shows how algebraic manipulation and semantic preservation work together.

Remark (Forward to functional completeness). Canonical normal forms point forward to functional completeness. If a truth function can be described by its true rows, then it can be represented by a disjunction of conjunctions of literals. If it can be described by its false rows, then it can be represented by a conjunction of disjunctions of literals. This is the bridge from truth tables to the question of which connectives are enough to express every truth function.

Remark (Forward to arguments and proof systems). Normal forms also prepare the transition from single formulas to arguments and proof systems. DNF gives explicit witnesses for satisfiability, while CNF turns formulas into collections of clauses that can be checked and transformed. The same syntactic regularity used here to classify formulas will later support validity tests, derivation systems, resolution methods, and comparisons between semantic consequence and formal proof.

1.8 Functional Completeness

Toolkit

This section uses Boolean functions, truth functions induced by formulas, logical equivalence, canonical normal forms, and connective rewrite laws.

Remark (Exposition). The functional-completeness section studies expressive power. A connective set is complete exactly when formulas built from it can represent every Boolean function, so normal forms and connective reductions become tests for the adequacy of propositional vocabularies.

Orientation

Remark (Section map). This section begins with Boolean functions and formula representation, then introduces connective bases and definability of connectives. It uses normal forms to

prove completeness of the standard connectives and then reduces the primitive vocabulary further. The final topics show that NAND and NOR each form a one-connective complete basis, while some familiar connective sets fail to be functionally complete.

Remark (Connection with normal forms). Normal forms explain why functional completeness is possible. Canonical DNF and canonical CNF show that the truth behavior of a formula can be reconstructed from finitely many truth-table rows. Once those canonical constructions are available, proving that a connective set is functionally complete amounts to showing that the connectives needed for those normal forms can be defined from that set.

Boolean Functions and Representation

Definition (Representation of a Boolean Function)

Definition 1.8.1 (Representation of a Boolean Function)**.** Let $f : \mathbb{B}^n \to \mathbb{B}$ be a Boolean function. A formula $\varphi \in \mathsf{WFF}$ with variables among $P_1, \dots, P_n$ *represents* f if for every truth assignment v,

$$\widehat{v}(\varphi) = f(v(P_1), \dots, v(P_n)).$$

Remark (Standard quantified statement).

$$\text{Represents}(\varphi, f; P_1, \dots, P_n) \iff (\forall v : \mathsf{Prop} \to \mathbb{B})\big(\widehat{v}(\varphi) = f(v(P_1), \dots, v(P_n))\big).$$

Remark (Definition predicate reading). $\text{Represents}(\varphi, f; P_1, \dots, P_n)$ means that φ has exactly the same truth table as the Boolean function f on the listed variables.

Remark (Interpretation). A formula represents a Boolean function when the formula computes the same truth value as the function for every input row. This turns a semantic table of values into a syntactic object inside the propositional language.

Remark (Dependencies).

- Boolean Function
- Truth Function Induced by a Formula
- Truth Assignment

Theorem (DNF Representation of Boolean Functions)

Theorem 1.8.2 (DNF Representation of Boolean Functions)**.** *Every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ is represented by a formula in disjunctive normal form using variables $P_1, \dots, P_n$.*
Go to proof.

Remark (Standard quantified statement).

$$(\forall f : \mathbb{B}^n \to \mathbb{B})(\exists \varphi \in \mathsf{WFF})\big(\varphi \text{ is in DNF} \wedge \text{Represents}(\varphi, f; P_1, \dots, P_n)\big).$$

Remark (Predicate reading). Every n-ary Boolean function has a propositional formula representative, and one may choose the representative to be in disjunctive normal form.

Remark (Interpretation). A truth table specifies which input rows make the Boolean function true. For each true row, form the corresponding conjunction of literals; the disjunction of those row conjunctions represents the entire function.

Remark (Dependencies).

- Representation of a Boolean Function
- Canonical Disjunctive Normal Form
- Canonical Disjunctive Normal Form Theorem
- Disjunctive Normal Form

Connective Bases

Definition (Connective Basis)

Definition 1.8.3 (Connective Basis)**.** A *connective basis* is a specified set B of propositional connectives allowed as primitive connectives for building formulas.

Remark (Standard quantified statement).

$$B \text{ is a connective basis} \iff B \subseteq \{\neg, \wedge, \vee, \to, \leftrightarrow, \ldots\}$$

and formulas over B are built using only connectives from B.

Remark (Definition predicate reading). ConnectiveBasis(B) means that B is the chosen primitive connective vocabulary for a propositional language or fragment.

Remark (Interpretation). A connective basis controls which connectives are primitive, not which truth functions are expressible. Different bases may have the same expressive power.

Remark (Dependencies).

- Logical Connective
- Propositional Language

Definition (Definable Connective)

Definition 1.8.4 (Definable Connective)**.** Let B be a connective basis. A connective $\circ$ is *definable from* B if there is a formula built using only connectives from B whose truth function agrees with the truth function of $\circ$.

Remark (Standard quantified statement).

$$\circ \text{ is definable from } B \iff \exists \theta \in \mathsf{WFF}_B \left(f_\theta = f_\circ\right),$$

where WFF_B denotes the formulas built using only connectives from B.

Remark (Definition predicate reading). DefinableFrom$(\circ, B)$ means that $\circ$ can be treated as an abbreviation for a formula pattern using only connectives from B.

Remark (Interpretation). Definability separates notation from expressive power. A connective can be omitted from the primitive syntax without losing its use, provided it is recoverable as a defined connective.

Remark (Dependencies).

- Connective Basis
- Truth Function Induced by a Formula
- Boolean Function

Definition (Functionally Complete Connective Set)

Definition 1.8.5 (Functionally Complete Connective Set)**.** A connective basis B is *functionally complete* if every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$, for every $n \geq 1$, is represented by some formula using only connectives from B.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(B) \iff (\forall n \geq 1)(\forall f : \mathbb{B}^n \to \mathbb{B})(\exists \varphi \in \mathsf{WFF}_B)\, \text{Represents}(\varphi, f; P_1, \ldots, P_n).$$

Remark (Definition predicate reading). FunctionallyComplete(B) means that formulas over B can represent every finite Boolean input-output table.

Remark (Interpretation). Functional completeness is the central expressive-power property for connective sets. It says that the basis can express all possible truth-functional behavior, not merely the connectives originally listed in the language.

Remark (Dependencies).

- Connective Basis
- Representation of a Boolean Function
- Boolean Function

Definition (Expressively Equivalent Connective Sets)

Definition 1.8.6 (Expressively Equivalent Connective Sets)**.** Two connective bases B and C are *expressively equivalent* if every connective in B is definable from C, and every connective in C is definable from B.

Remark (Standard quantified statement).

$$B \equiv_{\text{expr}} C \iff \big((\forall \circ \in B)\, \text{DefinableFrom}(\circ, C)\big) \wedge \big((\forall \star \in C)\, \text{DefinableFrom}(\star, B)\big).$$

Remark (Definition predicate reading). $B \equiv_{\text{expr}} C$ means that the two bases can define exactly the same connective behavior.

Remark (Interpretation). Expressive equivalence lets a proof replace one primitive vocabulary with another. This is why a language can be simplified syntactically without changing which truth functions can be expressed.

Remark (Dependencies).

- Connective Basis
- Definable Connective

Completeness of the Standard Connectives

Theorem (Standard Connectives are Functionally Complete)

Theorem 1.8.7 (Standard Connectives are Functionally Complete)**.** *The connective basis*

$$\{\neg, \wedge, \vee\}$$

is functionally complete.
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \wedge, \vee\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation, conjunction, and disjunction.

Remark (Interpretation). Canonical disjunctive normal form uses only literals, conjunctions, and disjunctions. Since literals use variables and negation, the normal-form theorem shows that $\{\neg, \wedge, \vee\}$ can express every Boolean function.

Remark (Dependencies).

- Functionally Complete Connective Set
- DNF Representation of Boolean Functions
- Disjunctive Normal Form

Theorem (Negation and Conjunction are Functionally Complete)

Theorem 1.8.8 (Negation and Conjunction are Functionally Complete)**.** *The connective basis*

$$\{\neg, \wedge\}$$

is functionally complete.
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \wedge\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation and conjunction.

Remark (Interpretation). Disjunction is definable from negation and conjunction by De Morgan's law:

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi).$$

Thus every formula using $\neg, \wedge, \vee$ can be translated into an equivalent formula using only $\neg$ and $\wedge$.

Remark (Dependencies).

- Standard Connectives are Functionally Complete
- De Morgan Laws for Propositional Logic
- Replacement of Logical Equivalents

Theorem (Negation and Disjunction are Functionally Complete)

Theorem 1.8.9 (Negation and Disjunction are Functionally Complete)**.** *The connective basis*

$$\{\neg, \vee\}$$

is functionally complete.
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \vee\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation and disjunction.

Remark (Interpretation). Conjunction is definable from negation and disjunction by De Morgan's law:

$$\varphi \wedge \psi \equiv \neg(\neg\varphi \vee \neg\psi).$$

Thus every formula using $\neg, \wedge, \vee$ can be translated into an equivalent formula using only $\neg$ and $\vee$.

Remark (Dependencies).

- Standard Connectives are Functionally Complete
- De Morgan Laws for Propositional Logic
- Replacement of Logical Equivalents

Reduction of Connective Sets

Definition (Connective Reduction)

Definition 1.8.10 (Connective Reduction)**.** A *connective reduction* replaces a formula using one connective basis by a logically equivalent formula using a smaller or different connective basis.

Remark (Standard quantified statement)*.*

$$\text{Reduction}_{B\to C}(\varphi,\psi) \iff \varphi \equiv \psi \quad \text{and} \quad \psi \in \mathsf{WFF}_C.$$

Remark (Definition predicate reading)*.* $\text{Reduction}_{B\to C}(\varphi,\psi)$ means that ψ is a translation of φ into the connective basis C without changing truth behavior.

Remark (Interpretation)*.* Reduction shows that some primitive connectives are optional. If a connective is definable from the remaining basis, then formulas using that connective can be translated away.

Remark (Exposition)*.* The standard reductions are:

$$\varphi \to \psi \equiv \neg\varphi \lor \psi,$$

$$\varphi \leftrightarrow \psi \equiv (\varphi \to \psi) \land (\psi \to \varphi),$$

$$\varphi \lor \psi \equiv \neg(\neg\varphi \land \neg\psi), \qquad \varphi \land \psi \equiv \neg(\neg\varphi \lor \neg\psi).$$

Each replacement preserves logical equivalence and may be performed inside a larger formula context.

Remark (Exposition)*.* Reducing a connective basis can make formulas longer. Functional completeness is therefore an expressive result, not an efficiency result. A small basis may be powerful enough to express every Boolean function while still being awkward for ordinary human-readable formulas.

Remark (Dependencies)*.*

- Connective Basis
- Definable Connective
- Logical Equivalence

NAND

Definition (NAND Connective)

Definition 1.8.11 (NAND Connective)**.** The *NAND* connective, written $\uparrow$, is the binary connective defined by

$$\varphi \uparrow \psi \equiv \neg(\varphi \land \psi).$$

Remark (Standard quantified statement).

$$\widehat{v}(\varphi \uparrow \psi) = \mathsf{True} \iff \text{not both } \widehat{v}(\varphi) = \mathsf{True} \text{ and } \widehat{v}(\psi) = \mathsf{True}.$$

Remark (Definition predicate reading). NAND(φ, ψ) means the negation of the conjunction of φ and ψ.

Remark (Interpretation). NAND is read as "not both." It is false only when both inputs are true. This single connective can reproduce negation, conjunction, and disjunction.

Remark (Dependencies).

- Logical Connective
- Truth Assignment

Definition (NAND-Only Formula)

Definition 1.8.12 (NAND-Only Formula)**.** A *NAND-only formula* is a well-formed formula whose only connective is $\uparrow$.

Remark (Standard quantified statement).

$$\varphi \text{ is NAND-only} \iff \varphi \in \mathsf{WFF}_{\{\uparrow\}}.$$

Remark (Definition predicate reading). NANDOnly(φ) means that every connective occurrence in φ is an occurrence of $\uparrow$.

Remark (Interpretation). A NAND-only formula is syntactically restricted but expressively powerful. The restriction is severe at the level of primitive symbols, but not at the level of truth functions.

Remark (Dependencies).

- NAND Connective
- Well-Formed Formula

Theorem (NAND is Functionally Complete)

Theorem 1.8.13 (NAND is Functionally Complete)**.** *The one-connective basis* $\{\uparrow\}$ *is functionally complete.*
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\uparrow\}).$$

Remark (Predicate reading). Every Boolean function can be represented by a NAND-only formula.

Remark (Interpretation). NAND can define negation and conjunction:

$$\neg\varphi \equiv \varphi \uparrow \varphi, \qquad \varphi \wedge \psi \equiv (\varphi \uparrow \psi) \uparrow (\varphi \uparrow \psi).$$

Once negation and conjunction are definable, functional completeness follows from the functional completeness of $\{\neg, \wedge\}$.

Remark (Dependencies).

- NAND Connective
- NAND-Only Formula
- Functionally Complete Connective Set
- Negation and Conjunction are Functionally Complete

NOR

Definition (NOR Connective)

Definition 1.8.14 (NOR Connective)**.** The *NOR* connective, written $\downarrow$, is the binary connective defined by

$$\varphi \downarrow \psi \equiv \neg(\varphi \vee \psi).$$

Remark (Standard quantified statement).

$$\widehat{v}(\varphi \downarrow \psi) = \mathsf{True} \iff \widehat{v}(\varphi) = \mathsf{False} \text{ and } \widehat{v}(\psi) = \mathsf{False}.$$

Remark (Definition predicate reading). $\mathrm{NOR}(\varphi, \psi)$ means the negation of the disjunction of φ and ψ.

Remark (Interpretation). NOR is read as "neither." It is true exactly when both inputs are false. Like NAND, this single connective can reproduce the standard functionally complete basis.

Remark (Dependencies).

- Logical Connective
- Truth Assignment

Definition (NOR-Only Formula)

Definition 1.8.15 (NOR-Only Formula)**.** A *NOR-only formula* is a well-formed formula whose only connective is $\downarrow$.

Remark (Standard quantified statement).

$$\varphi \text{ is NOR-only} \iff \varphi \in \mathsf{WFF}_{\{\downarrow\}}.$$

Remark (Definition predicate reading). NOROnly(φ) means that every connective occurrence in φ is an occurrence of $\downarrow$.

Remark (Interpretation). A NOR-only formula uses a single primitive connective. Its expressive power comes from the ability of NOR to define negation and disjunction.

Remark (Dependencies).

- NOR Connective
- Well-Formed Formula

Theorem (NOR is Functionally Complete)

Theorem 1.8.16 (NOR is Functionally Complete)**.** *The one-connective basis* $\{\downarrow\}$ *is functionally complete.*
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\downarrow\}).$$

Remark (Predicate reading). Every Boolean function can be represented by a NOR-only formula.

Remark (Interpretation). NOR can define negation and disjunction:

$$\neg\varphi \equiv \varphi \downarrow \varphi, \qquad \varphi \vee \psi \equiv (\varphi \downarrow \psi) \downarrow (\varphi \downarrow \psi).$$

Once negation and disjunction are definable, functional completeness follows from the functional completeness of $\{\neg, \vee\}$.

Remark (Dependencies).

- NOR Connective
- NOR-Only Formula
- Functionally Complete Connective Set
- Negation and Disjunction are Functionally Complete

Incomplete Connective Sets

Definition (Incomplete Connective Set)

Definition 1.8.17 (Incomplete Connective Set)**.** A connective basis is *incomplete* if it is not functionally complete.

Remark (Standard quantified statement).

$$\text{Incomplete}(B) \iff \neg\,\text{FunctionallyComplete}(B).$$

Remark (Definition predicate reading). Incomplete(B) means that some Boolean function cannot be represented by any formula using only connectives from B.

Remark (Interpretation). Incompleteness is usually proved by finding an invariant preserved by every formula built from the basis. If some Boolean function fails that invariant, the basis cannot express every Boolean function.

Remark (Dependencies).

- Functionally Complete Connective Set
- Boolean Function

Definition (Monotone Boolean Function)

Definition 1.8.18 (Monotone Boolean Function)**.** A Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ is *monotone* if changing input coordinates from False to True never changes the output from True to False.

Remark (Standard quantified statement).

$$\text{Monotone}(f) \iff (\forall a, b \in \mathbb{B}^n)\big(a \leq b \to f(a) \leq f(b)\big),$$

where False $\leq$ True coordinatewise.

Remark (Definition predicate reading). Monotone(f) means that increasing truth inputs cannot make the output decrease from true to false.

Remark (Interpretation). The connectives $\wedge$ and $\vee$ are monotone. Negation is not monotone, because changing P from false to true changes $\neg P$ from true to false.

Remark (Dependencies).

- Boolean Function

Theorem (Conjunction and Disjunction are not Functionally Complete)

Theorem 1.8.19 (Conjunction and Disjunction are not Functionally Complete)**.** *The connective basis $\{\wedge, \vee\}$ is not functionally complete.*
Go to proof.

Remark (Standard quantified statement).

$$\neg\,\text{FunctionallyComplete}(\{\wedge, \vee\}).$$

Remark (Predicate reading). There is at least one Boolean function that cannot be represented by any formula using only conjunction and disjunction.

Remark (Interpretation). Every formula built using only $\wedge$ and $\vee$ represents a monotone Boolean function. Since negation is not monotone, no formula using only $\wedge$ and $\vee$ can represent negation. Hence the basis is incomplete.

Remark (Dependencies).

- Incomplete Connective Set
- Monotone Boolean Function
- Functionally Complete Connective Set

Minimal Complete Bases

Definition (Minimal Complete Basis)

Definition 1.8.20 (Minimal Complete Basis)**.** A functionally complete connective basis B is *minimal complete* if no proper subset of B is functionally complete.

Remark (Standard quantified statement)*.*

$$\text{MinimalComplete}(B) \iff \text{FunctionallyComplete}(B) \wedge (\forall C \subsetneq B) \neg\, \text{FunctionallyComplete}(C).$$

Remark (Definition predicate reading)*.* MinimalComplete(B) means that B is complete, but removing any primitive connective from B destroys completeness.

Remark (Interpretation)*.* Minimality is relative to deletion of connectives from the basis. It does not mean that formulas using the basis are short or easy to read.

Remark (Dependencies)*.*

- Functionally Complete Connective Set
- Connective Basis

Theorem (NAND and NOR are Minimal Complete Bases)

Theorem 1.8.21 (NAND and NOR are Minimal Complete Bases)**.** *The one-connective bases* $\{\uparrow\}$ *and* $\{\downarrow\}$ *are minimal complete bases.*
Go to proof.

Remark (Standard quantified statement)*.*

$$\text{MinimalComplete}(\{\uparrow\}) \wedge \text{MinimalComplete}(\{\downarrow\}).$$

Remark (Predicate reading)*.* NAND alone is complete, NOR alone is complete, and deleting the sole connective from either basis leaves no functionally complete basis.

Remark (Interpretation)*.* A one-connective complete basis is automatically minimal once the empty basis is not functionally complete. NAND and NOR are therefore minimal complete bases because each is complete by itself.

Remark (Dependencies)*.*

- Minimal Complete Basis
- NAND is Functionally Complete
- NOR is Functionally Complete
- Incomplete Connective Set

Summary and Transition

Remark (Summary). Functional completeness measures the expressive strength of a connective basis. A functionally complete basis can represent every finite Boolean function by a formula using only the connectives in that basis. Normal forms supply the main construction: truth-table rows can be converted into formulas, and those formulas can be assembled into a representative for the desired Boolean function.

Remark (Section map). The standard connectives $\neg$, $\wedge$, and $\vee$ are functionally complete because they can build disjunctive normal forms. De Morgan laws reduce this basis to $\{\neg, \wedge\}$ or $\{\neg, \vee\}$. NAND and NOR go further: each single connective can define a complete two-connective basis and therefore can represent every Boolean function.

Remark (Transition). The next section moves from expressive power to argument forms and informal proof patterns. Functional completeness explains which truth functions can be expressed. Argument forms ask which patterns of inference preserve truth from premises to conclusions.

1.9 Argument Forms and Informal Proof Patterns

Toolkit

This section uses argument forms, inference patterns, replacement rules, informal proof patterns, and semantic validity.

Remark (Exposition). Argument forms organize recurring patterns of propositional reasoning. This section connects valid and invalid forms with replacement principles, informal proof practice, and the semantic tests that justify those patterns.

Orientation

Remark (Working vocabulary). This orientation uses truth assignments, logical consequence, tautology, logical equivalence, formula contexts, and counterexample assignments to justify common argument forms.

Remark (Exposition). Argument forms are reusable patterns of inference. The purpose of this section is to justify the basic inference tools that will be used throughout the rest of the project. The justification is semantic: an argument form is valid when no truth assignment makes every premise true while making the conclusion false.

Remark (Section map). This section introduces argument forms, substitution instances, validity, invalidity, and counterexample assignments. It then proves the validity of the standard inference patterns, isolates common invalid forms, explains rules of replacement, and connects informal proof patterns with semantic consequence. Formal proof systems are previewed but not developed here.

Remark (Boundary). The goal is rigorous senior-undergraduate propositional logic. The section proves that the proof moves used later are truth-preserving, but it does not build Hilbert systems, sequent calculi, cut elimination, or metatheoretic completeness machinery.

Argument Forms

Remark (Working vocabulary). This topic introduces argument forms, premises, conclusions, substitution instances, valid argument forms, invalid argument forms, and counterexample truth assignments.

Definition (Argument Form)

Definition 1.9.1 (Argument Form)**.** An *argument form* is a schematic pattern consisting of a finite list of premise formulas and one conclusion formula.

Remark (Standard quantified statement).

$$\text{ArgumentForm}(\Gamma, \varphi) \iff \Gamma \subseteq \mathsf{WFF} \text{ is finite and } \varphi \in \mathsf{WFF}.$$

Remark (Definition predicate reading). ArgumentForm(Γ, φ) means that Γ is the set or finite list of premises and φ is the conclusion.

Remark (Interpretation). An argument form records a pattern of inference independently of the particular subject matter. Its validity depends on truth preservation, not on whether the individual formulas happen to be true in one example.

Remark (Dependencies).

- Well-Formed Formula

Definition (Valid Argument Form)

Definition 1.9.2 (Valid Argument Form)**.** An argument form with premises Γ and conclusion φ is *valid* if every truth assignment that satisfies every formula in Γ also satisfies φ.

Remark (Standard quantified statement).

$$\text{ValidArgumentForm}(\Gamma, \varphi) \iff (\forall v)\big((\forall \gamma \in \Gamma)(v \models \gamma) \to v \models \varphi\big).$$

Remark (Definition predicate reading). ValidArgumentForm(Γ, φ) means that truth of all premises forces truth of the conclusion under every truth assignment.

Remark (Interpretation). Validity is a preservation property. A valid argument form may have false premises in a particular instance, but it cannot have true premises and a false conclusion under the same truth assignment.

Remark (Dependencies).

- Argument Form
- Satisfaction of a Formula
- Logical Consequence

Definition (Counterexample Assignment)

Definition 1.9.3 (Counterexample Assignment)**.** A *counterexample assignment* to an argument form (Γ, φ) is a truth assignment that satisfies every premise in Γ and does not satisfy φ.

Remark (Standard quantified statement)*.*

$$\text{CounterexampleAssignment}(v, \Gamma, \varphi) \iff (\forall \gamma \in \Gamma)(v \models \gamma) \wedge v \not\models \varphi.$$

Remark (Definition predicate reading)*.* $\text{CounterexampleAssignment}(v, \Gamma, \varphi)$ means that v shows the premises can all be true while the conclusion is false.

Remark (Interpretation)*.* A counterexample assignment is the semantic witness to invalidity. Finding one settles the question: the argument form is not truth-preserving.

Remark (Dependencies)*.*

- Valid Argument Form
- Truth Assignment

Theorem (Truth-Table Validity Test for Argument Forms)

Theorem 1.9.4 (Truth-Table Validity Test for Argument Forms)**.** *An argument form (Γ, φ) is valid if and only if no row of its truth table makes every formula in Γ true and φ false.*
Go to proof.

Remark (Standard quantified statement)*.*

$$\text{ValidArgumentForm}(\Gamma, \varphi) \iff \neg(\exists v)\, \text{CounterexampleAssignment}(v, \Gamma, \varphi).$$

Remark (Predicate reading)*.* The argument form is valid exactly when there is no counterexample truth assignment.

Remark (Interpretation)*.* The truth-table test is finite because only the variables appearing in the premises and conclusion matter. A failed row is a counterexample assignment; the absence of such rows proves semantic validity.

Remark (Dependencies)*.*

- Valid Argument Form
- Counterexample Assignment
- Truth Table
- Finite Truth-Table Theorem

Core Valid Inference Patterns

Remark (Working vocabulary). This topic proves the semantic validity of the standard propositional inference patterns used throughout later proof work.

Theorem (Modus Ponens is Valid)

Theorem 1.9.5 (Modus Ponens is Valid)**.** *The argument form*

$$\varphi, \qquad \varphi \to \psi \quad \therefore \quad \psi$$

is valid.
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi, \varphi \to \psi\}, \psi).$$

Remark (Predicate reading). Whenever φ and $\varphi \to \psi$ are both true, ψ must be true.

Remark (Interpretation). Modus ponens is the basic rule for using a conditional whose antecedent has already been established.

Remark (Dependencies).

- Valid Argument Form
- Truth Assignment

Theorem (Modus Tollens is Valid)

Theorem 1.9.6 (Modus Tollens is Valid)**.** *The argument form*

$$\varphi \to \psi, \qquad \neg\psi \quad \therefore \quad \neg\varphi$$

is valid.
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi \to \psi, \neg\psi\}, \neg\varphi).$$

Remark (Predicate reading). If φ would imply ψ, and ψ is false, then φ must be false.

Remark (Interpretation). Modus tollens is the semantic form of denying an antecedent by denying the conditional's consequent.

Remark (Dependencies).

- Valid Argument Form
- Conditional Equivalence Laws

Theorem (Hypothetical Syllogism is Valid)

Theorem 1.9.7 (Hypothetical Syllogism is Valid)**.** *The argument form*

$$\varphi \to \psi, \qquad \psi \to \chi \quad \therefore \quad \varphi \to \chi$$

is valid.
Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi, \psi, \chi \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi \to \psi, \psi \to \chi\}, \varphi \to \chi).$$

Remark (Predicate reading)*.* Conditional implication composes: if φ forces ψ, and ψ forces χ, then φ forces χ.

Remark (Interpretation)*.* Hypothetical syllogism justifies chaining conditional statements.

Remark (Dependencies)*.*

- Valid Argument Form
- Logical Consequence

Theorem (Disjunctive Syllogism is Valid)

Theorem 1.9.8 (Disjunctive Syllogism is Valid)**.** *The argument form*

$$\varphi \vee \psi, \qquad \neg\varphi \quad \therefore \quad \psi$$

is valid.
Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi, \psi \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi \vee \psi, \neg\varphi\}, \psi).$$

Remark (Predicate reading)*.* If at least one of φ or ψ is true and φ is false, then ψ must be true.

Remark (Interpretation)*.* Disjunctive syllogism eliminates one disjunct after its negation is known.

Remark (Dependencies)*.*

- Valid Argument Form
- Truth Assignment

Theorem (Addition is Valid)

Theorem 1.9.9 (Addition is Valid)**.** *The argument form*

$$\varphi \quad \therefore \quad \varphi \vee \psi$$

is valid.
Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi, \psi \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi\}, \varphi \vee \psi).$$

Remark (Predicate reading)*.* A true formula remains true when placed as one disjunct of a disjunction.

Remark (Interpretation)*.* Addition introduces a disjunction from a known true disjunct.

Remark (Dependencies)*.*

- Valid Argument Form

Theorem (Simplification is Valid)

Theorem 1.9.10 (Simplification is Valid)**.** *The argument form*

$$\varphi \wedge \psi \quad \therefore \quad \varphi$$

is valid.
Go to proof.

Remark (Standard quantified statement)*.*

$$(\forall \varphi, \psi \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi \wedge \psi\}, \varphi).$$

Remark (Predicate reading)*.* If a conjunction is true, then each conjunct is true.

Remark (Interpretation)*.* Simplification extracts a component from a true conjunction.

Remark (Dependencies)*.*

- Valid Argument Form

Theorem (Conjunction Introduction is Valid)

Theorem 1.9.11 (Conjunction Introduction is Valid)**.** *The argument form*

$$\varphi, \qquad \psi \quad \therefore \quad \varphi \wedge \psi$$

is valid.
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi, \psi\}, \varphi \wedge \psi).$$

Remark (Predicate reading). If two formulas are both true, then their conjunction is true.

Remark (Interpretation). Conjunction introduction combines independently established formulas into one conjunction.

Remark (Dependencies).

- Valid Argument Form

Theorem (Constructive Dilemma is Valid)

Theorem 1.9.12 (Constructive Dilemma is Valid)**.** *The argument form*

$$\varphi \to \chi, \qquad \psi \to \theta, \qquad \varphi \vee \psi \quad \therefore \quad \chi \vee \theta$$

is valid.
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi, \theta \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi \to \chi, \psi \to \theta, \varphi \vee \psi\}, \chi \vee \theta).$$

Remark (Predicate reading). If one of two antecedents holds, and each antecedent has its own consequence, then at least one corresponding consequence holds.

Remark (Interpretation). Constructive dilemma is a proof-by-cases pattern with conditionals attached to both cases.

Remark (Dependencies).

- Valid Argument Form
- Disjunctive Syllogism is Valid

Theorem (Destructive Dilemma is Valid)

Theorem 1.9.13 (Destructive Dilemma is Valid)**.** *The argument form*

$$\varphi \to \chi, \qquad \psi \to \theta, \qquad \neg\chi \vee \neg\theta \quad \therefore \quad \neg\varphi \vee \neg\psi$$

is valid.
Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi, \theta \in \mathsf{WFF})\ \text{ValidArgumentForm}(\{\varphi \to \chi, \psi \to \theta, \neg\chi \vee \neg\theta\}, \neg\varphi \vee \neg\psi).$$

Remark (Predicate reading). If at least one consequent fails, and each antecedent would force its consequent, then at least one antecedent must fail.

Remark (Interpretation). Destructive dilemma combines disjunction with modus tollens-style reasoning.

Remark (Dependencies).

- Valid Argument Form
- Modus Tollens is Valid

Invalid Argument Forms

Remark (Working vocabulary). This topic uses counterexample assignments to show why familiar invalid forms fail to preserve truth from premises to conclusion.

Theorem (Affirming the Consequent is Invalid)

Theorem 1.9.14 (Affirming the Consequent is Invalid)**.** *The argument form*

$$\varphi \to \psi, \qquad \psi \quad \therefore \quad \varphi$$

is invalid.
Go to proof.

Remark (Standard quantified statement).

$$\neg\,\mathrm{ValidArgumentForm}(\{\varphi \to \psi, \psi\}, \varphi).$$

Remark (Predicate reading). There is a truth assignment under which $\varphi \to \psi$ and ψ are true while φ is false.

Remark (Interpretation). A conditional can be true because both antecedent and consequent are true, but also because the antecedent is false. Therefore truth of the consequent does not force truth of the antecedent.

Remark (Dependencies).

- Counterexample Assignment
- Valid Argument Form

Theorem (Denying the Antecedent is Invalid)

Theorem 1.9.15 (Denying the Antecedent is Invalid)**.** *The argument form*

$$\varphi \to \psi, \qquad \neg\varphi \quad \therefore \quad \neg\psi$$

is invalid.
Go to proof.

Remark (Standard quantified statement).

$$\neg\,\mathrm{ValidArgumentForm}(\{\varphi \to \psi, \neg\varphi\}, \neg\psi).$$

Remark (Predicate reading). There is a truth assignment under which $\varphi \to \psi$ and $\neg\varphi$ are true while $\neg\psi$ is false.

Remark (Interpretation). A conditional does not say that the antecedent is the only way for the consequent to be true. The antecedent can fail while the consequent still holds.

Remark (Exposition). For both invalid forms, take a truth assignment with φ false and ψ true. Then $\varphi \to \psi$ is true. This assignment makes the premises true and the conclusion false in each corresponding invalid pattern.

Remark (Dependencies).

- Counterexample Assignment
- Valid Argument Form

Rules of Replacement

Remark (Working vocabulary). This topic uses formula contexts, logical equivalence, and replacement of logical equivalents to justify algebraic rewriting inside larger formulas.

Definition (Rule of Replacement)

Definition 1.9.16 (Rule of Replacement)**.** A *rule of replacement* is a schematic permission to replace a subformula by a logically equivalent formula inside a formula context.

Remark (Standard quantified statement).

$$\varphi \equiv \psi \Longrightarrow C[\varphi] \equiv C[\psi].$$

Remark (Definition predicate reading). A rule of replacement licenses an equivalence-preserving rewrite inside a larger formula.

Remark (Interpretation). Rules of replacement differ from rules of inference. Inference rules move from premises to conclusions; replacement rules rewrite formulas without changing truth behavior.

Remark (Dependencies).

- Formula Context
- Logical Equivalence
- Replacement of Logical Equivalents

Theorem (Replacement Rules Preserve Logical Equivalence)

Theorem 1.9.17 (Replacement Rules Preserve Logical Equivalence)**.** *If a formula ψ is obtained from a formula φ by finitely many applications of rules of replacement, then $\varphi \equiv \psi$.*
Go to proof.

Remark (Standard quantified statement)*.*

$$\text{ReplacementSequence}(\varphi, \psi) \Longrightarrow \varphi \equiv \psi.$$

Remark (Predicate reading)*.* A finite sequence of equivalence-preserving replacements preserves logical equivalence from the first formula to the last formula.

Remark (Interpretation)*.* Replacement rules justify algebraic manipulation of formulas. Each local rewrite preserves truth under every truth assignment, so the entire finite rewrite chain also preserves truth under every truth assignment.

Remark (Exposition)*.* Common replacement rules include double negation, De Morgan replacement, commutativity, associativity, distributivity, material implication, biconditional replacement, and contraposition. Each is justified by a logical equivalence law.

Remark (Dependencies).

- Rule of Replacement
- Replacement of Logical Equivalents
- Logical Equivalence is an Equivalence Relation

Informal Proof Patterns

Remark (Working vocabulary)*.* This topic connects valid argument forms with common informal proof patterns: direct proof, conditional proof, proof by cases, contradiction, contrapositive proof, implication chains, and equivalence chains.

Definition (Informal Proof Pattern)

Definition 1.9.18 (Informal Proof Pattern)**.** An *informal proof pattern* is a reusable truth-preserving strategy for organizing an ordinary mathematical proof.

Remark (Standard quantified statement)*.*

$$\text{InformalProofPattern}(\Pi) \Longrightarrow \Pi \text{ is justified by one or more valid argument forms.}$$

Remark (Definition predicate reading)*.* InformalProofPattern(Π) means that Π is a recognized proof strategy whose correctness rests on truth-preserving inference.

Remark (Interpretation)*.* Informal proof patterns are not a substitute for proof. They are templates for arranging valid moves so that a proof has a clear logical shape.

Remark (Exposition). A direct proof establishes a target conclusion by moving forward from known premises through valid argument forms. A conditional proof establishes $\varphi \to \psi$ by assuming φ and deriving ψ inside that assumption context. Proof by cases uses a disjunction of cases and shows that each case leads to the desired conclusion.

Remark (Exposition). Proof by contradiction assumes the negation of the desired conclusion and derives an impossible situation. Contrapositive proof establishes $\varphi \to \psi$ by showing $\neg\psi \to \neg\varphi$. Chain-of-implications proofs use repeated hypothetical syllogism, while equivalence-chain proofs use replacement and logical equivalence.

Remark (Exposition). If the target has the form $\varphi \to \psi$, a conditional proof is often natural. If the available information has the form $\varphi \vee \psi$, proof by cases is often natural. If the target is an equivalence, an equivalence chain or two conditional proofs are often natural.

Remark (Dependencies).

- Valid Argument Form
- Logical Consequence

Semantic Justification of Argument Forms

Remark (Working vocabulary). This topic uses tautology, logical consequence, valid argument forms, and truth-table reasoning to connect named inference patterns with semantic validity.

Theorem (Tautological Implication Gives a Valid Argument Form)

Theorem 1.9.19 (Tautological Implication Gives a Valid Argument Form). *Let* $\Gamma = \{\gamma_1, \dots, \gamma_n\} \subseteq \mathsf{WFF}$ *and let* $\varphi \in \mathsf{WFF}$. *If*

$$(\gamma_1 \wedge \cdots \wedge \gamma_n) \to \varphi$$

is a tautology, then the argument form (Γ, φ) *is valid.*
Go to proof.

Remark (Standard quantified statement).

$$\text{Tautology}((\gamma_1 \wedge \cdots \wedge \gamma_n) \to \varphi) \Longrightarrow \text{ValidArgumentForm}(\{\gamma_1, \dots, \gamma_n\}, \varphi).$$

Remark (Predicate reading). If the conditional from the conjunction of all premises to the conclusion is always true, then the premises cannot all be true while the conclusion is false.

Remark (Interpretation). This theorem converts a tautology check into an argument-validity check. It is one reason truth tables can validate named inference rules.

Remark (Exposition). The semantic proof pattern for a valid argument form is always the same. Let an arbitrary truth assignment make all premises true. Then show that the conclusion must also be true. If this can be done for an arbitrary truth assignment, then no counterexample assignment exists.

Remark (Exposition). This section does not yet define formal derivations. The word valid here is semantic: it refers to preservation of truth under truth assignments. Later proof systems will turn selected valid patterns into formal rules of inference.

Remark (Dependencies).

- Tautology
- Valid Argument Form
- Logical Consequence

Argument Forms as a Bridge to Formal Systems

Remark (Working vocabulary). This topic distinguishes semantic validity from formal derivability and explains why formal proof systems are deferred.

Remark (Exposition). A valid argument form is a semantic object: it preserves truth under all truth assignments. A formal rule of inference is a syntactic object inside a specified proof system. Many formal systems include rules corresponding to the valid argument forms justified in this section, but the semantic justification and the formal rule are not the same object.

Remark (Exposition). Formal proof systems require additional machinery: formal derivations, assumptions, discharge rules, axioms or axiom schemata, and precise proof-checking rules. Natural deduction, Hilbert systems, tableaux, resolution, and sequent calculus are examples of such systems. They are important, but they are not the main purpose of this introductory propositional-logic section.

Remark (Transition). The tools validated here are safe to use in ordinary mathematical proof because they are truth-preserving. Later, when formal proof systems are introduced, these same patterns can be represented as formal rules, derived rules, or admissible moves depending on the chosen system.

Summary and Transition

Remark (Working vocabulary). This summary reviews argument forms, validity, counterexamples, valid inference patterns, invalid forms, replacement rules, and the semantic foundation for ordinary proof moves.

Remark (Exposition). An argument form is valid when truth of the premises under an arbitrary truth assignment forces truth of the conclusion. The central negative test is a counterexample assignment: one assignment making all premises true and the conclusion false. This section justified both sides of the method by proving standard valid forms and refuting common invalid forms.

Remark (Exposition). The valid tools now grounded include modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, addition, simplification, conjunction introduction, constructive dilemma, and destructive dilemma. The invalid forms of affirming the consequent and denying the antecedent were separated by explicit counterexample assignments.

Remark (Transition). The next section turns to compactness and finite satisfiability as a preview of a deeper theme: infinite-looking logical questions can sometimes be controlled by finite witnesses. The semantic foundation developed here remains the base for that discussion.

1.10 Compactness and Finite Satisfiability Preview

Toolkit

This section uses truth assignments, satisfiable sets of formulas, finite subsets, finite satisfiability, finite unsatisfiability witnesses, and propositional compactness.

Remark (Exposition). The compactness preview records the finite-information pattern behind propositional satisfiability: when a set of formulas is unsatisfiable, the failure is already visible in some finite subset.

Orientation

Remark (Orientation). The semantic ideas developed earlier apply formula by formula and also to sets of formulas. This section isolates the finite-information pattern behind that viewpoint. A truth assignment can satisfy a whole set Γ only when it makes every member of Γ true. Failure will be shown, in propositional logic, to be detected by some finite part of Γ. This is the content of compactness.

The goal is not to replace truth tables. It is to record the principle that truth tables suggest: whenever inconsistency appears in propositional logic, it appears after only finitely many formulas have been selected.

Remark (Guiding question). For a possibly infinite set $\Gamma \subseteq \mathsf{WFF}$, when can the existence of truth assignments for every finite piece of Γ be promoted to a truth assignment for all of Γ?

Remark (Scope of the preview). Everything in this section is stated using truth assignments, formulas, sets of formulas, and finite subsets. Later chapters will need richer semantic data, but this preview stays inside propositional logic.

Satisfiable Sets

Remark (Recall). By satisfaction of a set of formulas, $v \models \Gamma$ means that $v \models \gamma$ for every $\gamma \in \Gamma$.

Definition (Satisfiable Set of Propositional Formulas)

Definition 1.10.1 (Satisfiable Set of Propositional Formulas)**.** Let $\Gamma \subseteq \mathsf{WFF}$. The set Γ is satisfiable if there exists a truth assignment v such that $v \models \Gamma$.

$$\mathrm{SatisfiableSet}(\Gamma) \iff \exists v \left(\mathrm{TruthAssignment}(v) \wedge v \models \Gamma\right).$$

Remark (Standard quantified statement).

$$\Gamma \text{ is satisfiable} \iff \exists v \left(\text{TruthAssignment}(v) \wedge \forall \varphi \in \Gamma \left(\widehat{v}(\varphi) = \text{True}\right)\right).$$

Remark (Definition predicate reading).

$$\text{SatisfiableSet}(\Gamma)$$

means that one truth assignment makes every formula in Γ true.

Remark (Negated quantified statement).

$$\neg\, \text{SatisfiableSet}(\Gamma) \iff \forall v \left(\text{TruthAssignment}(v) \Rightarrow \exists \varphi \in \Gamma \left(\widehat{v}(\varphi) = \text{False}\right)\right).$$

Remark (Negation predicate reading).

$$\neg\, \text{SatisfiableSet}(\Gamma)$$

means that every truth assignment fails on at least one formula in Γ.

Remark (Failure modes). Satisfiability fails when every truth assignment is blocked by at least one formula in Γ.

Remark (Failure mode decomposition).

$$\neg\, \text{SatisfiableSet}(\Gamma) \iff \forall v \left(\text{TruthAssignment}(v) \Rightarrow \exists \varphi \in \Gamma \left(\widehat{v}(\varphi) = \text{False}\right)\right).$$

Thus Γ is not satisfiable exactly when every truth assignment is defeated by at least one formula from Γ.

Remark (Interpretation). Satisfiability of a set is a compatibility condition. The formulas in Γ may impose many requirements, but they are satisfiable only if all of those requirements can be met by the same truth assignment.

Remark (Dependencies).

- Truth Assignment
- Satisfaction of a Set of Formulas

Definition (Unsatisfiable Set of Propositional Formulas)

Definition 1.10.2 (Unsatisfiable Set of Propositional Formulas)**.** Let $\Gamma \subseteq \mathsf{WFF}$. The set Γ is unsatisfiable if it is not satisfiable.

$$\text{UnsatisfiableSet}(\Gamma) \iff \neg\, \text{SatisfiableSet}(\Gamma).$$

Remark (Standard quantified statement).

$$\Gamma \text{ is unsatisfiable} \iff \forall v \left(\text{TruthAssignment}(v) \Rightarrow \exists \varphi \in \Gamma \left(\widehat{v}(\varphi) = \text{False}\right)\right).$$

Remark (Definition predicate reading).

$$\text{UnsatisfiableSet}(\Gamma)$$

means that no truth assignment makes every formula in Γ true.

Remark (Negated quantified statement).

$$\neg\,\text{UnsatisfiableSet}(\Gamma) \iff \text{SatisfiableSet}(\Gamma).$$

Remark (Negation predicate reading).

$$\neg\,\text{UnsatisfiableSet}(\Gamma)$$

means that Γ has at least one satisfying truth assignment.

Remark (Interpretation). Unsatisfiability is global failure of joint truth. It does not say that every formula in Γ is false; it says that no single truth assignment can make them all true at once.

Remark (Dependencies).

- Satisfiable Set of Propositional Formulas

Finitely Satisfiable Sets

Definition (Finite Subset of a Set of Propositional Formulas)

Definition 1.10.3 (Finite Subset of a Set of Propositional Formulas)**.** Let $\Gamma \subseteq \mathsf{WFF}$. A finite subset of Γ is a set $\Delta \subseteq \Gamma$ containing only finitely many formulas.

Remark (Standard quantified statement).

$$\Delta \text{ is a finite subset of } \Gamma \iff \Delta \subseteq \Gamma \wedge \Delta \text{ is finite.}$$

Remark (Definition predicate reading).

$$\Delta \subseteq \Gamma \quad \text{and} \quad \Delta \text{ is finite}$$

means that Δ is a finite selection of formulas from Γ.

Remark (Interpretation). A finite subset is a finite test case cut out of a larger set of formulas.

Remark (Dependencies).

- Well-Formed Formula

Definition (Finitely Satisfiable Set of Propositional Formulas)

Definition 1.10.4 (Finitely Satisfiable Set of Propositional Formulas)**.** Let $\Gamma \subseteq \mathsf{WFF}$. The set Γ is finitely satisfiable if every finite subset of Γ is satisfiable.

$$\text{FinitelySatisfiableSet}(\Gamma) \iff \forall \Delta \Big((\Delta \subseteq \Gamma \wedge \Delta \text{ is finite}) \Rightarrow \text{SatisfiableSet}(\Delta) \Big).$$

Remark (Standard quantified statement).

$$\Gamma \text{ is finitely satisfiable} \iff \forall \Delta \left((\Delta \subseteq \Gamma \wedge \Delta \text{ is finite}) \Rightarrow \exists v \left(\text{TruthAssignment}(v) \wedge v \models \Delta \right) \right).$$

Remark (Definition predicate reading).

$$\text{FinitelySatisfiableSet}(\Gamma)$$

means that every finite portion of Γ has at least one satisfying truth assignment.

Remark (Negated quantified statement).

$$\neg\, \text{FinitelySatisfiableSet}(\Gamma) \iff \exists \Delta \left(\Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta) \right).$$

Remark (Negation predicate reading).

$$\neg\, \text{FinitelySatisfiableSet}(\Gamma)$$

means that some finite subset of Γ already has no satisfying truth assignment.

Remark (Failure modes). Finite satisfiability fails when at least one finite subset of Γ is unsatisfiable.

Remark (Failure mode decomposition).

$$\neg\, \text{FinitelySatisfiableSet}(\Gamma) \iff \exists \Delta \left(\Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta) \right).$$

So failure of finite satisfiability is witnessed by one finite unsatisfiable subcollection.

Remark (Interpretation). Finite satisfiability says that no finite test has found a contradiction. It does not name a single truth assignment for all of Γ; it only says that each finite test can be passed.

Remark (Dependencies).

- Finite Subset of a Set of Propositional Formulas
- Satisfiable Set of Propositional Formulas
- Truth Assignment

Finite Unsatisfiability Witnesses

Definition (Unsatisfiability Witness)

Definition 1.10.5 (Unsatisfiability Witness)**.** Let $\Gamma \subseteq \mathsf{WFF}$. A subset $\Delta \subseteq \Gamma$ is an unsatisfiability witness for Γ if Δ is unsatisfiable.

Remark (Standard quantified statement).

$$\Delta \text{ is an unsatisfiability witness for } \Gamma \iff \Delta \subseteq \Gamma \wedge \text{UnsatisfiableSet}(\Delta).$$

Remark (Definition predicate reading).

$$\Delta \subseteq \Gamma \quad \text{and} \quad \text{UnsatisfiableSet}(\Delta)$$

means that the formulas in Δ already explain why the larger set Γ cannot be satisfied, provided Δ is part of Γ.

Remark (Interpretation). An unsatisfiability witness is a subcollection that already fails to be jointly satisfiable.

Remark (Dependencies).

- Unsatisfiable Set of Propositional Formulas

Definition (Finite Unsatisfiability Witness)

Definition 1.10.6 (Finite Unsatisfiability Witness)**.** Let $\Gamma \subseteq \mathsf{WFF}$. A subset $\Delta \subseteq \Gamma$ is a finite unsatisfiability witness for Γ if Δ is finite and unsatisfiable.

$$\text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma) \iff \Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta).$$

Remark (Standard quantified statement).

$$\Delta \text{ is a finite unsatisfiability witness for } \Gamma \iff \Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta).$$

Remark (Definition predicate reading).

$$\text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma)$$

means that Δ is a finite subset of Γ and that Δ already has no satisfying truth assignment.

Remark (Interpretation). A finite unsatisfiability witness is a finite obstruction. It says that the failure of Γ is already visible before one inspects all formulas in Γ.

Remark (Dependencies).

- Finite Subset of a Set of Propositional Formulas
- Unsatisfiable Set of Propositional Formulas

Theorem (Finite Unsatisfiability Witness)

Theorem 1.10.7 (Finite Unsatisfiability Witness)**.** *Let $\Gamma \subseteq \mathsf{WFF}$. If there exists a finite $\Delta \subseteq \Gamma$ such that Δ is unsatisfiable, then Γ is unsatisfiable.*

Go to proof.

Remark (Standard quantified statement).

$$\exists \Delta \ \text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma) \Rightarrow \text{UnsatisfiableSet}(\Gamma).$$

Remark (Predicate reading).

$$\exists \Delta \ \text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma) \Rightarrow \text{UnsatisfiableSet}(\Gamma)$$

means that a single finite unsatisfiable subcollection forces the whole set Γ to be unsatisfiable.

Remark (Failure modes). The theorem can fail only if a truth assignment satisfied Γ while a finite subset $\Delta \subseteq \Gamma$ remained unsatisfiable.

Remark (Failure mode decomposition).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \forall \Delta \left(\Delta \subseteq \Gamma \Rightarrow \text{SatisfiableSet}(\Delta)\right).$$

If the whole set has a satisfying truth assignment, each subset inherits one.

Remark (Contrapositive quantified statement).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \forall \Delta \left(\Delta \subseteq \Gamma \Rightarrow \text{SatisfiableSet}(\Delta)\right).$$

Remark (Contrapositive predicate reading).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \text{FinitelySatisfiableSet}(\Gamma)$$

means that one truth assignment for all of Γ also satisfies every finite subset of Γ.

Remark (Interpretation). An unsatisfiable finite subset is decisive. Adding more formulas cannot restore a truth assignment that already fails on the finite subset.

Remark (Dependencies).

- Finite Unsatisfiability Witness
- Unsatisfiable Set of Propositional Formulas
- Satisfaction of a Set of Formulas

Corollary (Finite Witness Contrapositive)

Corollary 1.10.8 (Finite Witness Contrapositive)**.** *Let* $\Gamma \subseteq \mathsf{WFF}$*. If* Γ *is satisfiable, then every finite subset of* Γ *is satisfiable.*

Go to proof.

Remark (Standard quantified statement).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \text{FinitelySatisfiableSet}(\Gamma).$$

Remark (Predicate reading).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \text{FinitelySatisfiableSet}(\Gamma)$$

means that full satisfiability automatically gives finite satisfiability.

Remark (Interpretation). This is the easy direction connecting full satisfiability to finite satisfiability: one truth assignment that works for all formulas also works for any finite selection.

Remark (Dependencies).

- Finite Unsatisfiability Witness
- Finitely Satisfiable Set of Propositional Formulas
- Satisfiable Set of Propositional Formulas

Propositional Compactness

Theorem (Propositional Compactness)

Theorem 1.10.9 (Propositional Compactness)**.** *Let* $\Gamma \subseteq \mathsf{WFF}$*. If every finite subset of* Γ *is satisfiable, then* Γ *is satisfiable.*

Go to proof.

Remark (Standard quantified statement).

$$\text{FinitelySatisfiableSet}(\Gamma) \Rightarrow \text{SatisfiableSet}(\Gamma).$$

Remark (Predicate reading).

$$\text{FinitelySatisfiableSet}(\Gamma) \Rightarrow \text{SatisfiableSet}(\Gamma)$$

means that passing every finite satisfiability test is enough to produce a truth assignment for all of Γ.

Remark (Negated quantified statement).

$$\text{FinitelySatisfiableSet}(\Gamma) \wedge \neg\, \text{SatisfiableSet}(\Gamma)$$

is the direct negation of the compactness implication.

Remark (Negation predicate reading).

$$\text{FinitelySatisfiableSet}(\Gamma) \wedge \neg\, \text{SatisfiableSet}(\Gamma)$$

would mean that all finite tests pass but the full set has no satisfying truth assignment.

Remark (Failure modes).

$$\neg\, \text{SatisfiableSet}(\Gamma) \Rightarrow \exists \Delta \left(\Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \neg\, \text{SatisfiableSet}(\Delta) \right).$$

Compactness says that unsatisfiability cannot first appear only at the infinite level.

Remark (Failure mode decomposition).

$$\text{UnsatisfiableSet}(\Gamma) \Rightarrow \exists \Delta\ \text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma).$$

Remark (Contrapositive quantified statement).

$$\text{UnsatisfiableSet}(\Gamma) \Rightarrow \exists \Delta\ \text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma).$$

Remark (Contrapositive predicate reading). The contrapositive says that if Γ is unsatisfiable, then the failure is already witnessed by some finite subset of Γ.

Remark (Interpretation). Propositional compactness turns infinitely many finite checks into one global truth assignment. Equivalently, every propositional inconsistency has a finite witness.

Remark (Dependencies).

- Finitely Satisfiable Set of Propositional Formulas
- Satisfiable Set of Propositional Formulas
- Finite Unsatisfiability Witness

Finite Truth Table Intuition

Remark (Finite truth table intuition). Truth tables are finite because a single formula contains only finitely many variables. More generally, a finite set of formulas contains only finitely many variables, so its satisfiability can be checked by a finite truth table over those variables.

Remark (Finite test). For a finite $\Delta \subseteq \mathsf{WFF}$, the variables appearing in Δ are finite in number. A truth table over those variables decides whether Δ is satisfiable.

Remark (Why compactness is surprising). Finite truth tables directly decide finite sets. Compactness says that, for propositional logic, passing all finite tests is enough to pass the entire possibly infinite test.

Remark (Dependencies).

- Finiteness of Variables
- Finite Truth Table
- Finitely Satisfiable Set of Propositional Formulas

Summary

Remark (Section summary). The compactness preview records three related ideas.

- A set of propositional formulas is satisfiable when one truth assignment makes all of its formulas true.
- A set is finitely satisfiable when every finite subset is satisfiable.
- Propositional compactness says that finite satisfiability is enough for satisfiability.

Remark (Conceptual takeaway). In propositional logic, an inconsistency always has a finite witness. If no finite witness exists, then the whole set of formulas can be satisfied.

Chapter 2

Predicate Logic

2.1 Model-Theoretic View

Toolkit

This section presents first-order logic as a syntax-plus-semantics model, with the language interpreted by structures and formulas evaluated by the shared free-algebra machinery.

Theory (First-Order Logic)

Depends on: `Signature; term; formula; sentence; bridge:consequence;` $\mathrm{Cn}_{\vdash}$

Language

$$\mathcal{L} = (\mathsf{Const}_{\mathcal{L}}, \mathsf{Func}_{\mathcal{L}}, \mathsf{Rel}_{\mathcal{L}}, =).$$

$$\mathsf{Term}_{\mathcal{L}}, \qquad \mathsf{Form}_{\mathcal{L}}, \qquad \mathsf{Sent}_{\mathcal{L}}.$$

Presented Theory

$$T := \mathrm{Cn}_{\vdash}(\Sigma), \qquad \Sigma \subseteq \mathsf{Sent}_{\mathcal{L}}.$$

Theorems and Consequences

1.
$$\Gamma \vdash_{\mathrm{FOL}} \varphi \iff \Gamma \models_{\mathrm{FOL}} \varphi \qquad \text{(completeness)}.$$

Model (First-Order Logic)

Model 2.1.1 (First-Order Logic). **Depends on:** `$\mathcal{L}$-structure; interpretation; assignment; bridge:free-sigma-algebra-evaluation; bridge:consequence`

Note

FOL runs the shared machine twice: once for terms, then once for quantifier-free formulas. Quantifiers sit above that fixed-seed machine.

Structure

$$\mathcal{A} = \langle A, I \rangle, \qquad A \neq \varnothing.$$

$$I(c) \in A, \qquad I(f) : A^n \to A, \qquad I(R) \subseteq A^n, \qquad = \text{ is logical equality on } A.$$

Binding

$$s : \mathsf{Var} \to A \xrightarrow{\text{eval}(\mathcal{A}_{\text{term}})} [\![\cdot]\!]_{\mathcal{A},s} \xrightarrow{\text{relation/equality tests}} \sigma_s \xrightarrow{\text{eval}(\mathbf{2})} \models_{\text{qf}} .$$

Binding

$$\Sigma_{\text{term}} := \mathsf{Const}_{\mathcal{L}} \cup \mathsf{Func}_{\mathcal{L}}, \qquad \mathcal{A}_{\text{term}} := (A, (If)_{f \in \mathsf{Func}}, (Ic)_{c \in \mathsf{Const}}).$$

$$[\![\cdot]\!]_{\mathcal{A},s} := \text{eval}(\mathcal{A}_{\text{term}})(s).$$

Binding

$$\sigma_s\big(R(\bar{t})\big) := \big[([\![t_1]\!]_{\mathcal{A},s}, \ldots, [\![t_n]\!]_{\mathcal{A},s}) \in I(R)\big],$$

$$\sigma_s(t_1 = t_2) := \big[[\![t_1]\!]_{\mathcal{A},s} = [\![t_2]\!]_{\mathcal{A},s}\big].$$

Syntax–Semantics Bridge

$$\mathcal{A}, s \models \varphi \quad :\Longleftrightarrow \quad \text{eval}(\mathbf{2})(\sigma_s)(\varphi) = 1 \qquad (\varphi \text{ quantifier-free}).$$

Syntax–Semantics Bridge

$$\mathcal{A}, s \models \exists x\, \varphi \quad \Longleftrightarrow \quad \exists a \in A,\ \mathcal{A}, s[x \mapsto a] \models \varphi.$$

$$\mathcal{A}, s \models \forall x\, \varphi \quad \Longleftrightarrow \quad \forall a \in A,\ \mathcal{A}, s[x \mapsto a] \models \varphi.$$

Syntax–Semantics Bridge

For $\sigma \in \mathsf{Sent}_{\mathcal{L}}$, $\mathcal{A}, s \models \sigma$ is independent of s by the coincidence lemma, so

$$\mathcal{A} \models \sigma := \mathcal{A}, s \models \sigma \qquad (s \text{ arbitrary}).$$

Theorems and Consequences

1. Semantic consequence, $\mathcal{A} \models T$, and $\mathrm{Th}(\mathcal{A})$ are inherited from *Consequence*.
2. Prop is the base case: seed $= v$. FOL is the inductive case: s is evaluated into term values, then relation tests compute σ_s.

Definitional Extension (First-Order Logic)

Depends on: `theory:first-order-logic; expansion; reduct; conservativity; unique existence`

Note

$$\exists! y\, \varphi(y) \;:\equiv\; \exists y\big(\varphi(y) \wedge \forall y'(\varphi(y') \to y' = y)\big).$$

Admissibility

Relation symbol R, arity n:

$$\forall \bar{x}\, (R(\bar{x}) \leftrightarrow \varphi(\bar{x})).$$

Function symbol f, arity n:

$$T \vdash \forall \bar{x}\, \exists! y\, \psi(\bar{x}, y), \qquad \forall \bar{x}\, \forall y\, (f(\bar{x}) = y \leftrightarrow \psi(\bar{x}, y)).$$

Constant symbol c:

$$T \vdash \exists! y\, \psi(y), \qquad \forall y\, (c = y \leftrightarrow \psi(y)).$$

Theorems and Consequences

1. $$T^+ \vdash \sigma \quad\Longleftrightarrow\quad T \vdash \sigma \qquad (\sigma \in \mathcal{L}).$$
2. $$\mathcal{A} \models T \Longrightarrow \exists! \mathcal{A}^+\, (\mathcal{A}^+ \restriction \mathcal{L} = \mathcal{A}).$$

2.2 Axiom Schemas

Predicate-Logic Axiom Schemas

The predicate-logic development extends propositional logic with the following quantifier axiom schemas. The side conditions prevent variable capture and ensure that the displayed instances are well formed.

Axiom Schema (Predicate Logic 1)

Axiom 2.2.1 (Universal Instantiation)**.** For every formula $\varphi(x)$ and every term t that is free for x in φ,

$$\forall x\, \varphi(x) \Longrightarrow \varphi(t).$$

Remark (Interpretation). A universally quantified statement may be specialized to any admissible instance.

Axiom Schema (Predicate Logic 2)

Axiom 2.2.2 (Quantifier Distribution)**.** For formulas φ and $\psi(x)$, if x is not free in φ, then

$$\forall x\, (\varphi \Longrightarrow \psi(x)) \Longrightarrow \big(\varphi \Longrightarrow \forall x\, \psi(x)\big).$$

Remark (Interpretation). When a hypothesis does not depend on the quantified variable, a universal conclusion may be drawn uniformly under that hypothesis.

2.3 Syntax

2.3.1 Syntax of First-Order Logic: Terms and Formulas

Remark (Section toolkit: Syntax ledger). This section fixes the syntactic inventory for first-order logic before any semantic machinery is introduced. The ledger proceeds from symbols to terms, from terms to atomic formulas, and from atomic formulas to all well-formed formulas.

Layer	Item	Role
Symbol stock	Variable	reusable placeholder symbol
Symbol stock	Constant Symbol	nullary name-forming symbol
Symbol stock	Function Symbol	operation-forming symbol
Symbol stock	Predicate Symbol	assertion-forming symbol
Object syntax	Term	expression that names an object syntactically
Formula syntax	Atomic Formula	predicate symbol applied to terms
Formula syntax	Well-Formed Formula	formula generated by the grammar
Formula syntax	Molecular Formula	non-atomic well-formed formula

Remark (Exposition). The grammar has two tracks. Terms are built first, because atomic formulas need terms as their inputs. Formulas are built second, because they begin with atomic formulas and then close under logical connectives and quantifiers. Semantic notions such as structures, assignments, satisfaction, truth, and models enter later; the present section records only which strings count as grammatical expressions.

Remark (Grammar boundary). Terms name objects. Formulas make assertions. A term is not a formula, and a formula is not a term. For example, $f(x)$ is a term, while $P(f(x))$ is a formula.

Remark (Predicate-symbol convention). Object-language predicate symbols, such as P, Q, R, are symbols that occur inside first-order formulas. Project metalanguage predicate readings, such as Term(t) or AtomicFormula(φ), are external classification predicates used by these notes to describe the grammar. The two levels should not be confused.

Definition (Variable)

Definition 2.3.1 (Variable)**.** For a first-order language $\mathcal{L}$, a *variable* is a symbol drawn from the distinguished variable stock of $\mathcal{L}$. Variables are usually written

$$x, y, z, x_0, x_1, x_2, \dots.$$

Remark (Standard quantified statement).

$$\mathsf{Var}_{\mathcal{L}} = \{x, y, z, x_0, x_1, x_2, \dots\}.$$

Remark (Definition predicate reading). Variable(x) means $x \in \mathsf{Var}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$x \notin \mathsf{Var}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg$ Variable(x) means that x is not one of the variable symbols of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a variable when it belongs to a different syntactic role, such as a constant symbol, function symbol, predicate symbol, connective, or punctuation mark.

Remark (Failure mode decomposition).

$$\underbrace{c}_{\text{constant symbol}} \qquad \underbrace{f}_{\text{function symbol}} \qquad \underbrace{P}_{\text{predicate symbol}} \qquad \underbrace{\forall}_{\text{logical symbol}}.$$

Remark (Interpretation). A variable is a basic syntactic placeholder. It can occur inside terms and formulas, and later sections explain how quantifiers bind occurrences of variables.

Remark (Examples). x, y, z, x_0, and x_1 are standard variable symbols.

Remark (Non-Examples). c, 0, f, P, $\forall$, and $\wedge$ are not variables merely because they are symbols of the language.

Remark (Dependencies).

- Language

Definition (Constant Symbol)

Definition 2.3.2 (Constant Symbol)**.** For a first-order language $\mathcal{L}$, a *constant symbol* is a non-logical symbol of arity 0. A constant symbol may occur as a term without taking input terms.

Remark (Standard quantified statement).

$$\mathsf{Const}_{\mathcal{L}} = \{c \in \mathcal{L} : \text{arity}(c) = 0\}.$$

Remark (Definition predicate reading). ConstantSymbol(c) means $c \in \mathsf{Const}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$c \notin \mathsf{Const}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg$ ConstantSymbol(c) means that c is not a nullary non-logical symbol of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a constant symbol when it is not in the language, is logical punctuation, or has positive arity.

Remark (Failure mode decomposition).

$$\underbrace{x}_{\text{variable}} \qquad \underbrace{f}_{\text{positive arity}} \qquad \underbrace{(}_{\text{punctuation}}.$$

Remark (Interpretation). A constant symbol is a simple name-forming symbol in the grammar. It supplies a base case for terms alongside variables.

Remark (Examples). In an arithmetic language, 0 and 1 are commonly used as constant symbols.

Remark (Non-Examples). $f(x)$ is not a constant symbol; it is a term built from a function symbol and an input term.

Remark (Dependencies).

- Variable

Definition (Function Symbol)

Definition 2.3.3 (Function Symbol)**.** For a first-order language $\mathcal{L}$, an n-ary *function symbol* is a non-logical symbol of arity $n \geq 1$. If f has arity n, then it forms terms by the pattern

$$f(t_1, \ldots, t_n).$$

Remark (Standard quantified statement).

$$\mathsf{Func}_{\mathcal{L}}^{(n)} = \{f \in \mathcal{L} : \text{arity}(f) = n \text{ and } n \geq 1\}.$$

Remark (Definition predicate reading). FunctionSymbol(f) means $f \in \mathsf{Func}_{\mathcal{L}}^{(n)}$ for some $n \geq 1$.

Remark (Negated quantified statement).

$$\forall n \geq 1 \; (f \notin \mathsf{Func}_{\mathcal{L}}^{(n)}).$$

Remark (Negation predicate reading). $\neg\,\text{FunctionSymbol}(f)$ means that f is not a positive-arity function symbol of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a function symbol when it is a variable, constant symbol, predicate symbol, logical symbol, punctuation mark, or has no assigned positive arity.

Remark (Failure mode decomposition).

$$\underbrace{c}_{\text{arity } 0} \qquad \underbrace{P}_{\text{predicate symbol}} \qquad \underbrace{\neg}_{\text{logical symbol}}.$$

Remark (Interpretation). A function symbol is an operation-forming symbol in the grammar. It is not a term by itself; it becomes part of a term only after it is supplied the required number of input terms.

Remark (Examples). In an arithmetic language, S may be unary, while $+$ and $\cdot$ may be binary function symbols.

Remark (Non-Examples). $f(x)$ is not a function symbol. It is a term formed by applying the function symbol f to the term x.

Remark (Dependencies).

- Constant Symbol

Definition (Predicate Symbol)

Definition 2.3.4 (Predicate Symbol)**.** For a first-order language $\mathcal{L}$, an n-ary *predicate symbol* is a non-logical symbol of arity $n \geq 1$. If P has arity n, then it forms atomic formulas by the pattern

$$P(t_1, \ldots, t_n).$$

Remark (Standard quantified statement).

$$\mathsf{Pred}_{\mathcal{L}}^{(n)} = \{P \in \mathcal{L} : \text{arity}(P) = n \text{ and } n \geq 1\}.$$

Remark (Definition predicate reading). $\text{PredicateSymbol}(P)$ means $P \in \mathsf{Pred}_{\mathcal{L}}^{(n)}$ for some $n \geq 1$.

Remark (Negated quantified statement).

$$\forall n \geq 1\ (P \notin \mathsf{Pred}_{\mathcal{L}}^{(n)}).$$

Remark (Negation predicate reading). $\neg\,\text{PredicateSymbol}(P)$ means that P is not a positive-arity predicate symbol of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a predicate symbol when it belongs to the term-forming side of the language, is a logical symbol, is punctuation, or has no assigned positive arity as a predicate symbol.

Remark (Failure mode decomposition).

$$\underbrace{x}_{\text{variable}} \qquad \underbrace{c}_{\text{constant symbol}} \qquad \underbrace{f}_{\text{function symbol}} \qquad \underbrace{\exists}_{\text{logical symbol}} .$$

Remark (Interpretation). Predicate symbols are the atomic-formula-forming vocabulary of the language. They are syntactic symbols, not project metalanguage predicates.

Remark (Examples). P, Q, R, $<$, and $=$ may be predicate symbols when the language declares the corresponding arities.

Remark (Non-Examples). $P(x)$ is not a predicate symbol. It is an atomic formula formed from the predicate symbol P and the term x.

Remark (Dependencies).

- Function Symbol

Definition (Term)

Definition 2.3.5 (Term)**.** The set of *terms* of a first-order language $\mathcal{L}$, denoted $\mathsf{Term}_{\mathcal{L}}$, is the smallest set of strings satisfying:

1. every variable is a term;
2. every constant symbol is a term;
3. if f is an n-ary function symbol and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term;
4. no string is a term unless it is generated by finitely many applications of the previous clauses.

Remark (Standard quantified statement).

$$\mathsf{Term}_{\mathcal{L}} = \bigcap \left\{ S : \mathsf{Var}_{\mathcal{L}} \subseteq S,\ \mathsf{Const}_{\mathcal{L}} \subseteq S,\ (\forall n \geq 1)(\forall f \in \mathsf{Func}_{\mathcal{L}}^{(n)})(\forall t_1, \dots, t_n \in S) f(t_1, \dots, t_n) \in S \right\}.$$

Remark (Definition predicate reading). $\mathrm{Term}(t)$ means $t \in \mathsf{Term}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$\sigma \notin \mathsf{Term}_{\mathcal{L}} \iff \exists S \begin{bmatrix} \mathsf{Var}_{\mathcal{L}} \subseteq S \wedge \mathsf{Const}_{\mathcal{L}} \subseteq S \\ \wedge\ (\forall n \geq 1)(\forall f \in \mathsf{Func}_{\mathcal{L}}^{(n)})(\forall t_1, \dots, t_n \in S) f(t_1, \dots, t_n) \in S \\ \wedge\ \ \sigma \notin S \end{bmatrix}.$$

Remark (Negation predicate reading). $\neg\,\mathrm{Term}(\sigma)$ means that σ is not generated by the term-formation clauses.

Remark (Failure modes). A string fails to be a term when it uses a predicate symbol, connective, or quantifier at the outer syntactic level, when a function symbol has too few or too many input terms, or when punctuation does not match the arity pattern.

Remark (Failure mode decomposition).

$$\underbrace{P(x)}_{\text{atomic formula, not a term}} \qquad \underbrace{f(x,y)}_{\text{wrong arity if } f \text{ is unary}} \qquad \underbrace{\forall x}_{\text{quantifier fragment}} .$$

Remark (Interpretation). Terms are the noun-like expressions of first-order syntax. They provide the inputs to predicate symbols but do not themselves make assertions.

Remark (Examples). x, c, $f(x)$, $g(c,x)$, and $h(f(x),c)$ are terms when the language assigns the displayed arities.

Remark (Non-Examples). $P(x)$, $\neg P(x)$, $\forall x\, P(x)$, and $x \wedge y$ are not terms.

Remark (Dependencies).

- Language

Definition (Atomic Formula)

Definition 2.3.6 (Atomic Formula)**.** An *atomic formula* of a first-order language $\mathcal{L}$ is a string of the form

$$P(t_1, \ldots, t_n),$$

where P is an n-ary predicate symbol and $t_1, \ldots, t_n$ are terms.

Remark (Standard quantified statement).

$$\mathsf{Atom}_{\mathcal{L}} = \left\{ P(t_1, \ldots, t_n) : P \in \mathsf{Pred}_{\mathcal{L}}^{(n)} \text{ and } t_1, \ldots, t_n \in \mathsf{Term}_{\mathcal{L}} \right\}.$$

Remark (Definition predicate reading). AtomicFormula(φ) means $\varphi \in \mathsf{Atom}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$\varphi \notin \mathsf{Atom}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg$ AtomicFormula(φ) means that φ is not a predicate symbol applied to the required number of terms.

Remark (Failure modes). A string fails to be atomic when it does not begin with a predicate symbol, when the predicate symbol receives the wrong number of term inputs, when an input is not a term, or when a logical connective or quantifier has already been applied.

Remark (Failure mode decomposition).

$$\underbrace{f(x)}_{\text{term, not formula}} \qquad \underbrace{P(x,y)}_{\text{wrong arity if } P \text{ is unary}} \qquad \underbrace{\neg P(x)}_{\text{molecular formula}} .$$

Remark (Interpretation). Atomic formulas are the base cases of formula formation. They are the first strings in the grammar that count as formulas.

Remark (Examples). $P(x)$, $R(x,c)$, $x = y$, and $x < y$ are atomic formulas when the language supplies the relevant predicate symbols and arities.

Remark (Non-Examples). x, $f(x)$, $\neg P(x)$, and $(P(x) \wedge Q(x))$ are not atomic formulas.

Remark (Dependencies).

- Predicate Symbol
- Term

Definition (Formula)

Definition 2.3.7 (Formula)**.** For a first-order language $\mathcal{L}$, a *formula* is an expression formed by the formula grammar of $\mathcal{L}$. The grammar starts from atomic formulas and closes under the logical connectives and quantifiers of the language.

Remark (Interpretation). The language supplies the symbols and formation rules; the formula notion is the syntactic category governed by those rules.

Remark (Dependencies).

- Language

Definition (Well-Formed Formula)

Definition 2.3.8 (Well-Formed Formula)**.** The set of *well-formed formulas* of a first-order language $\mathcal{L}$, denoted $\mathsf{Form}_{\mathcal{L}}$, is the smallest set of strings satisfying:

1. every atomic formula is a well-formed formula;
2. if φ is a well-formed formula, then $\neg\varphi$ is a well-formed formula;
3. if φ and ψ are well-formed formulas and $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$, then $(\varphi \circ \psi)$ is a well-formed formula;
4. if φ is a well-formed formula and x is a variable, then $\forall x\, \varphi$ and $\exists x\, \varphi$ are well-formed formulas;
5. no string is a well-formed formula unless it is generated by finitely many applications of the previous clauses.

Remark (Standard quantified statement).

$$\begin{aligned}\mathsf{Form}_{\mathcal{L}} = \bigcap \{S : {} & \mathsf{Atom}_{\mathcal{L}} \subseteq S, \\ & (\forall \varphi \in S)(\neg\varphi \in S), \\ & (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \to, \leftrightarrow\})((\varphi \circ \psi) \in S), \\ & (\forall x \in \mathsf{Var}_{\mathcal{L}})(\forall \varphi \in S)(\forall x\, \varphi \in S \wedge \exists x\, \varphi \in S)\}.\end{aligned}$$

Remark (Definition predicate reading). WellFormedFormula(φ) means $\varphi \in \mathsf{Form}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$\varphi \notin \mathsf{Form}_{\mathcal{L}} \iff \exists S \begin{bmatrix} \mathsf{Atom}_{\mathcal{L}} \subseteq S \wedge (\forall \psi \in S)(\neg\psi \in S) \\ \wedge\ (\forall \psi, \chi \in S)(\forall \circ \in \{\wedge, \vee, \to, \leftrightarrow\})((\psi \circ \chi) \in S) \\ \wedge\ (\forall x \in \mathsf{Var}_{\mathcal{L}})(\forall \psi \in S)(\forall x\, \psi \in S \wedge \exists x\, \psi \in S) \\ \wedge\ \ \varphi \notin S \end{bmatrix}.$$

Remark (Negation predicate reading). $\neg\,\mathrm{WellFormedFormula}(\varphi)$ means that φ is not generated by the formula-formation clauses.

Remark (Failure modes). A string fails to be well formed when it is a bare term, has a missing operand, uses a quantifier without a variable and formula, has unmatched punctuation, or cannot be produced by finitely many formation steps.

Remark (Failure mode decomposition).

$$\underbrace{f(x)}_{\text{term only}} \qquad \underbrace{P(x)\wedge}_{\text{missing right formula}} \qquad \underbrace{\forall P(x)}_{\text{missing bound variable}} \qquad \underbrace{(P(x)}_{\text{unmatched punctuation}}.$$

Remark (Interpretation). Well-formed formulas are exactly the grammatical formula strings of the language. Later semantic and proof-theoretic sections apply only to these strings.

Remark (Examples). $P(x)$, $\neg P(x)$, $(P(x) \to Q(x))$, and $\forall x\,(P(x) \to Q(x))$ are well-formed formulas.

Remark (Non-Examples). x, $f(x)$, $P(x)\wedge$, and $\forall P(x)$ are not well-formed formulas.

Remark (Dependencies).

- Formula

Definition (Molecular Formula)

Definition 2.3.9 (Molecular Formula)**.** A *molecular formula* is a well-formed formula that is not atomic. Equivalently, it is a well-formed formula whose final formation step uses a logical connective or a quantifier.

Remark (Standard quantified statement).

$$\varphi \in \mathsf{Form}_{\mathcal{L}} \setminus \mathsf{Atom}_{\mathcal{L}}.$$

Remark (Definition predicate reading).

$$\mathrm{MolecularFormula}(\varphi) := \mathrm{WellFormedFormula}(\varphi) \wedge \neg\,\mathrm{AtomicFormula}(\varphi).$$

Remark (Negated quantified statement).

$$\varphi \notin \mathsf{Form}_{\mathcal{L}} \quad \text{or} \quad \varphi \in \mathsf{Atom}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg\,\mathrm{MolecularFormula}(\varphi)$ means that φ is either not a well-formed formula or is atomic.

Remark (Failure modes). A string fails to be a molecular formula either because it is not a well-formed formula at all or because it is an atomic formula with no outer logical construction.

Remark (Failure mode decomposition).

$$\underbrace{x}_{\text{not a formula}} \qquad \underbrace{P(x)}_{\text{atomic formula}} \qquad \underbrace{P(x)\wedge}_{\text{not well formed}} .$$

Remark (Interpretation). Molecular formulas are the compound formulas of first-order syntax. They are formed after the atomic stage by applying a connective or quantifier.

Remark (Examples). $\neg P(x)$, $(P(x) \wedge Q(x))$, and $\forall x\, P(x)$ are molecular formulas.

Remark (Non-Examples). $P(x)$ is atomic rather than molecular, and $f(x)$ is a term rather than a formula.

Remark (Dependencies).

- Well-Formed Formula
- Atomic Formula

Variables and Substitution — Quick Reference

Concept	Meaning
Scope of a quantifier	Formula governed by the quantifier
Bound/free occurrence	Whether x is captured by a quantifier
Free variable $\mathrm{FV}(\varphi)$	Variables whose values affect truth
Sentence	Formula with $\mathrm{FV}(\varphi) = \varnothing$
Substitution $\varphi[t/x]$	Replace free x by term t
Free for substitution	No variable in t becomes bound
Alpha-equivalence	Renaming bound variables

2.3.2 Syntax: Free Variables, Bound Variables, and Substitution

Definition (Scope of a Quantifier)

Definition 2.3.10 (Scope of a Quantifier)**.** Let φ be a formula of a first-order language. If φ is of the form $\forall x\, \psi$ or $\exists x\, \psi$, then the formula ψ is called the *scope* of the quantifier. The quantifier is said to *bind* all occurrences of the variable x that appear within its scope.

Remark (Dependencies).

- Formula
- Variable
- Universal Quantifier

- Existential Quantifier

Remark (English reading). The scope is the reach of a quantifier — the subformula it governs. Scope is syntactically determined by the parenthesisation of the formula, not by proximity to the quantifier symbol.

Example. In $(\forall x\, P(x)) \wedge Q(x)$, the scope of $\forall x$ is $P(x)$ only. The occurrence of x in $Q(x)$ falls outside the scope and is free.

Definition (Bound and Free Occurrences)

Definition 2.3.11 (Bound and Free Occurrences)**.** An occurrence of a variable x in a formula φ is *bound* if it lies within the scope of a quantifier $\forall x$ or $\exists x$.
An occurrence of x is *free* if it is not bound by any quantifier in φ.

Remark (Dependencies).

- Formula
- Variable
- Universal Quantifier
- Existential Quantifier
- Scope of a Quantifier

Remark (English reading). The same variable can have both bound and free occurrences in a single formula. Each occurrence is classified independently by checking whether it falls within the scope of a binding quantifier.

Definition (Free Variables of a Formula)

Definition 2.3.12 (Free Variables of a Formula)**.** The set $\mathrm{FV}(\varphi)$ of *free variables* of a formula φ is defined recursively:

1. If $\varphi = P(t_1, \ldots, t_n)$ is atomic, then $\mathrm{FV}(\varphi) = \bigcup_{i=1}^{n} \mathrm{Var}(t_i)$, where $\mathrm{Var}(t_i)$ is the set of variables in term t_i.
2. $\mathrm{FV}(\neg\varphi) = \mathrm{FV}(\varphi)$.
3. $\mathrm{FV}(\varphi \circ \psi) = \mathrm{FV}(\varphi) \cup \mathrm{FV}(\psi)$ for any binary connective $\circ$.
4. $\mathrm{FV}(\forall x\, \varphi) = \mathrm{FV}(\varphi) \setminus \{x\}$.
5. $\mathrm{FV}(\exists x\, \varphi) = \mathrm{FV}(\varphi) \setminus \{x\}$.

Remark (Dependencies).

- Bound and Free Occurrences

- Atomic Formula
- Formula
- Term
- Variable
- Union
- Set Difference

Remark (English reading). $\mathrm{FV}(\varphi)$ collects exactly those variables whose values can influence the truth of φ under a structure. Quantifying over x removes x from the free set because the quantifier takes responsibility for ranging over all (or some) values of x.

Remark (Common error). Terms themselves contain only free variables. Variables become bound only through quantification in formulas. The interpretation of a formula depends exactly on the values assigned to its free variables.

Definition (Sentence)

Definition 2.3.13 (Sentence)**.** A *sentence* (or *closed formula*) is a formula with no free variables.
Formally, φ is a sentence if and only if $\mathrm{FV}(\varphi) = \varnothing$.

Remark (Dependencies).

- Formula
- Free Variables of a Formula
- Empty Set

Remark (Consequence). For sentences, truth depends only on the structure, not on the variable assignment. This makes sentences the natural objects to be called true or false in a model, without qualification.

Definition (Substitution Notation)

Definition 2.3.14 (Substitution Notation)**.** Let φ be a formula, x a variable, and t a term.
The expression $\varphi[t/x]$ denotes the formula obtained from φ by replacing every *free* occurrence of x with the term t, leaving bound occurrences of x unchanged.

Remark (Dependencies).

- Variable
- Term
- Formula
- Bound and Free Occurrences

Definition (Free for Substitution)

Definition 2.3.15 (Free for Substitution)**.** A term t is *free for* x in φ if no free occurrence of x in φ lies within the scope of a quantifier $\forall y$ or $\exists y$ where y is a variable occurring in t. Equivalently, t is free for x in φ if the substitution $\varphi[t/x]$ does not result in any variable in t becoming bound.

Remark (Standard quantified statement). The relation "t is free for x in φ" is defined by structural induction on formulas. It is automatic for atomic formulas. It is preserved through Boolean connectives:

$$t \text{ free for } x \text{ in } \neg\varphi \iff t \text{ free for } x \text{ in } \varphi,$$

and similarly componentwise for $\varphi \wedge \psi$, $\varphi \vee \psi$, and $\varphi \to \psi$. For a quantified formula, the capture-avoidance condition is

$$t \text{ free for } x \text{ in } Qy\,\varphi \iff \big(x \text{ is not free in } Qy\,\varphi\big) \text{ or } \big(y \notin \mathrm{Var}(t) \text{ and } t \text{ is free for } x \text{ in } \varphi\big),$$

where Q is either $\forall$ or $\exists$.

Remark (Dependencies).

- Variable
- Term
- Formula
- Bound and Free Occurrences
- Free Variables of a Formula

Remark (Capture-avoiding substitution). A substitution $\varphi[t/x]$ is admissible only if t is free

for x in φ. If this condition is violated, a variable occurring in t may become bound after substitution, silently changing the meaning of the formula. This is called *variable capture*.

Example (Variable capture). In the formula $\exists y\,(x < y)$, the term $y + 1$ is not free for x. A blind substitution would produce $\exists y\,(y + 1 < y)$, where the y introduced by the term has been captured by the existing quantifier.

Remark (Common error). Variable capture is one of the most common syntactic mistakes in predicate logic. Always check that the term being substituted introduces no variables that fall inside a binding quantifier in the target formula.

Definition (Alpha-Equivalence)

Definition 2.3.16 (Alpha-Equivalence)**.** Formulas that differ only by the names of bound variables are logically equivalent. This is called *alpha-equivalence* (or *alphabetic variance*):

$$\forall x\,\varphi \;\equiv\; \forall y\,\varphi[y/x] \quad \text{(provided } y \text{ is not free in } \varphi\text{)}.$$

Remark (Dependencies).

- Formula
- Bound and Free Occurrences
- Substitution Notation
- Free for Substitution

Remark (Consequence). Alpha-equivalence is the formal basis for the bound variable renaming used in prenex normal form conversion and in any proof where variable capture must be avoided. Two alpha-equivalent formulas are interchangeable in all contexts.

Formula Depth — Quick Reference

Concept	Meaning
Formula depth	Max formation steps from atomic formulas

2.3.3 Syntax: Formula Depth

Definition (Formula Depth)

Definition 2.3.17 (Formula Depth)**.** The *depth* (or *complexity*) of a formula φ, denoted $\text{depth}(\varphi)$, is a natural number defined recursively:

1. **Atomic case.** If φ is atomic, then $\text{depth}(\varphi) = 0$.
2. **Negation.** If $\varphi = \neg\psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.
3. **Binary connectives.** If $\varphi = (\psi \circ \chi)$ for $\circ \in \{\wedge, \vee, \to, \leftrightarrow\}$, then
$$\text{depth}(\varphi) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$
4. **Quantifiers.** If $\varphi = \forall x\, \psi$ or $\varphi = \exists x\, \psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.

Remark (Dependencies).

- Formula
- Natural Numbers
- Universal Quantifier
- Existential Quantifier

Remark (English reading). The depth of a formula measures the maximum number of logical formation steps required to build the formula from atomic formulas. It corresponds to the height of the formula's syntactic parse tree.

Remark (Proof strategy). Formula depth is the standard measure used in structural induction proofs over first-order formulas. The base case is depth 0 (atomic formulas); the inductive step handles each formation rule and assumes the result holds for sub-formulas of strictly smaller depth.

2.4 Semantics

2.4.1 Semantics: Structures and Variable Assignments

Remark (Section toolkit: Semantic setup). This section is the first point at which semantic objects enter the Predicate Logic chapter. A language supplies the symbols; a structure supplies a domain and interpretations for the non-logical symbols; an assignment supplies values for variables; term evaluation turns terms into domain elements.

Item	Notation	Role
First-order language	$\mathcal{L}$	symbol stock with arities
Structure	$\mathcal{M} = \langle D, I \rangle$	semantic object for $\mathcal{L}$
Variable assignment	$s : \mathsf{Var}_{\mathcal{L}} \to D$	values for variables
Modified assignment	$s[x \mapsto d]$	update at one variable
Term evaluation	$[\![t]\!]_{\mathcal{M},s}$	value of a term

Definition (First-Order Language)

Definition 2.4.1 (First-Order Language)**.** A *first-order language* $\mathcal{L}$ consists of variables, logical symbols, and a signature of non-logical symbols: constant symbols, function symbols with assigned arities, and predicate symbols with assigned arities. Equality may be included as a distinguished logical predicate.

Remark (Standard quantified statement).

$$\text{A first-order language } \mathcal{L} \text{ has symbol classes}$$
$$\mathsf{Var}_{\mathcal{L}}, \quad \mathsf{Const}_{\mathcal{L}}, \quad \mathsf{Func}_{\mathcal{L}}^{(n)}, \quad \mathsf{Pred}_{\mathcal{L}}^{(n)} \quad (n \geq 1),$$

together with the usual logical connectives, quantifiers, and punctuation.

Remark (Definition predicate reading). FirstOrderLanguage(L) means that L supplies the variable stock, logical symbols, and arity-indexed non-logical symbols used to form first-order terms and formulas.

Remark (Negated quantified statement).

$$L \text{ is not equipped with the required first-order symbol classes and arities.}$$

Remark (Negation predicate reading). $\neg$ FirstOrderLanguage(L) means that L lacks some required syntactic stock or arity assignment.

Remark (Failure modes). A proposed language fails when its non-logical symbols are not sorted by role, when positive-arity symbols have no arity, or when the logical grammar is not specified.

Remark (Failure mode decomposition).

$$\underbrace{\mathsf{Const}_L}_{\text{constant symbols}} \qquad \underbrace{\mathsf{Func}_L^{(n)}}_{\text{function symbols with arity}} \qquad \underbrace{\mathsf{Pred}_L^{(n)}}_{\text{predicate symbols with arity}} .$$

Remark (Interpretation). A first-order language is still syntax. It says which expressions can be formed, but it does not assign objects, functions, or relations to the non-logical symbols.

Remark (Examples). The arithmetic language with constant symbols $0, 1$, function symbols $S, +, \cdot$, and predicate symbol $<$ is a first-order language once the arities are fixed.

Remark (Dependencies).

- Variable

- Constant Symbol
- Function Symbol
- Predicate Symbol

Definition (Structure)

Definition 2.4.2 (Structure)**.** Let $\mathcal{L}$ be a first-order language. A *structure* for $\mathcal{L}$ is a pair

$$\mathcal{M} = \langle D, I \rangle$$

where D is a nonempty set and I assigns each constant symbol an element of D, each n-ary function symbol a function $D^n \to D$, and each n-ary predicate symbol a relation on D^n.

Remark (Standard quantified statement)*.*

$$\begin{aligned} &\text{Let } \mathcal{L} \text{ be a first-order language.} \\ &\mathcal{M} = \langle D, I \rangle \text{ is a structure for } \mathcal{L} \\ &\iff D \neq \varnothing,\ I(c) \in D,\ I(f) : D^n \to D,\ I(P) \subseteq D^n \end{aligned}$$

for each $c \in \mathsf{Const}_{\mathcal{L}}$, $f \in \mathsf{Func}_{\mathcal{L}}^{(n)}$, and $P \in \mathsf{Pred}_{\mathcal{L}}^{(n)}$.

Remark (Definition predicate reading)*.* Structure(M, L) means that M is a structure for the first-order language L.

Remark (Negated quantified statement)*.*

$$D = \varnothing \quad \text{or} \quad I \text{ fails to assign the required object, function, or relation to some symbol.}$$

Remark (Negation predicate reading)*.* $\neg$Structure(M, L) means that M is not a well-formed structure for L.

Remark (Failure modes)*.* A proposed structure fails if the domain is empty, if a symbol is left uninterpreted, or if the assigned object has the wrong arity or type.

Remark (Failure mode decomposition)*.*

$$\underbrace{D = \varnothing}_{\text{empty domain}} \quad \vee \quad \underbrace{I(c) \notin D}_{\text{bad constant value}} \quad \vee \quad \underbrace{I(f) : D^n \not\to D}_{\text{bad function arity}} \quad \vee \quad \underbrace{I(P) \not\subseteq D^n}_{\text{bad predicate relation}} .$$

Remark (Interpretation)*.* A structure is the minimal semantic object needed in Volume I. It gives meaning to the non-logical symbols of a language. Systematic topics such as substructures, expansions, reducts, and elementary classes belong to Volume VIII Model Theory.

Remark (Examples)*.* The natural numbers with their usual interpretations of $0, S, +, \cdot, <$ form a structure for the arithmetic language.

Remark (Dependencies)*.*

- First-Order Language

Definition (Variable Assignment)

Definition 2.4.3 (Variable Assignment)**.** Let $\mathcal{M} = \langle D, I \rangle$ be a structure for $\mathcal{L}$. A *variable assignment* for $\mathcal{M}$ is a function

$$s : \mathsf{Var}_{\mathcal{L}} \to D.$$

Remark (Standard quantified statement)*.*

$$s \text{ is a variable assignment for } \mathcal{M} \iff \mathrm{dom}(s) = \mathsf{Var}_{\mathcal{L}} \text{ and } \mathrm{ran}(s) \subseteq D.$$

Remark (Definition predicate reading)*.* VariableAssignment(s, M) means that s assigns a domain element of M to each variable of the language of M.

Remark (Negated quantified statement)*.*

$$\mathrm{dom}(s) \neq \mathsf{Var}_{\mathcal{L}} \quad \text{or} \quad \exists x \in \mathsf{Var}_{\mathcal{L}}\ (s(x) \notin D).$$

Remark (Negation predicate reading)*.* $\neg$ VariableAssignment(s, M) means that s is not a function assigning each variable a domain element of M.

Remark (Failure modes)*.* An assignment fails when it leaves a variable without a value or assigns a value outside the domain.

Remark (Failure mode decomposition)*.*

$$\underbrace{\mathrm{dom}(s) \neq \mathsf{Var}_{\mathcal{L}}}_{\text{missing variable value}} \quad \vee \quad \underbrace{s(x) \notin D}_{\text{value outside domain}} \quad .$$

Remark (Interpretation)*.* Structures interpret non-logical symbols; assignments interpret variables. Open formulas require assignments, while sentences eventually do not depend on the particular assignment.

Remark (Dependencies)*.*

- Structure

Definition (Modified Assignment)

Definition 2.4.4 (Modified Assignment)**.** Let $s : \mathsf{Var}_{\mathcal{L}} \to D$, let $x \in \mathsf{Var}_{\mathcal{L}}$, and let $d \in D$. The *modified assignment* $s[x \mapsto d]$ is the assignment defined by

$$s[x \mapsto d](y) = \begin{cases} d, & y = x, \\ s(y), & y \neq x. \end{cases}$$

Remark (Standard quantified statement)*.*

$$\forall y \in \mathsf{Var}_{\mathcal{L}} \quad s[x \mapsto d](y) = d \iff y = x, \qquad s[x \mapsto d](y) = s(y) \iff y \neq x.$$

Remark (Definition predicate reading). ModifiedAssignment(s, x, d, s') means that s' is the assignment obtained from s by sending x to d and leaving every other variable unchanged.

Remark (Negated quantified statement).

$$s'(x) \neq d \quad \text{or} \quad \exists y \in \mathsf{Var}_{\mathcal{L}}\ (y \neq x \wedge s'(y) \neq s(y)).$$

Remark (Negation predicate reading). $\neg$ ModifiedAssignment(s, x, d, s') means that s' is not the one-variable update of s at x with value d.

Remark (Failure modes). A proposed modified assignment fails if it does not change x to d, or if it changes some variable other than x.

Remark (Failure mode decomposition).

$$\underbrace{s'(x) \neq d}_{\text{target variable not updated}} \quad \vee \quad \underbrace{\exists y \neq x\ (s'(y) \neq s(y))}_{\text{unwanted extra change}}.$$

Remark (Interpretation). Modified assignments are the semantic mechanism behind quantifiers. To test a quantified formula, the assignment is varied at the bound variable while all other variable values remain fixed.

Remark (Dependencies).

- Variable Assignment

Definition (Term Evaluation)

Definition 2.4.5 (Term Evaluation)**.** Let $\mathcal{M} = \langle D, I \rangle$ be a structure and s a variable assignment. The *value* of a term t, denoted $[\![t]\!]_{\mathcal{M},s}$, is defined recursively by

$$[\![x]\!]_{\mathcal{M},s} = s(x), \qquad [\![c]\!]_{\mathcal{M},s} = I(c),$$

and

$$[\![f(t_1, \ldots, t_n)]\!]_{\mathcal{M},s} = I(f)\big([\![t_1]\!]_{\mathcal{M},s}, \ldots, [\![t_n]\!]_{\mathcal{M},s}\big).$$

Remark (Standard quantified statement).

$$\begin{aligned}
&[\![x]\!]_{\mathcal{M},s} = s(x),\\
&[\![c]\!]_{\mathcal{M},s} = I(c),\\
&[\![f(t_1, \ldots, t_n)]\!]_{\mathcal{M},s} = I(f)([\![t_1]\!]_{\mathcal{M},s}, \ldots, [\![t_n]\!]_{\mathcal{M},s}).
\end{aligned}$$

Remark (Definition predicate reading). TermEvaluation(M, s, t, a) means that the term t evaluates to the domain element a in M under s.

Remark (Negated quantified statement).

$$[\![t]\!]_{\mathcal{M},s} \neq a.$$

Remark (Negation predicate reading). $\neg$ TermEvaluation(M, s, t, a) means that a is not the value of t in M under s.

Remark (Failure modes). Term evaluation fails to have the proposed value when a variable, constant, or function-application clause computes a different domain element.

Remark (Failure mode decomposition).

$$\underbrace{s(x) \neq a}_{\text{variable case}} \quad \vee \quad \underbrace{I(c) \neq a}_{\text{constant case}} \quad \vee \quad \underbrace{I(f)(a_1, \ldots, a_n) \neq a}_{\text{function case}}.$$

Remark (Interpretation). Term evaluation is the semantic counterpart of term formation. It assigns a domain element to each grammatical term once a structure and assignment are fixed.

Remark (Dependencies).

- Term
- Structure
- Variable Assignment

2.4.2 Semantics: Satisfaction and Truth

Remark (Section toolkit: Satisfaction ledger). Satisfaction is the recursive semantic relation for formulas. Formulas with free variables are evaluated relative to an assignment. Sentences have no free variables, so their truth in a structure does not depend on which assignment is chosen.

Definition (Satisfaction)

Definition 2.4.6 (Satisfaction)**.** Let $\mathcal{M} = \langle D, I \rangle$ be a structure, let s be a variable assignment, and let φ be a well-formed formula. The relation $\mathcal{M}, s \models \varphi$ is defined recursively by the usual clauses: atomic formulas are evaluated by term values and interpreted predicate relations; equality compares term values; connectives follow the truth conditions for $\neg, \wedge, \vee, \to, \leftrightarrow$; and

$$\mathcal{M}, s \models \forall x\, \psi \iff \forall d \in D\ (\mathcal{M}, s[x \mapsto d] \models \psi),$$

$$\mathcal{M}, s \models \exists x\, \psi \iff \exists d \in D\ (\mathcal{M}, s[x \mapsto d] \models \psi).$$

Remark (Standard quantified statement).

$$\begin{aligned}
&\mathcal{M}, s \models P(t_1, \ldots, t_n) \iff (\llbracket t_1 \rrbracket_{\mathcal{M},s}, \ldots, \llbracket t_n \rrbracket_{\mathcal{M},s}) \in I(P),\\
&\mathcal{M}, s \models t_1 = t_2 \iff \llbracket t_1 \rrbracket_{\mathcal{M},s} = \llbracket t_2 \rrbracket_{\mathcal{M},s},\\
&\mathcal{M}, s \models \neg\psi \iff \mathcal{M}, s \not\models \psi,\\
&\mathcal{M}, s \models (\psi \wedge \chi) \iff \mathcal{M}, s \models \psi \text{ and } \mathcal{M}, s \models \chi,\\
&\mathcal{M}, s \models \forall x\, \psi \iff \forall d \in D\ (\mathcal{M}, s[x \mapsto d] \models \psi),\\
&\mathcal{M}, s \models \exists x\, \psi \iff \exists d \in D\ (\mathcal{M}, s[x \mapsto d] \models \psi).
\end{aligned}$$

Remark (Definition predicate reading). Satisfaction(M, s, φ) means $\mathcal{M}, s \models \varphi$.

Remark (Negated quantified statement).

$$\mathcal{M}, s \not\models \varphi.$$

Remark (Negation predicate reading). $\neg$ Satisfaction(M, s, φ) means that φ is not satisfied in M under s.

Remark (Failure modes). Satisfaction fails according to the outermost formation rule of the formula: an atomic tuple can miss the interpreted relation, equality can compare different term values, a connective can have the wrong truth pattern, or a quantifier can have a counterexample or lack a witness.

Remark (Failure mode decomposition).

$$\underbrace{(\llbracket t_i \rrbracket) \notin I(P)}_{\text{atomic failure}} \quad \underbrace{\llbracket t_1 \rrbracket \neq \llbracket t_2 \rrbracket}_{\text{equality failure}} \quad \underbrace{\exists d \in D\ (\mathcal{M}, s[x \mapsto d] \not\models \psi)}_{\text{universal failure}}.$$

Remark (Interpretation). Satisfaction is the bridge from grammar to semantic use. It is recursive because formulas are recursive. The assignment is essential exactly where free variables can affect the value of the formula.

Remark (Dependencies).

- Structure
- Variable Assignment
- Modified Assignment
- Term Evaluation
- Well-Formed Formula

Definition (Truth in a Structure)

Definition 2.4.7 (Truth in a Structure)**.** Let φ be a sentence and let $\mathcal{M}$ be a structure. The sentence φ is *true in* $\mathcal{M}$, written $\mathcal{M} \models \varphi$, exactly when $\mathcal{M}, s \models \varphi$ for every variable assignment s for $\mathcal{M}$.

Remark (Standard quantified statement).

$$\mathcal{M} \models \varphi \iff \forall s : \mathsf{Var}_{\mathcal{L}} \to D\ (\mathcal{M}, s \models \varphi), \qquad \varphi \text{ a sentence.}$$

Remark (Definition predicate reading). TruthInStructure(M, φ) means $\mathcal{M} \models \varphi$.

Remark (Negated quantified statement).

$$\mathcal{M} \not\models \varphi \iff \exists s : \mathsf{Var}_{\mathcal{L}} \to D\ (\mathcal{M}, s \not\models \varphi).$$

Remark (Negation predicate reading). $\neg$ TruthInStructure(M, φ) means that φ is false in M.

Remark (Failure modes). Truth in a structure fails when some assignment makes the sentence fail. For sentences, this is independent of assignment, but the quantified form keeps the definition uniform with satisfaction.

Remark (Failure mode decomposition).

$$\underbrace{\exists s}_{\text{assignment witness}} \qquad \underbrace{\mathcal{M}, s \not\models \varphi}_{\text{satisfaction failure}} .$$

Remark (Interpretation). Sentences have no free variables. Therefore the particular assignment does not carry mathematical information once the formula is a sentence; the notation $\mathcal{M} \models \varphi$ suppresses it.

Remark (Dependencies).

- Satisfaction

Definition (Validity)

Definition 2.4.8 (Validity)**.** A formula φ is *valid*, or *logically true*, exactly when $\mathcal{M}, s \models \varphi$ for every structure $\mathcal{M}$ for the language of φ and every variable assignment s for $\mathcal{M}$.

Remark (Standard quantified statement).

$$\models \varphi \iff \forall \mathcal{M}\ \forall s\ (\mathcal{M}, s \models \varphi).$$

Remark (Definition predicate reading). ValidFormula(φ) means that φ is satisfied in every structure under every assignment.

Remark (Negated quantified statement).

$$\not\models \varphi \iff \exists \mathcal{M}\ \exists s\ (\mathcal{M}, s \not\models \varphi).$$

Remark (Negation predicate reading). $\neg$ ValidFormula(φ) means that some structure and assignment make φ fail.

Remark (Failure modes). Validity fails by a counterexample structure together with an assignment.

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M}}_{\text{counterexample structure}} \qquad \underbrace{s}_{\text{counterexample assignment}} \qquad \underbrace{\mathcal{M}, s \not\models \varphi}_{\text{failure of satisfaction}} .$$

Remark (Interpretation). Validity is truth by logic alone. It does not depend on a special structure, special domain, or special interpretation of the non-logical symbols.

Remark (Dependencies).

- Satisfaction

Definition (Satisfiability)

Definition 2.4.9 (Satisfiability)**.** A formula φ is *satisfiable* exactly when there are a structure $\mathcal{M}$ and a variable assignment s such that $\mathcal{M}, s \models \varphi$.

Remark (Standard quantified statement)*.*

$$\varphi \text{ is satisfiable} \iff \exists \mathcal{M}\ \exists s\ (\mathcal{M}, s \models \varphi).$$

Remark (Definition predicate reading)*.* SatisfiableFormula(φ) means that φ is satisfied in at least one structure under at least one assignment.

Remark (Negated quantified statement)*.*

$$\varphi \text{ is unsatisfiable} \iff \forall \mathcal{M}\ \forall s\ (\mathcal{M}, s \not\models \varphi).$$

Remark (Negation predicate reading)*.* $\neg$ SatisfiableFormula(φ) means that no structure and assignment satisfy φ.

Remark (Failure modes)*.* Satisfiability fails only globally: every attempted structure and assignment fails to satisfy the formula.

Remark (Failure mode decomposition)*.*

$$\underbrace{\forall \mathcal{M}}_{\text{all structures}} \quad \underbrace{\forall s}_{\text{all assignments}} \quad \underbrace{\mathcal{M}, s \not\models \varphi}_{\text{no satisfying instance}}.$$

Remark (Interpretation)*.* Satisfiability asks whether a formula can be made true somewhere. This is the semantic counterpart of consistency only in the minimal Volume I sense; deeper proof-theoretic distinctions are deferred.

Remark (Dependencies)*.*

- Satisfaction

2.4.3 Semantics: Models, Theories, and Logical Consequence

Remark (Section toolkit: Minimal model notation)*.* Volume I uses theories only as sets of sentences so that the notation $\mathcal{M} \models T$ and $T \models \varphi$ can be read. The systematic study of theories and their classes of models belongs to Volume VIII Model Theory.

Definition (Model of a Sentence)

Definition 2.4.10 (Model of a Sentence)**.** Let φ be a sentence and let $\mathcal{M}$ be a structure for the language of φ. The structure $\mathcal{M}$ is a *model of* φ exactly when $\mathcal{M} \models \varphi$.

Remark (Standard quantified statement)*.*

$$\mathcal{M} \text{ is a model of } \varphi \iff \mathcal{M} \models \varphi.$$

Remark (Definition predicate reading). ModelOfSentence(M, φ) means that M is a model of the sentence φ.

Remark (Negated quantified statement).

$$\mathcal{M} \text{ is not a model of } \varphi \iff \mathcal{M} \not\models \varphi.$$

Remark (Negation predicate reading). $\neg$ ModelOfSentence(M, φ) means that φ is not true in M.

Remark (Failure modes). The only failure mode is semantic: the sentence is false in the proposed structure.

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M}}_{\text{proposed structure}} \qquad \underbrace{\mathcal{M} \not\models \varphi}_{\text{sentence false in the structure}}.$$

Remark (Interpretation). A model is not an extra kind of object. It is a structure viewed relative to a sentence that it makes true.

Remark (Dependencies).

- Truth in a Structure

Definition (Model of a Set of Sentences)

Definition 2.4.11 (Model of a Set of Sentences)**.** Let T be a set of sentences and let $\mathcal{M}$ be a structure for their common language. The structure $\mathcal{M}$ is a *model of* T, written $\mathcal{M} \models T$, exactly when $\mathcal{M} \models \theta$ for every $\theta \in T$.

Remark (Standard quantified statement).

$$\mathcal{M} \models T \iff \forall \theta \in T \; (\mathcal{M} \models \theta).$$

Remark (Definition predicate reading). ModelOfTheory(M, T) means that M is a model of every sentence in T.

Remark (Negated quantified statement).

$$\mathcal{M} \not\models T \iff \exists \theta \in T \; (\mathcal{M} \not\models \theta).$$

Remark (Negation predicate reading). $\neg$ ModelOfTheory(M, T) means that at least one sentence of T is false in M.

Remark (Failure modes). A proposed model of a set of sentences fails by a single sentence of the set that is false in the structure.

Remark (Failure mode decomposition).

$$\underbrace{\exists \theta \in T}_{\text{failed sentence}} \qquad \underbrace{\mathcal{M} \not\models \theta}_{\text{not true in the structure}}.$$

Remark (Interpretation). The notation $\mathcal{M} \models T$ abbreviates a universal check over the sentences in T. No additional theory of model classes is developed here.

Remark (Dependencies).

- Model of a Sentence

Definition (Theory)

Definition 2.4.12 (Theory)**.** In Volume I, a *theory* is a set of sentences in a fixed first-order language.

Remark (Standard quantified statement).

$$T \text{ is a theory} \iff \forall \theta \in T \ (\theta \text{ is a sentence of the fixed language}).$$

Remark (Definition predicate reading). Theory(T) means that T is a set of sentences in a fixed first-order language.

Remark (Negated quantified statement).

$$\exists \theta \in T \ (\theta \text{ is not a sentence of the fixed language}).$$

Remark (Negation predicate reading). $\neg$ Theory(T) means that T is not a set consisting only of sentences of the fixed language.

Remark (Failure modes). A proposed theory fails in this minimal sense when it contains something other than a sentence of the fixed language.

Remark (Failure mode decomposition).

$$\underbrace{\theta \in T}_{\text{member of the proposed theory}} \qquad \underbrace{\theta \text{ is not a sentence}}_{\text{bad member}}.$$

Remark (Interpretation). This is intentionally modest. Complete theories, elementary classes, categoricity, saturation, types, diagrams, and elementary extensions are Volume VIII topics. A theory may be called satisfiable in Volume I when it has at least one model.

Remark (Dependencies).

- Truth in a Structure

Definition (Logical Consequence)

Definition 2.4.13 (Logical Consequence)**.** Let T be a set of sentences and let φ be a sentence in the same language. The sentence φ is a *logical consequence* of T, written $T \models \varphi$, exactly when every model of T is a model of φ.

Remark (Standard quantified statement).

$$T \models \varphi \iff \forall \mathcal{M} \ (\mathcal{M} \models T \Rightarrow \mathcal{M} \models \varphi).$$

Remark (Definition predicate reading). LogicalConsequence(T, φ) means that every model of T is a model of φ.

Remark (Negated quantified statement).

$$T \not\models \varphi \iff \exists \mathcal{M}\ (\mathcal{M} \models T \wedge \mathcal{M} \not\models \varphi).$$

Remark (Negation predicate reading). $\neg$ LogicalConsequence(T, φ) means that some structure models T while failing φ.

Remark (Failure modes). Logical consequence fails by a countermodel: a structure satisfying all sentences of T but not satisfying φ.

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M} \models T}_{\text{premises true}} \qquad \underbrace{\mathcal{M} \not\models \varphi}_{\text{conclusion false}}.$$

Remark (Interpretation). Logical consequence is semantic entailment. The premises in T leave no room, across structures, for φ to fail.

Remark (Dependencies).

- Model of a Set of Sentences
- Theory

Definition (Logical Equivalence)

Definition 2.4.14 (Logical Equivalence)**.** Two sentences φ and ψ are *logically equivalent*, written $\varphi \equiv \psi$, exactly when each is a logical consequence of the other:

$$\varphi \models \psi \quad \text{and} \quad \psi \models \varphi.$$

Remark (Standard quantified statement).

$$\varphi \equiv \psi \iff \forall \mathcal{M}\ (\mathcal{M} \models \varphi \iff \mathcal{M} \models \psi).$$

Remark (Definition predicate reading). LogicalEquivalence(φ, ψ) means that φ and ψ have the same truth value in every structure.

Remark (Negated quantified statement).

$$\varphi \not\equiv \psi \iff \exists \mathcal{M}\ \big((\mathcal{M} \models \varphi \wedge \mathcal{M} \not\models \psi) \vee (\mathcal{M} \models \psi \wedge \mathcal{M} \not\models \varphi)\big).$$

Remark (Negation predicate reading). $\neg$ LogicalEquivalence(φ, ψ) means that some structure separates the two sentences.

Remark (Failure modes). Logical equivalence fails when one sentence is true and the other false in the same structure.

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M} \models \varphi \wedge \mathcal{M} \not\models \psi}_{\text{first separation}} \quad \vee \quad \underbrace{\mathcal{M} \models \psi \wedge \mathcal{M} \not\models \varphi}_{\text{second separation}}.$$

Remark (Interpretation). Logical equivalence is semantic interchangeability at the sentence level. It records sameness of truth across all structures in the relevant language.

Remark (Dependencies).

- Logical Consequence

2.4.4 Semantics: Open Formulas, Definable Relations, and Substitution Lemmas

Remark (Section toolkit: Predicate vocabulary). This section separates three uses that are often blurred. A predicate symbol is object-language syntax from the signature. An open formula is a formula with free variables. A definable relation is the set-theoretic relation in a structure determined by an open formula.

Definition (Open Formula)

Definition 2.4.15 (Open Formula)**.** An *open formula* is a well-formed formula with at least one free variable. When the free variables are among $x_1, \ldots, x_n$, it may be displayed as $\varphi(x_1, \ldots, x_n)$.

Remark (Standard quantified statement).

$$\varphi \text{ is open} \iff \varphi \in \mathsf{Form}_{\mathcal{L}} \text{ and } \mathsf{FV}(\varphi) \neq \varnothing.$$

Remark (Definition predicate reading). OpenFormula(φ) means that φ is a well-formed formula with at least one free variable.

Remark (Negated quantified statement).

$$\varphi \notin \mathsf{Form}_{\mathcal{L}} \quad \text{or} \quad \mathsf{FV}(\varphi) = \varnothing.$$

Remark (Negation predicate reading). $\neg$ OpenFormula(φ) means that φ is either not well formed or is a sentence.

Remark (Failure modes). An expression fails to be an open formula when it is not a well-formed formula or when all of its variables are bound.

Remark (Failure mode decomposition).

$$\underbrace{\varphi \notin \mathsf{Form}_{\mathcal{L}}}_{\text{not well formed}} \quad \vee \quad \underbrace{\mathsf{FV}(\varphi) = \varnothing}_{\text{sentence, not open}}.$$

Remark (Interpretation). Open formulas are syntactic predicates in the lightest Volume I sense. They do not by themselves name a set-theoretic relation until a structure is fixed.

Remark (Examples). $P(x)$, $x < y$, and $\exists z\, R(x, z)$ are open formulas when the displayed variables remain free as indicated.

Remark (Non-Examples). $\forall x\, P(x)$ is a sentence, not an open formula. A predicate symbol P alone is a symbol, not a formula.

Remark (Dependencies).

- Predicate Symbol
- Well-Formed Formula

Definition (Definable Relation)

Definition 2.4.16 (Definable Relation)**.** Let $\mathcal{M} = \langle D, I\rangle$ be a structure and let $\varphi(x_1, \ldots, x_n)$ be an open formula. The *relation defined by* φ in $\mathcal{M}$ is the set

$$R_\varphi^{\mathcal{M}} = \{(a_1, \ldots, a_n) \in D^n : \mathcal{M}, s[x_1 \mapsto a_1, \ldots, x_n \mapsto a_n] \models \varphi\}.$$

Remark (Standard quantified statement).

$$(a_1, \ldots, a_n) \in R_\varphi^{\mathcal{M}} \iff \mathcal{M}, s[x_1 \mapsto a_1, \ldots, x_n \mapsto a_n] \models \varphi.$$

Remark (Definition predicate reading). DefinableRelation(R, φ, M) means that R is the relation on the domain of M determined by φ in M.

Remark (Negated quantified statement).

$$R \neq R_\varphi^{\mathcal{M}}.$$

Remark (Negation predicate reading). $\neg$ DefinableRelation(R, φ, M) means that R is not the relation determined by φ in M.

Remark (Failure modes). A proposed definable relation fails when at least one tuple is included or excluded differently from the satisfaction condition imposed by the formula.

Remark (Failure mode decomposition).

$$\underbrace{(a_1, \ldots, a_n) \in R \wedge \mathcal{M}, s[\vec{x} \mapsto \vec{a}] \not\models \varphi}_{\text{extra tuple}} \quad \vee \quad \underbrace{(a_1, \ldots, a_n) \notin R \wedge \mathcal{M}, s[\vec{x} \mapsto \vec{a}] \models \varphi}_{\text{missing tuple}}.$$

Remark (Interpretation). A relation is a set-theoretic subset of D^n. A definable relation is one obtained from a formula after a structure supplies semantic meaning.

Remark (Dependencies).

- Open Formula
- Structure
- Satisfaction

Lemma (Substitution Lemma for Terms)

Lemma 2.4.17 (Substitution Lemma for Terms)**.** *Let $\mathcal{M}$ be a structure, let s be a variable assignment, let x be a variable, and let t and u be terms. Then*

$$[\![t[u/x]]\!]_{\mathcal{M},s} = [\![t]\!]_{\mathcal{M},\, s[x \mapsto [\![u]\!]_{\mathcal{M},s}]}.$$

Go to proof.

Remark (Standard quantified statement).

$$\forall t\, \forall u \quad [\![t[u/x]]\!]_{\mathcal{M},s} = [\![t]\!]_{\mathcal{M},\, s[x \mapsto [\![u]\!]_{\mathcal{M},s}]}.$$

Remark (Predicate reading). Term substitution commutes with semantic term evaluation in a fixed structure and assignment.

Remark (Interpretation). The lemma says that syntactically substituting a term before evaluation gives the same domain element as first evaluating the substituting term and then updating the assignment.

Remark (Dependencies).

- Substitution Notation
- Free for Substitution
- Term Evaluation

Lemma (Substitution Lemma for Formulas)

Lemma 2.4.18 (Substitution Lemma for Formulas)**.** *Let φ be a formula and let u be a term free for x in φ. For every structure $\mathcal{M}$ and assignment s,*

$$\mathcal{M}, s \models \varphi[u/x] \iff \mathcal{M}, s[x \mapsto [\![u]\!]_{\mathcal{M},s}] \models \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi\, \forall u \quad u \text{ free for } x \text{ in } \varphi \Rightarrow \big(\mathcal{M}, s \models \varphi[u/x] \iff \mathcal{M}, s[x \mapsto [\![u]\!]_{\mathcal{M},s}] \models \varphi\big).$$

Remark (Predicate reading). Formula substitution commutes with satisfaction when the substituted term is free for the variable being replaced.

Remark (Interpretation). This lemma connects the syntactic substitution operation from the syntax section with the semantic modified-assignment operation from the semantics section. The free-for condition prevents variable capture.

Remark (Dependencies).

- Substitution Notation

- Free for Substitution
- Satisfaction
- Substitution Lemma for Terms

2.5 Quantifiers

Toolkit: Quantifiers

Topic	Role
Basic syntax and semantics	universal, existential, bounded, sentence, open formula, scope
Quantifier laws	equivalence laws for negation, distribution, commutation, and renaming
Prenex normal form	syntactic normalization with quantifiers pulled forward
Logical strength	quantifier order, dependency, and non-equivalence examples

Remark (Exposition). Quantifiers extend propositional logic by allowing statements about all domain elements or about witnesses. The main new issue is dependency: later witnesses may depend on earlier universal choices.

Basic Syntax and Semantics

Remark (Section toolkit: Quantifier grammar). This section records the basic quantifier forms and the boundary between open formulas and sentences. Quantifiers bind variables. A formula with free variables requires an assignment for semantic evaluation; a sentence has no free variables and has a truth value in a structure.

Remark (Exposition). Sentences have no free variables and have truth values in structures. Open formulas may contain free variables and require assignments. Quantifiers turn some free occurrences into bound occurrences by placing them inside a scope.

Definition (Universal Formula)

Definition 2.5.1 (Universal Formula)**.** A *universal formula* is a well-formed formula of the form

$$\forall x\, \varphi,$$

where x is a variable and φ is a well-formed formula.

Remark (Standard quantified statement).

$$\forall x\, \varphi \in \mathsf{Form}_{\mathcal{L}} \iff x \in \mathsf{Var}_{\mathcal{L}} \text{ and } \varphi \in \mathsf{Form}_{\mathcal{L}}.$$

Remark (Definition predicate reading). UniversalFormula(Φ) means that Φ is a formula whose outermost constructor is a universal quantifier.

Remark (Negated quantified statement).

$$\Phi \text{ is not of the form } \forall x\, \varphi.$$

Remark (Interpretation). A universal formula asserts that its scope holds for every value assigned to the bound variable.

Remark (Examples). $\forall x\, P(x)$ and $\forall x\, \exists y\, R(x, y)$ are universal formulas.

Remark (Non-Examples). $P(x)$ and $\exists x\, P(x)$ are not universal formulas.

Remark (Dependencies).

- Variable

Definition (Existential Formula)

Definition 2.5.2 (Existential Formula)**.** An *existential formula* is a well-formed formula of the form

$$\exists x\, \varphi,$$

where x is a variable and φ is a well-formed formula.

Remark (Standard quantified statement).

$$\exists x\, \varphi \in \mathsf{Form}_{\mathcal{L}} \iff x \in \mathsf{Var}_{\mathcal{L}} \text{ and } \varphi \in \mathsf{Form}_{\mathcal{L}}.$$

Remark (Definition predicate reading). ExistentialFormula(Φ) means that Φ is a formula whose outermost constructor is an existential quantifier.

Remark (Negated quantified statement).

$$\Phi \text{ is not of the form } \exists x\, \varphi.$$

Remark (Interpretation). An existential formula asserts that its scope holds for at least one value of the bound variable.

Remark (Examples). $\exists x\, P(x)$ and $\exists x\, \forall y\, R(x, y)$ are existential formulas.

Remark (Non-Examples). $P(x)$ and $\forall x\, P(x)$ are not existential formulas.

Remark (Dependencies).

- Variable

Definition (Bounded Quantifier)

Definition 2.5.3 (Bounded Quantifier)**.** A *bounded quantifier* is an abbreviation that restricts a quantifier to a specified set or predicate:

$$\forall x \in A\, \varphi \quad :=\quad \forall x\, (x \in A \to \varphi),$$

$$\exists x \in A\, \varphi \quad :=\quad \exists x\, (x \in A \wedge \varphi).$$

Remark (Standard quantified statement).

$$\forall x \in A\, \varphi \equiv \forall x\, (x \in A \to \varphi), \qquad \exists x \in A\, \varphi \equiv \exists x\, (x \in A \wedge \varphi).$$

Remark (Definition predicate reading). BoundedQuantifier(Φ) means that Φ uses a restricted universal or existential quantifier.

Remark (Negated quantified statement).

$$\Phi \text{ is not an abbreviation of either displayed bounded form.}$$

Remark (Interpretation). The restricted universal uses implication, while the restricted existential uses conjunction. This preserves the usual behavior on empty sets.

Remark (Examples). $\forall x \in A\, P(x)$ and $\exists x \in A\, P(x)$ are bounded-quantifier formulas.

Remark (Non-Examples). $\forall x\, (x \in A \wedge P(x))$ is not the correct expansion of the bounded universal form.

Remark (Dependencies).

- Universal Formula
- Existential Formula

Definition (Sentence)

Definition 2.5.4 (Sentence)**.** A *sentence* is a well-formed formula with no free variables.

Remark (Standard quantified statement).

$$\varphi \text{ is a sentence} \iff \varphi \in \mathsf{Form}_{\mathcal{L}} \text{ and } \mathsf{FV}(\varphi) = \varnothing.$$

Remark (Definition predicate reading). Sentence(φ) means that φ is a well-formed formula with no free variables.

Remark (Negated quantified statement).

$$\varphi \notin \mathsf{Form}_{\mathcal{L}} \quad \text{or} \quad \mathsf{FV}(\varphi) \neq \varnothing.$$

Remark (Interpretation). Sentences have truth values in structures without reference to a particular assignment.

Remark (Examples). $\forall x\, P(x)$ and $\exists x\, \forall y\, R(x, y)$ are sentences when all variables shown are bound.

Remark (Non-Examples). $P(x)$ and $\exists y\, R(x, y)$ are not sentences when x remains free.

Remark (Dependencies).

- Well-Formed Formula

Definition (Open Formula)

Definition 2.5.5 (Open Formula)**.** An *open formula* is a well-formed formula with at least one free variable.

Remark (Standard quantified statement).

$$\varphi \text{ is open} \iff \varphi \in \mathsf{Form}_{\mathcal{L}} \text{ and } \mathsf{FV}(\varphi) \neq \varnothing.$$

Remark (Definition predicate reading). OpenFormula(φ) means that φ has at least one free variable.

Remark (Negated quantified statement).

$$\varphi \notin \mathsf{Form}_{\mathcal{L}} \quad \text{or} \quad \mathsf{FV}(\varphi) = \varnothing.$$

Remark (Interpretation). Open formulas may become true or false only after a structure and assignment are fixed.

Remark (Examples). $P(x)$ and $R(x, y)$ are open formulas.

Remark (Non-Examples). $\forall x\, P(x)$ is a sentence rather than an open formula.

Remark (Dependencies).

- Well-Formed Formula
- Open Formula

Definition (Quantifier Scope)

Definition 2.5.6 (Quantifier Scope)**.** In a formula $Qx\,\varphi$, where $Q \in \{\forall, \exists\}$, the *scope* of the quantifier Qx is the formula φ. Occurrences of x inside that scope are bound by the quantifier unless an inner quantifier for x intervenes.

Remark (Standard quantified statement).

$$Qx\,\varphi \text{ has scope } \varphi, \qquad Q \in \{\forall, \exists\}.$$

Remark (Definition predicate reading). Scope(x, φ) means that φ is the scope governed by the quantifier binding x.

Remark (Negated quantified statement).

$$\varphi \text{ is not the formula governed by the displayed quantifier on } x.$$

Remark (Interpretation). Scope determines which variable occurrences a quantifier controls. Scope errors are the main source of incorrect translations from English into predicate logic.

Remark (Examples). In $\forall x\,(P(x) \to Q(x))$, the scope is $(P(x) \to Q(x))$. In $(\forall x\, P(x)) \to Q(x)$, the final occurrence of x is outside the scope of the quantifier.

Remark (Dependencies).

- Universal Formula
- Existential Formula

Theorem (Negation of Bounded Quantifiers)

Theorem 2.5.7 (Negation of Bounded Quantifiers)**.** *Let A be a set and let φ be a formula. Then*

$$\neg(\forall x \in A\, \varphi) \equiv \exists x \in A\, \neg\varphi, \qquad \neg(\exists x \in A\, \varphi) \equiv \forall x \in A\, \neg\varphi.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\neg\forall x\, (x \in A \to \varphi) \equiv \exists x\, (x \in A \wedge \neg\varphi), \qquad \neg\exists x\, (x \in A \wedge \varphi) \equiv \forall x\, (x \in A \to \neg\varphi).$$

Remark (Predicate reading)*.* The theorem records the negation law for bounded universal and bounded existential formulas.

Remark (Interpretation)*.* The domain restriction is preserved under negation. What changes is the quantifier and the inner assertion.

Remark (Dependencies).

- Bounded Quantifier
- Quantifier Negation Laws

Quantifier Laws

Remark (Section toolkit: Quantifier algebra)*.* Quantifier algebra is the predicate-logic analogue of propositional equivalence laws. These laws justify rewriting quantified formulas while preserving logical equivalence.

Theorem (Quantifier Negation Laws)

Theorem 2.5.8 (Quantifier Negation Laws)**.** *For every formula φ,*

$$\neg\forall x\, \varphi \equiv \exists x\, \neg\varphi, \qquad \neg\exists x\, \varphi \equiv \forall x\, \neg\varphi.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\forall \varphi \in \mathsf{Form}_{\mathcal{L}} \quad \neg\forall x\, \varphi \equiv \exists x\, \neg\varphi \quad \text{and} \quad \neg\exists x\, \varphi \equiv \forall x\, \neg\varphi.$$

Remark (Predicate reading)*.* LogicalEquivalence$(\neg\forall x\, \varphi, \exists x\, \neg\varphi)$ and LogicalEquivalence$(\neg\exists x\, \varphi, \forall x\, \neg\varphi)$

Remark (Interpretation)*.* Negating a quantified statement flips the quantifier and pushes the negation into the scope.

Remark (Dependencies).

- Universal Formula
- Existential Formula

- Logical Equivalence

Theorem (Double Negation and Quantifiers)

Theorem 2.5.9 (Double Negation and Quantifiers)**.** *For every formula φ,*

$$\neg\neg\forall x\,\varphi \equiv \forall x\,\varphi, \qquad \neg\neg\exists x\,\varphi \equiv \exists x\,\varphi.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\forall \varphi \in \mathsf{Form}_{\mathcal{L}} \quad \neg\neg\forall x\,\varphi \equiv \forall x\,\varphi \quad \text{and} \quad \neg\neg\exists x\,\varphi \equiv \exists x\,\varphi.$$

Remark (Predicate reading)*.* LogicalEquivalence$(\neg\neg Qx\,\varphi, Qx\,\varphi)$ for $Q \in \{\forall, \exists\}$.

Remark (Interpretation)*.* Double negation does not change the quantifier form once the two negations are cancelled.

Remark (Dependencies)*.*

- Quantifier Negation Laws

Theorem (Same-Type Quantifier Commutation)

Theorem 2.5.10 (Same-Type Quantifier Commutation)**.** *Quantifiers of the same type commute:*

$$\forall x\,\forall y\,\varphi \equiv \forall y\,\forall x\,\varphi, \qquad \exists x\,\exists y\,\varphi \equiv \exists y\,\exists x\,\varphi.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\forall \varphi \in \mathsf{Form}_{\mathcal{L}} \quad \forall x\,\forall y\,\varphi \equiv \forall y\,\forall x\,\varphi \quad \text{and} \quad \exists x\,\exists y\,\varphi \equiv \exists y\,\exists x\,\varphi.$$

Remark (Predicate reading)*.* The theorem asserts logical equivalence after swapping adjacent quantifiers of the same type.

Remark (Interpretation)*.* Two universal choices can be checked in either order, and two existential witnesses can be selected in either order.

Remark (Dependencies)*.*

- Universal Formula
- Existential Formula
- Logical Equivalence

Theorem (Mixed Quantifiers Do Not Commute)

Theorem 2.5.11 (Mixed Quantifiers Do Not Commute)**.** *In general,*

$$\forall x\, \exists y\, \varphi \not\equiv \exists y\, \forall x\, \varphi.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\exists \varphi \in \mathsf{Form}_{\mathcal{L}} \quad \forall x\, \exists y\, \varphi \not\equiv \exists y\, \forall x\, \varphi.$$

Remark (Predicate reading)*.* $\neg\,\text{LogicalEquivalence}(\forall x\, \exists y\, \varphi, \exists y\, \forall x\, \varphi)$ in general.

Remark (Interpretation)*.* The first form permits the witness y to depend on x. The second form requires one uniform witness y that works for every x.

Remark (Dependencies)*.*

- Same-Type Quantifier Commutation

Theorem (Valid Quantifier Distribution)

Theorem 2.5.12 (Valid Quantifier Distribution)**.** *For all formulas* φ, ψ*,*

$$\forall x\, (\varphi \wedge \psi) \equiv (\forall x\, \varphi) \wedge (\forall x\, \psi),$$

$$\exists x\, (\varphi \vee \psi) \equiv (\exists x\, \varphi) \vee (\exists x\, \psi).$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\forall \varphi, \psi \in \mathsf{Form}_{\mathcal{L}} \quad \forall x\, (\varphi \wedge \psi) \equiv (\forall x\, \varphi) \wedge (\forall x\, \psi)$$

and

$$\exists x\, (\varphi \vee \psi) \equiv (\exists x\, \varphi) \vee (\exists x\, \psi).$$

Remark (Predicate reading)*.* The displayed universal-conjunction and existential-disjunction distributions are logical equivalences.

Remark (Interpretation)*.* Universal quantification distributes over conjunction because both conjuncts must hold for every value. Existential quantification distributes over disjunction because a witness for either disjunct witnesses the disjunction.

Remark (Dependencies)*.*

- Logical Equivalence
- Universal Formula
- Existential Formula

Theorem (Invalid Quantifier Distribution)

Theorem 2.5.13 (Invalid Quantifier Distribution)**.** *In general,*

$$\forall x\,(\varphi \lor \psi) \not\equiv (\forall x\,\varphi) \lor (\forall x\,\psi),$$

$$\exists x\,(\varphi \land \psi) \not\equiv (\exists x\,\varphi) \land (\exists x\,\psi).$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\exists \varphi, \psi \in \mathsf{Form}_{\mathcal{L}} \quad \forall x\,(\varphi \lor \psi) \not\equiv (\forall x\,\varphi) \lor (\forall x\,\psi),$$

$$\exists \varphi, \psi \in \mathsf{Form}_{\mathcal{L}} \quad \exists x\,(\varphi \land \psi) \not\equiv (\exists x\,\varphi) \land (\exists x\,\psi).$$

Remark (Predicate reading)*.* The displayed distribution patterns are not logical equivalences in general.

Remark (Interpretation)*.* The invalid laws fail because witnesses or counterexamples may vary between the two subformulas.

Remark (Dependencies)*.*

- Valid Quantifier Distribution

Theorem (Vacuous Quantification)

Theorem 2.5.14 (Vacuous Quantification)**.** *If x does not occur free in φ, then*

$$\forall x\,\varphi \equiv \varphi, \qquad \exists x\,\varphi \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement)*.*

$$x \notin \mathsf{FV}(\varphi) \Rightarrow (\forall x\,\varphi \equiv \varphi) \land (\exists x\,\varphi \equiv \varphi).$$

Remark (Predicate reading)*.* Vacuous quantification preserves logical equivalence when the quantified variable is not free in the scope.

Remark (Interpretation)*.* A quantifier that binds no occurrence has no semantic effect.

Remark (Dependencies)*.*

- Quantifier Scope

Theorem (Renaming Bound Variables)

Theorem 2.5.15 (Renaming Bound Variables)**.** *If y is free for x in φ and no capture is introduced, then*

$$\forall x\,\varphi \equiv \forall y\,\varphi[y/x], \qquad \exists x\,\varphi \equiv \exists y\,\varphi[y/x].$$

Go to proof.

Remark (Standard quantified statement).

$$y \text{ free for } x \text{ in } \varphi \Rightarrow (\forall x\, \varphi \equiv \forall y\, \varphi[y/x]) \wedge (\exists x\, \varphi \equiv \exists y\, \varphi[y/x]).$$

Remark (Predicate reading). Renaming bound variables preserves logical equivalence when the renaming is capture-avoiding.

Remark (Interpretation). Bound variable names are placeholders. Renaming them changes notation, not meaning, provided no free variable becomes captured.

Remark (Dependencies).

- Free for Substitution
- Substitution Notation

Prenex Normal Form

Remark (Section toolkit: Prenex normalization). Prenex normal form is the predicate-logic analogue of the normal-form procedures from propositional logic. It is a syntactic normalization procedure, not a semantic change: each step preserves logical equivalence.

Definition (Quantifier Prefix)

Definition 2.5.16 (Quantifier Prefix)**.** A *quantifier prefix* is a finite string

$$Q_1 x_1\, Q_2 x_2 \cdots Q_n x_n,$$

where each $Q_i \in \{\forall, \exists\}$ and each x_i is a variable.

Remark (Standard quantified statement).

$$\Pi = Q_1 x_1 \cdots Q_n x_n \iff \forall i \leq n\ (Q_i \in \{\forall, \exists\} \text{ and } x_i \in \mathsf{Var}_{\mathcal{L}}).$$

Remark (Definition predicate reading). QuantifierPrefix(Π) means that Π is a finite string of quantifier-variable pairs.

Remark (Interpretation). A prefix records all quantifier choices before the quantifier-free part of a prenex formula is read.

Remark (Dependencies).

- Universal Formula
- Existential Formula

Definition (Matrix)

Definition 2.5.17 (Matrix)**.** In a formula $Q_1 x_1 \cdots Q_n x_n\, \psi$, the *matrix* is the trailing formula ψ. For prenex normal form, the matrix is required to be quantifier-free.

Remark (Standard quantified statement).

$$Q_1 x_1 \cdots Q_n x_n \, \psi \text{ has matrix } \psi, \qquad \psi \text{ quantifier-free in prenex normal form.}$$

Remark (Definition predicate reading). Matrix(ψ, Φ) means that ψ is the matrix of the formula Φ.

Remark (Interpretation). The matrix is the part of a prenex formula where the propositional connectives and atomic formulas remain after all quantifiers have been pulled forward.

Remark (Dependencies).

- Well-Formed Formula

Definition (Prenex Formula)

Definition 2.5.18 (Prenex Formula)**.** A *prenex formula* is a formula of the form

$$Q_1 x_1 \, Q_2 x_2 \cdots Q_n x_n \, \psi,$$

where $Q_1 x_1 \cdots Q_n x_n$ is a quantifier prefix and ψ is quantifier-free.

Remark (Standard quantified statement).

$$\Phi \text{ is prenex} \iff \Phi = Q_1 x_1 \cdots Q_n x_n \, \psi \text{ with } \psi \text{ quantifier-free.}$$

Remark (Definition predicate reading). PrenexFormula(Φ) means that all quantifiers of Φ occur in an initial prefix.

Remark (Negated quantified statement).

$$\Phi \text{ is not of the form } Q_1 x_1 \cdots Q_n x_n \, \psi \text{ with } \psi \text{ quantifier-free.}$$

Remark (Interpretation). Prenex formulas isolate quantifier order from the quantifier-free matrix.

Remark (Examples). $\forall x \, \exists y \, (P(x) \to R(x, y))$ is prenex.

Remark (Non-Examples). $\forall x \, (P(x) \to \exists y \, R(x, y))$ is not prenex because a quantifier remains inside the matrix.

Remark (Dependencies).

- Quantifier Prefix
- Matrix

Definition (Prenex Normal Form)

Definition 2.5.19 (Prenex Normal Form)**.** A formula ψ is a *prenex normal form* of a formula φ exactly when ψ is prenex and $\varphi \equiv \psi$.

Remark (Standard quantified statement).

$$\psi \text{ is a prenex normal form of } \varphi \iff \psi \text{ is prenex and } \varphi \equiv \psi.$$

Remark (Definition predicate reading). PrenexNormalForm(ψ, φ) means that ψ is a prenex formula logically equivalent to φ.

Remark (Interpretation). Prenex normal form changes the placement of quantifiers but not the meaning of the formula.

Remark (Exposition). Prenex normal form is not unique. Different choices of bound-variable names and different orders of valid pulling steps can produce different but logically equivalent prenex formulas.

Remark (Exposition). A prenex conversion proceeds in stages: eliminate implications and biconditionals, push negations inward, rename bound variables as needed, pull quantifiers forward, and leave a quantifier-free matrix.

Remark (Examples). Starting with

$$\neg\big(\forall x\, P(x) \to \exists y\, Q(y)\big),$$

eliminate the implication, push the negation inward, and pull the quantifiers forward to obtain

$$\forall x\, \forall y\, (P(x) \wedge \neg Q(y)).$$

Remark (Dependencies).

- Prenex Formula
- Logical Equivalence

Theorem (Existence of Prenex Normal Form)

Theorem 2.5.20 (Existence of Prenex Normal Form)**.** *Every first-order formula is logically equivalent to some formula in prenex normal form. Go to proof.*

Remark (Standard quantified statement).

$$\forall \varphi \in \mathsf{Form}_{\mathcal{L}}\ \exists \psi \in \mathsf{Form}_{\mathcal{L}}\ (\psi \text{ is prenex} \wedge \varphi \equiv \psi).$$

Remark (Predicate reading). $\forall \varphi\, \exists \psi$ PrenexNormalForm(ψ, φ).

Remark (Interpretation). Every formula can be converted into an equivalent one with all quantifiers at the front and a quantifier-free matrix at the end.

Remark (Exposition). The proof proceeds by structural rewriting: remove implication and biconditional connectives, push negations inward, rename bound variables to avoid capture, and then pull quantifiers forward using equivalences whose side conditions have been made true by renaming.

Remark (Dependencies).

- Prenex Normal Form
- Quantifier Negation Laws
- Renaming Bound Variables
- Valid Quantifier Distribution

Logical Strength and Quantifier Order

Remark (Section toolkit: Strength and order). Quantifier order is one of the main new phenomena in predicate logic. In propositional logic, changing the order of independent connectives often keeps truth conditions recognizable. In predicate logic, moving $\forall$ and $\exists$ can change whether witnesses may depend on earlier choices.

Definition (Stronger Formula)

Definition 2.5.21 (Stronger Formula)**.** A sentence A is *stronger than* a sentence B exactly when

$$A \models B \quad \text{and} \quad B \not\models A.$$

Remark (Standard quantified statement).

$$A \text{ is stronger than } B \iff (\forall \mathcal{M}\ (\mathcal{M} \models A \Rightarrow \mathcal{M} \models B)) \wedge (\exists \mathcal{N}\ (\mathcal{N} \models B \wedge \mathcal{N} \not\models A)).$$

Remark (Definition predicate reading). StrongerFormula(A, B) means that A entails B, but B does not entail A.

Remark (Negated quantified statement).

$$A \not\models B \quad \text{or} \quad B \models A.$$

Remark (Interpretation). The stronger sentence has fewer possible structures in which it is true. It rules out more cases.

Remark (Examples). $\forall x\, P(x)$ is stronger than $\exists x\, P(x)$ over nonempty domains.

Remark (Dependencies).

- Logical Consequence

Definition (Weaker Formula)

Definition 2.5.22 (Weaker Formula)**.** A sentence B is *weaker than* a sentence A exactly when A is stronger than B.

Remark (Standard quantified statement).

$$B \text{ is weaker than } A \iff A \models B \text{ and } B \not\models A.$$

Remark (Definition predicate reading). WeakerFormula(B, A) means that B is entailed by A but does not entail A.

Remark (Interpretation). The weaker sentence permits more structures. It follows from the stronger sentence but carries less information.

Remark (Dependencies).

- Stronger Formula

Definition (Quantifier Order)

Definition 2.5.23 (Quantifier Order)**.** The *quantifier order* of a prenex formula is the ordered list of quantifiers in its prefix. A change of order is significant when it changes which witnesses may depend on earlier variables.

Remark (Standard quantified statement).

$$Q_1 x_1 \cdots Q_n x_n \, \psi \text{ has quantifier order } (Q_1, \ldots, Q_n).$$

Remark (Definition predicate reading). QuantifierOrder$(\Phi, (Q_1, \ldots, Q_n))$ records the order of the quantifiers in the prefix of Φ.

Remark (Interpretation). Quantifier order controls dependency. In $\forall x \, \exists y$, the witness y may depend on x. In $\exists y \, \forall x$, the same y must work for all x.

Remark (Dependencies).

- Quantifier Prefix
- Prenex Formula

Theorem (Universal Implies Existential)

Theorem 2.5.24 (Universal Implies Existential)**.** *Over nonempty domains,*

$$\forall x \, \varphi(x) \models \exists x \, \varphi(x).$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \mathcal{M} \, \big(\mathcal{M} \models \forall x \, \varphi(x) \Rightarrow \mathcal{M} \models \exists x \, \varphi(x) \big).$$

Remark (Predicate reading). LogicalConsequence$(\{\forall x \, \varphi(x)\}, \exists x \, \varphi(x))$.

Remark (Interpretation). If every domain element satisfies φ, then at least one does because first-order structures have nonempty domains.

Remark (Dependencies).

- Structure

- Logical Consequence

Theorem (Existential Does Not Imply Universal)

Theorem 2.5.25 (Existential Does Not Imply Universal)**.** *In general,*

$$\exists x\, \varphi(x) \not\models \forall x\, \varphi(x).$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\exists \mathcal{M}\, \big(\mathcal{M} \models \exists x\, \varphi(x) \wedge \mathcal{M} \not\models \forall x\, \varphi(x)\big).$$

Remark (Predicate reading)*.* $\neg\,\text{LogicalConsequence}(\{\exists x\, \varphi(x)\}, \forall x\, \varphi(x))$.

Remark (Interpretation)*.* One witness does not make a universal claim true.

Remark (Exposition)*.* In the real numbers, $\exists x\, (x = 0)$ is true, but $\forall x\, (x = 0)$ is false.

Remark (Dependencies).

- Universal Implies Existential

Theorem (Uniform Witness Implies Dependent Witness)

Theorem 2.5.26 (Uniform Witness Implies Dependent Witness)**.** *For every formula* $\varphi(x, y)$*,*

$$\exists y\, \forall x\, \varphi(x, y) \models \forall x\, \exists y\, \varphi(x, y).$$

Go to proof.

Remark (Standard quantified statement)*.*

$$\forall \mathcal{M}\, \big(\mathcal{M} \models \exists y\, \forall x\, \varphi(x, y) \Rightarrow \mathcal{M} \models \forall x\, \exists y\, \varphi(x, y)\big).$$

Remark (Predicate reading)*.* $\text{LogicalConsequence}(\{\exists y\, \forall x\, \varphi(x, y)\}, \forall x\, \exists y\, \varphi(x, y))$.

Remark (Interpretation)*.* A single witness y that works for every x can be reused as the witness required for each separate x.

Remark (Dependencies).

- Quantifier Order
- Logical Consequence

Theorem (Dependent Witness Need Not Be Uniform)

Theorem 2.5.27 (Dependent Witness Need Not Be Uniform)**.** *In general,*

$$\forall x\, \exists y\, \varphi(x, y) \not\models \exists y\, \forall x\, \varphi(x, y).$$

Go to proof.

Remark (Standard quantified statement).

$$\exists \mathcal{M}\ \big(\mathcal{M} \models \forall x\, \exists y\, \varphi(x,y) \wedge \mathcal{M} \not\models \exists y\, \forall x\, \varphi(x,y)\big).$$

Remark (Predicate reading). $\neg\,\mathrm{LogicalConsequence}(\{\forall x\, \exists y\, \varphi(x,y)\}, \exists y\, \forall x\, \varphi(x,y))$.

Remark (Interpretation). The dependent-witness statement allows y to vary with x. The uniform statement requires one y to handle all x at once.

Remark (Exposition). Over $\mathbb{R}$, let $\varphi(x,y)$ be $y > x$. Then $\forall x\, \exists y\, (y > x)$ is true, since $y = x+1$ works for each x. But $\exists y\, \forall x\, (y > x)$ is false, since no real number is larger than every real number.

Remark (Exposition). Quantifier order matters because it records dependency. The phrase "for every x, there exists y" permits a different y after x is known. The phrase "there exists y such that for every x" chooses y before seeing x. This is a genuine expressive jump beyond propositional logic.

Remark (Dependencies).

- Uniform Witness Implies Dependent Witness

2.6 Proof Theory

Quantifier Inference Rules — Quick Reference

Rule	Schema	Key restriction
UI	$\forall x\, \varphi \Rightarrow \varphi[t/x]$	t free for x
UG	$\varphi \Rightarrow \forall x\, \varphi$	x not free in assumptions
EI	$\exists x\, \varphi \Rightarrow \varphi[c/x]$	c fresh witness
EG	$\varphi[t/x] \Rightarrow \exists x\, \varphi$	none

2.6.1 Proof Theory: Quantifier Inference Rules

Definition (Universal Instantiation — UI)

Definition 2.6.1 (Universal Instantiation — UI)**.** From a universally quantified statement, infer any instance obtained by substituting a term for the bound variable:

$$\forall x\, \varphi \ \Rightarrow\ \varphi[t/x].$$

Remark (Dependencies).

- Universal Quantifier
- Substitution Notation
- Free for Substitution

Remark (Side condition). The term t must be free for x in φ. If t contains a variable that would become bound after substitution, the inference is invalid.

Definition (Universal Generalization — UG)

Definition 2.6.2 (Universal Generalization — UG)**.** From a formula, infer a universally quantified statement:

$$\varphi \;\Rightarrow\; \forall x\, \varphi.$$

Remark (Dependencies).

- Universal Quantifier
- Formula
- Bound and Free Occurrences

Remark (Restriction on UG). The variable x must not occur free in any undischarged assumption on which φ depends. This restriction ensures that x is truly arbitrary — not tied to a specific hypothesis about x.

Remark (Arbitrary element argument). To prove $\forall x\, \varphi(x)$, fix an arbitrary element x (introducing no special assumptions about x), prove $\varphi(x)$, and then apply UG. The variable x must remain unconstrained throughout.

Definition (Existential Instantiation — EI)

Definition 2.6.3 (Existential Instantiation — EI)**.** From an existentially quantified statement, infer an instance with a fresh constant (witness):

$$\exists x\, \varphi \;\Rightarrow\; \varphi[c/x].$$

Remark (Dependencies).

- Existential Quantifier
- Constant Symbol
- Substitution Notation
- Free for Substitution

Remark (Witness discipline for EI). The constant c must be fresh: it must not appear in φ, in any undischarged assumption, or in the final conclusion of the proof. The constant represents an arbitrary but fixed witness to the existential claim, not a specific known object.

Remark (Common error). Using a witness constant introduced by EI outside its allowed scope, or choosing c before establishing the existential premise, invalidates the proof.

Definition (Existential Generalization — EG)

Definition 2.6.4 (Existential Generalization — EG)**.** From a formula containing a term, infer an existential statement:

$$\varphi[t/x] \;\Rightarrow\; \exists x\, \varphi.$$

Remark (Dependencies).

- Existential Quantifier
- Term
- Substitution Notation

Remark (No side conditions). EG has no side conditions: any term t may be replaced by an existentially quantified variable. To prove $\exists x\, \varphi(x)$, explicitly exhibit a term t such that $\varphi(t)$ holds, then apply EG.

Remark (Counterexample argument). To refute $\forall x\, \varphi(x)$: produce a single term t such that $\neg\varphi(t)$ holds. To refute $\exists x\, \varphi(x)$: show that $\varphi(t)$ fails for every term t in the domain.

Quantifier Proof Strategies — Quick Reference

Goal shape	Strategy	Key rule
Prove $\forall x\, \varphi(x)$	Fix an arbitrary x; prove $\varphi(x)$ for that x	UG
Prove $\exists x\, \varphi(x)$	Produce a specific witness t; verify $\varphi(t)$	EG
Use $\forall x\, \varphi(x)$	Instantiate at any term t to get $\varphi(t)$	UI
Use $\exists x\, \varphi(x)$	Introduce a fresh name c for the witness	EI
Negate $\forall x\, \varphi$	Becomes $\exists x\, \neg\varphi$	Neg. law
Negate $\exists x\, \varphi$	Becomes $\forall x\, \neg\varphi$	Neg. law
Prove $\forall x\, \exists y\, \varphi(x, y)$	Fix x; then construct y depending on x	UG then EG
Prove $\exists y\, \forall x\, \varphi(x, y)$	Find y first; then verify for all x	EG then UG

2.6.2 Proof Strategies for Quantified Statements

The inference rules UI, UG, EI, and EG were defined in the previous section. That section answered the question: what does each rule license? This section answers a different question: when the goal of a proof has a particular quantifier structure, which rule should be used, and what does the proof skeleton look like? The two questions are distinct, and both must be understood clearly before reading analysis proofs.

The single most important organizing principle is this: *the shape of the goal determines the proof structure, and the shape of the hypothesis determines how to use it.* A $\forall$ goal calls for a different move than a $\exists$ goal; a $\forall$ hypothesis is used differently from a $\exists$ hypothesis. The four combinations are the four strategies below.

Strategy 1: Proving a Universal $\forall x\, \varphi(x)$

To prove $\forall x\, \varphi(x)$: introduce a fresh variable x_0 representing an *arbitrary, unspecified* element of the domain. Prove $\varphi(x_0)$ without making any assumption about x_0 beyond its being in the domain. Then apply Universal Generalization (UG) to conclude $\forall x\, \varphi(x)$.

> **Let** x_0 be an arbitrary element of the domain.
> [**prove** $\varphi(x_0)$ **using only the hypotheses, not any special property of** x_0]
> **Therefore:** $\forall x\, \varphi(x)$. (by UG)

The critical constraint is that x_0 must be *arbitrary*: no hypothesis in the proof may impose a condition on x_0 that is not universally true. Violating this constraint is the UG fallacy ("illicit generalization").

Remark (Why "let x be arbitrary" works). Fixing an arbitrary element x_0 is the formal counterpart of the English phrase "let x be any element." The word "arbitrary" means: the proof uses no information about x_0 that is specific to x_0 — only information that applies to every element of the domain. If the argument goes through for such an unspecified x_0, it goes through for every x, so the universal follows.

Remark (The UG side condition in practice). UG requires that x_0 not appear free in any undischarged assumption. In practice: do not use any hypothesis of the form "x_0 has property P" unless P holds of every domain element. If a hypothesis says "$x_0 > 0$," then x_0 is not arbitrary, and UG to $\forall x$ is invalid.

Strategy 2: Proving an Existential $\exists x\, \varphi(x)$

To prove $\exists x\, \varphi(x)$: exhibit a specific term t and verify that $\varphi(t)$ holds. Then apply Existential Generalization (EG) to conclude $\exists x\, \varphi(x)$.

> **Claim:** $\exists x\, \varphi(x)$.
> **Take:** $t :=$ [specific term].
> **Verify:** $\varphi(t)$ holds. [proof that t works]
> **Therefore:** $\exists x\, \varphi(x)$. (by EG)

The entire burden of an existential proof is finding the right t. Once t is identified and verified, the logic is trivial: one application of EG.

Remark (Where the difficulty lies). In existential proofs, the logical structure is simple but the mathematical insight is often deep. Finding the witness requires understanding the problem; verifying the witness is usually straightforward. This is why analysis proofs often say "take $\delta = \min(\varepsilon/2, 1)$" without explaining where that choice came from — the work happened before the formal proof was written.

Strategy 3: Using a Universal $\forall x\,\varphi(x)$

If a hypothesis or established fact has the form $\forall x\,\varphi(x)$, one can instantiate it at *any* specific term t to obtain $\varphi(t)$, by Universal Instantiation (UI). There are no side conditions beyond t being free for x in φ.

> **Available:** $\forall x\,\varphi(x)$.
> **Choose any term t.**
> **Obtain:** $\varphi(t)$. (by UI)

UI can be applied repeatedly to the same universal statement with different choices of t.

Strategy 4: Using an Existential $\exists x\,\varphi(x)$

If a hypothesis has the form $\exists x\,\varphi(x)$, introduce a *fresh constant* c to name the (unknown, unspecified) witness, obtaining $\varphi(c)$ by Existential Instantiation (EI). The constant c must be new: it must not appear anywhere else in the proof.

> **Available:** $\exists x\,\varphi(x)$.
> **Let c be a witness.** (fresh constant, not used elsewhere)
> **Obtain:** $\varphi(c)$. (by EI)

The freshness requirement is not a technicality. Using a constant that already appears in the proof — say, a constant a introduced under a different assumption — would illegally identify the witness with that specific element. The witness c is only known to satisfy $\varphi(c)$; nothing else about it is known, and it cannot be equated with anything specific.

Remark (The scope of a witness constant). Once c is introduced by EI, it names the witness for the duration of the argument. Conclusions derived using c depend on the existence of such a witness; they are valid (they do not depend on which specific element c actually names). However, c cannot be exported outside the scope in which the existential was instantiated, and no universal generalization may be applied to c.

The Central Asymmetry: Universal vs. Existential Goals

The following comparison is the most important thing to understand about quantifier proof structure. Universal and existential goals require fundamentally different strategies, and confusing them is one of the most common serious errors in mathematical writing.

Universal vs. Existential Goals: The Fundamental Difference

	$\forall x\,\varphi(x)$	$\exists x\,\varphi(x)$
To prove:	Fix *arbitrary* x; prove $\varphi(x)$	*Choose* specific t; verify $\varphi(t)$
Who chooses x?	The adversary (you cannot choose)	You (the prover)
Main rule:	UG	EG
Burden:	Works for every x without assumptions	A single witness suffices
Error:	Assuming something special about x	Failing to exhibit a concrete t

Remark (Who controls the variable). For a universal goal, the variable x is chosen by the "opponent" of the proof: the proof must work no matter what x is. The prover has no control over x; hence the term "arbitrary." For an existential goal, the prover chooses the witness t; the power is entirely on the prover's side, but so is the obligation to produce something concrete and verifiable.

Nested Quantifiers: The $\forall\exists$ Pattern in Analysis

Every ε-δ proof in analysis is a proof of a statement with the pattern $\forall\varepsilon\,\exists\delta\,\cdots$. Understanding the proof structure of such statements is the direct payoff of this section.

Proof Skeleton: $\forall x\,\exists y\,\varphi(x,y)$

Step 1. **Fix arbitrary** x**.** (Opening move for the outer $\forall$; x is unspecified. This is Strategy 1.)

Step 2. **Define** y **in terms of** x**.** (The witness for $\exists y$ may, and typically does, depend on x. This is Strategy 2 — choosing a witness.)

Step 3. **Verify** $\varphi(x,y)$ using the specific y from Step 2 and the arbitrary x from Step 1.

Step 4. **Conclude:** $\exists y\,\varphi(x,y)$. (by EG)

Step 5. **Conclude:** $\forall x\,\exists y\,\varphi(x,y)$. (by UG, since x was arbitrary in Step 1)

The key insight: y is chosen *after* x is fixed, so y is allowed to depend on x.

Proof Skeleton: $\exists y\,\forall x\,\varphi(x,y)$

Step 1. **Define** y**.** (The witness for $\exists y$ must be chosen *before* x is introduced. This y must work for *every* x.)

Step 2. **Fix arbitrary** x**.** (Now x is introduced.)

Step 3. **Verify** $\varphi(x,y)$ for the specific y from Step 1 and arbitrary x from Step 2.

Step 4. **Conclude:** $\forall x\,\varphi(x,y)$. (by UG)

Step 5. **Conclude:** $\exists y\,\forall x\,\varphi(x,y)$. (by EG)

The key contrast: y is chosen *before* x, so y cannot depend on x. This is why $\exists y\,\forall x$ is a much stronger claim.

Remark (Why order determines strength). In the $\forall\exists$ skeleton, the witness y is constructed from x, so a different y is allowed for each x. In the $\exists\forall$ skeleton, a single y must work for all x simultaneously. The first pattern appears in continuity (δ may depend on ε and x); the second in uniform continuity (δ may depend on ε but not on x). The difference between continuity and uniform continuity is exactly the difference between these two proof skeletons.

A Complete Worked Example

The following proof traces every UI/UG/EI/EG step explicitly. In practice, proofs are written more concisely, but the logical structure is always this one.

Example (A worked multi-quantifier proof). **Claim:** If $\forall x\, \exists y\, P(x,y)$ and $\forall x\, \forall y\, (P(x,y) \to Q(x,y))$, then $\forall x\, \exists y\, Q(x,y)$.

Proof:

1. Assume $H_1 := \forall x\, \exists y\, P(x,y)$ and $H_2 := \forall x\, \forall y\, (P(x,y) \to Q(x,y))$. (hypotheses)
2. Let a be an arbitrary element of the domain. (introduce arbitrary variable for $\forall x$ in the goal)
3. From H_1, instantiate at a: $\exists y\, P(a,y)$. (by UI applied to H_1 with $t = a$)
4. Let b be a witness: $P(a,b)$. (by EI applied to Step 3; b is fresh)
5. From H_2, instantiate at a: $\forall y\, (P(a,y) \to Q(a,y))$. (by UI applied to H_2 with $t = a$)
6. From Step 5, instantiate at b: $P(a,b) \to Q(a,b)$. (by UI applied to Step 5 with $t = b$)
7. From Step 4 ($P(a,b)$) and Step 6 ($P(a,b) \to Q(a,b)$): obtain $Q(a,b)$. (by Modus Ponens)
8. From Step 7: $\exists y\, Q(a,y)$. (by EG: b witnesses the existential)
9. Since a was arbitrary: $\forall x\, \exists y\, Q(x,y)$. (by UG: a was not constrained)

Annotation: The proof follows the $\forall\exists$ skeleton exactly. Step 2 fixes the arbitrary x. Steps 3–4 use the first hypothesis to obtain a witness b depending on a. Steps 5–7 apply the second hypothesis to get the desired conclusion about b. Steps 8–9 wrap everything back up with EG and UG.

Remark (Reading analysis proofs). A proof that begins "Let $\varepsilon > 0$" is executing Step 2 of the $\forall\exists$ skeleton with domain $\mathbb{R}_{>0}$. A sentence like "take $\delta = \varepsilon/2$" is executing Step 2 of the inner existential proof: it is the witness for $\exists\delta$. Everything between "let $\varepsilon > 0$" and "take $\delta = \ldots$" is scaffolding to find the right witness. Everything after "take $\delta = \ldots$" is the verification that the witness works. Once this structure is seen, every ε-δ proof becomes readable by template.

Equality Rules — Quick Reference

Rule	Schema	Status
Reflexivity (EqI)	$t = t$	Primitive rule
Equality Elim (EqE)	$t_1 = t_2,\ \varphi[t_1/x] \Rightarrow \varphi[t_2/x]$	Primitive rule
Symmetry	$t_1 = t_2 \Rightarrow t_2 = t_1$	Derived
Transitivity	$t_1 = t_2,\ t_2 = t_3 \Rightarrow t_1 = t_3$	Derived
Term substitution	$f(\ldots, t_1, \ldots) = f(\ldots, t_2, \ldots)$	Derived
Predicate substitution	$P(\ldots, t_1, \ldots) \Leftrightarrow P(\ldots, t_2, \ldots)$	Derived

2.6.3 Proof Theory: Equality Rules

Definition (Equality Introduction — Reflexivity)

Definition 2.6.5 (Equality Introduction — Reflexivity)**.** For any term t, one may infer $t = t$.

Remark (Dependencies).

- Term

Remark (English reading). Reflexivity requires no premises and holds for all terms under all interpretations. It is the foundational rule that allows equality reasoning to get off the ground.

Definition (Equality Elimination — Substitution of Equals)

Definition 2.6.6 (Equality Elimination — Substitution of Equals)**.** From $t_1 = t_2$ and $\varphi[t_1/x]$, one may infer $\varphi[t_2/x]$.

Remark (Dependencies).

- Term
- Formula
- Substitution Notation
- Free for Substitution

Remark (English reading). Equality elimination permits replacing a term by an equal term in any formula position, provided the substitution is capture-avoiding. This formalizes the intuition that equal things can be substituted for each other.

Theorem (Symmetry of Equality)

Theorem 2.6.7 (Symmetry of Equality)**.** *From $t_1 = t_2$, one may infer $t_2 = t_1$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2 : \ t_1 = t_2 \to t_2 = t_1$.

Remark (Dependencies).

- Equality Introduction
- Equality Elimination

Theorem (Transitivity of Equality)

Theorem 2.6.8 (Transitivity of Equality)**.** *From $t_1 = t_2$ and $t_2 = t_3$, one may infer $t_1 = t_3$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2, t_3 : \ t_1 = t_2 \wedge t_2 = t_3 \to t_1 = t_3$.

Remark (Dependencies).

- Equality Introduction
- Equality Elimination

Theorem (Term Substitution under Equality)

Theorem 2.6.9 (Term Substitution under Equality)**.** *If $t_1 = t_2$, then for any function symbol f, $f(\dots, t_1, \dots) = f(\dots, t_2, \dots)$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2 : \ t_1 = t_2 \to \forall$ function symbol f: $f(\dots, t_1, \dots) = f(\dots, t_2, \dots)$.

Remark (Dependencies).

- Equality Elimination

Theorem (Predicate Substitution under Equality)

Theorem 2.6.10 (Predicate Substitution under Equality)**.** *If $t_1 = t_2$ and P is an n-ary predicate symbol, then $P(\dots, t_1, \dots) \Leftrightarrow P(\dots, t_2, \dots)$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2 : \ t_1 = t_2 \to \forall$ predicate symbol P: $P(\dots, t_1, \dots) \leftrightarrow P(\dots, t_2, \dots)$.

Remark (Dependencies).

- Equality Elimination

Remark (Summary). These derived rules collectively express that functions and predicates *respect equality*: equal inputs produce equal outputs (for functions) or equivalent propositions (for predicates). Together they amount to the principle of Leibniz substitutivity.

Remark (Common error). When substituting equals for equals, the substitution must be made consistently. Replacing a term by an equal in one occurrence but not another in the same formula can produce incorrect conclusions.

Soundness and Completeness — Quick Reference

Property	Direction
Soundness	$\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi$
Completeness	$\Gamma \models \varphi \Rightarrow \Gamma \vdash \varphi$
Soundness Theorem (FOL)	All standard FOL rules are sound
Gödel's Completeness Theorem	FOL is complete

2.6.4 Proof Theory: Soundness and Completeness

Definition (Soundness)

Definition 2.6.11 (Soundness)**.** A proof system is *sound* if every provable formula is valid:

$$\Gamma \vdash \varphi \implies \Gamma \models \varphi.$$

Remark (Dependencies).

- Satisfaction
- Logical Consequence

Definition (Completeness)

Definition 2.6.12 (Completeness)**.** A proof system is *complete* if every valid formula is provable:

$$\Gamma \models \varphi \implies \Gamma \vdash \varphi.$$

Remark (Dependencies).

- Satisfaction
- Logical Consequence

Theorem (Soundness of First-Order Logic)

Theorem 2.6.13 (Soundness of First-Order Logic)**.** *The standard inference rules for first-order logic (UI, UG, EI, EG, and the propositional rules) are sound: they preserve truth in all structures.*
If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$. Go to proof.

Remark (Standard quantified statement). $\forall \Gamma, \varphi : \ \Gamma \vdash \varphi \ \to \ \forall \mathcal{M}, \forall s, \ (\forall \gamma \in \Gamma, \ \mathcal{M}, s \models \gamma) \to \mathcal{M}, s \models \varphi$.

Remark (Dependencies).

- Soundness
- Satisfaction
- Logical Consequence
- Universal Instantiation
- Quantifier Distribution

Theorem (Gödel's Completeness Theorem)

First-order logic is complete: every logically valid formula is provable.

$$\Gamma \models \varphi \implies \Gamma \vdash \varphi.$$

Equivalently: if a set of sentences Γ is consistent (has no proof of contradiction), then Γ has a model.

Remark (English reading). Soundness ensures proofs do not lead us astray: we cannot prove false statements from true premises. Completeness ensures proofs are powerful enough: every statement that is true in all models can be established by a formal proof.

Together, soundness and completeness show that syntactic provability ($\vdash$) and semantic entailment ($\models$) coincide for first-order logic.

Remark (Completeness vs. incompleteness). Gödel's completeness theorem (1930) should not be confused with his incompleteness theorems (1931). The completeness theorem concerns first-order logic as a proof system. The incompleteness theorems concern the limitations of formal theories capable of expressing arithmetic, and are a separate result.

2.7 Translation

Toolkit: Translation

Task	Question	Typical output
Translation key	What does each symbol mean?	Domain, constants, predicates, relations
Universal claim	Which objects are restricted?	$\forall x\,(A(x) \to B(x))$
Existential claim	Which properties are jointly required?	$\exists x\,(A(x) \wedge B(x))$
Scope analysis	Which quantifier is outermost?	$\forall x \exists y\, R(x,y)$ or $\exists y \forall x\, R(x,y)$

Remark (Exposition). This section records the applied translation layer for predicate logic: keys, domains, restricted quantifiers, scope ambiguity, and the modern first-order reading of categorical forms.

English to Logic and Scope Ambiguity

Definition (Translation Key)

Definition 2.7.1 (Translation Key)**.** A *translation key* is a finite record that fixes a domain of discourse and assigns intended English readings to the constant symbols, predicate symbols, and relation symbols used in a translation.

Remark (Standard quantified statement).

K is a translation key
$\iff$ K specifies a domain and readings for each non-logical symbol used.

Remark (Definition predicate reading). TranslationKey(K) means that K supplies the domain and symbol readings needed to parse a translated formula.

Remark (Interpretation). A translation key prevents hidden changes of meaning. The same symbol must keep the same reading throughout one exercise unless the key is explicitly replaced.

Remark (Examples). For the English sentence "Every applicant submitted some form," a key may take the domain to be people and forms, with $A(x)$ for "x is an applicant," $F(y)$ for "y is a form," and $S(x, y)$ for "x submitted y."

Remark (Non-Examples). Using $P(x)$ once for "x is a person" and later for "x passed" is not a translation key; it is a change of vocabulary inside one translation.

Remark (Dependencies).

- Constant Symbol
- Predicate Symbol

Definition (Domain of Discourse)

Definition 2.7.2 (Domain of Discourse)**.** The *domain of discourse* for a translation is the collection of objects over which the variables of the translated formulas range.

Remark (Standard quantified statement).

D is the domain of discourse $\iff$ each variable in the translation ranges over objects in D.

Remark (Definition predicate reading). DomainOfDiscourse(D, K) means that D is the domain fixed by the translation key K.

Remark (Interpretation). The domain decides what the quantifiers are allowed to talk about. If the domain is people, then an exam must either be named by a separate sort-like predicate or represented by a different translation choice.

Remark (Examples). For "All cats are mammals," one may use the domain of all animals and write $\forall x\,(C(x) \to M(x))$. With the smaller domain of cats, the same English claim may be written $\forall x\, M(x)$.

Remark (Non-Examples). Writing $\forall x\, M(x)$ while silently changing the domain from animals to cats halfway through the exercise is not a valid translation.

Remark (Dependencies).

- Translation Key
- Variable

Definition (Predicate and Relation Symbols in a Translation)

Definition 2.7.3 (Predicate and Relation Symbols in a Translation)**.** In a translation key, a unary predicate symbol names a property of one object, and an n-ary relation symbol names a relation among n objects.

Remark (Standard quantified statement)*.*

$$\begin{aligned} P \text{ is unary in } K &\iff P(x) \text{ has one argument,} \\ R \text{ is } n\text{-ary in } K &\iff R(x_1, \ldots, x_n) \text{ has } n \text{ arguments.} \end{aligned}$$

Remark (Definition predicate reading)*.* TranslationPredicateSymbol(P, K) records that P has a fixed predicate reading in K, and TranslationRelationSymbol(R, n, K) records that R is used with arity n in K.

Remark (Interpretation)*.* Object-language predicate symbols such as P, Q, R are symbols inside the formal language. Metalinguistic predicate readings such as Term(t) or AtomicFormula(φ) are repository descriptions of syntax; they are not formulas being translated.

Remark (Examples)*.* $A(x)$ may mean "x is an athlete." $L(x, y)$ may mean "x likes y." The arity is part of the key: $L(x)$ and $L(x, y)$ are not the same symbol use.

Remark (Non-Examples)*.* Writing $L(x, y, z)$ after declaring $L(x, y)$ as "x likes y" is an arity error.

Remark (Dependencies).

- Predicate Symbol
- Atomic Formula

Definition (Constants and Variables in a Translation)

Definition 2.7.4 (Constants and Variables in a Translation)**.** A *constant* names a fixed object from the translation key, while a *variable* marks a place bound by a quantifier or left open for later binding.

Remark (Standard quantified statement)*.*

$$\begin{aligned} c \text{ is a translation constant} &\iff c \text{ names one fixed object,} \\ x \text{ is a translation variable} &\iff x \text{ is available for quantification or free occurrence.} \end{aligned}$$

Remark (Definition predicate reading)*.* TranslationConstant(c, K) means that c names a fixed object in K, and TranslationVariable(x, K) means that x is used as a variable in translations governed by K.

Remark (Interpretation)*.* Constants do not range. Variables do range when bound by quantifiers. Confusing the two usually leaves a formula with the wrong dependency pattern.

Remark (Examples)*.* If a names Alice, then $P(a)$ says Alice is a participant. The formula $\forall x\,(S(x) \to P(x))$ uses x as a bound variable.

Remark (Non-Examples). The expression $\forall a\, S(a)$ is not appropriate if a has already been declared as a constant naming Alice.

Remark (Dependencies).

- Constant Symbol
- Variable

Proposition (Universal Translation Pattern)

Proposition 2.7.5 (Universal Translation Pattern)**.** *In a broad domain, an English statement of the form "All A are B" is translated by*

$$\forall x\,(A(x) \to B(x)).$$

Go to proof.

Remark (Standard quantified statement).

$$\forall x\,(A(x) \to B(x)).$$

Remark (Predicate reading). UniversalTranslationPattern(A, B) records the implication schema for translating universal restricted claims.

Remark (Interpretation). The antecedent $A(x)$ restricts attention to the relevant objects. Objects outside A do not have to satisfy B.

Remark (Exposition). "All cats are mammals" becomes $\forall x\,(C(x) \to M(x))$. "No cats are reptiles" becomes $\forall x\,(C(x) \to \neg R(x))$. The form $\forall x\,(C(x) \wedge M(x))$ says every object in the domain is both a cat and a mammal, which is far stronger than the intended restricted claim. In a broad domain, "all," "every," and "any" usually translate restricted claims with implication.

Remark (Dependencies).

- Universal Formula
- Molecular Formula

Proposition (Existential Translation Pattern)

Proposition 2.7.6 (Existential Translation Pattern)**.** *In a broad domain, an English statement of the form "Some A are B" is translated by*

$$\exists x\,(A(x) \wedge B(x)).$$

Go to proof.

Remark (Standard quantified statement).

$$\exists x\,(A(x) \wedge B(x)).$$

Remark (Predicate reading). ExistentialTranslationPattern(A, B) records the conjunction schema for translating existential restricted claims.

Remark (Interpretation). The witness must satisfy both restrictions at once. Existence is not merely a conditional promise about what would happen if an A existed.

Remark (Exposition). "Some cats are black" becomes $\exists x\,(C(x) \wedge B(x))$. "Some cats are not black" becomes $\exists x\,(C(x) \wedge \neg B(x))$. The form $\exists x\,(C(x) \to B(x))$ may be true because the witness is not a cat, so it does not translate "Some cats are black." In a broad domain, "some," "there is," and "at least one" usually translate restricted claims with conjunction.

Remark (Dependencies).

- Existential Formula
- Molecular Formula

Proposition (Conditional Translation Pattern)

Proposition 2.7.7 (Conditional Translation Pattern)**.** *An English statement of the form "If A, then B" is translated by*

$$A \to B,$$

with quantifiers placed according to the variables occurring in A and B.
Go to proof.

Remark (Standard quantified statement).

$$A \to B.$$

Remark (Predicate reading). ConditionalTranslationPattern(A, B) records the implication schema for conditional English.

Remark (Interpretation). Conditionals are especially common inside universal translations. The conditional separates the restricting condition from the asserted consequence.

Remark (Exposition). "Every applicant who is eligible is interviewed" becomes $\forall x\,((A(x) \wedge E(x)) \to I(x))$. The form $\forall x\,(A(x) \wedge E(x) \wedge I(x))$ says every object is an eligible applicant who is interviewed, which is not the intended conditional claim.

Remark (Dependencies).

- Universal Translation Pattern

Proposition (Negation Translation Pattern)

Proposition 2.7.8 (Negation Translation Pattern)**.** *The negation of a translated quantified statement is obtained by negating the whole formula and then pushing the negation inward using the quantifier negation laws.*
Go to proof.

Remark (Standard quantified statement).

$$\neg\forall x\,\varphi \iff \exists x\,\neg\varphi,$$
$$\neg\exists x\,\varphi \iff \forall x\,\neg\varphi.$$

Remark (Predicate reading). NegationTranslationPattern(φ, ψ) records that ψ is obtained from $\neg\varphi$ by pushing negation through the quantifiers and connectives.

Remark (Interpretation). The negation of "all" is "some not." The negation of "some" is "none." For restricted universal claims, negating the implication produces a witness that satisfies the restriction and fails the consequent.

Remark (Exposition). The negation of $\forall x\,(A(x) \to B(x))$ is $\exists x\,(A(x) \wedge \neg B(x))$. The negation of $\exists x\,(A(x) \wedge B(x))$ is $\forall x\,(A(x) \to \neg B(x))$. The negation of $\forall x\,(A(x) \to B(x))$ is not $\forall x\,(A(x) \to \neg B(x))$; that latter formula says every A fails to be B, not merely that some counterexample exists.

Remark (Dependencies).

- Quantifier Negation Laws
- Universal Translation Pattern
- Existential Translation Pattern

Definition (Scope Ambiguity)

Definition 2.7.9 (Scope Ambiguity)**.** A sentence has *scope ambiguity* when the same ordinary-language wording admits two or more translations with different quantifier nesting or different connective scope.

Remark (Standard quantified statement).

E has scope ambiguity
$\iff$ there exist distinct candidate translations φ, ψ of E with different scope.

Remark (Definition predicate reading). ScopeAmbiguity(E, φ, ψ) means that the English sentence E can reasonably be translated by φ or by ψ, and the difference is a scope difference.

Remark (Interpretation). Scope is not cosmetic. Changing the outer quantifier can change whether a single witness works for everyone or whether each object may have its own witness.

Remark (Exposition). The formulas $\forall x\exists y\,R(x, y)$ and $\exists y\forall x\,R(x, y)$ are not interchangeable. The first allows the witness y to depend on x; the second requires one y that works for every x. Symbols such as $A(x)$, $B(x)$, and $R(x, y)$ are object-language formula patterns used in translations. Expressions such as TranslationKey(K), Term(t), and AtomicFormula(φ) are metalinguistic predicate readings used by the project to describe mathematical objects.

All A are B	$\rightsquigarrow$	$\forall x\,(A(x) \to B(x))$
Some A are B	$\rightsquigarrow$	$\exists x\,(A(x) \wedge B(x))$
No A are B	$\rightsquigarrow$	$\forall x\,(A(x) \to \neg B(x))$
Some A are not B	$\rightsquigarrow$	$\exists x\,(A(x) \wedge \neg B(x))$
Every A has a B	$\rightsquigarrow$	$\forall x\,(A(x) \to \exists y\,(B(y) \wedge R(x, y)))$
There is a B for every A	$\rightsquigarrow$	$\exists y\,(B(y) \wedge \forall x\,(A(x) \to R(x, y)))$.

Remark (Examples). "Every A has a B" is usually $\forall x\,(A(x) \to \exists y\,(B(y) \wedge R(x,y)))$. "There is a B for every A" is usually $\exists y\,(B(y) \wedge \forall x\,(A(x) \to R(x,y)))$. The bare patterns $\forall x \exists y\, R(x,y)$ and $\exists y \forall x\, R(x,y)$ make the same scope contrast visible.

Remark (Non-Examples). Renaming a bound variable from x to z does not create a scope ambiguity. The scope is unchanged.

Remark (Dependencies).

- Quantifier Scope
- Quantifier Order

Square of Opposition

Definition (Universal Affirmative Form)

Definition 2.7.10 (Universal Affirmative Form)**.** The *universal affirmative* categorical form, called A, translates "All A are B" as

$$\forall x\,(A(x) \to B(x)).$$

Remark (Standard quantified statement).

$$\forall x\,(A(x) \to B(x)).$$

Remark (Definition predicate reading). CategoricalAForm(A, B, φ) means that φ is the universal affirmative translation from A to B.

Remark (Interpretation). The A-form says that every object satisfying the subject predicate also satisfies the predicate predicate. It does not by itself say that anything satisfies the subject predicate.

Remark (Dependencies).

- Universal Translation Pattern

Definition (Universal Negative Form)

Definition 2.7.11 (Universal Negative Form)**.** The *universal negative* categorical form, called E, translates "No A are B" as

$$\forall x\,(A(x) \to \neg B(x)).$$

Remark (Standard quantified statement).

$$\forall x\,(A(x) \to \neg B(x)).$$

Remark (Definition predicate reading). CategoricalEForm(A, B, φ) means that φ is the universal negative translation from A to B.

Remark (Interpretation). The E-form says that anything satisfying A fails to satisfy B. It is still universal, so it can be true when no object satisfies A.

Remark (Dependencies).

- Universal Translation Pattern
- Negation Translation Pattern

Definition (Particular Affirmative Form)

Definition 2.7.12 (Particular Affirmative Form)**.** The *particular affirmative* categorical form, called I, translates "Some A are B" as

$$\exists x\,(A(x) \wedge B(x)).$$

Remark (Standard quantified statement).

$$\exists x\,(A(x) \wedge B(x)).$$

Remark (Definition predicate reading). CategoricalIForm(A, B, φ) means that φ is the particular affirmative translation from A to B.

Remark (Interpretation). The I-form requires a witness satisfying both predicates. It carries existential force.

Remark (Dependencies).

- Existential Translation Pattern

Definition (Particular Negative Form)

Definition 2.7.13 (Particular Negative Form)**.** The *particular negative* categorical form, called O, translates "Some A are not B" as

$$\exists x\,(A(x) \wedge \neg B(x)).$$

Remark (Standard quantified statement).

$$\exists x\,(A(x) \wedge \neg B(x)).$$

Remark (Definition predicate reading). CategoricalOForm(A, B, φ) means that φ is the particular negative translation from A to B.

Remark (Interpretation). The O-form requires a witness satisfying the subject predicate and failing the predicate predicate.

Remark (Dependencies).

- Existential Translation Pattern
- Negation Translation Pattern

Proposition (A-O Contradiction)

Proposition 2.7.14 (A-O Contradiction)**.** *The A-form* $\forall x\,(A(x) \to B(x))$ *and the O-form* $\exists x\,(A(x) \wedge \neg B(x))$ *are contradictory: exactly one of them is true.*
Go to proof.

Remark (Standard quantified statement)*.*

$$\neg\forall x\,(A(x) \to B(x)) \iff \exists x\,(A(x) \wedge \neg B(x)).$$

Remark (Predicate reading)*.* ContradictoryPair(φ_A, φ_O) records that the A-form and O-form cannot share a truth value.

Remark (Interpretation)*.* The O-form is exactly the counterexample to the A-form: an object that is A but not B.

Remark (Dependencies)*.*

- Universal Affirmative Form
- Particular Negative Form
- Quantifier Negation Laws

Proposition (E-I Contradiction)

Proposition 2.7.15 (E-I Contradiction)**.** *The E-form* $\forall x\,(A(x) \to \neg B(x))$ *and the I-form* $\exists x\,(A(x) \wedge B(x))$ *are contradictory: exactly one of them is true.*
Go to proof.

Remark (Standard quantified statement)*.*

$$\neg\forall x\,(A(x) \to \neg B(x)) \iff \exists x\,(A(x) \wedge B(x)).$$

Remark (Predicate reading)*.* ContradictoryPair(φ_E, φ_I) records that the E-form and I-form cannot share a truth value.

Remark (Interpretation)*.* The I-form is exactly the counterexample to the E-form: an object that is both A and B.

Remark (Dependencies)*.*

- Universal Negative Form
- Particular Affirmative Form
- Quantifier Negation Laws

Proposition (Contrariety Requires Existential Import)

Proposition 2.7.16 (Contrariety Requires Existential Import)**.** *If* $\exists x\,A(x)$*, then the A-form and E-form cannot both be true.*
Go to proof.

Remark (Standard quantified statement).

$$\exists x\, A(x) \to \neg\big(\forall x\, (A(x) \to B(x)) \wedge \forall x\, (A(x) \to \neg B(x))\big).$$

Remark (Predicate reading). ContraryWithExistentialImport(φ_A, φ_E) records that contrariety holds only after the subject predicate is known to have a witness.

Remark (Interpretation). With an A-object available, the two universal claims force both B and $\neg B$ of that object. Without such an object, both universals may be vacuously true. The traditional square of opposition assumes existential import; modern first-order logic does not automatically assume that a universal statement implies existence.

Remark (Dependencies).

- Universal Affirmative Form
- Universal Negative Form
- Existential Formula

2.8 Reference

Toolkit: Predicate Logic Reference

Reference	Use
Common errors	Diagnose translation, scope, and inference mistakes.
Summary tables	Compare translation patterns, quantifier laws, categorical forms, and proof cues.

Remark (Exposition). This section collects compact reference material for translation, quantifier negation, common inference errors, and proof strategy cues.

Common Errors and Fallacies

Remark (Translation and proof errors).

Error	Incorrect form	Corrected for
Confusing $\forall$ and $\exists$	$\exists x\, P(x)$ for "every x has P"	$\forall x\, P(x)$
Universal conjunction	$\forall x\,(A(x) \wedge B(x))$	$\forall x\,(A(x) \to B(x$
Existential implication	$\exists x\,(A(x) \to B(x))$	$\exists x\,(A(x) \wedge B(x$
Scope error	$\exists y \forall x\, R(x, y)$	$\forall x \exists y\, R(x, y)$, each x may h own y
Variable capture	$\varphi[t/x]$ without checking bound variables	Rename bound ables before su tion
Free variable left in a sentence	$\forall x\, R(x, y)$ as a closed claim	$\forall x \forall y\, R(x, y)$, or as intended
Affirming the consequent	$P \to Q,\ Q\ \therefore P$	Invalid; use a terexample valua structure
Denying the antecedent	$P \to Q,\ \neg P\ \therefore \neg Q$	Invalid; use a terexample valua structure
Invalid quantifier reversal	$\forall x \exists y\, R(x, y) \Rightarrow \exists y \forall x\, R(x, y)$	Invalid without hypotheses

Remark (Failure-mode checklist). Before accepting a translation or inference, check the domain, each symbol reading, every bound-variable scope, and whether the final formula is intended to be a sentence. Then test suspicious implications by trying to build one case where the premise is true and the conclusion false.

Summary Tables

Remark (English phrase to logical pattern).

English phrase	Logical pattern
All A are B	$\forall x\,(A(x) \to B(x))$
Some A are B	$\exists x\,(A(x) \wedge B(x))$
No A are B	$\forall x\,(A(x) \to \neg B(x))$
Some A are not B	$\exists x\,(A(x) \wedge \neg B(x))$
Every A has a B	$\forall x\,(A(x) \to \exists y\,(B(y) \wedge R(x,y)$
There is a B for every A	$\exists y\,(B(y) \wedge \forall x\,(A(x) \to R(x,y)$

Remark (Quantifier negation laws).

$$\begin{aligned}
\neg\forall x\,\varphi &\iff \exists x\,\neg\varphi,\\
\neg\exists x\,\varphi &\iff \forall x\,\neg\varphi,\\
\neg\forall x\exists y\,\varphi &\iff \exists x\forall y\,\neg\varphi,\\
\neg\exists x\forall y\,\varphi &\iff \forall x\exists y\,\neg\varphi.
\end{aligned}$$

Remark (Categorical form translations).

Form	Name	Translation
A	Universal affirmative	$\forall x\,(A(x) \to B(x))$
E	Universal negative	$\forall x\,(A(x) \to \neg B(x))$
I	Particular affirmative	$\exists x\,(A(x) \wedge B(x))$
O	Particular negative	$\exists x\,(A(x) \wedge \neg B(x))$

Remark (Proof strategy cues).

Goal or premise	Useful cue
Prove $\forall x\,\varphi(x)$	Let x be arbitrary, then prove $\varphi(x)$.
Use $\forall x\,\varphi(x)$	Instantiate it with a term that is free for x.
Prove $\exists x\,\varphi(x)$	Produce a witness and verify φ.
Use $\exists x\,\varphi(x)$	Introduce a fresh witness and keep its scope cont
Disprove $\forall x\,\varphi(x)$	Find a counterexample satisfying $\neg\varphi$.
Disprove an implication	Make the hypothesis true and the conclusion fa

Chapter 3

Many-Sorted Logic

3.1 Introduction

3.1.1 Examples

3.1.2 Reduction to and comparison with first-order logic

3.1.3 Uses of many-sorted logic

3.1.4 Many-sorted logic as a unifier logic

3.2 Structures

3.2.1 Definition (signature)

3.2.2 Definition (structure)

3.3 Formal many-sorted language

3.3.1 Alphabet

3.3.2 Expressions: formulas and terms

3.3.3 Remarks on notation

3.3.4 Abbreviations

3.3.5 Induction

3.3.6 Free and bound variables

3.4 Semantics

3.4.1 Definitions

3.4.2 Satisfiability, validity, consequence and logical equivalence

3.5 Substitution of a term for a variable

3.6 Semantic theorems

3.6.1 Coincidence lemma

3.6.2 Substitution lemma

3.6.3 Equals substitution lemma

3.6.4 Isomorphism theorem

3.7 The completeness of many-sorted logic

3.7.1 Deductive calculus

Chapter 4

Standard Second-Order Logic

4.1 Introduction

4.1.1 General idea

4.1.2 Expressive power

4.1.3 Model-theoretic counterparts of expressiveness

4.1.4 Incompleteness

4.2 Second-order grammar

4.2.1 Definition (signature and alphabet)

4.2.2 Expressions: terms, predicates and formulas

4.2.3 Remarks on notation

4.2.4 Induction

4.2.5 Free and bound variables

4.2.6 Substitution

4.3 Standard structures

4.3.1 Definition of standard structures

4.3.2 Relations between standard structures established without the formal language

4.4 Standard semantics

4.4.1 Assignment

4.4.2 Interpretation

4.4.3 Consequence and validity

4.4.4 Logical equivalence

4.4.5 Simplifying our language

4.4.6 Alternative presentation of standard semantics

4.4.7 Definable sets and relations in a given structure

4.4.8 More about the expressive power of standard SOL

4.4.9 Negative results

4.5 Semantic theorems

Chapter 5

Languages, Axioms, and Models

Languages, Axioms, and Models Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.1 Signatures

Signatures Toolkit — Quick Reference

Concept	Role
Signature	non-logical vocabulary
Language	signature together with logical syntax
Constant symbol	names a distinguished object syntactically
Function symbol	names an operation syntactically
Relation symbol	names a relation syntactically
Arity	records number of input places

Signatures

Signature Presentation: A Simple Arithmetic Language

Let

$$\Sigma_{\mathrm{arith}} = \left(\mathcal{C}_{\mathrm{arith}}, \mathcal{F}_{\mathrm{arith}}, \mathcal{R}_{\mathrm{arith}}, \mathrm{ar}\right)$$

where

$$\mathcal{C}_{\text{arith}} = \{0, 1\}, \qquad \mathcal{F}_{\text{arith}} = \{+, \cdot, S\}, \qquad \mathcal{R}_{\text{arith}} = \{<\}.$$

The arity function is specified by

$$\text{ar}(0) = \text{ar}(1) = 0, \qquad \text{ar}(S) = 1, \qquad \text{ar}(+) = \text{ar}(\cdot) = 2, \qquad \text{ar}(<) = 2.$$

Symbol	Kind	Arity	Intended Reading
$0, 1$	constants	0	distinguished constant names
S	function symbol	1	successor operation symbol
$+, \cdot$	function symbols	2	binary operation symbols
$<$	relation symbol	2	binary order relation symbol

The signature records only the available non-logical symbols and their arities. It does not assert any axioms about those symbols.

Definitional Root (First-Order Signature)

Definition 5.1.1 (First-Order Signature)**.** A *first-order signature* is a tuple

$$\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \text{ar}),$$

where $\mathcal{C}$ is a set of constant symbols, $\mathcal{F}$ is a set of function symbols, $\mathcal{R}$ is a set of relation symbols, and

$$\text{ar} : \mathcal{F} \cup \mathcal{R} \to \mathbb{Z}_{>0}$$

assigns to each function or relation symbol its number of arguments.

Remark (Standard quantified statement)*.* For every $s \in \mathcal{F} \cup \mathcal{R}$, the value $\text{ar}(s)$ is a positive integer. If $s \in \mathcal{F}$, then s forms terms with $\text{ar}(s)$ inputs; if $s \in \mathcal{R}$, then s forms atomic formulas with $\text{ar}(s)$ inputs.

Remark (Interpretation)*.* A signature is the formal vocabulary available before any axioms are chosen. It says which symbols may occur and how many input places each symbol has.

Definition (Arity)

Definition 5.1.2 (Arity)**.** Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \text{ar})$ be a first-order signature. The *arity function* is the map

$$\text{ar} : \mathcal{F} \cup \mathcal{R} \to \mathbb{Z}_{>0}$$

that assigns to each function or relation symbol the number of arguments it requires. When constants are treated as 0-ary function symbols, the codomain is enlarged to include 0.

Remark (Standard quantified statement). For every $s \in \mathcal{F} \cup \mathcal{R}$, there is a unique positive integer n such that $\mathrm{ar}(s) = n$. If $f \in \mathcal{F}$ and $\mathrm{ar}(f) = n$, then an expression $f(t_1, \ldots, t_k)$ is a well-formed term exactly when $k = n$. If $R \in \mathcal{R}$ and $\mathrm{ar}(R) = n$, then an expression $R(t_1, \ldots, t_k)$ is a well-formed atomic formula exactly when $k = n$.

Remark (Interpretation). Arity is bookkeeping for grammatical well-formedness. It determines whether a symbol has been supplied with the correct number of arguments.

Remark (Dependencies).

- Signature

Definitional Framework (Language)

Definition 5.1.3 (Language)**.** A *first-order language* has two standard readings. As vocabulary, it is the same data as a signature,

$$\mathcal{L} = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \mathrm{ar}).$$

As a formal language generated by a signature Σ, it is the set of well-formed strings built from the alphabet

$$A = \Sigma \cup V \cup \Lambda,$$

where V is a countable set of variables and Λ is the fixed stock of logical symbols and punctuation. In this second reading,

$$\mathcal{L}(\Sigma) = \{s \in A^* \mid s \in \mathrm{Terms}(\Sigma) \text{ or } s \in \mathrm{WFF}(\Sigma)\}.$$

Remark (Standard quantified statement). The vocabulary reading records which non-logical symbols may occur. The generated-language reading then selects from all finite strings over A only the terms and well-formed formulas admitted by the formation rules determined by Σ.

Remark (Interpretation). The signature supplies the subject-specific vocabulary. The language adds the logical grammar needed to form expressions from that vocabulary.

Remark (Dependencies).

- Signature

Definition (Constant Symbol)

Definition 5.1.4 (Constant Symbol)**.** A *constant symbol* is a non-logical symbol that behaves syntactically as a 0-ary function symbol. In the streamlined convention where constants are included among function symbols, this is the condition

$$\mathrm{Constant}(c) \iff c \in \mathcal{F} \text{ and } \mathrm{ar}(c) = 0.$$

Because its arity is 0, the symbol c is already a well-formed term and takes no input terms.

Remark (Standard quantified statement). If $c \in \mathcal{F}$ and $\mathrm{ar}(c) = 0$, then $c \in \mathrm{Terms}(\mathcal{L})$. In a structure $\mathfrak{M} = (M, \mathcal{I})$, the interpretation of a constant is a unique element of the domain:

$$\forall c \in \mathcal{C},\ \exists! e \in M,\ \mathcal{I}(c) = e.$$

Remark (Interpretation). A constant symbol is syntactically rigid: unlike a variable, it does not vary with an assignment. In the arithmetic signature, the symbol 0 is a piece of formal syntax; under the standard interpretation, it denotes the number zero.

Remark (Dependencies).

- Signature
- Arity

Definition (Function Symbol)

Definition 5.1.5 (Function Symbol)**.** A *function symbol* is a non-logical symbol $f \in \mathcal{F}$ whose assigned arity determines how many terms it must take as inputs. If $\mathrm{ar}(f) = n$, then f is an n-ary term constructor: it takes exactly n terms and produces a new term, not a truth value.

Remark (Standard quantified statement). For every $f \in \mathcal{F}$, if $\mathrm{ar}(f) = n$, then

$$f(t_1, \ldots, t_k) \in \mathrm{Terms}(\mathcal{L}) \iff k = n$$

whenever $t_1, \ldots, t_k$ are terms. In a structure $\mathfrak{M} = (M, \mathcal{I})$, the interpretation of f is a total operation closed on the domain:

$$\mathcal{I}(f) : M^n \to M.$$

Remark (Interpretation). A function symbol is operation-forming syntax. In the arithmetic signature, S is unary, while $+$ and $\cdot$ are binary. Under an interpretation, each becomes an actual operation on the chosen domain.

Remark (Dependencies).

- Signature
- Arity

Definition (Relation Symbol)

Definition 5.1.6 (Relation Symbol)**.** A *relation symbol*, or predicate symbol, is a non-logical symbol $R \in \mathcal{R}$ whose assigned arity determines how many terms it must take as inputs. If $\mathrm{ar}(R) = n$, then R is an n-ary formula constructor: it takes exactly n terms and produces an atomic formula, hence something with a truth value rather than an object.

Remark (Standard quantified statement). For every $R \in \mathcal{R}$, if $\mathrm{ar}(R) = n$, then

$$R(t_1, \ldots, t_k) \in \mathrm{WFF}(\mathcal{L}) \iff k = n$$

whenever $t_1, \ldots, t_k$ are terms. In a structure $\mathfrak{M} = (M, \mathcal{I})$, the interpretation of R is a subset of n-tuples from the domain:

$$\mathcal{I}(R) \subseteq M^n.$$

Thus $R(t_1, \ldots, t_n)$ is true exactly when the tuple of interpreted terms lies in $\mathcal{I}(R)$.

Remark (Interpretation). A relation symbol is a statement-forming symbol once it is applied to the right number of terms. In the arithmetic signature, $+$ is a binary function symbol because it returns a number, while $<$ is a binary relation symbol because it returns a truth value. Under the standard interpretation, $<$ denotes the set of ordered pairs (a, b) such that a is less than b.

Remark (Dependencies).

- Signature
- Arity

Remark (Structural remarks). The same signature may support different choices of axioms later in the chapter. For example, a single arithmetic vocabulary can be used with stronger or weaker axiom sets.

The same signature may also be read over different mathematical universes once interpretations are introduced. At this stage, the signature itself records only the common vocabulary; it does not choose any particular interpretation.

5.2 Axioms

Axioms Toolkit — Quick Reference

Concept	Role
Axiom	formal assertion adopted without proof
Axiom system	collection of assertions in one language
Assumption	temporary assertion for reasoning
Consistency	absence of contradiction
Theory	deductive closure of axioms

Axioms

Axiom Presentation: A Simple Arithmetic Axiom System

Reuse the arithmetic signature

$$\Sigma_{\text{arith}} = \big(\{0,1\}, \{+,\cdot,S\}, \{<\}, \text{ar}\big)$$

from the Signatures section. The signature supplies the symbols. An axiom system supplies assertions formed with those symbols.
For example, the following are familiar arithmetic-style axioms:

$$\forall x\ (x + 0 = x),$$

$$\forall x\ (S(x) \neq 0),$$

$$\forall x \forall y\ (S(x) = S(y) \to x = y).$$

These examples are illustrative. The signature says that 0, $+$, and S are available symbols. The axioms assert formal relationships involving those symbols.

Remark (Structural remarks). **Vocabulary vs assertion.** A signature is vocabulary: it specifies what may be said. An axiom is an assertion: it specifies what is adopted as true within that vocabulary. The signature supplies symbols; the axioms supply assertions involving those symbols.

Same signature, different axiom systems. A single signature may support multiple collections of axioms. For example, geometric languages can be used for Euclidean or non-Euclidean axiom systems. The usual group signature can be used for group theory, while adding the commutativity axiom gives the stronger axiom system for abelian groups. The vocabulary can remain fixed while the assertions change.

Forward conceptual roadmap. The chapter proceeds from collections of adopted assertions to their deductive and semantic roles:

Axiom System
$\Downarrow$
Theory
$\Downarrow$
Models

The present section introduces the assertion layer. Later sections formalize the theory generated by axioms and the models in which those assertions hold.

Definition (Axiom)

Definition 5.2.1 (Axiom)**.** Let $\mathcal{L}$ be a first-order language, and let $\text{Sent}(\mathcal{L})$ denote the set of sentences of $\mathcal{L}$. If $T \subseteq \text{Sent}(\mathcal{L})$ is a theory, then an *axiom of* T is an element $\alpha \in T$.

Remark (Standard quantified statement). For every first-order language $\mathcal{L}$ and every theory $T \subseteq \text{Sent}(\mathcal{L})$, a sentence α is an axiom of T exactly when $\alpha \in T$.

Remark (Interpretation). An axiom is not part of the vocabulary itself. It is an assertion made using the vocabulary of a language. The signature determines which symbols may appear; the axiom states something with those symbols.

Remark (Proof-theoretic reading). In a deductive presentation, an axiom may be used as a derivation with no undischarged premises:

$$\frac{}{\vdash \alpha}.$$

Thus axioms are the starting assertions of the proof system.

Remark (Dependencies).

- Language

Definition (Axiom System)

Definition 5.2.2 (Axiom System)**.** An *axiom system* is a collection of axioms expressed in a common language.

Remark (Interpretation). An axiom system records the assertions chosen for a mathematical setting. Two axiom systems may use the same language while adopting different assertions. The shared language fixes the vocabulary; the axiom system fixes the adopted formal claims.

Remark (Dependencies).

- Axiom
- Language

Definition (Assumption)

Definition 5.2.3 (Assumption)**.** An *assumption* is a statement temporarily accepted for purposes of reasoning.

Remark (Interpretation). An axiom is adopted as part of the formal background of a system. An assumption is local to an argument: it may be introduced to prove a conditional, to reason within a case, or to derive a contradiction. Axioms belong to the system; assumptions belong to the proof context.

Remark (Dependencies).

- Axiom

5.3 Theories

Theories Toolkit — Quick Reference

Concept	Role
Theory	all consequences generated by axioms
Theorem	statement derivable from axioms
Consequence	statement following from a theory
Derivation	chain of justified steps
Deductive closure	all derivable statements

Theories

Theory Presentation: Arithmetic Consequences

Start with the arithmetic-style axiom system from the previous section. It uses the arithmetic signature and includes assertions such as

$$\forall x\ (x + 0 = x),$$

$$\forall x\ (S(x) \neq 0),$$

$$\forall x \forall y\ (S(x) = S(y) \to x = y).$$

Those axioms are the starting assertions. From them, together with other arithmetic axioms and ordinary logical reasoning, additional statements may be established. For instance, one may later prove new equations, cancellation facts, uniqueness facts, and order facts that were not listed as axioms.
A theory contains the original axioms. A theory also contains the statements derivable from those axioms. Thus the axioms are the generators, while the theory is the full collection of what those generators imply.

Remark (Structural remarks). **Axioms vs theory.** Axioms are starting assertions. A theory includes all logical consequences of those assertions. The distinction is the next step after the vocabulary and assertion distinction: a signature supplies vocabulary, axioms supply adopted assertions, and a theory contains everything derivable from those assertions.

Finite axioms, infinite theory. A small axiom system can generate an enormous collection of consequences. The listed axioms may be finite, but the statements derivable from them need not be. This is why a theory is usually larger than the axiom system that presents it.

Forward roadmap. The next conceptual step is semantic:

$$\begin{array}{c}\text{Theory}\\ \Downarrow\\ \text{Models}\end{array}$$

A model will later be introduced as a mathematical structure in which the statements of a theory hold.

Definition (Theory)

Definition 5.3.1 (Theory)**.** A *theory* over a first-order language $\mathcal{L}$ is a collection of closed sentences in that language:

$$T \subseteq \text{Sentences}(\mathcal{L}).$$

If Γ is a chosen set of axioms, the theory generated by Γ is the deductive closure

$$T = \{\varphi \in \text{Sentences}(\mathcal{L}) \mid \Gamma \vdash \varphi\}.$$

Remark (Standard quantified statement). Syntactically, a sentence φ belongs to the theory generated by Γ exactly when φ is derivable from Γ. Semantically, for an $\mathcal{L}$-structure $\mathfrak{M}$, the complete theory of $\mathfrak{M}$ is

$$\text{Th}(\mathfrak{M}) = \{\varphi \in \text{Sentences}(\mathcal{L}) \mid \mathfrak{M} \vDash \varphi\}.$$

For every sentence φ, either $\mathfrak{M} \vDash \varphi$ or $\mathfrak{M} \vDash \neg\varphi$, so $\text{Th}(\mathfrak{M})$ decides every sentence of the language.

Remark (Interpretation). A theory is not merely the list of axioms. It is the closure of those axioms under derivability: every statement that follows from the axioms belongs to the theory. The axioms start the system; the theory records everything the system proves.

Remark (Dependencies).

- Language
- Axiom System

Definition (Theorem)

Definition 5.3.2 (Theorem)**.** A *theorem* of a theory T is a sentence φ for which there is a finite formal derivation from T, written

$$T \vdash \varphi.$$

Equivalently, in proof-theoretic terms, φ is a theorem of T exactly when some derivation has leaves drawn from logical axioms or sentences of T, uses valid inference rules, and has φ as its root.

Remark (Standard quantified statement). Syntactically,

$$\text{Theorem}(T, \varphi) \iff \exists d\ (d : T \vdash \varphi).$$

Semantically, φ is entailed by T when it is true in every model of T:

$$T \vDash \varphi \iff \forall \mathfrak{M}\,\big(\mathfrak{M} \vDash T \implies \mathfrak{M} \vDash \varphi\big).$$

For first-order logic, the completeness theorem connects these readings: $T \vdash \varphi$ exactly when $T \vDash \varphi$.

Remark (Interpretation). Within an axiom system, a theorem is not an additional starting assumption. It is a statement reached from the adopted axioms by reasoning. Axioms are given; theorems are obtained.

Remark (Dependencies).

- Theory

Definition (Consequence)

Definition 5.3.3 (Consequence)**.** A sentence φ is a *semantic consequence* of a set of premises Γ, written $\Gamma \vDash \varphi$, if every model of Γ is also a model of φ. Equivalently, there is no structure in which all sentences of Γ are true while φ is false.

Remark (Standard quantified statement). Semantic consequence is the universal condition

$$\Gamma \vDash \varphi \iff \forall \mathfrak{M} \Big((\forall \gamma \in \Gamma,\ \mathfrak{M} \vDash \gamma) \implies \mathfrak{M} \vDash \varphi \Big).$$

Syntactic consequence, written $\Gamma \vdash \varphi$, means that there exists a finite valid deduction of φ from Γ. For first-order logic, the completeness theorem says that $\Gamma \vDash \varphi$ exactly when $\Gamma \vdash \varphi$.

Remark (Interpretation). Consequence is the relation that connects a theory to what it implies. If the axioms generate a theory, then consequences are the statements carried along by those axioms and by the statements already obtained from them.

Remark (Dependencies).

- Theory

Definition (Deductive Closure)

Definition 5.3.4 (Deductive Closure)**.** Let $\Gamma \subseteq \text{Sentences}(\mathcal{L})$ be a set of premises. The *deductive closure* of Γ, denoted $\text{Cn}(\Gamma)$, is the set of all sentences derivable from Γ:

$$\text{Cn}(\Gamma) = \{\varphi \in \text{Sentences}(\mathcal{L}) \mid \Gamma \vdash \varphi\}.$$

Remark (Standard quantified statement). A set of sentences T is deductively closed exactly when

$$T = \text{Cn}(T),$$

or equivalently, for every sentence φ,

$$T \vdash \varphi \implies \varphi \in T.$$

The closure operator satisfies the standard Tarskian closure laws:

$$\Gamma \subseteq \text{Cn}(\Gamma), \qquad \Gamma \subseteq \Delta \implies \text{Cn}(\Gamma) \subseteq \text{Cn}(\Delta), \qquad \text{Cn}(\text{Cn}(\Gamma)) = \text{Cn}(\Gamma).$$

By completeness for first-order logic, this syntactic closure agrees with the semantic closure

$$\{\varphi \in \text{Sentences}(\mathcal{L}) \mid \Gamma \vDash \varphi\}.$$

Remark (Interpretation). Deductive closure is the operation that turns an axiom system into the full theory it generates. Starting with the axioms, close under derivability; the result is the theory determined by those starting assertions.

Remark (Dependencies).

- Axiom System

5.4 Models

Models Toolkit — Quick Reference

Concept	Role
Interpretation	assigns meanings to symbols
Structure	domain with interpreted symbols
Model	structure satisfying the theory
Satisfaction	truth of a statement in a structure
Countermodel	model where a statement fails

Free Σ-Algebra and Evaluation

Depends on: `Signature; inductively generated set; unique readability`

Parameter

Shared bridge machinery. Parameter: a signature Σ. This card is instantiated by connective evaluation, term evaluation, parsing, and later by initial-algebra presentations such as $\mathbb{N}$.

Structure

$$\Sigma \text{ is a set of operation symbols,} \qquad \mathrm{ar} : \Sigma \to \mathbb{N}.$$

A Σ-algebra is a carrier $|\mathcal{A}|$ with operations

$$f^{\mathcal{A}} : |\mathcal{A}|^{\mathrm{ar}(f)} \to |\mathcal{A}| \qquad (f \in \Sigma).$$

Seeds land in the carrier; homomorphisms are maps of algebras.

Structure

$$T_\Sigma(X), \qquad f^T(t_1, \ldots, t_n) := f(t_1, \ldots, t_n), \qquad \eta : X \to T_\Sigma(X).$$

Syntax–Semantics Bridge

$$\mathrm{Hom}(h, \mathcal{A}, \mathcal{B}) \quad :\Longleftrightarrow \quad h(f^{\mathcal{A}}(\bar{a})) = f^{\mathcal{B}}(h(a_1), \ldots, h(a_n))$$

for every $f \in \Sigma$ and $\bar{a} \in |\mathcal{A}|^n$.

Axioms and Laws

No junk.

$$t \in T_\Sigma(X) \Longrightarrow t \in X \ \vee \ \exists f \in \Sigma\, \exists \bar{t}\, \big(t = f(\bar{t})\big).$$

No confusion.

$$x \neq f(\bar{t}), \qquad f(\bar{s}) = g(\bar{t}) \Longrightarrow f = g \wedge \bar{s} = \bar{t}.$$

Theorems and Consequences

1. **Universal property.**

$$\forall \mathcal{A}\, \forall \sigma : X \to |\mathcal{A}|\ \exists! \widehat{\sigma} : T_\Sigma(X) \to |\mathcal{A}|$$

$$\mathrm{Hom}(\widehat{\sigma}, T_\Sigma(X), \mathcal{A}) \wedge \widehat{\sigma} \circ \eta = \sigma.$$

2.

$$\mathrm{eval}(\mathcal{A})(\sigma) := \widehat{\sigma}.$$

Instances

use	Σ	X	target $\mathcal{A}$	seed σ
formula eval	Ω	atoms	$\mathbf{2}$	atom truth values
parse	Ω	X	$T_\Omega(X)$	η
term eval	Σ_{term}	Var	$\mathcal{A}_{\text{term}}$	assignment s
$\mathbb{N}$ preview	$\{0^{(0)}, S^{(1)}\}$	$\varnothing$	(C, c_0, succ)	none

Instances

$$\Omega = \{\neg^{(1)}, \wedge^{(2)}, \vee^{(2)}, \to^{(2)}, \leftrightarrow^{(2)}\},$$

$$\mathbf{2} = (\{0,1\}, \neg^{\mathbf{2}}, \wedge^{\mathbf{2}}, \vee^{\mathbf{2}}, \to^{\mathbf{2}}, \leftrightarrow^{\mathbf{2}}).$$

Boundary

The machine fixes one seed. Connective and term evaluation are exact instances. FOL quantifiers range over the assignment-indexed family $(\sigma_s)_s$:

$$\mathcal{A}, s \models \exists x\, \varphi \Longleftrightarrow \exists a \in A,\ \mathcal{A}, s[x \mapsto a] \models \varphi.$$

That is the downstream cylindric/polyadic layer. The absolutely free $T_\Sigma(X)$ is also distinct from the Lindenbaum–Tarski quotient by $\vdash$-equivalence.

Consequence

Depends on: `Satisfaction relation` $\models \subseteq \mathsf{Mod} \times \mathsf{Form}$

Parameter

A class of models Mod and a satisfaction relation

$$m \models \varphi \qquad (m \in \mathsf{Mod}).$$

Syntax–Semantics Bridge

$$\Gamma \models \varphi \quad :\Longleftrightarrow \quad \forall m \in \mathsf{Mod}\,\big((\forall \gamma \in \Gamma,\ m \models \gamma) \to m \models \varphi\big).$$

$$\mathrm{Cn}_{\models}(\Gamma) := \{\varphi : \Gamma \models \varphi\}, \qquad \models \varphi :\Longleftrightarrow \varnothing \models \varphi.$$

$$\mathrm{Th}(m) := \{\varphi : m \models \varphi\}.$$

Binding

logic	Mod	$m \models \varphi$
prop	valuations v	$\widehat{v}(\varphi) = 1$
FOL open	pairs $(\mathcal{A}, s)$	$\mathcal{A}, s \models \varphi$
FOL sentences	structures $\mathcal{A}$	$\mathcal{A} \models \sigma$

Models

Model Presentation: Arithmetic as a Model

Reuse the arithmetic signature and arithmetic-style axioms from the previous sections. A model begins by interpreting the symbols of that signature in an actual mathematical structure.

$$\begin{aligned} \text{Domain} &= \text{the natural numbers},\\ 0 &\mapsto \text{the number zero},\\ S &\mapsto \text{the successor operation},\\ + &\mapsto \text{addition}. \end{aligned}$$

In this structure, the statement

$$\forall x\ (x + 0 = x)$$

holds because adding zero to any natural number gives back that same natural number. The formal symbol 0 is only a symbol until it is interpreted as the number zero. The formal symbol + is only a symbol until it is interpreted as addition. The symbols acquire meaning only after interpretation.

Remark (Structural remarks). **Syntax vs meaning.** Signatures, axioms, and theories are formal objects. They belong to syntax: they specify vocabulary, adopted assertions, and consequences. Models provide semantic meaning by assigning mathematical content to the symbols and by determining which statements hold.

One theory, many models. The same theory may have multiple models. Euclidean

geometry may be realized in different coordinate systems. The group axioms have many models, including different concrete groups. The theory fixes the assertions; a model is one mathematical realization in which those assertions hold.

Countermodels. A statement fails to follow from a theory if there is a model of the theory in which that statement is false. Countermodels are therefore evidence that a claim is not forced by the axioms.

Forward roadmap. The next sections use models to explain consistency and independence:

$$\begin{array}{c}\text{Models}\\ \Downarrow\\ \text{Consistency}\\ \Downarrow\\ \text{Independence}\end{array}$$

Definition (Interpretation)

Definition 5.4.1 (Interpretation)**.** Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \mathrm{ar})$ be a first-order signature, and let M be a nonempty set. An *interpretation of* Σ *on* M is an assignment $\mathcal{I}$ that maps each symbol of Σ to concrete data over M: constants are assigned elements of M, function symbols are assigned operations on M, and relation symbols are assigned relations on M.

Remark (Standard quantified statement). For every $c \in \mathcal{C}$, $\mathcal{I}(c) \in M$. For every $f \in \mathcal{F}$, if $\mathrm{ar}(f) = n$, then

$$\mathcal{I}(f) : M^n \to M.$$

For every $R \in \mathcal{R}$, if $\mathrm{ar}(R) = n$, then

$$\mathcal{I}(R) \subseteq M^n.$$

Remark (Interpretation). An interpretation turns formal vocabulary into mathematical content. It tells what each constant, function symbol, and relation symbol means in a particular setting.

Remark (Dependencies).

- Signature

Definition ($\mathcal{L}$-Structure)

Definition 5.4.2 ($\mathcal{L}$-Structure)**.** Let $\mathcal{L}$ be a first-order signature. An $\mathcal{L}$*-structure* is a pair

$$\mathfrak{M} = (M, \mathcal{I}),$$

where M is a nonempty domain and $\mathcal{I}$ is an interpretation of the symbols of $\mathcal{L}$ on M.

Remark (Standard quantified statement). For every first-order signature $\mathcal{L}$, an $\mathcal{L}$-structure consists of a nonempty domain M and an interpretation $\mathcal{I}$ such that every constant symbol is assigned an element of M, every n-ary function symbol is assigned a function $M^n \to M$, and every n-ary relation symbol is assigned a relation on M^n.

Remark (Interpretation). A structure is the environment in which formal symbols receive meaning. The domain supplies the objects, and the interpretation explains how the symbols refer to objects, operations, and relations on that domain.

Remark (Type-theoretic reading). In dependent type theory, this bundled data can be viewed schematically as

$$\sum_{M:\mathrm{Type}} \left(\prod_{c \in \mathcal{C}} M \right) \times \left(\prod_{f \in \mathcal{F}} (M^{\mathrm{ar}(f)} \to M) \right) \times \left(\prod_{R \in \mathcal{R}} (M^{\mathrm{ar}(R)} \to \mathrm{Prop}) \right).$$

This is the same semantic package represented by a Lean structure carrying a carrier type, interpretations of constants and functions, and predicates for relations.

Remark (Dependencies).

- Signature
- Interpretation

Definition (Model)

Definition 5.4.3 (Model). Let $\mathcal{L}$ be a first-order signature, let T be a theory over $\mathcal{L}$, and let $\mathfrak{M} = (M, \mathcal{I})$ be an $\mathcal{L}$-structure. The structure $\mathfrak{M}$ is a *model of* T, written $\mathfrak{M} \vDash T$, if every sentence in T is true in $\mathfrak{M}$.

Remark (Standard quantified statement). For every theory T over $\mathcal{L}$ and every $\mathcal{L}$-structure $\mathfrak{M}$,

$$\mathfrak{M} \vDash T \quad \Longleftrightarrow \quad \forall \varphi \in T,\ \mathfrak{M} \vDash \varphi.$$

Remark (Interpretation). A model is a structure that makes the theory true. It is not just an interpretation of the vocabulary; it is an interpretation in which the adopted assertions and their consequences hold.

Remark (Dependencies).

- Structure
- Theory

Definition (Satisfaction)

Definition 5.4.4 (Satisfaction). Let $\mathfrak{M} = (M, \mathcal{I})$ be an $\mathcal{L}$-structure. A *variable assignment* is a function $s : \mathrm{Var} \to M$. The assignment extends recursively to a term-evaluation map $\bar{s}$ by

$$\bar{s}(x) = s(x), \qquad \bar{s}(c) = \mathcal{I}(c),$$

and

$$\bar{s}\big(f(t_1, \ldots, t_n)\big) = \mathcal{I}(f)\big(\bar{s}(t_1), \ldots, \bar{s}(t_n)\big).$$

The satisfaction relation $\mathfrak{M} \vDash \varphi[s]$ is then defined by induction on formulas: atomic relations are tested inside the interpreted relation, equality is interpreted as equality in M, Boolean connectives are interpreted by the corresponding truth operations, and quantifiers range over all elements of M.

Remark (Standard quantified statement). For terms $t_1, \dots, t_n$ and an n-ary relation symbol R,

$$\mathfrak{M} \vDash R(t_1, \dots, t_n)[s] \quad \Longleftrightarrow \quad \big(\bar{s}(t_1), \dots, \bar{s}(t_n)\big) \in \mathcal{I}(R).$$

For equality,

$$\mathfrak{M} \vDash (t_1 = t_2)[s] \quad \Longleftrightarrow \quad \bar{s}(t_1) = \bar{s}(t_2).$$

For quantifiers,

$$\mathfrak{M} \vDash (\forall x\, \varphi)[s] \quad \Longleftrightarrow \quad \forall a \in M,\ \mathfrak{M} \vDash \varphi[s[x \mapsto a]],$$

and

$$\mathfrak{M} \vDash (\exists x\, \varphi)[s] \quad \Longleftrightarrow \quad \exists a \in M,\ \mathfrak{M} \vDash \varphi[s[x \mapsto a]].$$

Remark (Interpretation). Satisfaction is the semantic test for a statement. Once the symbols have been interpreted in a structure, the statement can be checked as true or false in that setting.

Remark (Sentences). If φ is a sentence, then it has no free variables, so its truth does not depend on the initial assignment s. In that case the notation is shortened from $\mathfrak{M} \vDash \varphi[s]$ to $\mathfrak{M} \vDash \varphi$. This is the bridge from satisfaction of a single sentence to the definition of a model of a theory.

Remark (Dependencies).

- Structure

Definition (Countermodel)

Definition 5.4.5 (Countermodel)**.** Let $\Gamma \subseteq \mathrm{Sent}(\mathcal{L})$ be a set of premises, let $\varphi \in \mathrm{Sent}(\mathcal{L})$, and let $\mathfrak{M}$ be an $\mathcal{L}$-structure. A *countermodel to the entailment* $\Gamma \vDash \varphi$ is a structure $\mathfrak{M}$ that satisfies every sentence in Γ but does not satisfy φ.

Remark (Standard quantified statement). For $\Gamma \subseteq \mathrm{Sent}(\mathcal{L})$ and $\varphi \in \mathrm{Sent}(\mathcal{L})$,

$$\mathrm{CounterModel}(\mathfrak{M}, \Gamma, \varphi) \quad \Longleftrightarrow \quad \big(\forall \gamma \in \Gamma,\ \mathfrak{M} \vDash \gamma\big) \wedge \big(\mathfrak{M} \nvDash \varphi\big).$$

Equivalently, for classical closed sentences,

$$\mathrm{CounterModel}(\mathfrak{M}, \Gamma, \varphi) \quad \Longleftrightarrow \quad \big(\forall \gamma \in \Gamma,\ \mathfrak{M} \vDash \gamma\big) \wedge \big(\mathfrak{M} \vDash \neg\varphi\big).$$

Remark (Interpretation). A countermodel shows that the theory does not force the statement. It keeps the axioms true while making the proposed statement false.

Remark (Special cases). If $\Gamma = \emptyset$, then a countermodel to validity of φ is a structure $\mathfrak{M}$ such that $\mathfrak{M} \vDash \neg\varphi$. For a theory T, models of T satisfying φ and models of T satisfying $\neg\varphi$ witness the independence pattern: the theory alone does not settle φ.

Remark (Lean reading). In a mechanized setting, invalidity is witnessed by constructing semantic data:

$$\exists \mathfrak{M}\, \big((\forall \gamma \in \Gamma,\ \mathfrak{M} \vDash \gamma) \wedge \mathfrak{M} \vDash \neg\varphi\big).$$

The constructed structure and satisfaction proofs are the certificate that the entailment cannot be valid.

Remark (Dependencies).

- Model
- Satisfaction

5.5 Consistency

Consistency Toolkit — Quick Reference

Concept	Role
Contradiction	assertion that cannot be true in an interpretation
Consistency	absence of contradiction
Inconsistency	contradiction in or from axioms
Model existence	evidence that axioms fit together
Relative consistency	consistency transferred from another theory

Consistency

Consistency Presentation: A Simple Example

Consider two informal axiom systems.

Example A.

- All objects are red.
- Some object is not red.

These assertions cannot simultaneously hold. The first assertion says every object has the property; the second says at least one object lacks it.

Example B.

- All objects are red.
- Object a is red.

These assertions can hold simultaneously. The second assertion agrees with what the first assertion already requires.

These examples are only for intuition. A consistent axiom system is one whose assertions fit together without contradiction. An inconsistent axiom system is one whose assertions cannot be held together coherently.

Remark (Structural remarks). **Why contradictions matter.** Contradictory systems are unusable because their assertions do not fit together. If a system both requires and forbids the same situation, then it no longer gives a coherent basis for reasoning.

Models as evidence. If a model exists, the axioms hold together coherently in at least one interpretation. A model is therefore evidence that the axioms are mutually compatible.

Absolute vs relative consistency. Most consistency results in mathematics are relative consistency results. They show that one theory is consistent provided some other background theory is consistent. This section records the idea only; the technical development belongs elsewhere.

Forward roadmap. The next section uses models and consistency to introduce independence:

$$\text{Consistency} \Downarrow \text{Independence}$$

Definition (Contradiction)

Definition 5.5.1 (Contradiction)**.** A *contradiction* is a statement that cannot be true under a given interpretation.

Remark (Interpretation). A contradiction marks a breakdown in compatibility. It identifies an assertion that cannot be made true in the interpretation under consideration.

Remark (Dependencies).

- Satisfaction

Definition (Consistent Axiom System)

Definition 5.5.2 (Consistent Axiom System)**.** A *consistent axiom system* is an axiom system that does not lead to contradiction.

Remark (Interpretation). Consistency says that the adopted assertions fit together. A consistent axiom system avoids forcing an impossible situation.

Remark (Dependencies).

- Axiom System
- Contradiction

Definition (Inconsistent Axiom System)

Definition 5.5.3 (Inconsistent Axiom System)**.** An *inconsistent axiom system* is an axiom system containing or producing contradictory assertions.

Remark (Interpretation). Inconsistency means that the adopted assertions cannot all be maintained coherently. The system asks for incompatible requirements at once.

Remark (Dependencies).

- Axiom System
- Contradiction

Definition (Model Existence Criterion (Informal))

Definition 5.5.4 (Model Existence Criterion (Informal))**.** Informally, if an axiom system has a model, then the axioms are mutually compatible.

Remark (Interpretation)*.* This is an informal criterion in this section, not a formal theorem. The idea is that a model provides a setting in which all the axioms hold together, so the axioms have at least one coherent realization.

Remark (Dependencies)*.*

- Model

Definition (Relative Consistency)

Definition 5.5.5 (Relative Consistency)**.** Let T_0 and T_1 be theories. The theory T_1 is *consistent relative to* T_0 if

$$\mathrm{Con}(T_0) \implies \mathrm{Con}(T_1).$$

Equivalently, every contradiction in T_1 would imply a contradiction in T_0.

Remark (Standard quantified statement)*.* For theories T_0 and T_1,

$$\mathrm{RelCon}(T_1, T_0) \iff \big(\neg\exists d_0\,(d_0 : T_0 \vdash \bot)\big) \implies \big(\neg\exists d_1\,(d_1 : T_1 \vdash \bot)\big).$$

Equivalently, in contrapositive form,

$$\exists d_1\,(d_1 : T_1 \vdash \bot) \implies \exists d_0\,(d_0 : T_0 \vdash \bot).$$

Remark (Interpretation)*.* Relative consistency does not establish consistency from nothing. It says that the target theory introduces no new contradiction unless the base theory was already inconsistent. In practice, a relative consistency proof often gives a translation that turns any contradiction derived in T_1 into a contradiction derived in T_0.

Remark (Dependencies)*.*

- Consistent Axiom System

5.6 Independence

Independence Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.7 Comparing Theories

Comparing Theories Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.8 Examples And Previews

Examples And Previews Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.9 Summary

Summary Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

Chapter 6

Proof Techniques

6.1 Proof Architecture

The Two Questions Before You Write Anything. Before writing a single symbol, ask two questions in order.

Question 1. *What is the logical form of the statement?*
Read the claim and identify its outermost logical structure: universal, conditional, biconditional, existential, uniqueness, equality, or structural assertion.

Question 2. *What proof strategy does this form demand?*
The form determines the skeleton of the proof almost mechanically, before any mathematics is done.

Remark 6.1.1 (Why order matters — and why most students get it backwards). Question 2 cannot be answered before Question 1, but almost every beginner tries to answer it first. The default instinct is to reach for a familiar technique — proof by contradiction is the usual reflex — and start writing. This is the wrong order.

Reading the statement *is* the first mathematical act of the proof. The logical form of the statement is not preliminary bookkeeping; it is the information that determines the entire structure of the argument. A proof whose structure mismatches the statement's form is not just inelegant — it is often invalid or at best needlessly circular.

Until you are fluent enough to read the form automatically, slow down deliberately before every proof. Write the statement out. Identify its outermost connective. Then and only then choose a strategy.

Example 6.1.2 (Applying the two questions). **Statement.** *The identity element of a group is unique.*

Q1. Logical form. Strip the natural-language phrasing down to its logical skeleton. "The identity is unique" means: for every group G and for any two elements $e, e' \in G$ that both satisfy the identity axiom, we have $e = e'$. The outermost structure is a universal statement with a uniqueness conclusion: $\forall G,\ \forall e, e' \in G$, [both satisfy identity axiom] $\Rightarrow e = e'$.

Q2. Strategy. A universally quantified uniqueness claim demands the *assume-two-and-compare* pattern: introduce an arbitrary group G, assume two objects e and e' both satisfying the definition, and derive $e = e'$.

The strategy is forced entirely by the form. No creative insight is required to select it. The selection is a consequence of reading.

Remark 6.1.3 (What "logical form" means in practice). You do not need to write out the full first-order formula every time. You need to identify one thing: the *outermost* structure.

Is the statement "for all x, ..."? Then introduce an arbitrary x. Is it "P implies Q"? Then assume P. Is it "there exists x"? Then construct a witness. Is it "there exists a unique x"? Then split into existence and uniqueness. Is it "$A = B$ as sets"? Then prove double inclusion. Is it "$a = b$ as elements in an algebraic structure"? Then either do algebra or use satisfy-and-cite.

The statement map in the next section makes this correspondence explicit.

Statement Forms and Their Proof Strategies. The following table maps every common statement form to the proof strategy it demands. This is not a list of suggestions — it is a mechanical lookup. Once you have identified the outermost logical form of the statement you are trying to prove, the table tells you how to open your proof.

Statement Form	Proof Strategy	Opening Move
$\forall x,\ P(x)$	Introduce arbitrary element	"Let x be arbitrary."
$P \Rightarrow Q$	Direct proof	"Assume P."
$P \Rightarrow Q$	Contrapositive	"Assume $\neg Q$."
$P \Rightarrow Q$ (when negation more useful)	Contradiction	"Assume P and $\neg Q$."
$P \Leftrightarrow Q$	Two separate directions	Prove $P \Rightarrow Q$, then $Q \Rightarrow P$.
$\exists x,\ P(x)$	Construct a witness	"Define $x := \ldots$ and verify $P(x)$."
$\exists!\, x,\ P(x)$	Existence then uniqueness	Construct witness; then assume-two-and-compare.
$A = B$ (sets)	Double inclusion	Prove $A \subseteq B$ and $B \subseteq A$.
$a = b$ (algebraic elements)	Satisfy-and-cite or algebra	Show a satisfies the defining property of b.
$P(n)$ for all $n \in \mathbb{N}$	Induction	Base case; inductive step.
"X is unique"	Assume-two-and-compare	"Let x, y both satisfy the definition."

Remark 6.1.4 (Reading the table: defaults and alternatives). Some statement forms admit more than one strategy, and the table reflects this with multiple rows for the same form.

For $P \Rightarrow Q$, the default is direct proof: assume P and derive Q by forward reasoning. Contrapositive is reached for when $\neg Q$ is more useful than P as an assumption — this happens when the negation of the conclusion is a strong, concrete condition (for example, "n is odd" is concrete; "n is not even" is the same thing but sounds weaker). Contradiction is reached for when neither P alone nor $\neg Q$ alone gets you far, but combining them produces a quick absurdity.

The hierarchy is: try direct proof first. If the hypothesis is hard to use forward, try contrapositive. If neither works, try contradiction. Do not reach for contradiction as a default just because the proof feels difficult.

Remark 6.1.5 (Set equality versus element equality are genuinely different). The two equality rows in the table look similar but demand completely different techniques, and conflating them is a common error.

$A = B$ as *sets* is proved by double inclusion because sets are defined extensionally: two sets are equal if and only if they have the same elements. Proving $A = B$ by any other means — saying they are "clearly the same" or "follow from the definition" — is not a proof. The double inclusion must be made explicit.

$a = b$ as *elements in an algebraic structure* is different. Here b is typically a uniquely-determined object: an inverse, an identity, a limit, a fixed point. The strategy is to show a satisfies the same defining property as b, and then invoke the previously proved uniqueness

theorem to conclude $a = b$. This is the satisfy-and-cite pattern (Section 6.1).

The mistake is trying to use double inclusion on algebraic elements, or trying to use algebraic manipulation on sets. The form of the objects determines which technique applies.

Remark 6.1.6 (The biconditional is always two separate proofs). A biconditional $P \Leftrightarrow Q$ is *never* proved in a single argument that runs from P to Q and "loops back." It is always proved in two distinct proofs:

($\Rightarrow$) Assume P. Prove Q.
($\Leftarrow$) Assume Q. Prove P.

Label the two directions explicitly. The two directions often require completely different techniques and different amounts of work. In characterisation theorems in analysis, one direction is typically a two-line unraveling of a definition while the other requires a nontrivial inequality argument. Treating them as one argument is how gaps get hidden.

The Five Proof Archetypes. Every mathematical proof, at the highest level of compression, is an instance of one of five archetypes. The statement map tells you the specific opening move. The archetypes tell you the *global shape* of the argument — what kind of reasoning is being done, and why it constitutes a valid proof of the claim.

The Five Proof Archetypes

1. **Construct something.** Exhibit a concrete object and verify it has every required property. This is the proof structure for all existence claims.
2. **Assume two and compare.** Assume two objects both satisfy a definition; derive that they are equal. This is the proof structure for all uniqueness claims.
3. **Take a minimal element.** Apply the well-ordering principle to a nonempty set of counterexamples; exploit the minimality of the least element to derive a contradiction. Used in number theory, descent arguments, and equivalence with induction.
4. **Induct.** Verify a base case; show the property propagates to successors. Used whenever the claim concerns all natural numbers or any recursively defined structure.
5. **Chase an arbitrary element.** Take an arbitrary element satisfying one condition; track it through definitions and established facts until it satisfies the target condition. Used for set inclusions, function properties, and direct proofs where the logic flows in a straight line from hypothesis to conclusion.

Remark 6.1.7 (Why five archetypes, not more?). The five archetypes are not a classification chosen for convenience. They correspond to the five fundamental proof obligations that mathematical statements impose:

To prove something *exists*, you must produce it. To prove something is *unique*, you must show that any two instances are equal. To prove a statement holds *for all natural numbers*, the inductive structure of $\mathbb{N}$ provides the only available tool. To prove a *conditional* claim $P \Rightarrow Q$, you must trace an arbitrary element or object satisfying P to a situation where Q

holds. The minimal element archetype is the well-ordering counterpart to induction, available whenever you need to reason about the smallest counterexample.

Everything else in proof writing is refinement of these five. The specific proof structures in Section 6.3 (direct proof, contrapositive, contradiction, cases, existence-uniqueness) are instantiations of these archetypes adapted to specific logical forms. The tactics in Section 6.5 are the move-level tools used to execute whichever archetype applies.

Remark 6.1.8 (Archetypes can nest, and this is a feature). A single proof often uses more than one archetype. The most common nested pattern in abstract algebra is: construct a witness (archetype 1), then invoke a uniqueness theorem (archetype 2) to conclude that the constructed object equals a previously named one.

This is the satisfy-and-cite tactic (Section 6.1). It appears in almost every proof of the form "$a = b$" where b is an algebraically defined object. The nesting is not a complication — it is the standard pattern for proving equalities in algebraic structures, and once recognized, it becomes mechanical.

Another common nesting: induct (archetype 4), and inside the inductive step, use a direct proof argument (archetype 5) to move from $P(n)$ to $P(n+1)$. The outer structure is induction; the inner argument is element-chasing.

Remark 6.1.9 (The archetype identifies the proof's centre of gravity). When you read a finished proof in a textbook, the archetype is usually visible in the first line or two. "Suppose x, y both satisfy the definition" — that is archetype 2. "Define $f : X \to Y$ by $f(x) = \ldots$" — that is archetype 1. "Let $n \in \mathbb{N}$ be arbitrary" followed by induction markup — that is archetype 4. "Let $x \in A$" followed by a chain of implications — that is archetype 5.

Reading proofs actively means identifying the archetype at the start and then watching how the author executes it. This is far more efficient than trying to absorb a proof line by line without a sense of its overall shape.

The Satisfy-and-Cite Pattern. There is a proof pattern that does not appear in any of the standard proof archetypes by name, yet appears constantly — in every chapter of abstract algebra and analysis. Most undergraduates have never been taught it explicitly, which means they either reinvent it poorly each time or miss it entirely and write longer, less clear proofs.

The pattern is called **satisfy-and-cite**. Here is its logic.

The Satisfy-and-Cite Pattern

Setup. You want to prove $a = b$, where b is a *uniquely determined* object: the unique identity, the unique inverse, the unique limit, the unique fixed point, the unique element satisfying some algebraic condition.

Step 1. Show that a satisfies the defining property of b.
Step 2. Cite the previously proved uniqueness theorem: there is exactly one object satisfying that property.
Step 3. Conclude that $a = b$, since b is the unique such object and a is one too.

Remark 6.1.10 (Why this is not the same as assume-two-and-compare). The confusion between

these two patterns is extremely common, so it is worth being precise about what each one does.

The **assume-two-and-compare** pattern *proves* a uniqueness theorem. You use it when the goal is to establish that some object is unique. You say: let x and y both satisfy the definition; then show $x = y$.

The **satisfy-and-cite** pattern *uses* an already proved uniqueness theorem. You use it when you already know some object b is unique and you want to identify a with b. You show a satisfies b's defining property, then invoke uniqueness to collapse a to b.

Assume-two-and-compare appears in the proof that the identity is unique. Satisfy-and-cite appears in every proof that uses that fact afterward. The first creates the tool; the second uses it. Conflating them means writing a new uniqueness proof every time you want to identify two equal objects, which is circular at best and wrong at worst.

Example 6.1.11 (Socks-shoes property: $(ab)^{-1} = b^{-1}a^{-1}$). We want to show $(ab)^{-1} = b^{-1}a^{-1}$ in a group.

The inverse of ab is, by definition, the unique element x satisfying $(ab)x = e$ and $x(ab) = e$.

We compute directly, using only associativity and the inverse axiom:

$$(ab)(b^{-1}a^{-1}) = a(bb^{-1})a^{-1} = aea^{-1} = aa^{-1} = e.$$

Similarly $(b^{-1}a^{-1})(ab) = e$.

So $b^{-1}a^{-1}$ *satisfies the defining property of* $(ab)^{-1}$. By the uniqueness of inverses (which we have already proved), we conclude $(ab)^{-1} = b^{-1}a^{-1}$. □

Notice what did not happen: we never assumed two inverses of ab existed and compared them. We showed one specific element satisfies the inverse condition, then cited uniqueness to identify it with the named inverse.

Example 6.1.12 (Ring theorem: $a \cdot (-b) = -(ab)$). We want to show $a(-b) = -(ab)$ in a ring.

The additive inverse $-(ab)$ is, by definition, the unique element x satisfying $ab + x = 0$.

We compute using distributivity:

$$ab + a(-b) = a(b + (-b)) = a \cdot 0 = 0.$$

So $a(-b)$ *satisfies the defining property of* $-(ab)$. By the uniqueness of additive inverses (previously proved), $a(-b) = -(ab)$. □

Same pattern: compute, satisfy, cite.

Remark 6.1.13 (The pattern across mathematics). Once you see satisfy-and-cite, you will see it everywhere. It appears wherever uniqueness theorems have been proved:

In **algebra**: every proof of the form "this element equals the inverse/identity/zero" uses it. The four examples above are $(ab)^{-1} = b^{-1}a^{-1}$, $a \cdot (-b) = -(ab)$, $-(-a) = a$, and $a \cdot 0 = 0$.
In **analysis**: the uniqueness of limits means that to show $\lim_{n\to\infty} x_n = L$, you verify x_n converges to L by the definition; uniqueness then implies L equals any other expression you have shown to be the limit of the same sequence.
In **linear algebra**: the zero vector is unique; the additive inverse is unique; a linear transformation is uniquely determined by its values on a basis. Each uniqueness theorem creates a new application of satisfy-and-cite.

Each time a uniqueness theorem is proved, it creates a new proof tool that can be activated by satisfy-and-cite for every subsequent argument in that structure. The theorem is not just a fact — it is a lever.

6.2 The Proof Construction Algorithm

Remark 6.2.1 (What this section is for). Architecture (Section 6.1) tells you what kind of proof to write. This section tells you how to write it, line by line, once the architecture is chosen.

The procedure below is intentionally explicit and more detailed than anything you will find in a standard textbook. That is on purpose. With practice, these steps will become internalized and automatic. But until they are — until you have written enough proofs that the process feels natural — working through them consciously will prevent the most common errors: undefined objects, unjustified steps, and forgotten stop conditions.

A longer proof with every step explicit is always better than a shorter proof with implicit gaps.

Step 0: Classify the Statement. Before anything else, identify the logical form of the claim. Common forms:

- Universal: $\forall x\ P(x)$
- Conditional: $P \to Q$
- Biconditional: $P \leftrightarrow Q$
- Existential: $\exists x\ P(x)$
- Existence-and-uniqueness: $\exists!\, x\ P(x)$
- Set equality: $A = B$
- Algebraic equality: $a = b$
- Structural assertion: "f is injective," "R is an equivalence relation"

The logical form determines the skeleton of the proof. Consulting the statement map (Section 6.1) is the direct link from this classification to an opening strategy.

Step 1: Restate the Givens. Write out explicitly what is given. Introduce every set, relation, function, and algebraic structure, and record any properties assumed about them.

> Let G be a group. Let $a \in G$.

This is not a formality. By recording the givens, you license their use in the proof — you can now invoke associativity, the existence of inverses, and the identity element without reintroducing them each time. What is not in the givens cannot be used.

Step 2: Identify Objects and Their Types. Before reasoning begins, verify the *type* of each object you will work with: is it an element, a set, a function, or a relation? What is its ambient universe? What operations apply to it?

$$x \in A, \quad f : A \to B, \quad R \subseteq A \times A.$$

This step prevents the most common logical errors in undergraduate proofs: writing $x \in f$ when you mean $x \in \mathrm{dom}(f)$; writing $A = x$ when you mean $x \in A$; confusing membership ($\in$) with inclusion ($\subseteq$). Every object used in a proof must have a declared type. If you cannot say what type something is, you do not yet understand what you are working with.

Step 3: Introduce Arbitrary Elements. If the claim is universal ($\forall x\ P(x)$), immediately introduce an arbitrary element:

$$\text{Let } x \in A \text{ be arbitrary.}$$

The word "arbitrary" is not decorative. It means the element is not given any properties beyond those of the set A. Whatever you prove about this x will hold for all $x \in A$, precisely because you have assumed nothing about it beyond membership. The moment you specialize — "let $x = 0$" or "let x be such that ..." — you have lost the ability to conclude the universal statement.

Step 4: Expand the Goal. Rewrite the conclusion using definitions rather than named concepts.

- To prove f is injective: expand injectivity as "$f(a) = f(b) \Rightarrow a = b$" and take that as the new goal.
- To prove $A = B$ (sets): take as new goal "$A \subseteq B$ and $B \subseteq A$."
- To prove $x \in A \cup B$: rewrite the goal as "$x \in A$ or $x \in B$."
- To prove $\{x_n\}$ converges: expand to "$\forall\, \varepsilon > 0\ \exists\, N$ s.t. $n \geq N \Rightarrow d(x_n, L) < \varepsilon$."

This step is the one most students skip, and skipping it is the single most reliable predictor of a stuck proof. You cannot make progress toward a goal you have not named in operative terms. "Prove f is injective" is not an operative goal. "Prove that $f(a) = f(b)$ implies $a = b$" is.

Step 5: Introduce Helper Objects. Introduce any auxiliary objects needed for the argument: witnesses for existential claims, intermediate elements for indirect arguments, bounds for inequality proofs, function values.

$$\text{Let } N_1 \in \mathbb{N} \text{ s.t. } n \geq N_1 \Rightarrow d(x_n, L) < \varepsilon/2.$$

No object should appear in a proof without being explicitly introduced. If you write "let c be the cancellation element" without first establishing that such an element exists, your proof has an undefined object. Every name carries an obligation: the obligation to have said what the name refers to.

Step 6: Apply One Rule at a Time. Each line of a proof follows from exactly one of four sources:

1. A definition (unpacking what a term means),
2. A hypothesis (something assumed in the current proof),
3. A previously proved theorem (cited by name and reference),
4. A basic logical rule (modus ponens, universal instantiation, etc.).

Lines that are justified by "clearly" or "obviously" or no reason at all are gaps. At the learning stage, every step should have an explicit justification. When a proof stalls, the right question is always: which of the four sources has not been consulted? Almost always, the answer is that some definition has not been unpacked.

Step 7: Use Forward and Backward Reasoning. Proofs alternate between two modes of reasoning, and recognizing which mode is productive at each stage is a significant skill.

Forward reasoning: deduce consequences from what you have established so far. If you know $a^2 = e$ in a group, derive that $a(ae) = a \cdot a \cdot e = a^2 \cdot e = e$. You are moving away from the hypotheses, toward the goal.

Backward reasoning: examine the goal and ask what would suffice to establish it. If the goal is "$a = a^{-1}$," ask: by what definition would that follow? It follows if $aa = e$, since that is the condition for a to be its own inverse. Now your new goal is to prove $aa = e$, which may be more directly accessible.

In practice, most proofs proceed in both directions simultaneously: you derive forward from the hypotheses while simplifying the goal backward until the two sides meet. When you are stuck, try the direction you have not been working in.

Step 8: Handle Cases Explicitly. If a statement splits into mutually exclusive and exhaustive cases, enumerate all of them.

$$\text{Either } n \text{ is even, or } n \text{ is odd.}$$

Handle each case separately. Close each case before opening the next. Verify, explicitly, that the cases together cover every possibility. A proof with unlabeled cases — where it is unclear which case is being argued or whether all cases have been covered — is not a proof.

Step 9: Close the Argument. Once the desired conclusion has been established, state it explicitly:

- "Thus $x \in B$, and so $A \subseteq B$."
- "Hence $f(a) = f(b)$ implies $a = b$, so f is injective."
- "In both cases $P(n+1)$ holds."

The explicit closing step serves two functions. First, it tells the reader (and you) that the proof goal has been achieved. Second, it forces you to check that what you have proved actually *is* the goal. Students who reach the end of a proof and write □without stating a conclusion often discover, on re-reading, that they proved something adjacent to the goal but not the goal itself.

Step 10: Signal Completion. Conclude with □, ■, or "This completes the proof."

Legal Moves and Stop Conditions

What you may never do:

- Choose an existential witness before proving it exists.
- Introduce an "arbitrary" element and then give it special properties.
- Assume the conclusion at any point in the proof.
- Apply a definition partially (all conditions must be checked).

A proof is complete when:

- A universal claim has been proved for a genuinely arbitrary element.
- An existential claim has produced a verified witness.
- A set equality has established both inclusions explicitly.
- Each direction of a biconditional has been proved separately.
- An existence-uniqueness claim has done both parts.

Line-by-Line Discipline. Before writing each line, ask three silent questions:

(i) Has every object mentioned in this line been defined or introduced earlier?

(ii) Which definition, hypothesis, or theorem justifies this step?

(iii) Does this step move the argument toward the stated goal?

If any answer is no, stop and resolve it before continuing.

6.3 Proof Structures

Direct Proof. A direct proof of $P \Rightarrow Q$ assumes P and derives Q through a chain of valid inferences.

Template: Direct Proof

Given. P.
Goal. Q.

Assume P.
⋮ [expand definitions; apply lemmas; derive consequences]
Therefore Q. □

Remark 6.3.1 (Direct proof is the default — always try it first). Direct proof is not one option among equals. It is the default. Every time you sit down to prove $P \Rightarrow Q$, the first thing you write is "Assume P" and the first thing you do is unpack what P says. You use what P gives you, apply definitions and previously proved results, and derive Q.

Reach for contrapositive or contradiction only when direct proof has genuinely stalled — when you have unpacked the hypothesis and the goal and there is no visible path between them. Do not reach for contradiction just because the statement feels hard. Most statements that feel hard are hard only because a definition has not been unpacked.

Remark 6.3.2 (The anatomy of a direct proof). Every direct proof has three phases that correspond to the first four steps of the construction algorithm:

Phase 1 — Unpack the hypothesis. P is a named concept or a logical statement. Expand it into its operative form. If P says "G is a group," this licenses associativity, the existence of an identity, and the existence of inverses. Write those down as available tools.

Phase 2 — Expand the goal. Q is also a named concept. Expand it into what you need to establish. If Q says "$a = a^{-1}$," the operative target is showing a satisfies the defining property of an inverse: $aa = e$.

Phase 3 — Bridge the gap. Apply definitions, lemmas, and axioms to move from the expanded hypothesis to the expanded conclusion.

When Phase 3 stalls, the cause is almost always that Phase 1 or Phase 2 was not completed. The most common error is having a vague goal ("show $a = a^{-1}$") without having written out what that means in terms of the group axioms.

Example 6.3.3 (Direct proof in a group). **Claim.** In a group G, if $a^2 = e$ then $a = a^{-1}$.

Proof. Assume $a^2 = e$, meaning $aa = e$.

We want to show $a = a^{-1}$, which by definition of inverse means we need $aa = e$ and $ea = a$ (or equivalently, a is its own inverse).

Multiply both sides of $aa = e$ on the right by a^{-1}:

$$(aa)a^{-1} = e \cdot a^{-1}.$$

By associativity: $a(aa^{-1}) = a^{-1}$. By the inverse axiom: $ae = a^{-1}$. By the identity axiom: $a = a^{-1}$.

Hence $a^2 = e$ implies $a = a^{-1}$. □

Each step is justified by exactly one of: the hypothesis ($aa = e$), the inverse axiom ($aa^{-1} = e$), the identity axiom ($ae = a$), or associativity. No step is unjustified.

Proof by Contrapositive. The contrapositive of $P \Rightarrow Q$ is $\neg Q \Rightarrow \neg P$. These two statements are logically equivalent: one is true if and only if the other is. A proof by contrapositive establishes $\neg Q \Rightarrow \neg P$ by a direct argument, and since this is equivalent to $P \Rightarrow Q$, the original claim follows.

Template: Proof by Contrapositive

Goal. $P \Rightarrow Q$.
Equivalent goal. $\neg Q \Rightarrow \neg P$.

Assume $\neg Q$.

$\vdots$ [derive $\neg P$ from $\neg Q$]
Therefore $\neg P$.
Hence $P \Rightarrow Q$. □

Remark 6.3.4 (When contrapositive is the right tool). Contrapositive is useful when the negation of the conclusion is more concrete and easier to work with than the hypothesis.

The canonical example is injectivity. To prove $P \Rightarrow Q$ where P says "$f(a) = f(b)$" and Q says "$a = b$": the contrapositive is $a \neq b \Rightarrow f(a) \neq f(b)$, which may be directly accessible if the function is order-preserving or explicitly defined.

Another signal: when Q is the statement that some object exists or some set is non-empty. The negation $\neg Q$ says no such object exists, which may be a very restrictive and workable condition.

The general diagnostic: after writing $\neg Q$, look at what it says. If it gives you a strong, concrete assumption — something you can actually manipulate — contrapositive is likely the right tool. If $\neg Q$ is vague or complicated, try a different approach.

Remark 6.3.5 (Contrapositive versus contradiction: the essential difference). Students routinely conflate contrapositive and contradiction. They are not the same, and knowing the difference matters for clarity.

In a **contrapositive** proof, you assume *only* $\neg Q$ and derive $\neg P$. The hypothesis P is never mentioned. The logical engine is: you proved $\neg Q \Rightarrow \neg P$, which by equivalence gives $P \Rightarrow Q$.

In a **contradiction** proof, you assume *both* P and $\neg Q$ simultaneously and derive *any* absurdity — not necessarily $\neg P$. The logical engine is: the assumption $P \wedge \neg Q$ leads to a false statement, so $P \wedge \neg Q$ must itself be false, so $P \Rightarrow Q$.

When contrapositive works, it is cleaner: you need one assumption, not two, and the conclusion you derive ($\neg P$) is specific rather than "any contradiction." Always try contrapositive before reaching for contradiction.

Example 6.3.6 (Contrapositive for divisibility). **Claim.** If n^2 is even then n is even.

Proof (contrapositive). We prove the contrapositive: if n is odd then n^2 is odd.

Assume n is odd. Then $n = 2k + 1$ for some $k \in \mathbb{Z}$.

$$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which has the form $2m + 1$ and is therefore odd.

Hence n odd $\Rightarrow$ n^2 odd, which is the contrapositive of n^2 even $\Rightarrow$ n even. □

Notice: the hypothesis "n^2 is even" was never used. We proved the contrapositive directly, and the original implication follows by logical equivalence. There is no need to loop back and mention the original statement; the contrapositive suffices.

Proof by Contradiction. To prove P, assume $\neg P$ and derive a statement that contradicts a known truth: a hypothesis, an axiom, or a previously proved theorem. Since the assumption $\neg P$ led to a false statement, $\neg P$ is false, and therefore P is true.

Template: Proof by Contradiction

Goal. P.

Suppose for contradiction that $\neg P$.

⋮ [derive consequences; identify the absurdity]

This contradicts [name the violated fact].

Therefore P. □

Remark 6.3.7 (The logic: why it works). Contradiction relies on the law of the excluded middle: every statement is either true or false. To prove P, it suffices to show $\neg P$ is false. You show $\neg P$ is false by deriving from it a statement that is known to be false — a contradiction.

The contradiction can be anything: $0 = 1$, a set that is both empty and non-empty, a number that is simultaneously even and odd, an inequality $x < x$. The only requirement is that the contradicted fact was genuinely established before the current proof began.

This is the move that students most often get wrong: they think they have derived a contradiction when they have merely derived something surprising. A statement being counterintuitive is not a contradiction. The contradicted fact must be something you can point to: a hypothesis, an axiom, a definition, or a theorem with a name.

Remark 6.3.8 (When contradiction is the right tool). Contradiction is best reserved for claims whose negation is a strong, productive assumption. The archetypes where contradiction excels are:

Irrationality proofs. Assuming $\sqrt{2} \in \mathbb{Q}$ gives you a fraction in lowest terms; arithmetic then forces the numerator and denominator to be simultaneously even, contradicting the lowest-terms assumption.

Infinitude arguments. Assuming there are finitely many primes lets you list them; constructing a number not divisible by any of them contradicts the assumption that the list was complete.

Nonexistence results. If you want to show no object satisfies some property, assume one exists and derive an absurdity.

Uniqueness (via assume-two-and-compare). Assuming two distinct objects both satisfy a uniqueness condition produces a contradiction from the definitions.

Avoid contradiction as a default. When a direct proof or contrapositive works, prefer it: the argument is shorter and the logical structure is more transparent. Contradiction is a sign that the natural direction of inference does not work, and that you need to reason from an impossible assumption instead.

Remark 6.3.9 (Always name the contradiction explicitly). The closing line of a contradiction proof must name what has been contradicted. "This is a contradiction" with no further specification is a red flag: it suggests the student either does not know what was contradicted, or is hoping the reader will not check.

Write: "This contradicts the assumption that $\gcd(p, q) = 1$." Write: "This contradicts Theorem X, which asserts ..." Write: "This contradicts $m \in S$ being the least element."

Naming the violated fact performs two functions: it confirms that the contradiction is genuine, and it locates the logical break in the argument so that anyone reading the proof can verify it.

Example 6.3.10 ($\sqrt{2}$ is irrational). *Proof.* Suppose for contradiction that $\sqrt{2} \in \mathbb{Q}$. Write $\sqrt{2} = p/q$ where $p, q \in \mathbb{Z}$, $q \neq 0$, and $\gcd(p, q) = 1$ (in lowest terms; this is possible by the definition of $\mathbb{Q}$).

Squaring: $2q^2 = p^2$. So p^2 is even, which forces p to be even (by the contrapositive example above). Write $p = 2k$.

Then $2q^2 = (2k)^2 = 4k^2$, so $q^2 = 2k^2$. So q^2 is even, which forces q to be even.

But then both p and q are even, so $\gcd(p, q) \geq 2$. This contradicts $\gcd(p, q) = 1$.

Therefore $\sqrt{2} \notin \mathbb{Q}$. □

The structure is: assume $\neg P$ (i.e., $\sqrt{2} \in \mathbb{Q}$); extract information from this assumption (lowest-terms representation); derive a consequence (both p and q even); identify the contradiction (conflicts with lowest terms); conclude P.

Proof by Cases. If the hypotheses or conclusion naturally divide into a finite number of mutually exclusive and collectively exhaustive alternatives, handle each case separately.

Template: Proof by Cases

Goal: Prove Q, given that exactly one of $P_1, P_2, \ldots, P_k$ holds (and one of them always holds).

Verify exhaustiveness: Every possibility falls under some P_i.
Verify mutual exclusivity (when needed).

Case 1: Assume P_1. [Argue.] Hence Q.

Case 2: Assume P_2. [Argue.] Hence Q.

⋮

Case k: Assume P_k. [Argue.] Hence Q.

Since $P_1, \ldots, P_k$ are exhaustive and Q holds in each case, Q holds unconditionally. □

Remark 6.3.11 (The exhaustiveness obligation is non-negotiable). A case proof is only valid when the cases together cover every possibility. Always verify exhaustiveness explicitly, even when it feels obvious.

The most common exhaustive splits:

- n is even, or n is odd.

- $a > 0$, $a = 0$, or $a < 0$.
- $a \mid b$, or $a \nmid b$.
- $x \in A$, or $x \notin A$.
- $x = y$, or $x \neq y$.

Writing "we consider two cases" without naming what they are, or naming cases that do not cover all possibilities, is a structural error. The proof is not valid until exhaustiveness is established.

Remark 6.3.12 (Each case is a self-contained argument). Inside each case, you have an additional assumption: P_i is true. That assumption is local to the case. You may use it within Case i, but you may not carry it into Case j.

When you close a case — "hence Q, completing Case 1" — you are signalling that the local assumption is no longer in scope. Case 2 starts fresh, with only the original given plus the assumption P_2.

This discipline prevents the most common case-analysis error: drifting from one case into another by accidentally retaining a local assumption beyond its scope.

Remark 6.3.13 (Symmetric cases). When two cases are exactly symmetric — when the argument for Case 2 is obtained from the argument for Case 1 by renaming variables, with no other change — it is acceptable to write "Case 2 is symmetric to Case 1 by swapping a and b." But be careful: only write this when the symmetry is complete. A partial symmetry that requires a different step at one point must be written out in full.

Example 6.3.14 (Parity of $n(n+1)$). **Claim.** For every $n \in \mathbb{Z}$, $n(n+1)$ is even.

Proof. Every integer is either even or odd (these cases are exhaustive and mutually exclusive).

Case 1: n is even. Write $n = 2k$ for some $k \in \mathbb{Z}$. Then $n(n+1) = 2k(n+1)$. Since this is 2 times an integer, $n(n+1)$ is even.

Case 2: n is odd. Write $n = 2k+1$ for some $k \in \mathbb{Z}$. Then $n+1 = 2k+2 = 2(k+1)$, so $n(n+1) = (2k+1) \cdot 2(k+1)$. This is 2 times an integer, so $n(n+1)$ is even.

In both cases $n(n+1)$ is even. Since every integer falls into one of these cases, the proof is complete. □

Notice that the cases were named before the argument, the exhaustiveness was verified ("every integer is even or odd"), and each case was closed explicitly before moving to the next.

Example 6.3.15 (Case split induced by a definition). **Claim.** The discrete metric satisfies the triangle inequality.

Proof. Let $x, y, z \in X$. We must show $d(x,z) \leq d(x,y) + d(y,z)$.

Case 1: $x = z$. Then $d(x,z) = 0$. Since $d(x,y) \geq 0$ and $d(y,z) \geq 0$, we have $0 \leq d(x,y) + d(y,z)$.

Case 2: $x \neq z$. Then $d(x,z) = 1$. We cannot have both $x = y$ and $y = z$, for that would force $x = z$. So at least one of $d(x,y), d(y,z)$ equals 1, hence $d(x,y) + d(y,z) \geq 1 = d(x,z)$.

The cases $x = z$ and $x \neq z$ are exhaustive. The triangle inequality holds in both cases. □

The case split here is induced by the two-value structure of the discrete metric: distances are 0 or 1. The right case split to use is the one naturally suggested by the definition you are working with.

Existence and Uniqueness. The statement $\exists!\, x\ P(x)$ asserts two things simultaneously: that some object satisfying P exists, and that it is the *only* such object. These two obligations are proved separately, in order.

Template: Existence and Uniqueness ($\exists!\, x\ P(x)$)

Step 1 — Existence.
Define $x :=$ [explicit construction].
Verify that $P(x)$ holds. (Check every condition in P.)

Step 2 — Uniqueness (assume-two-and-compare).
Let x and y be arbitrary objects both satisfying P.
Apply the definition of x with y as input; apply the definition of y with x as input. The two applications collapse.
Conclude $x = y$.

Step 3 — Conclude.
By Steps 1 and 2, there exists exactly one object satisfying P. □

Remark 6.3.16 (Existence must be proved before uniqueness). The uniqueness argument has the form: "let x and y both satisfy P." This assumption is only meaningful if we already know that at least one object satisfying P exists. Without Step 1, the uniqueness argument is vacuous: we are asserting something about objects that might not exist.

This is not pedantry. There are statements where the existence fails and the uniqueness argument would appear to go through — but the whole proof would then be establishing uniqueness for an empty set of objects, which is trivially true and useless. Existence first is not just a convention; it is logically necessary.

Remark 6.3.17 (The key move in assume-two-and-compare). The assume-two-and-compare argument has a characteristic structure that appears across mathematics. The critical move is this: if x and y both satisfy the defining property P, apply x's property with y as the argument, and simultaneously apply y's property with x as the argument. The two applications collapse to $x = y$.

Here is the pattern in the group identity example: if e satisfies the identity axiom, then $e \cdot e' = e'$ (using e as identity with argument e'). If e' satisfies the identity axiom, then $e \cdot e' = e$ (using e' as identity with argument e). Combining: $e' = e$.

The same move works for: uniqueness of inverses, uniqueness of the zero vector, uniqueness of limits, uniqueness of the supremum. In every case, the two assumed objects are "played against each other": each one's defining property is applied with the other as input.

Example 6.3.18 (Uniqueness of the group identity). *Proof.* Let G be a group. Suppose e and e' both satisfy the identity axiom:

$$e \cdot g = g \cdot e = g \quad \text{for all } g \in G,$$

$$e' \cdot g = g \cdot e' = g \quad \text{for all } g \in G.$$

Apply e''s identity property with $g = e$:

$$e' \cdot e = e.$$

Apply e's identity property with $g = e'$:

$$e \cdot e' = e'.$$

But $e' \cdot e = e \cdot e'$ (both equal the product $e \cdot e'$ in G by commutativity — actually, both equal $e \cdot e'$ directly). Actually, more directly: $e = e' \cdot e = e'$ where the first step uses e' as identity and the second uses e as identity. Hence $e = e'$. □

The single line is: $e = e \cdot e' = e'$. First equality: e acts as identity on e'. Second equality: e' acts as identity on e. Both identities applied to the same product; both sides collapse to the same expression; $e = e'$.

Remark 6.3.19 (Why proving uniqueness creates a proof tool). Every uniqueness theorem, once proved, creates a new application of the satisfy-and-cite tactic. This is the connection that most students miss.

Proving uniqueness says: there is exactly one object satisfying P. This means: if you can ever show that some element a satisfies P, you automatically get $a = b$, where b is the canonical name for the unique object satisfying P.

The group inverse provides the clearest example. Once you have proved that inverses are unique, the pattern for proving $(ab)^{-1} = b^{-1}a^{-1}$ is immediate: show $b^{-1}a^{-1}$ satisfies the defining property of the inverse of ab, then cite uniqueness. You never need to set up another assume-two-and-compare argument. The uniqueness theorem absorbs all future comparisons.

This is why uniqueness results are not just facts — they are *tools*. Prove them early; use them everywhere afterward.

6.4 Mathematical Induction

Why Induction Works. Mathematical induction is often presented as a technique: base case, inductive step, done. That presentation is useful but incomplete. Induction *feels* like a procedure, but it is actually a theorem, and it is worth understanding why it is true before using it.

Remark 6.4.1 (The Peano axiom behind induction). The natural numbers are characterized by the Peano axioms. The critical one for induction is Axiom P5:

If $A \subseteq \mathbb{N}$ satisfies (i) $0 \in A$ and (ii) $n \in A \Rightarrow S(n) \in A$, then $A = \mathbb{N}$.

This axiom says that $\mathbb{N}$ is the *smallest* set containing 0 and closed under the successor function S. There is no proper subset of $\mathbb{N}$ with these two properties.

The induction principle is a direct translation. Let $P(n)$ be any statement. Define $A = \{n \in \mathbb{N} : P(n)\}$. If $P(0)$ holds, then $0 \in A$. If $P(n) \Rightarrow P(S(n))$ for every n, then A is closed under S. By P5, $A = \mathbb{N}$. Therefore $P(n)$ holds for every $n \in \mathbb{N}$.

The full development of this from the Peano axioms is in the Axiom Systems chapter. Here, the takeaway is: induction is not a trick. It is a theorem derivable from the definition of $\mathbb{N}$.

Remark 6.4.2 (Why the base case is not optional). The inductive step alone says: *if* $P(n)$ holds for some n, then $P(n+1)$ holds. This is a conditional, not an assertion. Without a

base case, there is no n for which $P(n)$ is known to hold. The chain of conditionals has no starting point, and the proof produces nothing.

The most famous illustration is the false theorem "all horses are the same colour." A flawed argument runs: base case trivial ($n = 1$), inductive step [fatally flawed reasoning]. The lesson is not just that the step is wrong — it is that even a correct base case does not rescue a bad step, and even a correct step does not rescue a missing base case. Each part is independent; both are necessary.

In terms of P5: the base case establishes $0 \in A$. The inductive step establishes closure under S. Both conditions of P5 are required for the conclusion $A = \mathbb{N}$.

Remark 6.4.3 (The domino picture and its limitations). The common picture of induction as an infinite row of falling dominoes is a useful mnemonic but a misleading explanation. It suggests that induction requires each domino to knock over the next, which captures the inductive step. But it does not explain why the argument is valid — the validity comes from P5, not from a physical metaphor.

More importantly, the domino picture suggests that the steps happen *in time*: first domino 0 falls, then 1, then 2, and so on. But induction proves a statement about *all* natural numbers simultaneously, not by a sequential process. The proof is instantaneous: once you have established the base case and the inductive step, the conclusion $P(n)$ for all n follows at once from P5.

Use the domino picture as memory aid. For understanding, use P5.

Remark 6.4.4 (Induction and well-ordering are equivalent). The induction principle and the well-ordering principle for $\mathbb{N}$ (every nonempty subset has a least element) are logically equivalent. This means every induction argument can be rephrased as a minimal-counterexample argument, and vice versa. Both are valid proof tools. The well-ordering approach is sometimes more natural when you want to reason about the *first* failure rather than climbing from a base case. The equivalence is proved in Section 6.4.

Theorem: Weak (Standard) Induction

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$. Suppose:

1. **Base case.** $P(n_0)$ is true for some starting value n_0.
2. **Inductive step.** For all $n \geq n_0$, $P(n) \Rightarrow P(n+1)$.

Then $P(n)$ is true for all $n \geq n_0$.

Template: Weak Induction Proof

Let $P(n)$ denote the statement: [*write it out completely as a sentence*].

Base case ($n = [n_0]$). [*Verify $P(n_0)$ directly.*] Hence $P(n_0)$ holds.

Inductive step. Let $n \geq n_0$ and assume $P(n)$ holds. [*State explicitly what $P(n)$ says.*] We show $P(n+1)$ holds.
[*Rewrite $P(n+1)$ to expose $P(n)$ inside it. Substitute the inductive hypothesis. Simplify.*] Hence $P(n+1)$ holds.

By induction, $P(n)$ holds for all $n \geq n_0$. □

Remark 6.4.5 (The most important discipline: write $P(n)$ as a sentence). The single most impactful habit in induction proofs is writing the inductive hypothesis as a *complete mathematical sentence* before the inductive step begins.

"Assume $P(n)$ holds" is not sufficient if you have not first written out what $P(n)$ says. Write: "Assume $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$." That sentence is your inductive hypothesis. It is the precise tool you have available in the inductive step.

Without this sentence, students typically make one of two errors: they try to use $P(n)$ without knowing what it says, guessing at it from context; or they prove the inductive step using information beyond what $P(n)$ supplies, making the proof circular or invalid. Write the sentence. It takes five seconds and eliminates the most common induction error.

Remark 6.4.6 (The core skill of the inductive step). Every inductive step involves the same core action: *rewrite $P(n+1)$ to expose $P(n)$ inside it, then substitute the inductive hypothesis.*

For sum formulas, this means: split off the last term.

$$\sum_{k=1}^{n+1} k = \left(\sum_{k=1}^{n} k\right) + (n+1).$$

Now substitute $P(n)$: replace $\sum_{k=1}^{n} k$ with $\frac{n(n+1)}{2}$. Simplify to get the $P(n+1)$ form.

For divisibility, this means: write $4^{n+1} - 1 = 4(4^n - 1) + 3$ to expose $4^n - 1$ inside $4^{n+1} - 1$.

In both cases, the step is: factor out the $P(n)$ term, substitute, and verify the arithmetic produces $P(n+1)$.

When the rewriting step is not obvious, it usually means $P(n)$ needs to be strengthened (see Section 6.4).

Example 6.4.7 (Sum formula). **Proposition.** For all $n \geq 1$: $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

Proof. Let $P(n)$ denote: $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

Base case ($n = 1$). $\sum_{k=1}^{1} k = 1 = \frac{1 \cdot 2}{2}$. Hence $P(1)$ holds.

Inductive step. Let $n \geq 1$ and assume $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$. *(Inductive hypothesis.)*

We show $\displaystyle\sum_{k=1}^{n+1} k = \frac{(n+1)(n+2)}{2}$.

Split off the last term:

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \left(\sum_{k=1}^{n} k\right) + (n+1) \\ &= \frac{n(n+1)}{2} + (n+1) && \text{(inductive hypothesis)} \\ &= (n+1)\left(\frac{n}{2}+1\right) = \frac{(n+1)(n+2)}{2}.\end{aligned}$$

Hence $P(n+1)$ holds. By induction, $P(n)$ holds for all $n \geq 1$. □

Example 6.4.8 (Divisibility). **Proposition.** For all $n \geq 0$, $3 \mid (4^n - 1)$.

Proof. Let $P(n)$ denote: $3 \mid (4^n - 1)$.

Base case ($n = 0$). $4^0 - 1 = 0 = 3 \cdot 0$. Hence $3 \mid 0$ and $P(0)$ holds.

Inductive step. Assume $3 \mid (4^n - 1)$, i.e., $4^n - 1 = 3m$ for some $m \in \mathbb{Z}$.

Expose $4^n - 1$ inside $4^{n+1} - 1$:

$$4^{n+1} - 1 = 4 \cdot 4^n - 1 = 4(4^n - 1) + 4 - 1 = 4(4^n - 1) + 3 = 4 \cdot 3m + 3 = 3(4m + 1).$$

Hence $3 \mid (4^{n+1} - 1)$ and $P(n+1)$ holds.

By induction, $P(n)$ holds for all $n \geq 0$. □

The rewriting step $4^{n+1} - 1 = 4(4^n - 1) + 3$ is the key move: it exposes the inductive hypothesis $3 \mid (4^n - 1)$ inside the goal $3 \mid (4^{n+1} - 1)$.

Choosing $P(n)$: The Hard Part. In textbook problems, the statement $P(n)$ is usually handed to you: the claim is explicitly "prove that $\sum_{k=1}^{n} k = n(n+1)/2$", and $P(n)$ is that equality. In original proofs, you must construct $P(n)$ from the statement of the goal. And sometimes even when $P(n)$ is given, the straightforward choice fails and must be strengthened.

Remark 6.4.9 (Rule 1: $P(n)$ mirrors the logical form of the goal). Every claim to be proved by induction has the form $\forall n,\ \Phi(n)$ for some predicate Φ. Strip the universal quantifier: what remains is $P(n)$.

Goal form	**Shape of $P(n)$**
$\forall n,\ f(n) = g(n)$	Equality: $P(n) :\equiv f(n) = g(n)$
$\forall n,\ A(n) \Rightarrow B(n)$	Conditional: $P(n) :\equiv A(n) \Rightarrow B(n)$
$\forall n,\ A(n) \Leftrightarrow B(n)$	Biconditional: $P(n) :\equiv A(n) \Leftrightarrow B(n)$

Do not simplify. Do not add conditions. Transcribe the logical form of $\Phi(n)$ exactly into $P(n)$.

Remark 6.4.10 (Rule 2: induct on the recursive variable). When the goal involves a recursively defined operation, identify which variable the definition recurses on. Induct on that variable and hold all others fixed as arbitrary.

To find the recursive variable: locate the definition of the operation and identify which variable is reduced in the recursive clause.

Operation	**Recursive clause**	**Induct on**
Addition (Peano)	$(n\texttt{++}) + m := (n + m)\texttt{++}$	n
Multiplication (Peano)	$(n\texttt{++}) \times m := (n \times m) + m$	n
Exponentiation	$m^{n\texttt{++}} := m^n \times m$	n

Remark 6.4.11 (The strengthening principle: when $P(n)$ is too weak). Sometimes the natural $P(n)$ fails not because the statement is false but because the inductive hypothesis is insufficient to prove the step. The fix is counterintuitive: *add more to* $P(n)$, making the claim stronger.

This works because a stronger $P(n)$ gives you more in the inductive hypothesis, which is what you actually need to prove $P(n+1)$. The stronger $P(n)$ is harder to prove overall but easier to propagate.

The signal that strengthening is needed: you write the inductive step, you need to use some fact about the sequence that $P(n)$ alone does not provide, and you find yourself thinking "if only I also knew $P(n-1)$." That thought is the signal to switch to strong induction (which implicitly strengthens $P(n)$ to "$P(k)$ for all $k \leq n$").

Example 6.4.12 (Strengthening: Fibonacci bound requires two prior values). **Goal.** Show that the Fibonacci sequence satisfies $F_n < 2^n$ for all $n \geq 1$.

Naive attempt. Let $P(n)$: "$F_n < 2^n$."

Inductive step: assume $F_n < 2^n$; show $F_{n+1} < 2^{n+1}$.

We have $F_{n+1} = F_n + F_{n-1}$. We know $F_n < 2^n$ by the inductive hypothesis. But what do we know about F_{n-1}? Nothing: $P(n)$ says nothing about F_{n-1}. The step is stuck.

Fix: strong induction gives two terms. Use the strong inductive hypothesis: $F_k < 2^k$ for all $k \leq n$. Then:

$$F_{n+1} = F_n + F_{n-1} < 2^n + 2^{n-1} < 2^n + 2^n = 2^{n+1}.$$

The step completes because strong induction provides $F_{n-1} < 2^{n-1}$. This is the canonical signal that strong induction is needed: the recursive step requires knowing $P(k)$ for some $k < n$ other than $k = n - 1$.

Remark 6.4.13 (Vacuous base cases in conditional statements). When $P(n)$ is a conditional and its hypothesis is false at the base value, the base case is vacuously true. This is expected and requires no further argument.

For example, if the claim is "for all $n \geq 1$, if $n > 0$ and $m > 0$ then $nm > 0$" and you induct starting at $n = 0$, then $P(0)$: "if $0 > 0$ and $m > 0$ then $0 \times m > 0$" has a false hypothesis ($0 > 0$ is false), so $P(0)$ is vacuously true.

This signals that the claim only has content for values of n where the hypothesis holds. The induction then establishes the claim for all n where the hypothesis is satisfied.

Remark 6.4.14 (Diagnostic questions when stuck choosing $P(n)$).

1. What exactly do I need to prove for $n = 0, 1, 2, 3$? Write it out. The pattern of $P(n)$ will appear.
2. Does the inductive step require knowing $P(n-1)$ as well as $P(n)$? If yes, switch to strong induction.
3. Does the inductive step produce a bound or equation slightly weaker than $P(n+1)$? If yes, strengthen $P(n)$.
4. After writing $P(n\texttt{++})$, can I apply the recursive definition to expose $P(n)$? If not, the wrong variable may have been chosen.

Theorem: Strong Induction

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$. Suppose that for every $n \in \mathbb{N}$:

$$\Big(\forall k < n,\ P(k)\Big) \Rightarrow P(n).$$

Then $P(n)$ holds for all $n \in \mathbb{N}$.

Remark 6.4.15 (Strong and weak induction prove the same statements). Strong induction is sometimes described as "more powerful" than weak induction, but this is imprecise. Both principles are logically equivalent (they both follow from P5 and each implies the other). Strong induction is not more powerful in the sense of proving statements that weak induction cannot — every statement provable by one is provable by the other.

What strong induction offers is convenience: when the proof of $P(n+1)$ genuinely requires knowing $P(k)$ for some $k < n$ that is not $n-1$, strong induction supplies that information directly. Rephrasing the proof as weak induction would require artificially strengthening $P(n)$ to "$P(k)$ for all $k \leq n$", which is precisely the hypothesis that strong induction builds in automatically.

Use strong induction when the step needs information from more than one earlier case. Use weak induction when $P(n)$ alone suffices.

Remark 6.4.16 (The base case in strong induction). When $n = 0$, the hypothesis $\forall k < 0,\ P(k)$ is vacuously true: there are no natural numbers less than 0. So the step for $n = 0$ requires proving $P(0)$ with no inductive hypothesis at all — this is the base case, and it must be proved directly.

In practice, always write the base case explicitly, even though it is implicit in the universal statement. Students who skip the base case in strong induction often write "by the inductive hypothesis applied to $n-1$" when $n = 0$, which is an error: $n - 1 = -1$ is not a natural number and has no inductive hypothesis.

Remark 6.4.17 (When to use strong induction: the diagnostic). The diagnostic for strong induction is simple. Write out the inductive step: $P(n+1) = f(P(?), P(?), \ldots)$, and ask what earlier values of P you need. If the answer includes anything other than exactly $P(n)$, use strong induction.

Canonical situations where strong induction is needed:

- Recursive sequences with two prior terms: $a_{n+1} = a_n + a_{n-1}$ requires both $P(n)$ and $P(n-1)$.
- Prime factorization: writing $n = ab$ with $a, b < n$ requires $P(a)$ and $P(b)$ where a and b are arbitrary.
- Any argument that splits $n+1$ into parts: you need P at the part values, which are not predetermined.

Template: Strong Induction Proof

Let $P(n)$ denote: [*write it out*].

Base case ($n =$ [start]). [*Verify directly, with no inductive hypothesis.*]

Inductive step. Let $n >$ [start] and assume $P(k)$ holds for all [start] $\leq k < n$. *(Strong inductive hypothesis.)* We show $P(n)$ holds.
[*Use $P(k)$ for some specific $k < n$.*]
Hence $P(n)$ holds.
By strong induction, $P(n)$ holds for all $n \geq$ [start]. □

Example 6.4.18 (Every integer $n \geq 2$ has a prime factor). *Proof.* Let $P(n)$: "n has a prime factor."

Base case ($n = 2$). 2 is prime, so 2 is a prime factor of itself. $P(2)$ holds.

Inductive step. Let $n > 2$ and assume $P(k)$ holds for all $2 \leq k < n$.

Case 1: n is prime. Then n is its own prime factor, and $P(n)$ holds.

Case 2: n is composite. Write $n = ab$ with $2 \leq a, b < n$. By the strong inductive hypothesis applied to a, a has a prime factor p. Then $p \mid a$ and $a \mid n$, so $p \mid n$. Hence $P(n)$ holds.

In both cases $P(n)$ holds. By strong induction, $P(n)$ holds for all $n \geq 2$. □

Why weak induction cannot work directly: in Case 2, the value a satisfies $2 \leq a < n$ but is otherwise arbitrary. It is not necessarily $n-1$. Weak induction supplies $P(n-1)$, which is useless when $a \neq n-1$. Strong induction supplies $P(k)$ for all $k < n$, covering whatever value a takes.

Well-Ordering Principle

Every nonempty subset of $\mathbb{N}$ has a least element.

Remark 6.4.19 (The minimal counterexample argument). Well-ordering drives a proof strategy that is the structural dual of induction: the **minimal counterexample argument**.

Instead of climbing up from a base case, you assume the claim fails somewhere, take the smallest failure, and derive a contradiction from its minimality. The logic:

1. Suppose $P(n)$ fails for some n. Let $S = \{n \in \mathbb{N} : \neg P(n)\}$. By assumption $S \neq \emptyset$.
2. By well-ordering, S has a least element m.
3. Derive a contradiction: either show $P(m)$ holds (contradicting $m \in S$), or find some $k < m$ with $\neg P(k)$ (contradicting the minimality of m).

The key tool is minimality: the fact that m is the *smallest* failure means all smaller values are successes, which is a powerful assumption. It is equivalent to the strong inductive hypothesis.

Proposition (Induction and well-ordering are equivalent)

Proposition 6.4.20 (Induction and well-ordering are equivalent)**.** *Over the axioms of* $\mathbb{N}$ *without P5, the following three principles are logically equivalent:*

1. *The (weak) induction principle.*
2. *The well-ordering principle.*
3. *The strong induction principle.*

Remark 6.4.21 (Proof sketch of the equivalence). **Induction $\Rightarrow$ Well-ordering.** Suppose $S \subseteq \mathbb{N}$ has no least element. Let $P(n)$: "$n \notin S$." Then $P(0)$ holds (otherwise $0 \in S$ would be the least element). If $P(k)$ holds for all $k \leq n$, then $n+1 \notin S$ (otherwise $n+1$ would be the least element of S). By induction, $P(n)$ holds for all n, so $S = \emptyset$.

Well-ordering $\Rightarrow$ Induction. Suppose the base case $P(n_0)$ holds and the inductive step holds. Let $S = \{n \geq n_0 : \neg P(n)\}$. If $S \neq \emptyset$, let m be its least element. Since $P(n_0)$ holds, $m \neq n_0$, so $m > n_0$ and $m - 1 \geq n_0$. By minimality of m, $P(m-1)$ holds. But by the inductive step, $P(m-1) \Rightarrow P(m)$, contradicting $m \in S$. Hence $S = \emptyset$.

Template: Minimal Counterexample

Suppose for contradiction that $P(n)$ fails for some $n \in \mathbb{N}$.
Let $S = \{n \in \mathbb{N} : \neg P(n)\}$. By assumption $S \neq \emptyset$. By well-ordering, S has a least element m.
[*Derive a contradiction using minimality of m: either show $P(m)$ holds, or construct a smaller counterexample.*]
This contradicts [minimality of m / the base case]. Therefore $P(n)$ holds for all $n \in \mathbb{N}$. □

Example 6.4.22 (Minimal counterexample: exponential bound). **Claim.** For all $n \geq 1$, $2^n > n$.

Proof (minimal counterexample). Suppose the claim fails. Let $S = \{n \geq 1 : 2^n \leq n\}$. By assumption $S \neq \emptyset$; let m be its least element.

Since $2^1 = 2 > 1$, we have $m \neq 1$, so $m \geq 2$. Then $m - 1 \geq 1$ and $m - 1 < m$, so $m - 1 \notin S$ (by minimality of m). Thus $2^{m-1} > m - 1$.

Then:

$$2^m = 2 \cdot 2^{m-1} > 2(m-1) = 2m - 2 \geq m$$

(using $m \geq 2$ for the last inequality).

This contradicts $m \in S$ (which requires $2^m \leq m$). Therefore $S = \emptyset$ and the claim holds for all $n \geq 1$. □

The minimal counterexample argument proceeds by taking the smallest failure and showing it cannot be the smallest failure. The contradiction is always of the same form: either the "failure" was not a failure, or there is a smaller one.

Structural Induction (Preview). Induction on $\mathbb{N}$ is a special case of a general principle that applies to any recursively defined structure. You do not need $\mathbb{N}$ to do induction; you need a recursive definition.

Remark 6.4.23 (The general principle). Let X be a set defined by:

1. A collection of *base cases* (atomic elements not built from other elements), and
2. A collection of *constructor rules* (ways of building larger elements from smaller ones).

Structural induction on X: to prove $P(x)$ for all $x \in X$, prove P for all base cases, then prove that if P holds for the parts, it holds for the whole.

This is exactly standard induction where the "less than" ordering on $\mathbb{N}$ is replaced by the "proper sub-element" ordering on X.

Example 6.4.24 (Induction on algebraic terms). Define an *algebraic term* over a ring R by:

- Base case: any element $r \in R$ is a term.
- Constructor: if s, t are terms, so are $s + t$ and $s \cdot t$.

To prove a property P holds for all terms:

1. Prove $P(r)$ for all $r \in R$ (base cases).
2. Prove: if $P(s)$ and $P(t)$ hold, then $P(s + t)$ holds.
3. Prove: if $P(s)$ and $P(t)$ hold, then $P(s \cdot t)$ holds.

The recursive structure of terms determines the structure of the proof.

Remark (Exposition). Structural induction is the proof principle paired with structural recursion. Both follow the constructors of the object being studied. Structural recursion defines a function by giving its value on the base cases and then giving its value on each constructed object in terms of the values already defined on the immediate parts. Structural induction proves a predicate or property by showing that it holds for the base cases and is preserved by every constructor.

Thus structural induction is like ordinary mathematical induction: it proves a statement $P(x)$ for every object x in a recursively generated class. The difference is that the next step is not always the Peano successor $n \mapsto n{+}{+}$. The next step is whatever constructor builds the object. For natural numbers the constructors are 0 and successor. For formulas the constructors are atomic formulas, negation, binary connectives, and quantifiers. For trees the constructors are leaves and nodes.

A useful way to see this is to attach a syntax tree to each well-formed formula. Define trees for propositional formulas recursively:

- if $P \in \mathsf{Prop}$, then $\mathrm{leaf}(P)$ is a tree;
- if T is the tree for φ, then $\mathrm{node}(\neg, T)$ is the tree for $\neg\varphi$;
- if T_φ and T_ψ are the trees for φ and ψ, then $\mathrm{node}(\circ, T_\varphi, T_\psi)$ is the tree for $(\varphi \circ \psi)$.

Operations on this tree are recursive in the same sense. The depth of a leaf is 0; the depth of a negation node is one plus the depth of its child; the depth of a binary node is one plus the maximum of the depths of its two children. Structural induction then proves statements about all such trees, and hence about all formulas, by checking the leaf case, the negation-node case, and the binary-node case. The proof follows the same recursive blueprint as the definition.

Remark 6.4.25 (Where structural induction appears in this project). Induction on $\mathbb{N}$ is structural induction on the Peano structure $(\mathbb{N}, 0, S)$: the base case is $n = 0$, the constructor is $n \mapsto n{+}{+}$.

In the Abstract Algebra chapter, polynomial properties are proved by induction on degree, which is structural induction on the recursive definition of polynomials over a ring.

In computer science, correctness of recursive algorithms is proved by structural induction on the input data structure (lists, trees, expressions). The full treatment of structural induction appears in the Abstract Algebra chapter.

Common Induction Mistakes.

Remark 6.4.26 (Mistake 1: Missing base case). A proof that establishes the inductive step but not the base case proves nothing. The step says: *if* $P(n)$ holds for some n, *then* $P(n+1)$ holds. Without a base case, there is no n for which $P(n)$ is established, so the chain never starts.

The classic illustration: the false theorem "all horses are the same colour." The base case for $n = 1$ is trivially true (one horse has one colour). The inductive step has a subtle flaw in the overlap argument. The lesson is that both parts are essential and independent. Always verify the base case, even when it seems trivial. A trivial base case takes one line; write the one line.

Remark 6.4.27 (Mistake 2: Circular inductive step). The inductive step proves $P(n+1)$ using only $P(n)$ and previously established facts. It is a fatal error to assume $P(n+1)$ at any point in the proof of $P(n+1)$.

This error most often appears when a student "derives" both sides of an equation and meets in the middle, inadvertently assuming what they are trying to prove. The symptom: the final few lines of the step are not deductions from $P(n)$ but rather manipulations of $P(n+1)$ itself.

The cure: always work in one direction. Start with the left side of $P(n+1)$ (or whatever the goal is), apply the inductive hypothesis once, and algebraically arrive at the right side. Never write the target as an intermediate line and work toward it from both sides.

Remark 6.4.28 (Mistake 3: Unstated inductive hypothesis). The most common error in practice is writing "assume $P(n)$" without saying what $P(n)$ actually says. This is a sign that $P(n)$ has not been written out as a full sentence.

The effect is that the inductive step becomes vague at the crucial moment: the student needs to use the inductive hypothesis but cannot cite it precisely, so they either paraphrase it loosely (which may introduce errors) or implicitly use a stronger version than what was stated.

Write $P(n)$ as a complete sentence at the top of the proof. Then write "assume $P(n)$: [full sentence]." This forces precision and prevents the most common errors in the body of the proof.

Remark 6.4.29 (Mistake 4: Wrong base case). Induction proves $P(n)$ for all $n \geq n_0$, where n_0 is whatever value you use as the base case. If the claim is only true for $n \geq 2$, the base case $n = 0$ will fail, and the proof is invalid.

Always check: what is the smallest n for which the claim is true? That is your base case. Do not default to $n = 0$ or $n = 1$ without verifying that the claim holds there. For divisibility statements, $n = 0$ often gives $0 = 0 \cdot k$, which works. For growth bounds, $n = 1$ or $n = 2$ may be needed. For Fibonacci identities, specific small values may require individual checking.

Remark 6.4.30 (Mistake 5: Using strong induction where weak suffices, and vice versa). Strong induction is not automatically better than weak. A proof that invokes the full strong hypothesis "$P(k)$ for all $k < n$" when only $P(n-1)$ is actually used is unnecessarily complicated and obscures the proof's logic.

Conversely, a proof that tries to use weak induction on a recursion requiring two prior terms will stall at the step.

The diagnostic: write the step and check explicitly which prior values you use. If only $P(n)$, use weak. If $P(k)$ for some $k < n$ that is not $n - 1$, use strong.

Remark 6.4.31 (Mistake 6: Forgetting vacuity in strong induction). In strong induction, the case $n = 0$ has a vacuously true hypothesis $\forall k < 0,\ P(k)$, since no natural number is less than 0. This means $P(0)$ must be proved *directly*, not from the inductive hypothesis. Many students write "by the inductive hypothesis applied to $n - 1$" for $n = 0$, which refers to $n - 1 = -1$, a non-existent value.

The base case in strong induction is always proved from scratch. The strong hypothesis is only available for $n \geq 1$.

6.5 Algebraic Tactics

The DU/TA/AM Framework. Every line of an algebraic proof is justified by exactly one of three sources. This framework makes that obligation explicit and provides the tagging system used throughout these notes.

The Three Line Tags

- DU **(Definition / Unpack).** The step applies a definition or unpacks a named concept into its explicit conditions. *Example*: expanding "e is an identity" into "$ea = ae = a$ for all $a \in G$".
- TA **(Theorem / Axiom).** The step cites a previously proved proposition or a structural axiom (group axiom, ring axiom, field axiom, Peano axiom). *Example*: citing G1 (associativity) to regroup $(ab)c = a(bc)$.
- TA **(Algebraic Manipulation, abbreviated AM).** The step performs a computation whose justification is itself a combination of DU and TA moves, compressed for readability. *Example*: simplifying $a \cdot e = a$ in one step after the identity axiom has already been cited.

Remark 6.5.1 (Why the framework matters: finding where proofs break). The DU/TA/AM

system serves a diagnostic purpose. Every unjustified step in a proof is either a DU step that has not been taken (some definition has not been unpacked) or a TA step that has not been cited (some axiom or theorem is being used silently).

When an algebraic proof stalls, the right question is: which definition have I not yet unpacked? Which axiom am I using but not citing? The tags make these omissions visible.

In practice, stalling almost always traces to a definition that has been left in its named form ("G is a group") when it needed to be expanded into operative conditions ("associativity holds; an identity exists; every element has an inverse"). Expand the definition and the stuck proof typically becomes unstuck.

Remark 6.5.2 (The hierarchy: DU and TA are atomic; AM is shorthand). DU and TA are the atomic justifications. There is nothing more basic. AM is not a separate category of reasoning — it is a compressed sequence of DU and TA steps that have been verified and labeled as a single unit for concision.

A fully formal proof contains only DU and TA steps, explicitly tagged. The three-column proof format used in this project — tag in the left column, statement in the middle, commentary in the right — enforces explicit justification of every line and is the recommended format while you are learning.

As fluency develops, AM steps can absorb more and more of the lower-level manipulations. But the underlying DU/TA structure is always present, even when not written. The ability to decompose any AM step into its DU and TA components is the hallmark of genuine understanding.

Remark 6.5.3 (A proof is a chain of citations). The deepest way to understand the DU/TA/AM system is this: a mathematical proof is a sequence of statements, each of which is justified by citing something that is already known (a definition, an axiom, or a previously proved theorem). The proof is valid if and only if every step has a valid citation.

When you read a proof and feel uncertain whether it is correct, the right move is to label every step. If you can assign a DU or TA justification to every line, the proof is valid. If any line has no justification, there is a gap.

This is not just a learning technique. Working mathematicians who find a suspicious step in a proof do exactly this: they ask "what justifies this?" and try to answer. If no answer is available, the step is an error.

Catalog of Algebraic Tactics. The following catalog lists the specific algebraic moves available once a proof is in progress and the architecture has been chosen. Each entry states the tactic, its precondition (what must already be established or assumed), and its effect (what it achieves in the proof).

Tactic	Precondition	Effect
Multiply by inverse	$a \neq 0$ in a field; or $a \in G$ in a group	Multiply both sides by a^{-1}; use associativity to collapse $a^{-1}a = e$; isolates the target variable.
Cancellation	$ac = bc$ (or $ca = cb$) in a group or integral domain	Conclude $a = b$. In a group: multiply by c^{-1}. In an integral domain: cite no-zero-divisors.
Distributivity bridge	Multiplication by 0, -1, or a negative element	Rewrite the product as a sum using distributivity, then use additive group structure to collapse. Crosses from $\times$ to $+$.
Add zero	Need to introduce a term without changing the expression	Write $a = a + 0 = a + (b + (-b))$; regroup to expose the needed structure.
Satisfy-and-cite	A uniqueness theorem for some named object b is in scope	Show a satisfies b's defining property; cite uniqueness; conclude $a = b$. Avoids assume-two-and-compare entirely.
Double negation	$-(-a)$ appears or is the goal	$-a$ is the unique element x with $a + x = 0$; $-(-a)$ satisfies this; by uniqueness, $-(-a) = a$.
Absorb into axiom	A subexpression matches the LHS of an axiom	Rewrite using the axiom. Tag as TA.

Remark 6.5.4 (The distributivity bridge in detail). The distributivity bridge is the central tactic for ring and field proofs involving zero and negatives. Every instance of "$a \cdot 0 = 0$" and "$a(-b) = -(ab)$" uses it. Once you have seen the pattern, it is immediately recognizable.

The pattern always has four steps:

1. Introduce $a \cdot 0$ or $a \cdot (-b)$ into the argument.
2. Expand: write $0 = x + (-x)$ or $-b = b + (-b) + (-b)$ using the additive inverse definition.
3. Apply distributivity: $a(x + y) = ax + ay$.
4. Use uniqueness of the additive inverse to identify the result with $-(ab)$ or 0.

The reason this requires distributivity as a bridge is that the additive structure (with its identity 0 and its inverses $-a$) and the multiplicative structure (with its products ab) are separate. The only connection between them is distributivity. To get from the multiplicative side $(a \cdot 0)$ to the additive side $(0 = 0 + 0)$, you must cross the bridge.

Remark 6.5.5 (Cancellation: two theorems that must not be conflated). "Cancellation" names two distinct results with different preconditions.

Group cancellation. In any group, $ac = bc \Rightarrow a = b$. Proved by multiplying both sides on the right by c^{-1}, which always exists in a group. The hypothesis $c \neq 0$ is not needed because inverses always exist in a group.

Domain cancellation (or integral domain cancellation). In an integral domain, $ac = bc$ and $c \neq 0$ imply $a = b$. Proved by rewriting as $(a - b)c = 0$ and invoking the no-zero-divisors property. The hypothesis $c \neq 0$ is essential: without it, cancellation can fail (e.g., in $\mathbb{Z}_6$, $2 \cdot 3 = 4 \cdot 3$ but $2 \neq 4$).

The error to avoid: using group cancellation in an integral domain argument (it works but uses stronger machinery than needed and may not be available), or trying to use domain cancellation without verifying $c \neq 0$.

Remark 6.5.6 (Add zero: the simplest but most overlooked tactic). The add-zero tactic sounds trivial but appears in almost every algebraic proof where an intermediate term needs to be introduced without changing the value of an expression. The template is:

$$a = a + 0 = a + (b + (-b)) = (a + b) + (-b).$$

This exposes $a + b$ as a subexpression of a, which can then be worked with. It is the algebraic equivalent of adding and subtracting the same term in a real analysis inequality argument.

When you need a term to appear in an expression and cannot see how to introduce it, ask: can I write some zero as $x + (-x)$ and distribute?

6.6 Proof Strategies Reference

Lookup Table: Stuck Situations and Strategies.

Remark 6.6.1 (How to use this table). When progress on a proof stalls, locate the row that describes your situation. The middle column names the strategy. The right column gives the specific first move and a cross-reference to the section where that strategy is developed in detail.

This table is not a substitute for understanding the strategies; it is a navigation aid for when you know what you are trying to do but have forgotten the precise machinery. The full explanations and worked examples are in the sections cited.

Stuck because...	Strategy	First move / See
I need to prove P but don't know where to start	Two-question check	Write the logical form of P. Consult the statement map. §6.1
I need to show two things are equal and one is uniquely defined	Satisfy-and-cite	Show the first satisfies the definition of the second. Cite the uniqueness theorem. §6.1
I need to show something is the only object of its kind	Assume-two-and-compare	"Let x, y both satisfy the definition. Show $x = y$." §6.3
I have a conditional $P \Rightarrow Q$ and P is hard to use forward	Contrapositive	"Assume $\neg Q$." Derive $\neg P$. §6.3
I have a conditional and the conclusion seems impossible to reach directly	Contradiction	"Assume P and $\neg Q$." Derive any contradiction. §6.3
The claim splits naturally into a finite number of cases	Case analysis	List all cases explicitly. Verify exhaustiveness first. §6.3
The claim is about all $n \in \mathbb{N}$	Weak induction	Write $P(n)$ as a full sentence. Prove the base case directly. §6.4
The inductive step needs $P(k)$ for some $k < n$ (not just $n - 1$)	Strong induction	Replace "assume $P(n)$" with "assume $P(k)$ for all $k < n$." §6.4
The claim is about all $n \geq n_0$ and feels easier to negate	Minimal counterexample	"Let $S = \{n : \neg P(n)\} \neq \emptyset$. Let $m = \min S$." §6.4
I need to prove $a \cdot 0 = 0$ or $a(-b) = -(ab)$ in a ring	Distributivity bridge	Write $0 = x + (-x)$; apply distributivity; use uniqueness of additive inverse. §6.5
I have $ac = bc$ and need $a = b$	Cancellation	Group: multiply by c^{-1}. Domain: write $(a - b)c = 0$; cite no zero divisors. §6.5
I need to prove two sets are equal	Double inclusion	Prove $A \subseteq B$: let $x \in A$ arbitrary, derive $x \in B$. Then prove $B \subseteq A$. §6.1
A definition has not been unpacked	DU move	Identify the named concept and write out its definition. Tag the line DU. §6.5
A step uses an axiom or theorem by name	TA move	Cite the axiom or theorem explicitly by label or full name. Tag the line

Remark 6.6.2 (Cross-references to primary worked examples). The following table maps each strategy to a worked example in these notes where it is the primary tool.

Strategy	**Primary worked examples**
Assume-two-and-compare	Uniqueness of group identity (this section)
Satisfy-and-cite	$(ab)^{-1} = b^{-1}a^{-1}$; $a(-b) = -(ab)$ (this section)
Distributivity bridge	$a \cdot 0 = 0$; $a(-b) = -(ab)$ (Tactics section)
Cancellation (group)	Group left-cancellation (Tactics section)
Cancellation (domain)	Integral domain cancellation (Tactics section)
Weak induction	Sum formula $\sum k = n(n+1)/2$; $3 \mid 4^n - 1$
Strong induction	Fibonacci bound; prime factor theorem
Minimal counterexample	$2^n > n$
Contrapositive	n^2 even $\Rightarrow$ n even
Contradiction	$\sqrt{2} \notin \mathbb{Q}$

6.7 Analysis: What Changes and Why

Why Analysis Proofs Feel Different. Everything in the preceding sections applies to mathematics broadly: direct proof, induction, contradiction, and the rest are the universal grammar of mathematical argument. But real analysis — sequences, metric spaces, limits, continuity — introduces a particular *flavour* of proof work that surprises many students when they first encounter it.

This section prepares you for that surprise before you arrive at Volume III. The goal is not to teach you analysis; it is to give you the framework so that when you get there, the proofs do not feel alien.

Remark 6.7.1 (The shift from computation to verification). In calculus, most tasks are *computational*: differentiate this function, evaluate this integral, find this limit by l'Hôpital's rule. You apply known procedures and the answer is a number or a function.

In analysis, most tasks are *verificational*: prove that this sequence converges, show that this space is complete, establish that this function is continuous. The answer is a logical argument. There is no procedure to apply; there is a structure to find.

This shift is not just psychological. It changes what "progress" means. In a computation, you know you are making progress when the expressions simplify. In a proof, you know you are making progress when the definitions are unpacked, the hypotheses are applied, and the conclusion comes into reach. The habits of mind required are different.

Remark 6.7.2 (Definitions are not background; they are the machinery). The single most important habit in analysis is **returning to definitions**. Every concept — convergence, Cauchy, compactness, continuity — has a precise definition in terms of quantifiers and

inequalities. That definition is not background knowledge. It is the working machinery of every proof involving that concept.

Here is the canonical example. To prove that $1/n \to 0$, a calculus student says: "obviously it approaches zero." An analysis student unpacks the definition:

For every $\varepsilon > 0$, there exists $N \in \mathbb{N}$ such that $n \geq N$ implies $|1/n - 0| < \varepsilon$.

Then they find N: choose $N > 1/\varepsilon$ (possible by the Archimedean property). For $n \geq N$: $1/n \leq 1/N < \varepsilon$.

The proof is three lines, and every line is either the definition (unpacked), the Archimedean property (a theorem), or arithmetic. There is no mystery. The entire proof is the definition plus two lines of calculation. This is the prototype of an analysis proof.

The lesson: when a proof stalls, the first question to ask is always: *which definition have I not yet unpacked?*

Remark 6.7.3 (Analysis proofs are structured, not creative). Here is perhaps the most liberating thing to understand about analysis: **proofs are far less creative than they look.**

Experienced analysts do not invent proofs from scratch. They recognize the type of exercise, recall the standard move for that type, and apply it. The vocabulary of standard moves is small — twelve moves cover virtually all of the proofs in a first course in metric spaces. The creativity, if any, is in recognizing which move to apply.

This is the same observation that governs the Architecture section of this chapter (Section 6.1), scaled down to the specific landscape of analysis. The statement map tells you the global structure. In analysis, a more fine-grained map is available: there are specific named moves for specific recurring situations. Learning to name them is the same skill as learning to read a statement and immediately identify its proof type.

The rest of this section makes that vocabulary explicit.

The Six Exercise Types. Every exercise in sequences and metric spaces belongs to one of six structural types. Identifying the type is the first step before writing anything. The type determines the shape of the solution — not the details, but the overall structure.

Quick Reference: The Six Types

Type	Signal words	Shape	First move
Proof	*Prove, show, establish*	Hypotheses to conclusion	Unpack all definitions
Verification	*Verify that . . . is a metric, show . . . satisfies*	Check each condition	List every condition
Prove/Counterexample	*True? Prove or find a counterexample*	Decide, then justify	Decide first
Construction	*Find, construct, exhibit, give an example of*	Name the object; verify it	Name the object
Computation	*Compute, calculate, find the diameter of*	Evaluate from a definition	Write the definition
Characterization	*If and only if, characterize all . . . that*	Two separate directions	Split two directions

Remark 6.7.4 (Type 1 (Proof): you are told the statement is true). In a proof exercise, the statement is true and your job is to trace the logical path from the hypotheses to the conclusion. There is no choice to make about truth value.

The opening move is always the same: unpack every definition in the hypotheses and in the conclusion. The definitions turn abstract concepts into concrete quantifier-and-inequality statements that can be manipulated. Without this step, you have nothing to work with.

Example. *Prove that every convergent sequence is Cauchy.* Unpack "convergent": $\forall\, \varepsilon > 0\ \exists\, N$ s.t. $n \geq N \Rightarrow d(x_n, p) < \varepsilon$. Unpack "Cauchy": $\forall\, \varepsilon > 0\ \exists\, M$ s.t. $m, n \geq M \Rightarrow d(x_m, x_n) < \varepsilon$. Now the proof goal is visible: use the convergence condition (which controls $d(x_n, p)$) to establish the Cauchy condition (which asks for control over $d(x_m, x_n)$). The triangle inequality bridges them.

Remark 6.7.5 (Type 2 (Verification): check every condition). In a verification exercise, a specific named object is given and you must confirm it satisfies every condition of a definition. The definition has a finite list of conditions. Your job is to work through every one, explicitly and separately.

The error students make: checking four out of five conditions and writing QED. A verification is complete only when every condition is checked. When you begin a verification, write the complete list of conditions first. Then work through them in order.

Example. *Verify that the discrete metric is a metric.* A metric has four conditions: non-negativity, identity of indiscernibles, symmetry, triangle inequality. List all four. Check each one. The triangle inequality requires a case split (on whether $x = z$ or $x \neq z$), which is itself a standard move in analysis.

Remark 6.7.6 (Type 3 (Prove or counterexample): decide before you argue). This is the most dangerous type for students who rush. The exercise gives you a statement and asks you to determine its truth value, then justify. If you skip the determination stage and try to prove a false statement, you will waste effort and grow confused.

Stage 1 is always: decide. Test the statement on simple cases — $\mathbb{R}$ with the standard metric first, then discrete spaces, then other special cases. One counterexample falsifies; failure to find a counterexample after testing several cases suggests the statement is true.

Stage 2: justify. If true, write a proof. If false, exhibit a specific counterexample and verify it fails.

Example. *Is the open ball always equal to the closed ball?* Test in $\mathbb{R}$: $B(0,1) = (-1,1)$ and $C(0,1) = [-1,1]$. The point -1 is in $C(0,1)$ but not $B(0,1)$. False. Exhibit this specific counterexample in the solution, with the witness (-1) and the calculation ($d(-1,0) = 1$, so $-1 \in C(0,1)$ and $-1 \notin B(0,1)$).

Remark 6.7.7 (Type 4 (Construction): name first, verify second). A construction exercise asks you to produce an object with given properties. The solution has two parts: stating the object, and verifying it has every required property. Both parts are required.

State the object completely and unambiguously before any verification. If you cannot state it clearly, you do not yet have a solution. Then verify every required property from scratch — do not assume properties that were not established.

Example. *Find a metric on* $\mathbb{N}$ *such that* $\mathbb{N}$ *is bounded.* The idea: embed $\mathbb{N}$ into $(0,1] \subset \mathbb{R}$ via $n \mapsto 1/n$ and pull back the metric. Define $d(m,n) = |1/m - 1/n|$. Then verify: (i) it is a metric (four conditions), and (ii) it is bounded ($d(m,n) \leq 1$ for all m, n).

Remark 6.7.8 (Type 5 (Computation): the definition is the answer). In a computation exercise, a specific numerical or set-theoretic value is asked for. The answer always comes from writing out the relevant definition as a supremum, infimum, or set, and then evaluating it.

Do not skip straight to the answer. Write the definition first. This is not just for clarity — it prevents errors that arise from misremembering what is being computed.

Example. *Compute* $\operatorname{diam}([a,b])$. Write: $\operatorname{diam}([a,b]) = \sup\{|x-y| : x, y \in [a,b]\}$. Upper bound: $|x-y| \leq b-a$ for all $x, y \in [a,b]$. Attained: $|b-a| = b-a$. Therefore $\operatorname{diam}([a,b]) = b-a$.

Remark 6.7.9 (Type 6 (Characterization): two proofs, not one). A characterization exercise asks for an equivalence or an if-and-only-if. The solution is always two separate proofs, labeled ($\Rightarrow$) and ($\Leftarrow$). Label them before writing a single line.

The two directions often require completely different techniques and different amounts of work. The ($\Rightarrow$) direction may be three lines while the ($\Leftarrow$) direction requires a sequence argument. Writing them separately prevents the error of running arguments in both directions simultaneously and losing track of which assumptions are in play.

Example. *Show that a sequence converges if and only if it is eventually constant (in the discrete metric).* ($\Leftarrow$): Suppose eventually constant; then convergence holds trivially. ($\Rightarrow$): Suppose it converges; apply the definition with $\varepsilon = 1/2$ and use the fact that distances in the discrete metric are either 0 or 1.

The Twelve Canonical Proof Moves. A *proof move* is a recurring logical or computational technique that appears across many different proofs in analysis. Once you learn to name these moves, reading and writing proofs becomes a matter of recognizing which move applies — not inventing an argument from scratch.

The moves below are not tricks. Each one encodes a genuine mathematical idea. But once the idea is understood, the move can be deployed almost mechanically in the right situations.

The Twelve Moves at a Glance

#	Move	Use when ...	Level
1	Unpack a definition	starting any proof	L1
2	Triangle inequality	need to bound $d(x, z)$ via y	L2
3	ε-splitting	triangle ineq. produces a sum	L2
4	Tail argument	combining two N's	L3
5	Case splitting	discrete or binary structure	L2
6	Choose N explicitly	ε-N arguments	L3
7	Contradiction	uniqueness or non-existence	L4
8	Subsequence extraction	sequence misbehaves globally	L3
9	Squeeze (sandwich)	bound $d(x_n, p)$ from above	L2
10	Inherited metric / embedding	constructing a new metric	L4
11	Open cover / finite subcover	compactness arguments	L4
12	Diagonal argument	simultaneous subsequence conditions	L3/4

Remark 6.7.10 (Move 1: Unpack a definition). This is always the first move. Every analysis concept — convergence, Cauchy, compactness, closure, diameter — is a compressed form of a precise statement with quantifiers and inequalities. Unpacking the definition turns the abstract concept into something you can work with.

Without this step, you have named objects but no machinery. With it, you have the raw material for every subsequent move.

Template. Write out the definition of every term in the hypothesis and in the goal. The hypotheses become your assumptions; the goal becomes your target inequality or quantifier statement.

Remark 6.7.11 (Moves 2 and 3: Triangle inequality and ε-splitting). These two moves almost always appear together. The triangle inequality lets you insert an intermediate point y and split $d(x, z)$ into $d(x, y) + d(y, z)$. ε-splitting means choosing the bounds on each piece so they add up to ε.

The intermediate point y is not arbitrary: it is chosen to be something you already have control over (a limit, a fixed center, a sequence element). The fractions $\varepsilon/2$, $\varepsilon/3$ are chosen *before* picking N, so that the final bound comes out to ε exactly.

Template. When you need $d(x_m, x_n) < \varepsilon$ and you know each term is close to a limit p: insert p via the triangle inequality, budget $\varepsilon/2$ for each piece, and choose N so both pieces are bounded.

Remark 6.7.12 (Moves 4 and 6: Tail argument and choosing N). The ε-N definition of convergence requires you to *produce* an N. This is Move 6: work backwards from the target inequality to determine what N must satisfy, then assert its existence (typically by the Archimedean property: for any $\varepsilon > 0$ there is an integer greater than $1/\varepsilon$).

Move 4 (the tail argument) handles the situation where two separate conditions each give their own N_i. The resolution: take $N = \max(N_1, N_2, \ldots)$. For $n \geq N$, all conditions hold simultaneously.

These two moves are the engine of every ε-N argument. They appear in virtually every proof about convergent sequences.

Remark 6.7.13 (Move 5: Case splitting in analysis). Case splitting is the same structural technique described in Section 6.3, now applied to the specific context of metric spaces. The most common case split in metric space proofs is: $x = y$ (distance is 0) versus $x \neq y$ (distance is 1 in the discrete metric, or some positive value in other spaces).

Case splitting is particularly important in verification exercises: the triangle inequality almost always requires splitting on whether the two outer points coincide.

Remark 6.7.14 (Move 7: Contradiction in analysis). In analysis, contradiction is used most characteristically for *uniqueness* results: suppose two limits exist; assume they are different; derive a contradiction from the triangle inequality and the convergence hypothesis.

The template: suppose the goal fails (e.g., two limits exist with $d(x, y) > 0$). Choose $\varepsilon = d(x, y)/2$. Then for large enough n, both $d(x_n, x) < \varepsilon$ and $d(x_n, y) < \varepsilon$. The triangle inequality gives $d(x, y) \leq 2\varepsilon = d(x, y)$, a contradiction.

Notice: contradiction here uses the triangle inequality (Move 2) and the tail argument (Move 4) internally. Proof moves nest and combine.

Remark 6.7.15 (Moves 8 and 12: Subsequence extraction and diagonal argument). Subsequence extraction is the operation of selecting indices $n_1 < n_2 < \cdots$ so that the restricted sequence $\{x_{n_k}\}$ has some desired property (being monotone, being close to a given point, etc.). It is a Level 3 operation: you are selecting from an infinite sequence, which requires genuine sequence machinery.

The diagonal argument applies subsequence extraction infinitely many times and diagonalizes the result: the k-th term of the diagonal sequence is $x_k^{(k)}$, where $\{x_n^{(k)}\}$ is the k-th extracted subsequence. The diagonal sequence satisfies all conditions simultaneously. This is the technique behind Arzelà–Ascoli and related compactness results.

Remark 6.7.16 (Move 9: The squeeze (sandwich)). If you can show $0 \leq d(x_n, p) \leq c_n$ where $c_n \to 0$, then $x_n \to p$ follows immediately: any ε that works for c_n also works for $d(x_n, p)$.

The squeeze is particularly useful when $d(x_n, p)$ is hard to bound directly but can be dominated by a simpler sequence whose convergence is known.

Remark 6.7.17 (Move 10: Inherited metric / embedding). To construct a metric on a new space X, it is often easiest to find an injective function $f : X \to Y$ into a space Y whose metric is known, and define $d_X(x, z) := d_Y(f(x), f(z))$. All four metric axioms are then inherited from d_Y via f. The only things to verify are injectivity of f (to ensure the identity axiom) and whatever additional properties are required.

This is a Level 4 move because it operates on the global structure of the space: you are choosing an ambient space and an embedding, not just manipulating inequalities.

Remark 6.7.18 (Move 11: Open cover / finite subcover). Compactness says: every open cover of a compact set K has a finite subcover. The power of this is that it converts infinite problems into finite ones. If you need a single bound δ that works everywhere on K, you find a δ_x at each x (infinitely many), then extract a finite subcover and take the minimum of finitely many δ_x. A finite minimum exists; a minimum over infinitely many values might not.

This move appears in all compactness arguments: uniform continuity on compact sets, compactness implies boundedness, compactness implies completeness in complete metric spaces.

The Four-Layer Hierarchy. The twelve moves are not all at the same level. They form a four-layer hierarchy that reflects the logical structure of metric space proofs: each layer depends on the one below it, and together the four layers describe a progression from the most basic (unpacking definitions) to the most structural (global topological reasoning).

Understanding the hierarchy is not an organizational convenience. It tells you something deep: *why* metric spaces are defined the way they are, and why nearly every proof in the subject follows the same basic upward shape. Once you see this, metric space theory stops looking like a collection of unrelated tricks and starts looking like a small, elegant machine built from a few core mechanisms.

The Four Levels

L	Name	Moves	What it does
L1	Definition Expansion	Move 1	Turns abstract concepts into inequalities
L2	Inequality Mechanics	Moves 2, 3, 5, 9	Combines and bounds distances
L3	Sequence Control	Moves 4, 6, 8, 12	Controls asymptotic and tail behavior
L4	Structural / Topological	Moves 7, 10, 11	Reasons about the space as a whole

Remark 6.7.19 (Level 1: Definition Expansion — the foundation of everything). Every other move operates on inequalities. Those inequalities do not appear until you unpack a definition. Before Level 1, you have a statement about abstract objects. After Level 1, you have quantifiers and specific inequality conditions that can be manipulated.

This is why Level 1 is always the first step in any analysis proof, regardless of how complex the argument becomes. The inequality $d(x_n, p) < \varepsilon$ does not exist in the proof until you write it down. You write it down by unpacking the definition of convergence.

What Level 1 produces:

- "$x_n \to p$" $\mapsto \forall\, \varepsilon > 0\ \exists\, N$ s.t. $n \geq N \Rightarrow d(x_n, p) < \varepsilon$
- "$\{x_n\}$ is Cauchy" $\mapsto \forall\, \varepsilon > 0\ \exists\, M$ s.t. $m, n \geq M \Rightarrow d(x_m, x_n) < \varepsilon$
- "K is compact" $\mapsto$ every open cover of K has a finite subcover
- "E is closed" $\mapsto$ every convergent sequence in E converges to a point of E

The abstract word is a compressed form of the inequality. Unpacking it is not simplification; it is *activation*.

Remark 6.7.20 (Level 2: Inequality Mechanics — the algebra of metric spaces). Once Level 1 has produced inequalities, Level 2 manipulates them. This is the algebra of metric spaces: you combine, bound, split, and chain inequalities to move from what you know toward what you need.

The triangle inequality is a *metric axiom* for a reason: it was included in the definition of a metric precisely because it is the tool needed to chain distance bounds. Without it, you cannot get from control over $d(x, y)$ and $d(y, z)$ to control over $d(x, z)$. The axiom is there because it enables Level 2.

Notice that Level 2 does not involve sequences, indices, or tails. It is purely about bounding distances between specific points. The sequence machinery comes at the next level.

Remark 6.7.21 (Level 3: Sequence Control — the engine of limit arguments). Analysis is fundamentally about sequences approaching limits, distances becoming arbitrarily small, behavior stabilizing as an index grows large. Level 3 controls this asymptotic behavior.

The key idea: sequence control is about managing the word "eventually." The definition of convergence says that for large enough n, the distance $d(x_n, p)$ is small. The tail argument combines multiple "large enough n" conditions by taking the maximum index. Choosing N explicitly produces the specific threshold. Subsequence extraction selects which terms of the sequence to keep.

These moves require genuine sequence machinery — you are reasoning about indexed infinite processes, not just about individual distances between points. That is what places them at Level 3 rather than Level 2.

Remark 6.7.22 (Level 4: Structural / Topological — global reasoning). Level 4 moves operate on the global structure of the metric space, not on individual sequences or inequalities. Contradiction reasons about the logical structure of the whole proof. Embedding reasons about how the space sits inside a larger ambient space. Compactness reasons about the global covering properties of the space.

Compactness is the flagship Level 4 property. It converts infinite problems into finite ones: an infinite family of open sets reduces to a finite subcover, and a finite collection has a minimum, a maximum, a single controlling constant. This is the most powerful tool in metric space theory.

Level 4 and Level 3 interact bidirectionally. Sequential compactness (every sequence has a convergent subsequence) is a Level 3 characterization of a Level 4 property. Many deep results in analysis are precisely about these interactions.

Remark 6.7.23 (Reading a proof using the hierarchy). When you read a proof in the textbook, label each step by level. This is not a mechanical exercise; it reveals the logical architecture and tells you why each step appears where it does.

Here is the proof that convergent implies Cauchy, labeled:

Let $\varepsilon > 0$. *[L1: introduce ε from the Cauchy definition]*

Since $x_n \to p$, there exists N such that $n \geq N \Rightarrow d(x_n, p) < \varepsilon/2$. *[L3: choose N from the convergence hypothesis]*

For $m, n \geq N$: *[L3: tail condition]*

$d(x_m, x_n) \leq d(x_m, p) + d(p, x_n)$ *[L2: triangle inequality]*

$< \varepsilon/2 + \varepsilon/2 = \varepsilon.$ *[L2: ε-splitting]*

Hence $\{x_n\}$ is Cauchy. *[L1: conclusion matches Cauchy definition]*

This proof uses Levels 1, 2, and 3, but not Level 4. It is a local argument about sequences, not a global structural argument. To reach Level 4, you would need compactness or an embedding argument.

The levels a proof reaches tell you its depth and what tools it uses. A verification exercise stays at Levels 1–2. A limit argument reaches Level 3. A compactness argument reaches Level 4.

Remark 6.7.24 (Why the metric axioms are the axioms they are). The metric axioms — non-negativity, identity of indiscernibles, symmetry, triangle inequality — are not arbitrary. They are exactly the conditions needed to make Level 2 possible.

Without the triangle inequality, you cannot chain distance bounds. Without identity of indiscernibles, the notion of a limit point degenerates. Without symmetry, the definitions of balls and convergence become asymmetric and unworkable.

The axioms are the *minimal conditions on a distance function that support the full four-level proof hierarchy.* When you study a new property of metric spaces, ask: which level does it live at? The answer tells you what tools to expect and what kind of proofs will be required.

How to Read an Analysis Exercise Before Writing Anything. The preceding sections have given you the vocabulary: six exercise types, twelve proof moves, four levels. This section shows how to put them together in the five minutes before you write your first symbol.

Pre-Writing Checklist

1. **Identify the type.** Which of the six types is this: Proof, Verification, Prove-or-Counterexample, Construction, Computation, Characterization? Write the type at the top of your scratch paper before anything else.
2. **State the hypotheses and the goal.** In plain English, what are you assuming? What are you trying to show? Do not proceed until both are stated explicitly.
3. **Unpack every definition in the goal.** If the goal involves a named concept — convergence, compactness, closed, bounded — write out its definition in full. The unpacked goal is your actual target.
4. **Identify the key move.** Which of the twelve moves is most likely to bridge the hypothesis to the unpacked goal? Which level does that move belong to? Knowing the level tells you roughly how deep the proof will go and what other moves will likely appear alongside it.
5. **Sketch the shape in words.** Write in plain language: "I will choose $\varepsilon > 0$. I will use convergence of $\{x_n\}$ to find N_1. I will use convergence of $\{y_n\}$ to find N_2. I will take $N = \max(N_1, N_2)$. Then I will apply the triangle inequality." Only after this sketch do you fill in the symbols.

Remark 6.7.25 (The shape tells you what you need before you need it). Step 5 is the most important and the most skipped. Students who jump directly to symbols frequently paint

themselves into corners: they choose N for one condition and then need it to satisfy a second condition that their choice does not support.

If you write the shape of the proof in words first, you see *how many conditions N must satisfy* before you commit to a specific value. Then you choose N to satisfy all of them simultaneously. The tail argument tells you how: take the maximum of however many separate N_i's the conditions require.

The shape also tells you how many terms the triangle inequality will produce and therefore what the right ε-splitting fractions are. These are not things you discover mid-proof; they are visible from the sketch.

Remark 6.7.26 (Common proofs and their shapes). The following table summarizes the shape of the most common proof patterns in a first analysis course, listed by the move sequence and the levels they traverse.

Goal	**Move sequence**	**Levels**
Convergent $\Rightarrow$ Cauchy	Unpack both definitions; triangle inequality; ε-splitting; tail argument (max of two N's).	L1–L3
Uniqueness of limits	Contradiction (assume two limits); triangle inequality; choose $\varepsilon = d(x,y)/2$; tail argument.	L2–L4
Convergent $\Rightarrow$ bounded	Unpack convergence; choose $\varepsilon = 1$; tail argument (bound $n \geq N$); take max of first N terms.	L1–L3
Verify d is a metric	List four conditions; check non-negativity, identity, symmetry directly; case split for triangle inequality.	L1–L2
Compute diameter of $[a,b]$	Write diameter as supremum; give upper bound; exhibit achieving pair.	L1–L2

Remark 6.7.27 (A note on templates). For each of the six exercise types, there is a standard template structure. These templates live in Appendix A of the companion *Primer for Real Analysis Exercises*, which you will receive at the start of Volume III. The templates are starting-point structures, not scripts to be memorized. Their purpose is to ensure that every solution has all required parts and that no structural element is accidentally omitted.

What matters at this stage is the vocabulary: six types, twelve moves, four levels. You now have the map. Volume III will fill in the terrain.

Bibliography